GUÍA DE APRENDIZAJE

Construir el frontend de IT Service & Demand Manager

React 19 · TypeScript · Vite · Tailwind CSS · shadcn/ui · TanStack Query · React Hook Form · Zod

Edición completa · 26 capítulos y 5 anexos

Preparada para Alberto de Rafael Fernández
Octubre de 2026

Índice

Cada entrada es un enlace. Desde cualquier capítulo o sección, el enlace ↑ **Índice** junto al título te devuelve a su entrada en esta página.

0 Cómo usar esta guía 10

Parte I Fundamentos 13

1 Cómo funciona una aplicación web moderna 14
- 1.1 Del servidor que pinta páginas a la aplicación en el navegador 14
- 1.2 Las piezas del navegador: HTML, CSS, JavaScript y el DOM 16
- 1.3 Qué ocurre al abrir la aplicación 17
- 1.4 Enrutado en el cliente 19
- 1.5 El bundle: del código fuente a lo que descarga el navegador 19
- 1.6 Decisión: SPA con Vite frente a un framework con renderizado en servidor 21
- 1.7 Errores típicos y depuración 21
- 1.8 Ejercicios 22
- 1.9 Autoevaluación 22

2 JavaScript moderno y TypeScript para un desarrollador Java 24
- 2.1 JavaScript y TypeScript: qué es cada uno 24
- 2.2 Lo esencial de JavaScript moderno 25
- 2.3 Asincronía: promesas y `async` / `await` 29
- 2.4 Módulos: `import` y `export` 30
- 2.5 El sistema de tipos de TypeScript 30
- 2.6 Los tipos desaparecen en ejecución 34
- 2.7 Errores típicos y depuración 35
- 2.8 Ejercicios 35
- 2.9 Autoevaluación 36

3 Node.js, npm y el ecosistema de paquetes 37
- 3.1 Qué es Node.js y para qué lo usamos 37
- 3.2 `package.json`: el `pom.xml` del frontend 38
- 3.3 Versiones semánticas y el fichero de bloqueo 40
- 3.4 `node_modules` y la resolución de dependencias 41
- 3.5 Módulos ES frente a CommonJS 41
- 3.6 Los scripts del proyecto 41
- 3.7 Gestionar dependencias en el día a día 42
- 3.8 Errores típicos y depuración 43
- 3.9 Ejercicios 43
- 3.10 Autoevaluación 43

Parte II Herramientas del proyecto 45

4 Vite: el entorno de desarrollo y construcción 46
- 4.1 Qué es Vite y qué problema resuelve 46
- 4.2 El modo desarrollo 47
- 4.3 El modo producción 48

4.4 Vite no comprueba los tipos ... 48
4.5 `vite.config.ts` línea a línea ... 49
4.6 Variables de entorno ... 50
4.7 El proxy de desarrollo ... 51
4.8 Decisión: Vite frente a otras herramientas ... 52
4.9 Errores típicos y depuración ... 52
4.10 Ejercicios ... 52
4.11 Autoevaluación ... 53

5 Configuración de TypeScript, ESLint y calidad del código ... 54
5.1 Tres ficheros `tsconfig`: referencias de proyecto ... 54
5.2 `tsconfig.app.json`, opción a opción ... 55
5.3 `tsconfig.node.json` y `tsc -b` ... 56
5.4 El modo estricto en la práctica ... 57
5.5 ESLint: lo que el compilador no ve ... 57
5.6 Integración con el editor ... 59
5.7 Errores típicos y depuración ... 59
5.8 Ejercicios ... 59
5.9 Autoevaluación ... 60

Parte III React 19 a fondo ... 61

6 Componentes, JSX y props ... 62
6.1 Qué es React ... 62
6.2 Componentes: funciones que devuelven interfaz ... 63
6.3 JSX en detalle ... 64
6.4 Props: los parámetros de un componente ... 64
6.5 Renderizado condicional ... 66
6.6 Listas y claves ... 67
6.7 Eventos ... 68
6.8 Componentes como valores ... 68
6.9 Accesibilidad en JSX ... 69
6.10 Cómo se organizan los componentes en el proyecto ... 70
6.11 Errores típicos y depuración ... 70
6.12 Ejercicios ... 70
6.13 Autoevaluación ... 71

7 Estado, renderizado y reconciliación ... 72
7.1 Props frente a estado ... 72
7.2 `useState`: la memoria de un componente ... 72
7.3 Qué provoca un renderizado ... 73
7.4 Del renderizado al DOM: la reconciliación ... 74
7.5 Inmutabilidad: por qué no se modifica el estado ... 76
7.6 Dónde vive cada estado ... 76
7.7 Identidad del estado y la prop `key` ... 77
7.8 Componentes controlados ... 78
7.9 Rendimiento: cuándo preocuparse ... 78
7.10 Errores típicos y depuración ... 79
7.11 Ejercicios ... 79
7.12 Autoevaluación ... 80

8 Hooks: efectos, memorización y lógica reutilizable 81
8.1 Qué es un hook y por qué tiene reglas 81
8.2 `useEffect`: sincronizar con el exterior 82
8.3 `StrictMode` y la doble ejecución en desarrollo 84
8.4 `useMemo` y `useCallback`: estabilidad de referencias 84
8.5 Hooks personalizados 86
8.6 Los hooks de las librerías 87
8.7 Otros hooks de React 88
8.8 Errores típicos y depuración 88
8.9 Ejercicios 88
8.10 Autoevaluación 89

9 Composición, contexto y límites de error 90
9.1 Composición en lugar de herencia 90
9.2 El problema de pasar props de mano en mano 91
9.3 Contexto: datos disponibles para todo un subárbol 91
9.4 Los proveedores globales de `app.tsx` 93
9.5 Cuándo usar contexto y cuándo no 94
9.6 Más patrones de composición del proyecto 94
9.7 Límites de error 95
9.8 Carga diferida y espera 96
9.9 Errores típicos y depuración 96
9.10 Ejercicios 97
9.11 Autoevaluación 97

Parte IV Interfaz y diseño 99

10 Tailwind CSS 4: estilos con clases de utilidad 100
10.1 El problema que resuelve 100
10.2 Cómo leer las clases de utilidad 101
10.3 Diseño *responsive*: primero el móvil 102
10.4 Estados y variantes 103
10.5 Cómo funciona Tailwind 4 por dentro 104
10.6 El tema: tokens de diseño en `index.css` 105
10.7 Componer clases: `cn()` 107
10.8 Animaciones y transiciones 108
10.9 Decisión y alternativas 109
10.10 Errores típicos y depuración 109
10.11 Ejercicios 109
10.12 Autoevaluación 110

11 shadcn/ui y Radix: componentes accesibles que son tuyos 111
11.1 El problema de las librerías de componentes clásicas 111
11.2 Radix UI: comportamiento y accesibilidad sin estilos 112
11.3 shadcn/ui: un catálogo que se copia 113
11.4 Anatomía de un componente sencillo: `Button` 114
11.5 Anatomía de un componente compuesto: `Select` 115
11.6 Los componentes del proyecto 116
11.7 Iconos, notificaciones y fuente 117
11.8 Mantener componentes copiados 117

11.9 Errores típicos y depuración ... 118
11.10 Ejercicios ... 118
11.11 Autoevaluación ... 119

12 El layout y el sistema visual ... 120
12.1 La estructura de la pantalla ... 120
12.2 `AppLayout`: el armazón ... 120
12.3 El menú lateral declarativo ... 121
12.4 La cabecera y el menú móvil ... 123
12.5 La marca ... 124
12.6 El sistema visual ... 124
12.7 Los componentes de estado ... 126
12.8 Errores típicos y depuración ... 126
12.9 Ejercicios ... 127
12.10 Autoevaluación ... 127

Parte V Arquitectura ... 128

13 Organización por funcionalidad y reglas de dependencia ... 129
13.1 Dos formas de organizar un proyecto ... 129
13.2 El árbol de `src/` ... 130
13.3 Reglas de dependencia ... 130
13.4 Qué va en `common` y qué va en un módulo ... 132
13.5 `lib/`: utilidades sin interfaz ... 133
13.6 Convenciones de nombres ... 133
13.7 Decisión y evolución ... 134
13.8 Errores típicos y depuración ... 134
13.9 Ejercicios ... 134
13.10 Autoevaluación ... 135

14 Anatomía de un módulo ... 136
14.1 Vista general ... 136
14.2 `model/`: el dominio sin dependencias ... 137
14.3 `api/`: acceso a los datos ... 139
14.4 `components/`: presentación del módulo ... 140
14.5 `pages/`: la orquestación ... 140
14.6 `routes.ts`: el punto de montaje ... 141
14.7 Dos módulos lado a lado ... 141
14.8 Receta: crear un módulo nuevo ... 142
14.9 Errores típicos y depuración ... 142
14.10 Ejercicios ... 142
14.11 Autoevaluación ... 143

Parte VI Datos y backend ... 144

15 El cliente HTTP y la gestión de errores ... 145
15.1 Por qué un cliente propio ... 145
15.2 `fetch` en detalle ... 145
15.3 `apiRequest`, línea a línea ... 146
15.4 La cadena de consulta ... 147

15.5 El modelo de errores ... 148
15.6 Los formatos de error del backend ... 149
15.7 Cómo llegan los errores a la pantalla ... 151
15.8 Decisión y alternativas ... 152
15.9 Errores típicos y depuración ... 152
15.10 Ejercicios ... 153
15.11 Autoevaluación ... 153

16 CORS, el proxy de desarrollo y el despliegue ... 154
16.1 La política del mismo origen ... 154
16.2 CORS: cómo autoriza un servidor a otro origen ... 154
16.3 El proyecto en desarrollo: tres procesos y un proxy ... 155
16.4 Opciones para producción ... 156
16.5 Un *build* para todos los entornos ... 158
16.6 Seguridad básica de una SPA desplegada ... 159
16.7 Errores típicos y depuración ... 159
16.8 Ejercicios ... 159
16.9 Autoevaluación ... 160

17 TanStack Query: el estado del servidor ... 161
17.1 Estado del servidor frente a estado del cliente ... 161
17.2 El `QueryClient` y la caché ... 162
17.3 Anatomía de `useQuery` ... 162
17.4 El ciclo de vida de los datos ... 163
17.5 La configuración global del proyecto ... 164
17.6 Claves jerárquicas ... 164
17.7 Paginación sin parpadeos ... 165
17.8 Mutaciones: escribir y mantener la caché coherente ... 166
17.9 Consultas dependientes y desactivadas ... 167
17.10 Cancelación y condiciones de carrera ... 167
17.11 Decisión y alternativas ... 168
17.12 Errores típicos y depuración ... 168
17.13 Ejercicios ... 168
17.14 Autoevaluación ... 169

Parte VII Navegación y formularios ... 170

18 React Router: rutas, navegación y carga diferida ... 171
18.1 Qué hace un router en una SPA ... 171
18.2 Tres formas de usar React Router ... 172
18.3 La tabla de rutas ... 172
18.4 Rutas anidadas y `<Outlet />` ... 173
18.5 Parámetros de ruta ... 174
18.6 Navegar ... 174
18.7 Carga diferida de las páginas ... 175
18.8 `handle` y `useMatches`: metadatos de las rutas ... 176
18.9 Errores y páginas inexistentes ... 176
18.10 El diseño de las URL ... 177
18.11 Errores típicos y depuración ... 177
18.12 Ejercicios ... 178

18.13 Autoevaluación ... 178

19 El estado del listado en la URL ... 179
19.1 Por qué en la URL ... 179
19.2 El diseño de los parámetros ... 180
19.3 El ciclo completo ... 180
19.4 Leer y validar ... 181
19.5 Escribir ... 182
19.6 Actualizar desde la página ... 183
19.7 El historial: añadir o sustituir ... 184
19.8 La URL y la caché, juntas ... 184
19.9 Enlaces profundos entre pantallas ... 184
19.10 Pruebas ... 184
19.11 Errores típicos y depuración ... 185
19.12 Ejercicios ... 185
19.13 Autoevaluación ... 186

20 Formularios con React Hook Form y Zod ... 187
20.1 El problema de los formularios ... 187
20.2 Zod: describir y validar datos ... 187
20.3 React Hook Form: el estado del formulario ... 190
20.4 La integración con los componentes de shadcn/ui ... 190
20.5 De los valores del formulario al DTO ... 192
20.6 El ciclo de envío completo ... 194
20.7 Esquemas que dependen de los datos ... 194
20.8 Accesibilidad de los formularios ... 195
20.9 Pruebas de los esquemas ... 195
20.10 Errores típicos y depuración ... 196
20.11 Ejercicios ... 196
20.12 Autoevaluación ... 197

Parte VIII Recorridos completos ... 198

21 El módulo de aplicaciones de principio a fin ... 199
21.1 El mapa del módulo ... 199
21.2 Recorrido 1: consultar el catálogo ... 199
21.3 Recorrido 2: ver el detalle ... 201
21.4 Recorrido 3: dar de alta ... 202
21.5 Recorrido 4: modificar ... 202
21.6 La caché a lo largo de una sesión ... 203
21.7 El contrato con el backend ... 204
21.8 Errores típicos y depuración ... 204
21.9 Ejercicios ... 204
21.10 Autoevaluación ... 205

22 El módulo de demandas y sus reglas de negocio ... 206
22.1 Qué añade el dominio de las demandas ... 206
22.2 El ciclo de vida ... 207
22.3 Tres capas de defensa ... 208
22.4 El selector de aplicación ... 210
22.5 Las fechas ... 211

22.6 Navegación entre módulos ... 212
22.7 Diferencias con el contrato del backend ... 212
22.8 Errores típicos y depuración ... 213
22.9 Ejercicios ... 213
22.10 Autoevaluación ... 214

Parte IX Calidad y evolución ... 215

23 Pruebas con Vitest ... 216
23.1 Qué merece la pena probar en un frontend ... 216
23.2 Vitest ... 217
23.3 Anatomía de una prueba ... 218
23.4 Inventario de las pruebas ... 219
23.5 Patrones que usan las pruebas ... 219
23.6 Dobles de prueba ... 220
23.7 Ampliar la estrategia ... 220
23.8 Cobertura e integración continua ... 221
23.9 Errores típicos y depuración ... 222
23.10 Ejercicios ... 223
23.11 Autoevaluación ... 223

24 Registro de decisiones de arquitectura ... 224
24.1 Qué es un ADR y para qué sirve ... 224
24.2 El registro del proyecto ... 225
24.3 Decisiones propuestas ... 228
24.4 Cómo escribir los siguientes ADR ... 228
24.5 Errores típicos ... 229
24.6 Ejercicios ... 229
24.7 Autoevaluación ... 230

25 Notas de integración con el backend ... 231
25.1 La frontera actual ... 231
25.2 Inventario priorizado ... 231
25.3 Las notas, una a una ... 232
25.4 Deudas internas del frontend ... 235
25.5 Procedimiento: cuando cambia el backend ... 236
25.6 Ejercicios ... 236
25.7 Autoevaluación ... 237

26 Ejercicio guiado: el módulo de Incidencias ... 238
26.1 El objetivo ... 238
26.2 Paso 0 · El contrato de la API ... 238
26.3 Paso 1 · Conectar el servicio ... 240
26.4 Paso 2 · `model/` : tipos, constantes y reglas ... 240
26.5 Paso 3 · `api/` : endpoints y caché ... 242
26.6 Paso 4 · `components/` ... 242
26.7 Paso 5 · `pages/` y `routes.ts` ... 243
26.8 Paso 6 · Integrar en la aplicación ... 243
26.9 Paso 7 · Verificación final ... 244
26.10 Retos adicionales ... 244
26.11 Ejercicios ... 244

26.12 Autoevaluación 245

Anexos Material de consulta

A Soluciones y respuestas 246

B Glosario 285

C Equivalencias Java ↔ frontend 291

D Chuleta de comandos y atajos 295

E Mapa comentado de ficheros 299

ANTES DE EMPEZAR

Cómo usar esta guía

↑ Volver al índice

Esta guía explica, desde los fundamentos, cómo está construido el frontend de **IT Service & Demand Manager**. No es un manual genérico de React: cada concepto se explica con el código real del proyecto, y cada decisión de diseño se justifica frente a sus alternativas.

A quién va dirigida

Está escrita para un desarrollador con experiencia sólida en backend Java/JEE y Spring que empieza en el frontend moderno. Por eso se dan por sabidos conceptos como HTTP, REST, JSON, inyección de dependencias, capas, transacciones o pruebas unitarias. En cambio, se explican desde cero JavaScript, TypeScript, React y el resto del ecosistema del navegador.

Cómo está organizada

La guía avanza de lo general a lo particular, en nueve partes:

Parte	Contenido	Capítulos
I. Fundamentos	Qué es una SPA, JavaScript y TypeScript, Node y npm	1-3
II. Herramientas	Vite, la configuración de TypeScript, ESLint y los scripts	4-5
III. React 19	Componentes, estado, renderizado, hooks y contexto	6-9
IV. Interfaz y diseño	Tailwind CSS, shadcn/ui, Radix y el layout	10-12
V. Arquitectura	Organización por funcionalidad y anatomía de un módulo	13-14
VI. Datos y backend	Cliente HTTP, CORS y proxy, TanStack Query	15-17
VII. Navegación y formularios	React Router, estado en la URL, React Hook Form y Zod	18-20
VIII. Recorridos completos	Aplicaciones y demandas de principio a fin	21-22
IX. Calidad y evolución	Pruebas, decisiones, integración con el backend y el módulo de Incidencias	23-26

Cinco anexos completan la guía: **A**, soluciones de los ejercicios y respuestas de autoevaluación; **B**, glosario; **C**, equivalencias Java ↔ frontend; **D**, chuleta de comandos y atajos; y **E**, mapa comentado de todos los ficheros del proyecto.

Cómo está construido cada capítulo

Todos los capítulos siguen el mismo esquema, para que sepas siempre dónde buscar:

1. **Qué es**: el concepto, sin dar nada por sabido.
2. **Cómo funciona por dentro**: lo justo para que la tecnología deje de ser una caja negra.
3. **Cómo se aplica en el proyecto**: fragmentos de código real, con el fichero de origen indicado sobre cada bloque.
4. **Decisión y alternativas**: por qué se eligió esto y qué se descartó.
5. **Errores típicos y depuración**: fallos frecuentes y cómo diagnosticarlos.
6. **Ejercicios** y **autoevaluación**, con las soluciones en el anexo A.

Los recuadros

Si vienes de Java...

Relaciona el concepto con algo que ya conoces del mundo Java/Spring. Las analogías ayudan a arrancar, pero también se señala dónde dejan de funcionar.

En el proyecto

Señala exactamente dónde y cómo se usa el concepto en `it-service-manager-web`.

Decisión de diseño

Recoge una decisión tomada, el motivo y las alternativas que se valoraron.

Cuidado

Avisa de un error habitual o de una trampa del lenguaje o de la herramienta.

Cómo moverte por el PDF

- **Índice enlazado.** Cada parte, capítulo y sección del índice es un enlace a su página.
- **Vuelta al índice.** Junto al título de cada capítulo y de cada sección hay un enlace ↑ **Índice** que te lleva exactamente a su entrada en el índice, para que puedas saltar a otro punto.
- **Índice de capítulo.** Al principio de cada capítulo tienes la lista de sus secciones, también enlazada.
- **Ejercicios y soluciones.** Cada ejercicio enlaza con su solución en el anexo, y cada solución enlaza de vuelta con su ejercicio. Lo mismo ocurre con las preguntas de autoevaluación.
- **Marcadores.** El panel de marcadores de tu lector de PDF muestra la estructura completa de la guía.

Cómo sacarle el máximo partido

Lee con el proyecto abierto en el editor. Cuando aparezca un fragmento, abre el fichero completo y busca el contexto. Haz los ejercicios sobre una rama de Git aparte: la mayoría consisten en modificar el proyecto, romper algo a propósito y observar qué ocurre. Es la forma más rápida de pasar de entender a saber hacer.

> **Versiones**
>
> La guía describe el proyecto tal como está en octubre de 2026: React 19.3, TypeScript 6.0, Vite 8.3, Tailwind CSS 4.3, React Router 8.4, TanStack Query 5, React Hook Form 7, Zod 3, Vitest 5 y ESLint 10, sobre Node.js 22. Los conceptos son estables; los detalles de configuración pueden cambiar entre versiones mayores.

PARTE I

Fundamentos

1	Cómo funciona una aplicación web moderna	pág. 14
2	JavaScript moderno y TypeScript para un desarrollador Java	pág. 24
3	Node.js, npm y el ecosistema de paquetes	pág. 37

↑ Volver al índice

CAPÍTULO 1

Cómo funciona una aplicación web moderna

↑ Volver al índice

EN ESTE CAPÍTULO

1.1 Del servidor que pinta páginas a la aplicación en el navegador
1.2 Las piezas del navegador: HTML, CSS, JavaScript y el DOM
1.3 Qué ocurre al abrir la aplicación
1.4 Enrutado en el cliente
1.5 El bundle: del código fuente a lo que descarga el navegador
1.6 Decisión: SPA con Vite frente a un framework con renderizado en servidor
1.7 Errores típicos y depuración
1.8 Ejercicios
1.9 Autoevaluación

Antes de escribir una sola línea de React conviene entender qué tipo de aplicación estamos construyendo, dónde se ejecuta y qué ocurre desde que el usuario escribe la URL hasta que ve el listado de aplicaciones. Este capítulo da el mapa general que usaremos en el resto de la guía.

1.1 Del servidor que pinta páginas a la aplicación en el navegador

↑ Índice

Durante años, el modelo dominante en el mundo Java fue la **aplicación multipágina** (MPA, *Multi-Page Application*): cada vez que el usuario pulsa un enlace o envía un formulario, el navegador pide una página nueva al servidor, el servidor ejecuta lógica (un servlet, un controlador de Spring MVC), genera el HTML completo con una plantilla (JSP, JSF, Thymeleaf) y lo devuelve. El navegador tira la página anterior y pinta la nueva.

Una **aplicación de una sola página** (SPA, *Single-Page Application*) invierte el reparto de responsabilidades:

- El servidor entrega **una única vez** un HTML casi vacío y un paquete de JavaScript.
- A partir de ahí, **el JavaScript se ejecuta en el navegador** y construye toda la interfaz.
- Cuando necesita datos, no pide páginas: pide **JSON** a una API REST.
- Al navegar entre pantallas, la URL cambia, pero **la página no se recarga**: el JavaScript sustituye la parte de la interfaz que corresponde.



Figura 1.1. Aplicación multipágina frente a SPA. En una SPA el servidor se limita a exponer datos.

En nuestro caso, el backend `it-service-manager` (Spring Boot) **no genera ninguna pantalla**: expone recursos como `GET /api/applications` o `PUT /api/demands/{id}` y devuelve JSON. Todo lo que ve el usuario —el menú lateral, las tablas, los filtros, los formularios— lo construye el frontend en el navegador.

> **Si vienes de Java…**
>
> Piensa en el frontend como **otro cliente de tu API**, igual que lo sería una aplicación móvil o un proceso batch que consume REST con `RestClient`. El contrato entre ambos son los DTO (`ApplicationResponse`, `ApplicationCreateRequest`…) y los códigos HTTP. La consecuencia práctica es importante: **toda validación que haga el frontend es una comodidad para el usuario, nunca una garantía**. La autoridad sigue siendo el backend, con su Bean Validation y sus reglas de dominio.

1.1.1 Ventajas e inconvenientes

Aspecto	Multipágina	SPA
Primera carga	Rápida: llega HTML ya pintado	Más lenta: hay que descargar y ejecutar el JavaScript
Navegación posterior	Recarga completa en cada clic	Instantánea; solo se piden datos
Experiencia de usuario	Más «documento»	Más «aplicación de escritorio»
Separación frontend/backend	Acoplados en el mismo proyecto	Totalmente independientes; se comunican por HTTP
SEO e indexación	Natural	Requiere técnicas adicionales

Para una aplicación interna de gestión, con usuarios autenticados que trabajan durante horas con tablas y formularios, la SPA es la opción natural: el coste de la primera carga se paga una vez y el SEO es irrelevante.

1.2 Las piezas del navegador: HTML, CSS, JavaScript y el DOM

↑ Índice

El navegador entiende tres lenguajes, y conviene tener clara la función de cada uno:

- **HTML** describe la **estructura** del contenido: «esto es una tabla, esto un botón, esto un campo de texto».
- **CSS** describe la **presentación**: colores, tamaños, márgenes, disposición en rejilla o en columna.
- **JavaScript** aporta el **comportamiento**: reaccionar a un clic, pedir datos, cambiar lo que se ve.

1.2.1 El DOM

Cuando el navegador lee el HTML, no lo guarda como texto: construye un **árbol de objetos** en memoria llamado **DOM** (*Document Object Model*). Cada etiqueta es un nodo con propiedades y nodos hijos. Lo que ves en pantalla es siempre la representación del DOM *actual*, no del HTML original.

JavaScript puede leer y modificar ese árbol en cualquier momento:

```
const boton = document.createElement('button')
boton.textContent = 'Nueva aplicación'
boton.addEventListener('click', () => alert('¡Clic!'))
document.getElementById('root').appendChild(boton)
```

Esto es exactamente lo que hace React, solo que de forma automática y eficiente: tú describes **cómo debe ser** la interfaz y React calcula **qué operaciones del DOM** hacen falta para llegar a ese estado. Lo veremos a fondo en el capítulo 7.

1.2.2 Un solo hilo y el bucle de eventos

El JavaScript de una página se ejecuta en **un único hilo**. No hay `Thread`, ni `ExecutorService`, ni `synchronized`. ¿Cómo puede entonces hacer una petición HTTP sin congelar la interfaz?

La respuesta es el **bucle de eventos** (*event loop*). Las operaciones lentas —una petición de red, un temporizador— se delegan al navegador, que las ejecuta fuera del hilo de JavaScript. Cuando terminan, el navegador deja en una cola la función que hay que ejecutar con el resultado. El hilo de JavaScript, cuando termina lo que estaba haciendo, toma la siguiente tarea de la cola.

```
console.log('1. Antes de la petición')
fetch('/api/applications').then(() => console.log('3. Ha llegado la respuesta'))
console.log('2. Después de lanzar la petición')
```

La salida es 1, 2, 3: `fetch` no bloquea; registra qué hacer cuando llegue la respuesta y continúa. Toda la programación asíncrona de JavaScript (*promesas*, `async` / `await`, que veremos en el capítulo 2) se apoya en este mecanismo.

> **Si vienes de Java…**
>
> El modelo se parece más a un servidor reactivo (WebFlux, Netty) que a un servidor de servlets clásico con un hilo por petición. La regla de oro es la misma: **nunca bloquees el hilo**. Un bucle que tarde dos segundos en el navegador congela la página entera: no responde a clics ni repinta.

1.3 Qué ocurre al abrir la aplicación

↑ Índice

Veamos el recorrido completo, desde que el navegador pide `http://localhost:5173/aplicaciones` hasta que aparece el listado. Es el mejor mapa para orientarse en el código.

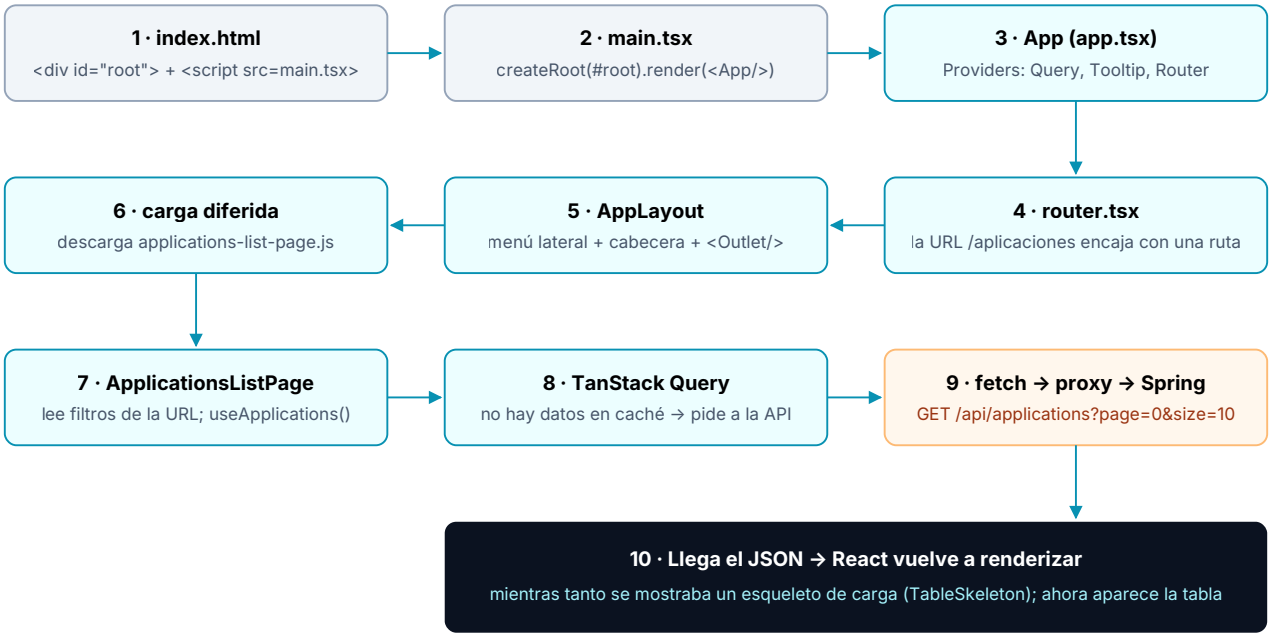


Figura 1.2. Secuencia de arranque de la aplicación hasta mostrar el listado de aplicaciones.

1.3.1 Paso 1: el HTML de partida

El servidor de desarrollo (Vite, capítulo 4) devuelve siempre el mismo `index.html`. Fíjate en lo poco que contiene:

`index.html`

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#0b1220" />
    <title>IT Service Manager</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Dos líneas lo explican todo. `<div id="root"></div>` es un contenedor vacío donde React dibujará la aplicación. `<script type="module" src="/src/main.tsx">` carga el punto de entrada. El atributo `type="module"` indica que es un **módulo ES** que puede importar otros módulos (capítulo 3). La etiqueta `viewport` es la que permite que el diseño se adapte a móviles.

1.3.2 Paso 2: el punto de entrada

src/main.tsx

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'

import './index.css'
import { App } from './app/app'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`createRoot` crea una **raíz de React** asociada al `div#root` y `render` le dice qué dibujar: el componente `App`. A partir de este momento React es el dueño de todo lo que hay dentro de ese `div`. La importación `'./index.css'` incorpora los estilos globales (Tailwind y el tema, Parte IV). `StrictMode` es una ayuda de desarrollo que estudiaremos en el capítulo 8: no pinta nada, pero ejecuta comprobaciones adicionales.

El signo `!` tras `getElementById('root')` es TypeScript: le estamos asegurando al compilador que el elemento existe y no es `null` (capítulo 2).

1.3.3 Paso 3: los proveedores globales

src/app/app.tsx

```tsx
export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <TooltipProvider>
        <RouterProvider router={router} />
        <Toaster position="top-right" />
      </TooltipProvider>
    </QueryClientProvider>
  )
}
```

`App` no pinta nada visible por sí mismo: **envuelve** la aplicación en *proveedores* que ofrecen servicios a todos los componentes que hay por debajo. `QueryClientProvider` da acceso a la caché de datos del servidor (TanStack Query, Parte VI), `TooltipProvider` coordina los tooltips, `RouterProvider` decide qué pantalla corresponde a la URL actual y `Toaster` es el contenedor de las notificaciones emergentes («Cambios guardados»). El mecanismo de los proveedores, el *contexto* de React, es el tema del capítulo 9.

> **Si vienes de Java…**
>
> `App` cumple un papel parecido a la configuración de arranque de una aplicación Spring (`@SpringBootApplication` más las clases `@Configuration`): no contiene lógica de negocio, pero registra la infraestructura que el resto del código usará sin tener que construirla. La diferencia es que aquí no hay contenedor de inversión de control: la «inyección» se hace por posición en el árbol de componentes.

1.3.4 Pasos 4 a 10: rutas, layout, página y datos

El router compara la URL con su tabla de rutas (`src/app/router.tsx`). La ruta raíz pinta `AppLayout` —el armazón con menú lateral y cabecera— y, dentro de su hueco `<Outlet />`, la página que corresponda. Las

páginas de cada módulo se cargan **bajo demanda**: el código de `ApplicationsListPage` no se descarga hasta que alguien visita `/aplicaciones`.

La página lee los filtros de la URL (`?estado=ACTIVE&pagina=2`), llama a `useApplications(params)` y TanStack Query lanza la petición `GET /api/applications`. Mientras llega la respuesta se muestra un esqueleto animado; cuando llega, React vuelve a dibujar la página con la tabla. Cada uno de estos pasos tiene su capítulo; por ahora basta con que te quedes con la secuencia.

1.4 Enrutado en el cliente

↑ Índice

En una SPA, cambiar de pantalla no implica pedir un HTML nuevo. ¿Cómo cambia entonces la URL de la barra de direcciones? Gracias a la **History API** del navegador: JavaScript puede llamar a `history.pushState()` para modificar la URL y añadir una entrada al historial **sin recargar la página**. Cuando el usuario pulsa *atrás*, el navegador emite un evento `popstate` y el router vuelve a pintar la pantalla correspondiente.

React Router (Parte VII) envuelve todo esto. Por eso en el código no verás enlaces `<a href>` para navegar dentro de la aplicación, sino el componente `Link`:

`src/features/applications/pages/applications-list-page.tsx`

```
<Button asChild variant="brand">
  <Link to="/aplicaciones/nueva">
    <Plus /> Nueva aplicación
  </Link>
</Button>
```

`Link` genera un `<a>` normal (para que funcione el clic con el botón central, la accesibilidad, etc.), pero intercepta el clic, evita la recarga y navega mediante la History API.

> **Cuidado**
>
> **El servidor debe devolver `index.html` para cualquier ruta.** Si el usuario recarga la página estando en `/aplicaciones/42/editar`, el navegador pedirá esa ruta al servidor. En desarrollo Vite ya lo resuelve, pero en producción el servidor web (Nginx, un *bucket* estático, el propio Spring Boot) debe configurarse con una regla de *fallback*: «si no existe un fichero con ese nombre, devuelve `index.html`». Es el error número uno al desplegar una SPA: todo funciona hasta que alguien pulsa F5 y recibe un 404.

1.5 El bundle: del código fuente a lo que descarga el navegador

↑ Índice

El navegador no entiende TypeScript ni JSX (la sintaxis tipo HTML que ves dentro de los componentes). Además, el proyecto tiene más de cien ficheros fuente y cientos de módulos en `node_modules`. Antes de llegar al navegador, el código pasa por un proceso de construcción (*build*):

1. **Transpilación.** TypeScript y JSX se convierten a JavaScript estándar. Los tipos desaparecen por completo.
2. **Resolución de módulos.** Se sigue cada `import` para saber qué ficheros forman parte de la aplicación.
3. **Empaquetado (*bundling*).** Se agrupan los módulos en unos pocos ficheros, los *chunks*, para no hacer cientos de peticiones.

4. **Eliminación de código muerto (*tree shaking*).** Lo que se importa de una librería pero no se usa no se incluye.
5. **Minificación.** Se eliminan espacios y se acortan nombres de variables.
6. **Huella en el nombre (*hashing*).** Cada fichero se llama, por ejemplo, `index-B7f3kQ2a.js`. Si el contenido cambia, cambia el nombre, de modo que el navegador puede guardar los ficheros en caché indefinidamente sin riesgo de servir versiones antiguas.

El resultado es la carpeta `dist/`: HTML, CSS, JavaScript e imágenes estáticos, que pueden servirse desde cualquier servidor web. No hay nada que ejecutar en el servidor.

1.5.1 División del código (*code splitting*)

Si todo el código fuera en un único fichero, el usuario descargaría el módulo de demandas aunque solo vaya a consultar aplicaciones. Por eso las rutas usan **importaciones dinámicas**:

`src/features/applications/routes.ts`

```
{
  index: true,
  handle: { title: 'Consultar aplicaciones' },
  lazy: () =>
    import('./pages/applications-list-page').then((m) => ({ Component: m.ApplicationsListPage })),
},
```

La función `import('…')` —con paréntesis— no se resuelve al construir, sino **en tiempo de ejecución**, y devuelve una promesa. El empaquetador lo detecta y genera un *chunk* separado para esa página, que solo se descarga cuando el router ejecuta `lazy()`. El resultado: un *bundle* inicial pequeño y páginas que llegan bajo demanda.

> **Si vienes de Java…**
>
> Es la misma idea que la carga perezosa de clases de la JVM o una relación `FetchType.LAZY` de JPA: no pagas el coste de algo hasta que lo necesitas. Y tiene el mismo matiz: la primera vez que se accede hay una pequeña espera, que aquí cubrimos con el indicador de carga (`hydrateFallbackElement` en `router.tsx`).

1.6 Decisión: SPA con Vite frente a un framework con renderizado en servidor

↑ Índice

Decisión de diseño: SPA pura construida con Vite

Contexto. En el ecosistema React existen *meta-frameworks* como Next.js o el modo framework de React Router, que renderizan las páginas en el servidor (SSR) o en tiempo de construcción y luego las «hidratan» en el navegador.

Decisión. Construir una SPA pura, servida como ficheros estáticos, con Vite como herramienta de construcción.

Motivos.

- El backend ya existe y es independiente (Spring Boot). Un framework con SSR añadiría un **segundo servidor** (Node.js) que mantener, desplegar y monitorizar.
- Es una aplicación interna y autenticada: el SEO y el tiempo de la primera visita no son críticos.
- Separación limpia: el frontend es un conjunto de ficheros estáticos; el backend, una API. Cada uno se despliega a su ritmo.
- Menos conceptos nuevos de golpe para quien aprende: sin componentes de servidor, sin hidratación, sin acciones de servidor.

Alternativas descartadas. Next.js (potente, pero orientado a SSR y a su propia plataforma de despliegue); React Router en modo framework (misma objeción del servidor Node); Angular (framework completo y muy válido en entornos empresariales, pero el proyecto apostó por el ecosistema React por su tamaño y su flexibilidad).

1.7 Errores típicos y depuración

↑ Índice

Las **herramientas de desarrollo del navegador** (*DevTools*, tecla F12) son tu depurador, tu visor de logs y tu analizador de red. Familiarízate desde ya con cinco pestañas:

Pestaña	Para qué la usarás
Elements	Ver el DOM real que ha generado React y los estilos aplicados a cada elemento.
Console	Errores de JavaScript, avisos de React y tus `console.log`.
Network	Cada petición HTTP: URL, método, código de estado, cabeceras, cuerpo y tiempos. Filtra por *Fetch/XHR* para ver solo las llamadas a la API.
Sources	Depurador con puntos de ruptura sobre tu código TypeScript original (gracias a los *source maps*).
Application	Almacenamiento local, cookies y caché del navegador.

Algunos síntomas frecuentes y dónde mirar:

- **Página en blanco.** Casi siempre es un error de JavaScript durante el arranque. Abre *Console*: verás el error y el fichero y la línea de origen.
- **«No se han podido cargar los datos» con un error HTTP 500 o 502.** Si la petición a `/api/...` devuelve un 5xx y la terminal de Vite muestra `http proxy error`, el proxy no llega al backend: comprueba que está arrancado y en el puerto configurado. Si en cambio el mensaje es «No se puede

conectar con el servidor», ha fallado la propia conexión de red (por ejemplo, porque el servidor de Vite se ha detenido).

- **404 al recargar en producción.** Falta la regla de *fallback* a `index.html` en el servidor web (sección 1.4).
- **Los cambios no aparecen.** En desarrollo, Vite recarga automáticamente; si no lo hace, mira la terminal donde corre `npm run dev`: probablemente hay un error de compilación.

> **Nota**
>
> Instala también la extensión **React Developer Tools** para Chrome, Edge o Firefox. Añade dos pestañas: *Components*, donde ves el árbol de componentes de React con sus *props* y su estado, y *Profiler*, que mide cuánto tarda cada renderizado. La usaremos a partir del capítulo 6.

1.8 Ejercicios

↑ Índice

Ejercicio 1.1 · Observa el arranque en la pestaña Network Ver solución →

Arranca el backend y el frontend (`npm run dev`). Abre las DevTools, pestaña *Network*, marca *Disable cache* y recarga `http://localhost:5173/aplicaciones`. Identifica:

1. La petición del documento HTML.
2. La petición de `main.tsx` y algunas de las que la siguen.
3. La petición a `/api/applications`: su URL completa con parámetros, el código de estado y el JSON de respuesta.

Navega después a *Demandas*. ¿Se vuelve a pedir `index.html`? ¿Qué peticiones nuevas aparecen?

Ejercicio 1.2 · Provoca y diagnostica errores Ver solución →

1. Detén el backend y recarga el listado. ¿Qué ve el usuario? ¿Qué aparece en *Network*?
2. Con el backend arrancado, visita `http://localhost:5173/esto-no-existe`. ¿Quién genera la página de error: el servidor o React? Razónalo con lo que ves en *Network*.

Ejercicio 1.3 · Mide el efecto de la división del código Ver solución →

Ejecuta `npm run build` y examina la carpeta `dist/assets`. Localiza los ficheros JavaScript que corresponden a las páginas del módulo de aplicaciones (los nombres contienen el nombre del módulo de origen). Después ejecuta `npm run preview`, abre la aplicación en `/demandas` con *Network* abierto y comprueba que el *chunk* de la página de listado de aplicaciones **no** se descarga hasta que navegas allí.

1.9 Autoevaluación

↑ Índice

Autoevaluación del capítulo 1 Ver respuestas →

1. En una SPA, ¿qué devuelve el servidor la primera vez y qué devuelve después?
2. ¿Por qué una validación hecha en el frontend no sustituye a la del backend?
3. ¿Qué es el DOM y qué relación tiene con lo que se ve en pantalla?

4. Si JavaScript tiene un solo hilo, ¿cómo puede esperar una respuesta HTTP sin congelar la interfaz?
5. ¿Qué hace `createRoot(...).render(...)` en `main.tsx`?
6. ¿Por qué hay que configurar un *fallback* a `index.html` en el servidor de producción?
7. ¿Qué diferencia hay entre `import x from './a'` e `import('./a')`?
8. ¿Para qué sirve incluir una huella (*hash*) en el nombre de los ficheros del *build*?

CAPÍTULO 2

JavaScript moderno y TypeScript para un desarrollador Java

↑ Volver al índice

EN ESTE CAPÍTULO

2.1 JavaScript y TypeScript: qué es cada uno
2.2 Lo esencial de JavaScript moderno
2.3 Asincronía: promesas y `async` / `await`
2.4 Módulos: `import` y `export`
2.5 El sistema de tipos de TypeScript
2.6 Los tipos desaparecen en ejecución
2.7 Errores típicos y depuración
2.8 Ejercicios
2.9 Autoevaluación

Todo el proyecto está escrito en TypeScript. Este capítulo recorre lo imprescindible del lenguaje con ejemplos sacados del propio código, y se detiene en las diferencias con Java que más confunden al principio: el tipado estructural, las uniones de literales, `null` frente a `undefined` y, sobre todo, que los tipos desaparecen al ejecutar.

2.1 JavaScript y TypeScript: qué es cada uno

↑ Índice

JavaScript es el único lenguaje de programación que ejecutan los navegadores. Su nombre oficial es ECMAScript; desde 2015 (ES2015, también llamado ES6) recibe una versión nueva cada año, y el JavaScript «moderno» de este capítulo es el de esas versiones: `const`, funciones flecha, módulos, clases, `async` / `await`... Es un lenguaje **dinámicamente tipado**: una variable puede contener un número y, tres líneas después, un texto.

TypeScript es un **superconjunto** de JavaScript creado por Microsoft: todo JavaScript válido es TypeScript válido, y TypeScript añade una capa de **tipos estáticos** que el compilador comprueba. Al compilar, los tipos se **eliminan** y el resultado es JavaScript corriente.



Figura 2.1. El borrado de tipos: TypeScript protege en tiempo de compilación, no en tiempo de ejecución.

> **Si vienes de Java…**
>
> Java también borra información de tipos (los genéricos, por *type erasure*), pero conserva las clases en ejecución: puedes hacer `instanceof Application` o leer anotaciones por reflexión. En TypeScript **una `interface` no existe en ejecución**: no hay `instanceof Application`, ni reflexión, ni nada que Jackson pueda usar para validar al deserializar. Es la idea más importante del capítulo, y volveremos a ella en la sección 2.6.

2.2 Lo esencial de JavaScript moderno

↑ Índice

2.2.1 Variables: `const` y `let`

```
const BASE_PATH = '/applications'   // no se puede reasignar
let response: Response              // se puede reasignar
```

Usa `const` por defecto y `let` solo si necesitas reasignar. Verás `var` en código antiguo: no lo uses, porque su ámbito es la función entera y no el bloque. `const` impide **reasignar la variable**, no modificar el objeto: `const lista = []` admite `lista.push(1)`, igual que una referencia `final` en Java.

2.2.2 Tipos primitivos, `null` y `undefined`

JavaScript tiene pocos tipos primitivos: `string`, `number` (no hay `int`, `long` ni `double`: todos los números son de coma flotante de 64 bits), `boolean`, `bigint`, `symbol`, y **dos** valores de ausencia:

- `undefined`: «no se ha asignado nada». Es lo que vale una variable sin inicializar, un parámetro que no se ha pasado o una propiedad que no existe.
- `null`: «se ha asignado explícitamente un vacío».

El backend serializa a `null` los campos sin valor (`description: null`), mientras que en el frontend usamos `undefined` para «filtro no seleccionado». Ambas convenciones conviven en el proyecto, como veremos en los tipos.

> **Cuidado**
>
> Compara siempre con `===` y `!==` (igualdad estricta). El operador `==` convierte tipos antes de comparar y produce resultados sorprendentes: `0 == ''` es `true`. La única excepción aceptada por muchos equipos es `x == null`, que es cierto tanto para `null` como para `undefined`.

2.2.3 Valores «verdaderos» y «falsos»

En una condición, JavaScript convierte cualquier valor a booleano. Son **falsos** (*falsy*): `false`, `0`, `''`, `null`, `undefined` y `NaN`. Todo lo demás es **verdadero** (*truthy*), incluidos `[]` y `{}`. El proyecto lo aprovecha a menudo:

`src/features/applications/pages/applications-list-page.tsx`

```
const hasFilters = !!params.status || !!params.criticality
```

`!!x` convierte cualquier valor a su booleano equivalente: `!!undefined` es `false`; `!!'ACTIVE'` es `true`.

2.2.4 Objetos y arrays literales

Un objeto se crea directamente, sin clase:

src/features/applications/model/constants.ts

```
export const DEFAULT_LIST_PARAMS: ApplicationListParams = {
  page: 0,
  size: 10,
  sortField: 'id',
  sortDirection: 'asc',
}
```

2.2.5 Desestructuración

Extrae propiedades de un objeto o posiciones de un array en variables:

src/lib/api-client.ts

```
const { method = 'GET', query, body, signal } = options
```

Equivale a cuatro asignaciones `const method = options.method ?? 'GET'`, etc.; el `= 'GET'` es un valor por defecto si la propiedad es `undefined`. Con arrays se desestructura por posición:

src/lib/list-params.ts

```
const [field, direction] = (value ?? '').split(',')
```

Verás la desestructuración constantemente en React: en los parámetros de los componentes (`function PageHeader({ title, description })`) y en los hooks (`const [open, setOpen] = useState(true)`).

2.2.6 El operador de propagación (*spread*)

Los tres puntos copian las propiedades de un objeto (o los elementos de un array) dentro de otro:

src/features/applications/pages/applications-list-page.tsx

```
setSearchParams(serializeListParams({ ...params, ...patch }), options)
```

`{ ...params, ...patch }` crea un **objeto nuevo** con todas las propiedades de `params` y, encima, las de `patch`: si ambos tienen `page`, gana la de `patch`. Es la forma idiomática de «actualizar» un objeto sin modificar el original, algo esencial en React (capítulo 7).

> **Si vienes de Java…**
>
> Es el equivalente de un *builder* `toBuilder().page(2).build()` o de un `record` con un método *wither*: no se muta el objeto; se crea uno nuevo con los cambios. Los objetos de JavaScript son mutables, pero en el código de React se tratan **como si fueran inmutables**.

2.2.7 Funciones flecha y funciones como valores

En JavaScript las funciones son valores: se guardan en variables, se pasan como argumentos y se devuelven. La sintaxis más usada es la **función flecha**:

```
const doble = (n: number) => n * 2              // cuerpo de una expresión: return implícito
const saluda = (nombre: string) => {             // cuerpo de bloque: return explícito
```

```
  const texto = `Hola, ${nombre}`
  return texto
}
```

Todo el objeto `applicationsApi` es un conjunto de funciones flecha guardadas en propiedades:

src/features/applications/api/applications-api.ts

```
export const applicationsApi = {
  list: (params: ApplicationListParams, signal?: AbortSignal) =>
    apiClient.get<ApplicationPage>(BASE_PATH, toApiQuery(params), signal),

  getById: (id: number, signal?: AbortSignal) =>
    apiClient.get<Application>(`${BASE_PATH}/${id}`, undefined, signal),
  // …
}
```

Y los componentes reciben funciones como propiedades para avisar de lo que ocurre (*callbacks*):

src/features/applications/pages/applications-list-page.tsx

```
<PaginationBar
  onPageChange={(page) => updateParams({ page })}
  onSizeChange={(size) => updateParams({ size, page: 0 })}
/>
```

`{ page }` es una abreviatura de `{ page: page }`: si la variable se llama igual que la propiedad, basta con escribirla una vez.

> **Si vienes de Java…**
>
> Las funciones flecha son las lambdas de Java, con dos diferencias: no hace falta ninguna interfaz funcional (`Function<T,R>`, `Consumer<T>`…) porque el tipo es la propia firma, `(page: number) => void`; y pueden capturar y **modificar** variables del ámbito exterior, no solo leer variables *efectivamente finales*.

2.2.8 Plantillas de texto

Las comillas invertidas permiten interpolar expresiones y escribir varias líneas:

src/lib/api-client.ts

```
const url = `${API_BASE_URL}${path}${buildQueryString(query)}`
```

2.2.9 Métodos de array

Los arrays traen métodos equivalentes a los de `java.util.stream`, pero sin `stream()` ni `collect()`: cada uno devuelve directamente un nuevo array o un valor.

JavaScript	Java Streams	Uso en el proyecto
`map(fn)`	`map(fn).toList()`	Construir las opciones de un filtro a partir de los valores del enum
`filter(fn)`	`filter(fn).toList()`	Descartar elementos
`find(fn)`	`filter(fn).findFirst()`	Encontrar el título de la ruta activa

JavaScript	Java Streams	Uso en el proyecto
`some(fn)`, `every(fn)`	`anyMatch`, `allMatch`	Comprobaciones
`includes(x)`	`contains(x)`	Validar que un valor de la URL es un estado permitido
`reduce(fn, ini)`	`reduce(ini, fn)`	Acumular

`src/features/applications/components/application-filters.tsx`

```
const STATUS_OPTIONS = APPLICATION_STATUSES.map((value) => ({
  value,
  label: STATUS_LABELS[value],
  hint: value,
}))
```

Fíjate en los paréntesis alrededor de `{ … }`: sin ellos, las llaves se interpretarían como el cuerpo de la función y no como un objeto devuelto.

Un ejemplo encadenado, del layout:

`src/components/layout/app-layout.tsx`

```
const title = [...matches]
  .reverse()
  .map((m) => (m.handle as { title?: string } | undefined)?.title)
  .find(Boolean)
```

`[...matches]` copia el array (porque `reverse()` modifica el original, una excepción incómoda en la API de arrays), lo invierte, extrae el título de cada ruta y se queda con el primero que no sea vacío: `find(Boolean)` usa la propia función `Boolean` como predicado.

2.2.10 Encadenamiento opcional y fusión de nulos

Dos operadores ahorran gran cantidad de comprobaciones de nulos:

```
const totalPages = query.data?.totalPages        // undefined si data es null/undefined
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? '/api'   // valor por defecto
```

- `a?.b` devuelve `undefined` si `a` es `null` o `undefined`, en lugar de lanzar un error. También funciona con llamadas: `onNavigate?.()`.
- `a ?? b` devuelve `b` solo si `a` es `null` o `undefined`. Es distinto de `a || b`, que también sustituye `0`, `''` y `false`. Para tamaños de página o números de página, que pueden valer `0`, `??` es el operador correcto.

> **Si vienes de Java...**
>
> `?.` y `??` son lo que `Optional.map(...)` y `Optional.orElse(...)` intentan expresar, pero integrados en el lenguaje y sin envolver el valor.

2.3 Asincronía: promesas y `async`/`await`

↑ Índice

Una **promesa** (`Promise<T>`) representa un valor que llegará en el futuro: el resultado de una petición HTTP, por ejemplo. Puede estar pendiente, cumplida (con un valor) o rechazada (con un error).

Las funciones marcadas con `async` devuelven siempre una promesa, y dentro de ellas `await` **suspende la función** —sin bloquear el hilo— hasta que la promesa se resuelve. El corazón del cliente HTTP del proyecto lo muestra:

`src/lib/api-client.ts (simplificado)`

```
export async function apiRequest<T>(path: string, options: RequestOptions = {}): Promise<T> {
  const { method = 'GET', query, body, signal } = options
  const url = `${API_BASE_URL}${path}${buildQueryString(query)}`

  let response: Response
  try {
    response = await fetch(url, { method, signal, headers: { Accept: 'application/json' } })
  } catch (error) {
    throw new ApiError({ message: 'No se puede conectar con el servidor…', status: 0, kind: 'network
' })
  }

  if (!response.ok) {
    throw await toApiError(response)
  }
  const text = await response.text()
  return (text ? JSON.parse(text) : undefined) as T
}
```

Se lee de arriba abajo como código síncrono, y los errores se capturan con `try` / `catch`. Cuando `fetch` falla por un problema de red, la promesa se **rechaza** y el `await` lanza la excepción dentro del `try`. Si la respuesta es un 4xx o un 5xx, `fetch` **no** falla: la promesa se cumple con una respuesta cuyo `ok` vale `false`. Por eso hay que comprobar `response.ok` explícitamente.

> **Si vienes de Java…**
>
> `Promise<T>` es el equivalente de `CompletableFuture<T>`, y `await` es lo que harías con `join()`, con una diferencia fundamental: `await` no bloquea el hilo. Libera el hilo para que el bucle de eventos atienda otras tareas y retoma la función cuando la promesa se resuelve. Se parece más a los hilos virtuales de Java 21 que a `join()` sobre un hilo de plataforma.

2.3.1 Cancelación con `AbortSignal`

Si el usuario cambia de página antes de que llegue la respuesta, la petición anterior ya no interesa. El parámetro `signal` permite cancelarla: TanStack Query crea un `AbortController` por consulta y pasa su `signal` a `fetch`. Por eso en el bloque `catch` se deja pasar el error de cancelación sin convertirlo en un error de red:

`src/lib/api-client.ts`

```
if (error instanceof DOMException && error.name === 'AbortError') throw error
```

2.4 Módulos: `import` y `export`

↑ Índice

Cada fichero `.ts` o `.tsx` es un **módulo** con su propio ámbito: lo que no se exporta, no se ve desde fuera.

```
export const PAGE_SIZE_OPTIONS = [5, 10, 20, 50] as const      // exportación con nombre
export function nextSort(...) { ... }

import { nextSort, PAGE_SIZE_OPTIONS } from '@/lib/list-params' // importación con nombre
import type { SortDirection } from '@/lib/list-params'          // importación solo de tipos
```

Tres detalles del proyecto:

- **Solo exportaciones con nombre.** El proyecto evita `export default`, de modo que cada cosa se importa siempre con el mismo nombre y el editor puede renombrarla de forma segura.
- **`import type`.** Importa solo información de tipos, que desaparece al compilar. La opción `verbatimModuleSyntax` del `tsconfig` lo exige cuando lo importado es solo un tipo (capítulo 5).
- **El alias `@/`.** `@/lib/list-params` significa `src/lib/list-params`. Evita rutas relativas del tipo `../../../lib`. Se configura en `vite.config.ts` y en `tsconfig.app.json`.

> **Si vienes de Java...**
> Un módulo es lo más parecido a una clase con miembros `public` (lo exportado) y privados (lo no exportado), pero sin clase: puedes exportar funciones, constantes y tipos sueltos. El alias `@/` cumple la función de la raíz de paquetes `com.albertoderafael.itmanager`.

2.5 El sistema de tipos de TypeScript

↑ Índice

2.5.1 Anotaciones e inferencia

Los tipos se escriben después del nombre, con dos puntos. Pero en la mayoría de los casos **no hace falta escribirlos**: TypeScript los infiere.

```
const size = 10                       // inferido: number
function cn(...inputs: ClassValue[]) { return twMerge(clsx(inputs)) }  // retorno inferido: string
```

La regla práctica del proyecto: se anotan los **parámetros** de las funciones y las **estructuras públicas** (tipos de DTO, *props* de componentes); el resto se deja a la inferencia.

2.5.2 `interface` y `type`

Hay dos formas de dar nombre a un tipo:

`src/features/applications/model/types.ts`

```
/** ApplicationResponse */
export interface Application {
  id: number
  name: string
  description: string | null
  criticality: ApplicationCriticality
  status: ApplicationStatus
}
```

```ts
export type ApplicationStatus = (typeof APPLICATION_STATUSES)[number]
```

`interface` describe la forma de un objeto y puede extenderse (`interface B extends A`). `type` puede nombrar **cualquier** tipo: uniones, tuplas, funciones, tipos calculados. En la práctica son intercambiables para objetos; el proyecto usa `interface` para formas de objeto (DTO y *props*) y `type` para todo lo demás.

2.5.3 Tipado estructural

Esta es la diferencia conceptual más grande con Java. En Java el tipado es **nominal**: un objeto es un `Application` solo si su clase se declaró así. En TypeScript es **estructural**: un valor es compatible con un tipo si **tiene la forma** que el tipo exige, se llame como se llame.

```ts
interface Paginable { page: number; size: number }

const params: ApplicationListParams = { page: 0, size: 10, sortField: 'id', sortDirection: 'asc' }
const p: Paginable = params   // válido: params tiene page y size (y más cosas)
```

`writePaginationParams` en `src/lib/list-params.ts` se aprovecha de ello: acepta `{ page, size, sortField, sortDirection }`, y por eso sirve tanto para los parámetros de aplicaciones como para los de demandas sin que ninguno «implemente» nada.

> **Si vienes de Java…**
>
> Es como si todas las interfaces se implementaran automáticamente: no hay `implements`. Un objeto literal con las propiedades correctas **es** un `Application`. Esto hace innecesarios muchos patrones de Java (adaptadores, mapeadores triviales) y es la razón de que en el frontend no haya clases DTO: basta con objetos planos y tipos que los describen.

2.5.4 Uniones y tipos literales

Un tipo puede ser una **unión** de varios: `string | null` admite un texto o `null`. Y un tipo puede ser un **valor literal concreto**: el tipo `'asc'` solo admite la cadena `'asc'`. Combinando ambas cosas se obtienen enumeraciones muy ligeras:

`src/lib/list-params.ts`

```ts
export type SortDirection = 'asc' | 'desc'
```

Si escribes `sortDirection: 'ascending'`, el compilador lo rechaza, y el editor te ofrece `'asc'` y `'desc'` al autocompletar.

2.5.5 El patrón `as const` + `(typeof X)[number]`

El proyecto necesita los valores de cada enumeración **dos veces**: como tipo (para el compilador) y como lista en ejecución (para pintar las opciones de un desplegable o validar la URL). Para no escribirlos dos veces, se declaran una sola vez como array y se **deriva** el tipo:

`src/features/applications/model/types.ts`

```ts
export const APPLICATION_STATUSES = ['ACTIVE', 'INACTIVE'] as const
export type ApplicationStatus = (typeof APPLICATION_STATUSES)[number]
```

Paso a paso:

1. Sin `as const`, el tipo del array sería `string[]`: se perderían los valores concretos.
2. Con `as const`, el tipo es `readonly ['ACTIVE', 'INACTIVE']`: una tupla de solo lectura con literales exactos.
3. `typeof APPLICATION_STATUSES` toma el **tipo** de esa constante.
4. `[number]` significa «el tipo de cualquier elemento accesible con un índice numérico», es decir, `'ACTIVE' | 'INACTIVE'`.

Si mañana el backend añade `RETIRED`, se añade en un solo sitio y el tipo, los desplegables, la validación de la URL y los esquemas de Zod se actualizan solos.

> **Decisión de diseño: Uniones de literales en lugar de `enum`**
>
> TypeScript tiene una palabra reservada `enum`, pero el proyecto no la usa. Un `enum` de TypeScript genera código en ejecución (un objeto), tiene particularidades con los valores numéricos y no es compatible con la opción `erasableSyntaxOnly`, que exige que todo lo de TypeScript sea borrable sin más. Las uniones de literales son simples cadenas en ejecución —exactamente lo que viaja en el JSON del backend— y el patrón `as const` da además la lista de valores.

2.5.6 `Record` y la exhaustividad

`Record<K, V>` es el tipo de un objeto cuyas claves son `K` y cuyos valores son `V`. Combinado con una unión de literales, obliga a cubrir **todos** los casos:

`src/features/applications/model/constants.ts`

```ts
export const STATUS_LABELS: Record<ApplicationStatus, string> = {
  ACTIVE: 'Activa',
  INACTIVE: 'Inactiva',
}
```

Si se añade `RETIRED` a `APPLICATION_STATUSES` y no se añade aquí su etiqueta, el proyecto **no compila**. Es el equivalente de un `switch` exhaustivo sobre un `enum` sellado de Java 21.

2.5.7 Propiedades opcionales y nulas

Hay dos formas de expresar «puede no estar», y el proyecto usa cada una con un significado:

`src/features/applications/model/types.ts`

```ts
export interface Application {
  description: string | null        // siempre presente; puede valer null (así lo envía el backend)
}
export interface ApplicationListParams {
  status?: ApplicationStatus        // puede no existir: «sin filtro de estado»
}
```

`status?: ApplicationStatus` equivale a `status?: ApplicationStatus | undefined`: la propiedad puede omitirse. Con `strict` activado (lo está), TypeScript **no deja usar un valor que puede ser `null` o `undefined` sin comprobarlo antes**. Es el mayor salto de seguridad respecto a Java: el equivalente a que el compilador detectara todos los `NullPointerException` posibles.

2.5.8 Estrechamiento y guardas de tipo

Cuando una variable tiene un tipo unión, TypeScript **estrecha** su tipo según las comprobaciones que haces:

src/lib/format.ts

```
export function formatDateTime(iso: string | null | undefined, style: 'short' | 'long' = 'short'):
string {
  if (!iso) return '—'
  const date = new Date(iso)     // aquí iso ya es string: null y undefined se descartaron arriba
  // …
}
```

Para tipos que el compilador no puede deducir solo, se escriben **guardas de tipo**: funciones que devuelven `valor is Tipo`.

src/lib/api-client.ts

```
export function isApiError(error: unknown): error is ApiError {
  return error instanceof ApiError
}
```

Tras `if (isApiError(error))`, el compilador sabe que `error` es un `ApiError` y permite acceder a `error.status` o `error.kind`. Observa también el tipo `unknown`: es el tipo de algo de lo que no se sabe nada, y obliga a comprobarlo antes de usarlo. Es el tipo de lo que se captura en un `catch`, porque en JavaScript se puede lanzar cualquier cosa, no solo errores.

> **Cuidado**
>
> Existe también el tipo `any`, que **desactiva** la comprobación de tipos para esa variable. Es una vía de escape que contagia: lo que sale de un `any` también es `any`. El proyecto no lo usa; cuando no se conoce un tipo se usa `unknown` y se comprueba.

2.5.9 Genéricos

Funcionan como en Java, con una sintaxis parecida:

src/lib/list-params.ts

```
/** Devuelve el valor si pertenece a la lista; si no, undefined. */
export function parseEnumParam<T extends string>(
  allowed: readonly T[],
  value: string | null,
): T | undefined {
  return value !== null && (allowed as readonly string[]).includes(value) ? (value as T) : undefined
}
```

`<T extends string>` declara un parámetro de tipo restringido a subtipos de `string`. Al llamar a `parseEnumParam(APPLICATION_STATUSES, search.get('estado'))`, TypeScript infiere que `T` es `'ACTIVE' | 'INACTIVE'` y la función devuelve `ApplicationStatus | undefined`. Una sola función valida estados, criticidades, prioridades y campos de ordenación.

Los componentes también pueden ser genéricos. `FilterSelect<V extends string>` (`src/components/common/filter-select.tsx`) recibe opciones de tipo `V` y llama a `onChange` con un `V`, de modo que el filtro de estado devuelve un `ApplicationStatus` y el de prioridad un `DemandPriority`, sin conversiones.

2.5.10 Tipos utilitarios

TypeScript permite **calcular tipos a partir de otros**. Los más usados en el proyecto:

Tipo	Significado	Ejemplo
`Partial<T>`	Todas las propiedades de `T`, opcionales	`updateParams(patch: Partial<ApplicationListParams>)`: se pasa solo lo que cambia
`Record<K, V>`	Objeto con claves `K` y valores `V`	Etiquetas de estado, estilos por tono
`readonly T[]`	Array que no se puede modificar	Listas de opciones y transiciones permitidas
`ReturnType<F>`	Tipo devuelto por una función	Útil con funciones de librerías
`z.infer`, `z.input`	Tipo derivado de un esquema de Zod	Tipos de los formularios (Parte VII)

2.5.11 Clases

TypeScript admite clases, pero el código de React apenas las usa: componentes, hooks y utilidades son funciones. La excepción es `ApiError`, que extiende `Error` para poder lanzarse, capturarse e identificarse con `instanceof`:

`src/lib/api-client.ts`

```
export class ApiError extends Error {
  readonly status: number
  readonly kind: ApiErrorKind
  readonly fieldErrors: Record<string, string>
  // constructor…
}
```

2.5.12 Aserciones: as y !

`valor as Tipo` y `valor!` le dicen al compilador «confía en mí». No convierten nada ni comprueban nada en ejecución; solo silencian al compilador. Por eso se usan con mucha moderación y, cuando aparecen, conviene preguntarse qué garantiza que sean ciertas:

```
createRoot(document.getElementById('root')!)   // garantizado: index.html contiene el div
return (text ? JSON.parse(text) : undefined) as T   // confianza en el contrato con el backend
```

2.6 Los tipos desaparecen en ejecución

↑ Índice

Volvamos a la idea de la figura 2.1. Cuando el cliente HTTP hace `JSON.parse(text) as T`, TypeScript **cree** que el resultado es un `ApplicationPage`, pero nadie lo ha comprobado. Si el backend cambiara el nombre de `totalElements` a `total`, el proyecto seguiría compilando y el fallo aparecería en ejecución como un `undefined` inesperado.

Por eso los tipos de `model/types.ts` llevan un comentario con el DTO de Spring al que corresponden: son un **espejo manual del contrato**, y mantenerlo sincronizado es responsabilidad del equipo.

Decisión de diseño: Tipos de DTO escritos a mano y validación en ejecución solo en la entrada del usuario

Decisión. Los tipos de los DTO se escriben a mano, alineados con el backend, y las respuestas de la API no se validan en ejecución. Lo que sí se valida en ejecución, con Zod, es lo que introduce el usuario en los formularios.

Motivos. El backend es propio y su contrato está controlado; validar cada respuesta añadiría código y coste sin un beneficio claro. La entrada del usuario, en cambio, es la fuente de datos menos fiable y la que necesita mensajes de error comprensibles.

Alternativas. Generar los tipos desde la especificación OpenAPI del backend (con herramientas como `openapi-typescript` u `orval`) eliminaría la sincronización manual; es una evolución natural si la API crece o la desarrollan equipos distintos. Validar las respuestas con esquemas de Zod sería lo indicado para APIs de terceros poco fiables.

2.7 Errores típicos y depuración

↑ Índice

- **«Object is possibly 'undefined'».** El compilador te avisa de que no has contemplado el caso vacío. No lo silencies con `!`: añade la comprobación (`if (!data) return …`) o usa `?.` y `??`.
- **«Type 'string' is not assignable to type 'ApplicationStatus'».** Estás pasando un texto cualquiera donde se espera uno de los literales. Valídalo antes (como hace `parseEnumParam`) en lugar de forzarlo con `as`.
- **Mutar en lugar de copiar.** `params.page = 2` modifica el objeto original y React puede no enterarse del cambio. Crea uno nuevo: `{ ...params, page: 2 }` (capítulo 7).
- **Olvidar `await`.** Sin `await`, obtienes una promesa en lugar del valor. Si ves en consola `Promise {<pending>}` o un `[object Promise]` en pantalla, falta un `await`.
- **`fetch` no lanza en un 404 ni en un 500.** Hay que comprobar `response.ok`. El cliente del proyecto ya lo hace, pero tenlo en cuenta si escribes llamadas a mano.
- **Errores de tipos que no ves al desarrollar.** Vite no comprueba tipos (sección 4.4). Mira los subrayados del editor y ejecuta `npm run typecheck` antes de dar algo por terminado.

Para depurar, `console.log` sigue siendo útil, pero el depurador es mucho más potente: en la pestaña *Sources* de las DevTools, abre un fichero con Ctrl+P, haz clic en un número de línea para poner un punto de ruptura y podrás inspeccionar variables, avanzar paso a paso y ver la pila de llamadas, como en IntelliJ. También puedes escribir la sentencia `debugger;` en el código para detenerte en ese punto.

2.8 Ejercicios

↑ Índice

Ejercicio 2.1 · Añade un estado a la enumeración Ver solución →

En una rama de pruebas, añade `'RETIRED'` a `APPLICATION_STATUSES` en `src/features/applications/model/types.ts` y ejecuta `npm run typecheck`. ¿Qué ficheros dejan de compilar y por qué? Corrígelos y observa en qué lugares de la interfaz aparece el nuevo estado sin haber tocado nada más. Después deshaz el cambio: el backend no conoce ese estado.

Ejercicio 2.2 · Escribe una función genérica Ver solución →

Escribe en `src/lib/` una función `groupBy<T, K extends string>(items: readonly T[], key: (item: T) => K): Record<K, T[]>` que agrupe una lista. Úsala mentalmente sobre una página de aplicaciones para agruparlas por criticidad. ¿Qué tipo infiere TypeScript para el resultado? Pista: `reduce` o un bucle `for…of`.

Ejercicio 2.3 · Estrechamiento con `unknown` Ver solución →

Escribe una función `describeError(error: unknown): string` que devuelva: «Error de red» si es un `ApiError` de tipo `network`; «Datos no válidos: campo1, campo2» si es un `ApiError` con `fieldErrors`; el mensaje si es un `Error` cualquiera; y «Error desconocido» en otro caso. Comprueba que el compilador no te deja acceder a `error.kind` antes de estrechar el tipo.

2.9 Autoevaluación

↑ Índice

Autoevaluación del capítulo 2 Ver respuestas →

1. ¿Qué queda de los tipos de TypeScript cuando el código llega al navegador?
2. ¿Cuál es la diferencia entre `null` y `undefined`, y cómo se usa cada uno en el proyecto?
3. ¿Qué diferencia hay entre `a ?? b` y `a || b`? Pon un ejemplo en el que den resultados distintos.
4. ¿Qué significa que el tipado de TypeScript sea estructural? ¿Qué consecuencia tiene frente a Java?
5. Explica qué hace `(typeof APPLICATION_STATUSES)[number]` y para qué sirve `as const`.
6. ¿Por qué `Record<ApplicationStatus, string>` ayuda a no olvidar casos?
7. ¿Por qué `fetch` exige comprobar `response.ok`?
8. ¿Qué diferencia hay entre `unknown` y `any`?
9. ¿Qué garantiza realmente `JSON.parse(text) as T`?

CAPÍTULO 3

Node.js, npm y el ecosistema de paquetes

↑ Volver al índice

EN ESTE CAPÍTULO

3.1 Qué es Node.js y para qué lo usamos
3.2 `package.json` : el `pom.xml` del frontend
3.3 Versiones semánticas y el fichero de bloqueo
3.4 `node_modules` y la resolución de dependencias
3.5 Módulos ES frente a CommonJS
3.6 Los scripts del proyecto
3.7 Gestionar dependencias en el día a día
3.8 Errores típicos y depuración
3.9 Ejercicios
3.10 Autoevaluación

El frontend se ejecuta en el navegador, pero se desarrolla y se construye con herramientas que se ejecutan en Node.js y se instalan con npm. Este capítulo explica esas piezas con el mismo nivel de detalle con el que conoces Maven: qué describe el `package.json`, cómo se resuelven las versiones, qué hay en `node_modules` y qué hace cada script.

3.1 Qué es Node.js y para qué lo usamos

↑ Índice

Node.js es un entorno de ejecución de JavaScript fuera del navegador. Toma el motor V8 de Chrome y le añade acceso al sistema de ficheros, a la red y a los procesos. Con él se pueden escribir servidores, pero en este proyecto **no hay ningún servidor Node en producción**: Node solo ejecuta las **herramientas de desarrollo**.

Herramienta	Qué hace	Cuándo se ejecuta
Vite	Servidor de desarrollo y construcción del *bundle*	`npm run dev`, `npm run build`
TypeScript (`tsc`)	Comprobación de tipos	`npm run typecheck`, `npm run build`, el editor
ESLint	Análisis estático del código	`npm run lint`, el editor
Vitest	Pruebas unitarias	`npm run test`
CLI de shadcn	Añadir componentes de interfaz al proyecto	`npx shadcn add …`

El resultado de `npm run build` son ficheros estáticos que cualquier servidor web puede servir; Node ya no interviene.

Si vienes de Java…

Node aquí cumple el papel de la **JDK en tu máquina de desarrollo**: la necesitas para compilar, probar y empaquetar, pero lo que despliegas es el resultado. La diferencia es que en Java desplegabas un JAR que necesitaba una JVM para ejecutarse; aquí despliegas ficheros que ejecuta el navegador del usuario.

El proyecto exige una versión mínima de Node:

package.json

```
"engines": {
  "node": ">=22.12.0"
}
```

Node publica una versión par cada año que se convierte en LTS (soporte a largo plazo). Usa siempre una LTS: la 22 o la 24. Si trabajas con varios proyectos que necesitan versiones distintas, un gestor de versiones como `nvm` (o `nvm-windows`, `fnm` o `volta`) te permite cambiar de una a otra.

3.2 `package.json`: el `pom.xml` del frontend

↑ Índice

El fichero `package.json` describe el proyecto: su nombre, sus dependencias y sus tareas.

package.json (extracto)

```
{
  "name": "it-service-manager-web",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "typecheck": "tsc -b",
    "lint": "eslint .",
    "test": "vitest run",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.3.0",
    "@tanstack/react-query": "^5.103.2",
    "zod": "^3.25.76"
  },
  "devDependencies": {
    "vite": "^8.3.0",
    "typescript": "^6.0.3",
    "vitest": "^5.0.1"
  }
}
```

- `"private": true` evita publicar el proyecto por error en el registro público de npm.
- `"type": "module"` indica que los ficheros `.js` del proyecto usan módulos ES (`import` / `export`), no el sistema antiguo de Node (sección 3.5).
- `scripts` define tareas que se lanzan con `npm run <nombre>` (sección 3.6).
- `dependencies` y `devDependencies` son las dependencias, con una distinción importante.

3.2.1 `dependencies` frente a `devDependencies`

En un proyecto de navegador, la frontera es: **¿acaba este código dentro del *bundle* que descarga el usuario?**

- **`dependencies`**: librerías que se ejecutan en el navegador (React, TanStack Query, Zod, los iconos...). Vite las incluye en el *bundle*.

- **`devDependencies`**: herramientas que solo se usan al desarrollar (Vite, TypeScript, ESLint, Vitest, los tipos `@types/*`). Nunca llegan al usuario.

> **Si vienes de Java…**
>
> Es la misma idea que los ámbitos de Maven: `dependencies` sería `compile` y `devDependencies` sería `test` o `provided`, más los *plugins* de construcción. En la práctica, Vite incluye en el *bundle* todo lo que se importa desde `src/`, esté donde esté declarado; la separación es sobre todo documental y sirve para instalaciones de producción en proyectos de servidor.

3.2.2 Las dependencias del proyecto, una a una

Paquete	Para qué sirve	Parte
`react`, `react-dom`	La librería de interfaz y su conexión con el DOM del navegador	III
`react-router`	Navegación entre pantallas y rutas	VII
`@tanstack/react-query`	Caché y sincronización de los datos del servidor	VI
`react-hook-form`	Estado y envío de formularios	VII
`zod`, `@hookform/resolvers`	Esquemas de validación y su integración con los formularios	VII
`radix-ui`	Componentes accesibles sin estilos (desplegables, diálogos…) sobre los que se construye shadcn/ui	IV
`class-variance-authority`, `clsx`, `tailwind-merge`	Composición de clases de Tailwind según variantes	IV
`lucide-react`	Iconos como componentes de React	IV
`sonner`	Notificaciones emergentes (*toasts*)	IV
`@fontsource-variable/inter`	La fuente Inter, servida desde el propio proyecto	IV
`vite`, `@vitejs/plugin-react`	Servidor de desarrollo y construcción; soporte de React	II
`tailwindcss`, `@tailwindcss/vite`, `tw-animate-css`	Motor de estilos y animaciones	IV
`typescript`, `@types/react`, `@types/react-dom`, `@types/node`	Compilador y definiciones de tipos	II
`eslint`, `typescript-eslint`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`, `@eslint/js`, `globals`	Análisis estático	II
`vitest`	Pruebas	IX

Los paquetes `@types/*` merecen una explicación: contienen **solo definiciones de tipos** (ficheros `.d.ts`) para librerías escritas en JavaScript que no incluyen las suyas. Muchas librerías modernas ya traen sus tipos y no los necesitan.

3.3 Versiones semánticas y el fichero de bloqueo

↑ Índice

Los paquetes de npm siguen el **versionado semántico** (*semver*): `MAYOR.MENOR.PARCHE`.

- **PARCHE** (`5.103.2 → 5.103.3`): correcciones compatibles.
- **MENOR** (`5.103.2 → 5.104.0`): funcionalidad nueva compatible.
- **MAYOR** (`5.x → 6.0`): cambios incompatibles.

En `package.json` no se fijan versiones exactas, sino **rangos**:

Rango	Significado	Ejemplo con `^19.3.0`
`^19.3.0`	Compatible: misma versión mayor	Admite `19.3.1`, `19.9.0`; no `20.0.0`
`~19.3.0`	Misma versión menor	Admite `19.3.5`; no `19.4.0`
`19.3.0`	Exacta	Solo `19.3.0`

Si el `package.json` solo tiene rangos, ¿cómo se garantiza que todo el equipo, y el servidor de integración continua, instalan **exactamente** las mismas versiones? Con el fichero **`package-lock.json`**: npm anota en él la versión concreta, la URL de descarga y una suma de verificación de **cada** paquete instalado, incluidos los transitivos. Ese fichero **se versiona en Git** y no se edita a mano.

Comando	Qué hace	Cuándo
`npm install`	Instala respetando el *lockfile*; si `package.json` cambió, lo actualiza	Desarrollo diario
`npm ci`	Borra `node_modules` e instala **exactamente** lo del *lockfile*; falla si no coincide con `package.json`	Integración continua, instalaciones limpias
`npm install paquete`	Añade una dependencia	Al incorporar una librería
`npm install -D paquete`	Añade una dependencia de desarrollo	Herramientas

Si vienes de Java…

Maven resuelve versiones exactas declaradas en el `pom.xml` (salvo rangos, que casi nadie usa), así que no necesita *lockfile*. En npm los rangos son la norma y el *lockfile* es lo que da reproducibilidad. Piensa en él como en un `dependency:tree` congelado y firmado.

En el proyecto

Fíjate en `"zod": "^3.25.76"`. Zod ya tiene una versión mayor 4, con cambios en la forma de personalizar mensajes de error. El proyecto usa la API de la rama 3 (por ejemplo, la opción `errorMap` en `schemas.ts`), y el `^` garantiza que `npm install` nunca saltará a la 4 por sorpresa. Actualizar a una nueva versión mayor es siempre una decisión explícita, con su lectura de la guía de migración.

3.4 `node_modules` y la resolución de dependencias

↑ Índice

`npm install` descarga los paquetes en la carpeta `node_modules`, dentro del propio proyecto (no hay un repositorio local compartido como `~/.m2`, aunque npm sí guarda una caché de descargas). Allí están también las dependencias transitivas, en una estructura lo más plana posible.

Algunas consecuencias prácticas:

- `node_modules` es **grande** (cientos de megas) y **nunca** se versiona: está en `.gitignore`. Se regenera siempre con `npm install`.
- Si algo se comporta de forma extraña tras cambiar de rama, la solución clásica es borrar `node_modules` y reinstalar.
- Los ejecutables de las herramientas quedan en `node_modules/.bin`. Por eso los scripts pueden llamar a `vite` o `tsc` sin tenerlos instalados globalmente, y `npx <herramienta>` ejecuta la versión local del proyecto.

3.5 Módulos ES frente a CommonJS

↑ Índice

Node nació antes de que JavaScript tuviera módulos estándar e inventó el suyo, **CommonJS**: `require()` y `module.exports`. Desde ES2015, el estándar son los **módulos ES** (*ESM*): `import` y `export`. Ambos sistemas conviven todavía en el ecosistema.

```
// CommonJS (antiguo)
const react = require('react')
module.exports = { miFuncion }

// Módulos ES (el estándar, el que usa el proyecto)
import react from 'react'
export { miFuncion }
```

Con `"type": "module"`, incluso los ficheros de configuración que ejecuta Node, como `eslint.config.js`, usan `import`. Si ves un error del tipo `require is not defined in ES module scope`, estás mezclando ambos sistemas.

3.6 Los scripts del proyecto

↑ Índice

Comando	Qué ejecuta	Qué hace
`npm run dev`	`vite`	Arranca el servidor de desarrollo en `http://localhost:5173`, con recarga en caliente y proxy a la API
`npm run build`	`tsc -b && vite build`	Comprueba los tipos y, **solo si no hay errores**, genera la versión de producción en `dist/`
`npm run typecheck`	`tsc -b`	Solo comprueba los tipos, sin generar nada
`npm run lint`	`eslint .`	Analiza el código en busca de errores y malas prácticas
`npm run test`	`vitest run`	Ejecuta las pruebas una vez

Comando	Qué ejecuta	Qué hace
`npm run preview`	`vite preview`	Sirve la carpeta `dist/` para probar la versión de producción en local

El `&&` de `build` es de la *shell*: ejecuta el segundo comando solo si el primero termina sin error. Así, un error de tipos impide generar una versión de producción, igual que un fallo de compilación impide a Maven empaquetar el JAR.

Si vienes de Java...

npm	Maven
`npm install`	`mvn dependency:resolve`
`npm run dev`	`mvn spring-boot:run` (con *devtools*)
`npm run build`	`mvn package`
`npm run typecheck`	`mvn compile`
`npm run test`	`mvn test`
`npm run lint`	Checkstyle, PMD o SpotBugs

3.7 Gestionar dependencias en el día a día

↑ Índice

- **Ver qué está desactualizado:** `npm outdated` muestra la versión instalada, la última compatible con tu rango y la última publicada.
- **Actualizar dentro de los rangos:** `npm update` actualiza a la última versión compatible y refresca el *lockfile*.
- **Saltar de versión mayor:** se hace paquete a paquete, con `npm install paquete@latest`, leyendo las notas de la versión y ejecutando `typecheck`, `lint`, `test` y una prueba manual.
- **Vulnerabilidades:** `npm audit` cruza tus dependencias con una base de datos de vulnerabilidades conocidas.

Antes de añadir una librería, hazte las mismas preguntas que con una dependencia de Maven: ¿está mantenida?, ¿cuántas dependencias arrastra?, ¿cuánto pesa en el *bundle*?, ¿lo resuelvo con veinte líneas propias? El proyecto, por ejemplo, prescinde de axios porque `fetch` ya cubre lo necesario (Parte VI).

Decisión de diseño: npm como gestor de paquetes

Decisión. Usar npm, el gestor que viene con Node.

Alternativas. pnpm (más rápido y eficiente en disco gracias a un almacén compartido con enlaces) y Yarn. Son excelentes, sobre todo en repositorios con muchos proyectos, pero para un único proyecto la diferencia es pequeña y npm no requiere instalar nada adicional. Cambiar más adelante es sencillo: lo importante es que todo el equipo use el mismo y que solo exista un fichero de bloqueo.

3.8 Errores típicos y depuración

↑ Índice

- **`'vite' no se reconoce como un comando` o `Cannot find module`.** Faltan las dependencias: ejecuta `npm install`.
- **Errores raros tras cambiar de rama o actualizar.** Borra `node_modules` (y, si persiste, también la caché de Vite en `node_modules/.vite`) y ejecuta `npm install`.
- **`EBADENGINE` o errores de sintaxis en herramientas.** Tu versión de Node es más antigua que la que exige `engines`. Compruébala con `node -v`.
- **Conflictos en `package-lock.json` al fusionar ramas.** No lo edites a mano: resuelve el conflicto en `package.json` y ejecuta `npm install` para regenerarlo.
- **`ERESOLVE unable to resolve dependency tree`.** Dos paquetes exigen versiones incompatibles de una tercera dependencia, típicamente de `react`. Lee el mensaje: dice quién pide qué. Evita `--force` como solución por defecto.

3.9 Ejercicios

↑ Índice

Ejercicio 3.1 · Lee el árbol de dependencias Ver solución →

Ejecuta `npm ls react` y `npm ls zod`. ¿Qué paquetes dependen de cada uno? Después ejecuta `npm explain @radix-ui/react-select` y localiza qué paquete de `package.json` lo introduce en el proyecto.

Ejercicio 3.2 · Comprueba qué está desactualizado Ver solución →

Ejecuta `npm outdated`. Clasifica cada fila en una de tres categorías: actualizable con `npm update`, requiere un salto de versión menor fuera del rango o requiere un salto de versión mayor. Para Zod, lee por encima la guía de migración de la versión 4 y apunta qué ficheros del proyecto habría que cambiar.

Ejercicio 3.3 · Rompe el build a propósito Ver solución →

Introduce un error de tipos (por ejemplo, asigna `'ACTIVA'` a una variable de tipo `ApplicationStatus`) y ejecuta `npm run dev` y después `npm run build`. ¿Se queja el servidor de desarrollo? ¿Y el *build*? Explica la diferencia con lo que sabes del script `build`.

3.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 3 Ver respuestas →

1. ¿Se ejecuta Node.js en producción en este proyecto? ¿Para qué se usa?
2. ¿Qué criterio separa `dependencies` de `devDependencies` en una aplicación de navegador?
3. ¿Qué versiones admite el rango `^5.103.2`? ¿Y `~5.103.2`?
4. ¿Para qué sirve `package-lock.json` y por qué se versiona en Git?
5. ¿Cuándo usarías `npm ci` en lugar de `npm install`?
6. ¿Por qué no se versiona `node_modules`?
7. ¿Qué indica `"type": "module"` en `package.json`?

8. ¿Qué ocurre al ejecutar `npm run build` si hay un error de tipos?

PARTE II

Herramientas del proyecto

4	Vite: el entorno de desarrollo y construcción	pág. 46
5	Configuración de TypeScript, ESLint y calidad del código	pág. 54

↑ Volver al índice

CAPÍTULO 4

Vite: el entorno de desarrollo y construcción

↑ Volver al índice

EN ESTE CAPÍTULO

4.1 Qué es Vite y qué problema resuelve
4.2 El modo desarrollo
4.3 El modo producción
4.4 Vite no comprueba los tipos
4.5 `vite.config.ts` línea a línea
4.6 Variables de entorno
4.7 El proxy de desarrollo
4.8 Decisión: Vite frente a otras herramientas
4.9 Errores típicos y depuración
4.10 Ejercicios
4.11 Autoevaluación

Vite es la herramienta que convierte la carpeta `src/` en una aplicación funcionando: en desarrollo sirve los módulos al navegador con recarga instantánea y reenvía las llamadas a la API al backend; en producción genera el *bundle* optimizado. Entender sus dos modos de funcionamiento explica muchos comportamientos que, de otro modo, parecen magia.

4.1 Qué es Vite y qué problema resuelve

↑ Índice

Hasta hace unos años, la herramienta habitual era **webpack**: antes de poder ver nada en el navegador, empaquetaba **toda** la aplicación. En un proyecto grande, arrancar el servidor de desarrollo tardaba decenas de segundos y cada cambio, varios más.

Vite («rápido», en francés) parte de una observación: los navegadores modernos ya entienden módulos ES (`<script type="module">` e `import`). En desarrollo no hace falta empaquetar nada; basta con **servir cada fichero por separado, transformándolo al vuelo** cuando el navegador lo pide. Solo al construir para producción, donde cientos de peticiones serían lentas, se empaqueta.

Vite tiene, por tanto, **dos modos con motores distintos**:

	Desarrollo (`npm run dev`)	Producción (`npm run build`)
Objetivo	Arranque y cambios instantáneos	Ficheros mínimos y cacheables
Empaqueta tu código	No: un módulo por petición	Sí: *chunks* optimizados
Motor	Servidor propio + transformación con Oxc	Rolldown
Resultado	En memoria, servido en `localhost:5173`	Carpeta `dist/`

> **Nota**
>
> Desde **Vite 8**, tanto el empaquetado de producción como la preparación de dependencias usan **Rolldown**, un empaquetador escrito en Rust por el mismo equipo, y las transformaciones de TypeScript y JSX usan **Oxc**. Hasta Vite 7 esas tareas se repartían entre esbuild y Rollup. Si lees documentación antigua, encontrarás esos nombres; el concepto es el mismo.

4.2 El modo desarrollo

↑ Índice

Cuando ejecutas `npm run dev` y abres `http://localhost:5173`:

1. El navegador pide `/` (o cualquier ruta) y Vite responde con `index.html`, al que añade un pequeño cliente para la recarga en caliente.
2. El navegador encuentra `<script type="module" src="/src/main.tsx">` y pide `/src/main.tsx`.
3. Vite lee el fichero, **elimina los tipos y transforma el JSX** a JavaScript, y lo devuelve. Tarda milisegundos, porque transforma un único fichero.
4. El navegador ve los `import` de `main.tsx` y pide cada módulo: `/src/app/app.tsx`, `/src/index.css`... y así sucesivamente, solo con los módulos que realmente se usan en la pantalla actual.

4.2.1 La preparación de dependencias

Las librerías de `node_modules` son un caso especial. Muchas están formadas por cientos de ficheros pequeños (por ejemplo, `radix-ui` incluye decenas de subpaquetes), y algunas todavía se publican en formato CommonJS, que el navegador no entiende. La primera vez que arrancas, Vite **pre-empaqueta** cada dependencia en un único módulo ES y lo guarda en `node_modules/.vite/deps`. Las siguientes veces reutiliza esa caché.

Por eso, la primera ejecución de `npm run dev` tras instalar dependencias tarda un poco más, y por eso a veces aparece el mensaje *new dependencies optimized, reloading*: Vite ha descubierto una dependencia nueva y recarga la página tras prepararla.

4.2.2 Recarga en caliente (HMR) y React Fast Refresh

Cuando guardas un fichero, Vite no recarga la página entera. Su servidor mantiene una conexión WebSocket abierta con el navegador y le avisa de **qué módulo ha cambiado**. El navegador descarga solo ese módulo y lo sustituye en caliente (*Hot Module Replacement*, HMR).

Para los componentes de React, el plugin `@vitejs/plugin-react` añade **React Fast Refresh**: el componente modificado se vuelve a pintar **conservando su estado**. Puedes estar a mitad de un formulario, cambiar el color de un botón en el código y ver el cambio sin perder lo que habías escrito.

> **Cuidado**
>
> Fast Refresh solo funciona bien si un fichero `.tsx` exporta **únicamente componentes**. Si además exporta constantes o funciones, Vite no puede saber si el cambio afecta a otros módulos y recarga la página entera. La regla de ESLint `react-refresh/only-export-components`, activa en el proyecto, te avisa de ello (capítulo 5). Por eso las constantes y utilidades viven en ficheros `.ts` aparte: `constants.ts`, `list-params.ts`, `navigation.ts`.

> **Si vienes de Java...**
>
> El efecto es parecido a Spring Boot DevTools o al *hot swap* del depurador de IntelliJ, pero bastante más fino: no se reinicia el contexto de la aplicación, solo se sustituye un módulo y el estado de la pantalla sobrevive.

4.3 El modo producción

↑ Índice

`vite build` hace lo contrario: parte de `index.html`, sigue todos los `import`, y con Rolldown genera la carpeta `dist/`:

```
dist/
├── index.html                          ← con las rutas a los ficheros con huella
├── favicon.svg                         ← copiado tal cual desde public/
└── assets/
    ├── index-<hash>.js                 ← bundle inicial: React, router, layout…
    ├── index-<hash>.css                ← todo el CSS (Tailwind ya procesado)
    ├── applications-list-page-<hash>.js  ← un chunk por página con carga diferida
    ├── demands-list-page-<hash>.js
    └── …
```

Aquí se aplican todas las optimizaciones del capítulo 1: eliminación de código muerto, minificación, división en *chunks* y huellas en los nombres. También se sustituyen las expresiones `import.meta.env.VITE_*` por sus valores literales (sección 4.6).

La carpeta `public/` contiene ficheros que se copian sin procesar a la raíz de `dist/` (aquí, el `favicon.svg`). Se referencian con ruta absoluta (`/favicon.svg`).

`npm run preview` arranca un servidor sencillo sobre `dist/` para comprobar el resultado de producción en local. No lo uses para desplegar: es solo para probar.

4.4 Vite no comprueba los tipos

↑ Índice

Este punto sorprende a todo el que llega desde Java: **Vite borra los tipos sin comprobarlos**. Oxc trata cada fichero de forma aislada y no tiene la visión global que necesita una comprobación de tipos. Es lo que le permite transformar un fichero en milisegundos.

Consecuencias:

- `npm run dev` arranca y funciona **aunque haya errores de tipos**. El navegador recibe el JavaScript igualmente.
- Los errores de tipos los ves en el **editor** (que ejecuta el servicio de lenguaje de TypeScript continuamente) y al ejecutar `tsc`.
- Por eso `npm run build` ejecuta `tsc -b` antes de `vite build`: así ninguna versión de producción sale con errores de tipos.

> **Decisión de diseño: Separar transformación y comprobación de tipos**
>
> Es el diseño estándar de Vite y del ecosistema actual: transformación rápida y aislada para el día a día, y comprobación completa en el editor, en el *build* y en la integración continua. La alternativa, comprobar tipos en el propio servidor de desarrollo con un plugin como `vite-plugin-checker`, es posible, pero ralentiza el ciclo y duplica lo que ya hace el editor. El proyecto no lo usa; a cambio, conviene ejecutar `npm run type-check` antes de cada *commit*.

4.5 `vite.config.ts` línea a línea

↑ Índice

Toda la configuración de Vite está en un fichero TypeScript que se ejecuta en Node:

`vite.config.ts`

```ts
import { fileURLToPath, URL } from 'node:url'
import { defineConfig, loadEnv } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '')

  // Microservicio de demandas
  const demandTarget =
    env.VITE_DEMAND_API_TARGET || 'http://localhost:8080'

  // Microservicio de aplicaciones
  const applicationTarget =
    env.VITE_APPLICATION_API_TARGET || 'http://localhost:8081'

  return {
    plugins: [react(), tailwindcss()],

    resolve: {
      alias: {
        '@': fileURLToPath(new URL('./src', import.meta.url)),
      },
    },

    server: {
      port: 5173,
      proxy: {
        '/api/demands': { target: demandTarget, changeOrigin: true },
        '/api/applications': { target: applicationTarget, changeOrigin: true },
      },
    },

    test: {
      environment: 'node',
      include: ['src/**/*.test.ts'],
    },
  }
})
```

Lo recorremos por bloques:

- **`defineConfig(({ mode }) => ({ … }))`.** La configuración puede ser un objeto o, como aquí, una **función** que recibe el modo (`development`, `production`) y devuelve el objeto. `defineConfig` no hace nada en ejecución: solo da tipos y autocompletado.
- **`loadEnv(mode, process.cwd(), '')`.** Lee los ficheros `.env` del proyecto. El tercer argumento es el prefijo de las variables que se cargan; con `''` se cargan **todas**, pero solo para usarlas aquí, en la configuración, que se ejecuta en Node. No llegan al navegador.
- **`plugins`.** `react()` añade la transformación de JSX y React Fast Refresh. `tailwindcss()` integra el motor de Tailwind CSS 4, que genera el CSS a partir de las clases que encuentra en el código (Parte IV).

- **`resolve.alias`**. Define que `@` apunta a la carpeta `src`. `import.meta.url` es la URL del propio fichero de configuración, y `fileURLToPath` la convierte en una ruta del sistema. El mismo alias está declarado en `tsconfig.app.json` para que **TypeScript** también lo entienda: Vite resuelve los ficheros y TypeScript comprueba los tipos, y cada uno necesita saberlo.
- **`server`**. El puerto del servidor de desarrollo y el **proxy**, que explicamos en la sección 4.7.
- **`test`**. La configuración de Vitest, que reutiliza la de Vite: las pruebas se ejecutan en un entorno Node (sin navegador simulado) e incluyen los ficheros `*.test.ts` (Parte IX).

En el proyecto

La configuración refleja que el backend se ha separado en **dos servicios**: las demandas en el puerto 8080 y las aplicaciones en el 8081, cada uno con su propia variable de entorno para cambiar el destino. El navegador no lo nota: sigue llamando a `/api/demands` y `/api/applications` en el mismo origen, y el proxy decide a qué servicio reenviar. Ten en cuenta que `.env.example` todavía documenta la variable anterior, `VITE_API_PROXY_TARGET`, que la configuración ya no usa; conviene actualizarlo con `VITE_DEMAND_API_TARGET` y `VITE_APPLICATION_API_TARGET` (ejercicio 4.2).

4.6 Variables de entorno

↑ Índice

Una aplicación necesita valores que cambian según el entorno, como la URL base de la API. Vite los lee de ficheros `.env`, en este orden de prioridad creciente:

Fichero	Cuándo se carga	¿Se versiona?
`.env`	Siempre	Sí, si no tiene secretos
`.env.local`	Siempre, salvo en pruebas	**No** (está en `.gitignore`)
`.env.development`, `.env.production`	Solo en ese modo	Sí
`.env.development.local`, `.env.production.local`	Solo en ese modo	**No**

En el código del navegador **solo están disponibles las variables que empiezan por `VITE_`**, a través de `import.meta.env`:

src/lib/api-client.ts

```ts
export const API_BASE_URL = (import.meta.env.VITE_API_BASE_URL as string | undefined) ?? '/api'
```

Y para que TypeScript conozca esas variables, se declaran en `src/vite-env.d.ts`:

src/vite-env.d.ts

```ts
/// <reference types="vite/client" />

interface ImportMetaEnv {
  readonly VITE_API_BASE_URL?: string
}
```

Cuidado

Las variables `VITE_*` no son secretas. Al construir, Vite sustituye `import.meta.env.VITE_API_BASE_URL` por su valor literal dentro del JavaScript que descarga cualquier usuario. Nunca pongas en ellas contraseñas, claves de API privadas ni nada que no publicarías en una página web. El prefijo existe precisamente para que no se filtren por accidente las demás variables del sistema. Además, el valor se fija **en el momento del *build***: para cambiarlo hay que volver a construir.

Si vienes de Java...

Es el equivalente de `application.properties` y los perfiles de Spring (`application-dev.properties`), con una diferencia crucial: en Spring las propiedades se leen en el servidor al arrancar; aquí se **incrustan en el código** al construir y viajan al navegador del usuario.

4.7 El proxy de desarrollo

↑ Índice

En desarrollo, el frontend se sirve desde `http://localhost:5173` y el backend escucha en `http://localhost:8080` y `8081`. Para el navegador son **orígenes distintos** (cambia el puerto), y por seguridad no permite que una página lea las respuestas de otro origen salvo que ese servidor lo autorice expresamente con cabeceras **CORS**. El backend no tiene CORS configurado.

El proxy de Vite evita el problema: el navegador **solo habla con Vite** (mismo origen), y es Vite, desde Node, quien reenvía la petición al backend. Entre servidores no existen restricciones de origen.

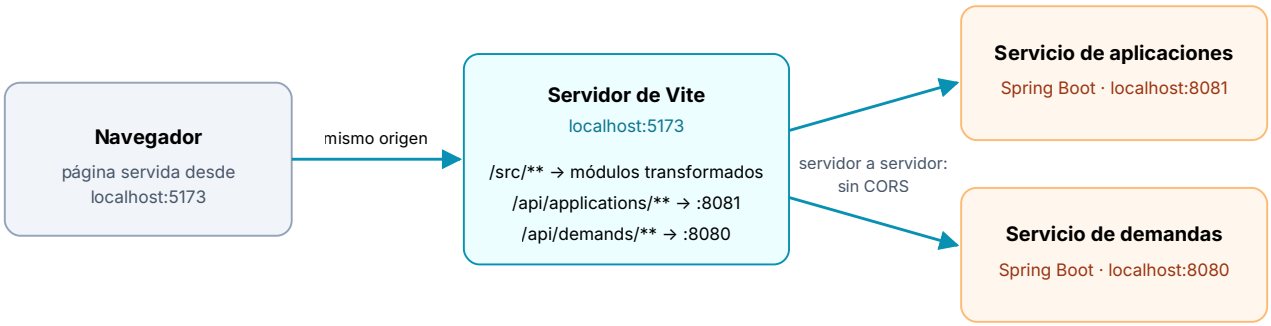


Figura 4.1. El proxy de Vite reenvía las llamadas a la API según su prefijo. El navegador nunca habla directamente con el backend.

Cada entrada de `proxy` asocia un **prefijo de ruta** a un destino. La ruta se reenvía completa: `GET /api/applications?page=0` llega al servicio como `GET http://localhost:8081/api/applications?page=0`. La opción `changeOrigin: true` reescribe la cabecera `Host` de la petición con la del destino, algo que algunos servidores necesitan para responder correctamente.

Cuidado

El proxy es una función **del servidor de Vite**: existe con `npm run dev` y, por defecto, también con `npm run preview`, que reutiliza la misma configuración. Pero en producción no hay ningún servidor de Vite, así que hay que resolver lo mismo de otra forma: con un proxy inverso que sirva el frontend y la API bajo el mismo dominio, o configurando CORS en el backend. Lo veremos con detalle en la Parte VI.

4.8 Decisión: Vite frente a otras herramientas

↑ Índice

Decisión de diseño: Vite como herramienta de desarrollo y construcción

Contexto. Durante años, la forma estándar de crear un proyecto React fue Create React App, basado en webpack. Hoy está abandonado y la propia documentación de React desaconseja usarlo.

Decisión. Vite, con el plugin oficial de React.

Motivos. Arranque y recarga casi instantáneos; configuración mínima y legible; integración directa con Tailwind CSS 4 y con Vitest (las pruebas reutilizan la misma configuración); proxy de desarrollo incluido; y es la base recomendada para SPA en la documentación de React y de la mayoría de librerías.

Alternativas. webpack (muy flexible, pero lento y con configuración extensa); Rspack (compatible con webpack y rápido, útil para migrar proyectos webpack existentes); Parcel (cero configuración, menos extendido).

4.9 Errores típicos y depuración

↑ Índice

- **`Port 5173 is in use`.** Ya hay otro `npm run dev` en marcha. Vite elegirá otro puerto (5174…) y lo indicará en la terminal; fíjate en la URL que muestra.
- **`http proxy error: /api/applications … ECONNREFUSED`** en la terminal de Vite. El servicio de destino no está arrancado o escucha en otro puerto. Comprueba los puertos y las variables `VITE_*_API_TARGET`.
- **Una petición a la API devuelve el `index.html`.** La ruta no encaja con ningún prefijo del proxy (por ejemplo, `/api/teams` cuando solo hay reglas para `applications` y `demands`), así que Vite la trata como una ruta de la SPA. Añade la regla correspondiente.
- **Cambiaste `.env.local` y no se nota.** Las variables se leen al arrancar: reinicia `npm run dev`.
- **`import.meta.env.MI_VARIABLE` es `undefined`.** Le falta el prefijo `VITE_`.
- **Comportamiento extraño tras actualizar dependencias.** Arranca con `npx vite --force` para regenerar la caché de `node_modules/.vite`.

4.10 Ejercicios

↑ Índice

Ejercicio 4.1 · Observa HMR y Fast Refresh Ver solución →

Con la aplicación abierta en *Nueva aplicación*, escribe algo en el campo *Nombre*. Sin enviar el formulario, cambia en el código el texto del título de la página (`src/features/applications/pages/application-create-page.tsx`) y guarda. ¿Se conserva lo que habías escrito? Ahora añade a ese fichero una línea `export const PRUEBA = 1`, guarda, y repite el experimento. ¿Qué cambia? Relaciónalo con la regla `react-refresh/only-export-components`. Deshaz el cambio.

Ejercicio 4.2 · Actualiza `.env.example` Ver solución →

Reescribe `.env.example` para que documente las variables que la configuración usa realmente (`VITE_API_BASE_URL`, `VITE_DEMAND_API_TARGET` y `VITE_APPLICATION_API_TARGET`), con un comentario para cada una. Crea un `.env.local` que apunte el servicio de aplicaciones a un puerto incorrecto, arranca y comprueba en la terminal y en *Network* qué ocurre.

Ejercicio 4.3 · Inspecciona el resultado del build Ver solución →

Ejecuta `npm run build`. En la salida de la terminal, Vite lista cada fichero generado con su tamaño y su tamaño comprimido con gzip. ¿Cuál es el *chunk* más grande y qué crees que contiene? Abre `dist/index.html` en un editor y localiza cómo se referencia el JavaScript inicial. Busca en `dist/assets/index-*.js` el texto `/api` para comprobar cómo quedó incrustada la URL base.

4.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 4 Ver respuestas →

1. ¿Por qué Vite no empaqueta tu código en desarrollo y sí en producción?
2. ¿Qué es la preparación de dependencias y dónde se guarda?
3. ¿Qué diferencia hay entre HMR y React Fast Refresh?
4. ¿Por qué `npm run dev` funciona aunque haya errores de tipos? ¿Qué lo impide en producción?
5. ¿Por qué el alias `@` se declara tanto en `vite.config.ts` como en `tsconfig.app.json`?
6. ¿Qué variables de entorno llegan al navegador y por qué no deben contener secretos?
7. ¿Qué problema resuelve el proxy de desarrollo y por qué no sirve en producción?
8. ¿A qué servicio y con qué URL llega la petición `GET /api/demands/7` en desarrollo?

CAPÍTULO 5

Configuración de TypeScript, ESLint y calidad del código

↑ Volver al índice

EN ESTE CAPÍTULO

5.1 Tres ficheros `tsconfig`: referencias de proyecto
5.2 `tsconfig.app.json`, opción a opción
5.3 `tsconfig.node.json` y `tsc -b`
5.4 El modo estricto en la práctica
5.5 ESLint: lo que el compilador no ve
5.6 Integración con el editor
5.7 Errores típicos y depuración
5.8 Ejercicios
5.9 Autoevaluación

El compilador de TypeScript y ESLint son las dos redes de seguridad del proyecto. Este capítulo explica opción a opción cómo están configurados, por qué el modo estricto es innegociable y qué detecta cada regla de ESLint que no detectaría el compilador.

5.1 Tres ficheros `tsconfig`: referencias de proyecto

↑ Índice

En la raíz hay tres ficheros de configuración de TypeScript. Puede parecer excesivo, pero responde a un problema real: el proyecto contiene código que se ejecuta en **dos entornos distintos**.

- El código de `src/` se ejecuta en el **navegador**: tiene `document`, `window` y `fetch`, pero no `process` ni el sistema de ficheros.
- `vite.config.ts` se ejecuta en **Node**: tiene `process` y los módulos `node:*`, pero no el DOM.

Si todo compartiera una configuración, TypeScript permitiría usar `document` en la configuración de Vite o `process.env` en un componente, y ambos fallarían en ejecución. La solución son las **referencias de proyecto**:

tsconfig.json

```
{
  "files": [],
  "references": [{ "path": "./tsconfig.app.json" }, { "path": "./tsconfig.node.json" }]
}
```

El `tsconfig.json` raíz no compila nada (`"files": []`); solo enumera los dos subproyectos. `tsconfig.app.json` cubre `src/` con los tipos del navegador, y `tsconfig.node.json` cubre `vite.config.ts` con los tipos de Node.

Si vienes de Java...

Es como un proyecto Maven multimódulo con un `pom.xml` padre de tipo `pom` y dos módulos con dependencias distintas. Cada módulo ve solo lo que le corresponde, y el padre los construye juntos.

5.2 `tsconfig.app.json`, opción a opción

↑ Índice

`tsconfig.app.json`

```jsonc
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "types": ["vite/client"],
    "skipLibCheck": true,

    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true,

    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["src"]
}
```

5.2.1 Qué JavaScript y qué entorno

Opción	Qué significa
`target: "ES2022"`	Versión de JavaScript que se considera disponible. Afecta a qué sintaxis se acepta sin transformar.
`lib`	Qué APIs existen: las de ES2022 más las del navegador (`DOM`, `DOM.Iterable`). Por eso `document` tiene tipos aquí y no en `vite.config.ts`.
`types: ["vite/client"]`	Incluye solo los tipos globales de Vite (`import.meta.env`, importación de CSS e imágenes). Evita que se cuelen tipos globales de otros paquetes `@types`.
`useDefineForClassFields`	Semántica estándar de los campos de clase. Irrelevante en la práctica para este proyecto.
`skipLibCheck`	No comprueba los ficheros de tipos (`.d.ts`) de `node_modules`. Acelera mucho la compilación y evita errores en librerías que no son nuestras.

5.2.2 Cómo se resuelven los módulos

Opción	Qué significa
`module: "ESNext"`	El código usa módulos ES.
`moduleResolution: "bundler"`	Resuelve los `import` como lo hace un empaquetador como Vite: sin exigir extensiones y respetando el campo `exports` de los `package.json`.

Opción	Qué significa
`allowImportingTsExtensions`	Permite escribir `import './x.ts'`. Solo es válido porque no se genera código.
`noEmit: true`	**TypeScript no genera ningún fichero**: solo comprueba. La transformación la hace Vite (capítulo 4).
`verbatimModuleSyntax`	Cada `import` se conserva o se elimina tal cual está escrito. Obliga a usar `import type` cuando lo importado es solo un tipo, de modo que una herramienta que procesa ficheros sueltos, como Vite, sabe qué importaciones eliminar sin analizar el resto del proyecto.
`moduleDetection: "force"`	Trata cada fichero como un módulo, aunque no tenga `import` ni `export`.
`jsx: "react-jsx"`	Usa la transformación moderna de JSX: no hace falta escribir `import React from 'react'` en cada componente.
`paths`	El alias `@/*` → `src/*`, igual que en `vite.config.ts`.

5.2.3 Lo estricto

Opción	Qué detecta
`strict: true`	Activa a la vez un conjunto de comprobaciones; las más importantes se explican abajo.
`noUnusedLocals`, `noUnusedParameters`	Variables, importaciones y parámetros declarados pero no usados.
`noFallthroughCasesInSwitch`	Un `case` que continúa en el siguiente por olvidar el `break` o el `return`.
`noUncheckedSideEffectImports`	Una importación sin nombres (`import './index.css'`) que apunta a un fichero inexistente.

`strict` engloba, entre otras:

- **`strictNullChecks`**: `null` y `undefined` no son asignables a cualquier tipo. Es la que convierte los `NullPointerException` en errores de compilación.
- **`noImplicitAny`**: si TypeScript no puede inferir un tipo, exige que lo escribas en lugar de suponer `any`.
- **`strictFunctionTypes`**: comprueba con rigor la compatibilidad de los parámetros de las funciones que se pasan como valores.
- **`useUnknownInCatchVariables`**: lo capturado en un `catch` es `unknown`, no `any`.

5.3 `tsconfig.node.json` y `tsc -b`

↑ Índice

`tsconfig.node.json` es casi idéntico, pero con `lib: ["ES2023"]` (sin DOM), `types: ["node", "vitest/config"]` y un único fichero incluido, `vite.config.ts`. El tipo `vitest/config` es el que permite escribir la sección `test` dentro de la configuración de Vite.

El script `typecheck` ejecuta `tsc -b` (*build mode*): recorre las referencias, comprueba cada subproyecto y guarda información incremental en los ficheros `*.tsbuildinfo` que indica `tsBuildInfoFile` (dentro de `node_modules/.tmp`, para no ensuciar el proyecto). La segunda ejecución solo vuelve a comprobar lo que ha cambiado.

> **Nota**
>
> **TypeScript 6.0** es una versión de transición: prepara el terreno para TypeScript 7, una reescritura del compilador en Go, mucho más rápida. Algunas opciones antiguas quedan obsoletas y varios valores por defecto cambian para alinearse con la configuración moderna que ya usa este proyecto, así que el salto debería ser poco traumático.

5.4 El modo estricto en la práctica

↑ Índice

El modo estricto se nota sobre todo en el tratamiento de los valores que pueden faltar. Un ejemplo real: el hook `useApplication` recibe un `id` que puede ser `undefined` (si la URL contiene un identificador no válido).

`src/features/applications/api/applications-queries.ts`

```
export function useApplication(id: number | undefined) {
  return useQuery({
    queryKey: applicationKeys.detail(id ?? -1),
    queryFn: ({ signal }) => applicationsApi.getById(id!, signal),
    enabled: id !== undefined,
    // …
  })
}
```

Sin modo estricto, `getById(id, signal)` compilaría aunque `id` pudiera ser `undefined`, y el error sería una petición a `/api/applications/undefined`. Con el modo estricto, el compilador obliga a decidir qué pasa. Aquí la consulta se desactiva con `enabled: id !== undefined`, de modo que `queryFn` nunca se ejecuta sin identificador; como el compilador no puede relacionar ambas cosas, se usa `id!` para afirmarlo. Es un uso justificado del operador `!`, y el comentario mental que lo acompaña es: «`enabled` lo garantiza».

5.5 ESLint: lo que el compilador no ve

↑ Índice

El compilador comprueba que los tipos encajan. **ESLint** analiza el código en busca de **patrones problemáticos** que son perfectamente válidos para el compilador: variables que deberían ser `const`, *hooks* llamados dentro de un `if`, dependencias olvidadas, código inalcanzable…

La configuración usa el formato plano (*flat config*), un array de objetos que se aplican en orden:

`eslint.config.js`

```
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2022,
      globals: globals.browser,
    },
```

```
  },
  {
    // Los componentes de shadcn/ui exportan variantes junto a los componentes.
    files: ['src/components/ui/**/*.{ts,tsx}'],
    rules: { 'react-refresh/only-export-components': 'off' },
  },
])
```

Bloque	Qué aporta
`globalIgnores(['dist'])`	No analiza el resultado del *build*.
`js.configs.recommended`	Reglas básicas de JavaScript: variables no definidas, código inalcanzable, comparaciones sospechosas...
`tseslint.configs.recommended`	Reglas específicas de TypeScript: prohíbe `any` explícito, `@ts-ignore` sin justificación, espacios de nombres antiguos...
`reactHooks.configs.flat.recommended`	Las reglas de los *hooks* de React (abajo).
`reactRefresh.configs.vite`	Avisa cuando un fichero de componentes exporta otras cosas e impide el Fast Refresh (capítulo 4).
`globals.browser`	Declara como conocidas las variables globales del navegador (`window`, `document`...).
Último bloque	Excepción para `src/components/ui`: los componentes generados por shadcn/ui exportan también sus variantes (`buttonVariants`), y son código de terceros que no conviene reescribir.

5.5.1 Las reglas de los *hooks*

`eslint-plugin-react-hooks` es el plugin **más importante** de un proyecto React. Dos de sus reglas existen desde siempre:

- **`rules-of-hooks`** (error): los *hooks* (`useState`, `useQuery`...) solo pueden llamarse en el nivel superior de un componente o de otro *hook*, nunca dentro de condiciones, bucles o funciones anidadas. Veremos el porqué en el capítulo 8.
- **`exhaustive-deps`** (aviso): en `useEffect`, `useMemo` y `useCallback`, la lista de dependencias debe incluir todo valor del componente que se use dentro. Olvidar una dependencia produce errores sutiles con datos antiguos.

La versión 7 del plugin, la del proyecto, añade al conjunto recomendado reglas derivadas del **React Compiler** que detectan patrones que rompen las suposiciones de React: modificar valores que deberían ser inmutables (`immutability`), leer referencias durante el renderizado (`refs`), llamar a funciones impuras al renderizar (`purity`) o actualizar el estado de forma síncrona dentro de un efecto (`set-state-in-effect`). Todas tendrán sentido tras la Parte III.

> **Si vienes de Java...**
>
> ESLint ocupa el lugar de Checkstyle, PMD y SpotBugs juntos, y las reglas de los *hooks* son comparables a las inspecciones de IntelliJ que detectan, por ejemplo, un `@Transactional` sobre un método privado: el código compila, pero no hace lo que crees.

5.6 Integración con el editor

↑ Índice

Ni el compilador ni ESLint deberían ser algo que solo se ejecuta al final. Configura tu editor para que los muestre mientras escribes:

- **VS Code**: instala las extensiones *ESLint* y, si quieres ver mejor las clases de Tailwind, *Tailwind CSS IntelliSense*. Usa la versión de TypeScript del proyecto (paleta de comandos → *TypeScript: Select TypeScript Version* → *Use Workspace Version*).
- **IntelliJ IDEA / WebStorm**: el soporte de TypeScript viene incluido; activa ESLint en *Settings* → *Languages & Frameworks* → *JavaScript* → *Code Quality Tools* → *ESLint* con la configuración automática.

El `.gitignore` del proyecto excluye `.vscode/*` salvo `extensions.json`: es el fichero donde un equipo recomienda las extensiones que deben instalarse.

Decisión de diseño: Modo estricto completo y reglas recomendadas, sin formateador automático todavía

Decisión. TypeScript en modo estricto con las comprobaciones adicionales de variables y parámetros sin usar; ESLint con los conjuntos recomendados de JavaScript, TypeScript, *hooks* y React Refresh, sin reglas personalizadas.

Motivos. Los conjuntos recomendados recogen el consenso de la comunidad y se mantienen solos al actualizar. El modo estricto es lo que hace que TypeScript merezca la pena: sin él, buena parte de la seguridad desaparece.

Pendiente. El proyecto no incluye todavía un formateador automático como **Prettier**. Añadirlo (con `eslint-config-prettier` para evitar conflictos) y ejecutarlo al guardar eliminaría las discusiones de estilo en las revisiones. Es una mejora pequeña y recomendable.

5.7 Errores típicos y depuración

↑ Índice

- **El editor no muestra errores que `tsc` sí muestra, o al revés.** El editor está usando otra versión de TypeScript; selecciona la del proyecto.
- **`'X' is declared but its value is never read.`** `noUnusedLocals`. Borra la importación o la variable; si es un parámetro que no puedes quitar, prefija su nombre con `_`.
- **`… is a type and must be imported using a type-only import when 'verbatimModuleSyntax' is enabled.`** Cambia a `import type { … }` o marca ese nombre con `type` dentro de la importación: `import { useQuery, type QueryKey } from …`.
- **`React Hook useEffect has a missing dependency.`** No la silencies con un comentario `eslint-disable`: casi siempre señala un error real. Añade la dependencia o reestructura el código (capítulo 8).
- **`Fast refresh only works when a file only exports components.`** Mueve las constantes o funciones exportadas a un fichero `.ts` aparte.

5.8 Ejercicios

↑ Índice

Ejercicio 5.1 · Desactiva el modo estricto Ver solución →

En una rama, cambia `"strict": true` por `false` en `tsconfig.app.json` y ejecuta `npm run typecheck`. Después vuelve a activarlo y escribe en cualquier página una línea que acceda a `query.data.name` sin comprobar que `data` existe. Compara el resultado con y sin modo estricto. ¿Qué error se habría producido en ejecución?

Ejercicio 5.2 · Provoca errores de ESLint Ver solución →

Escribe en un componente un `useState` dentro de un `if` y un `useEffect` que use una variable del componente sin incluirla en las dependencias. Ejecuta `npm run lint` y lee los mensajes. Luego consulta en la documentación de React la página *Rules of Hooks* y explica con tus palabras por qué existe la primera regla.

Ejercicio 5.3 · Comprueba la separación de entornos Ver solución →

Escribe `document.title = 'x'` dentro de `vite.config.ts` y `process.env.HOME` dentro de `src/main.tsx`. Ejecuta `npm run typecheck`. ¿Qué subproyecto se queja de cada línea y por qué? Deshaz los cambios.

5.9 Autoevaluación

↑ Índice

Autoevaluación del capítulo 5 Ver respuestas →

1. ¿Por qué hay dos subproyectos de TypeScript en lugar de uno?
2. Si TypeScript tiene `noEmit: true`, ¿quién genera el JavaScript?
3. ¿Qué obliga a hacer `verbatimModuleSyntax` y por qué es útil para Vite?
4. ¿Qué comprobación de `strict` es la que más reduce los errores en ejecución?
5. ¿Qué tipo de errores detecta ESLint que no detecta el compilador? Pon dos ejemplos.
6. ¿Por qué hay una excepción de ESLint para `src/components/ui`?
7. ¿Qué hace `tsc -b` que no haga `tsc` a secas?

PARTE III

React 19 a fondo

6	Componentes, JSX y props	pág. 62
7	Estado, renderizado y reconciliación	pág. 72
8	Hooks: efectos, memorización y lógica reutilizable	pág. 81
9	Composición, contexto y límites de error	pág. 90

↑ Volver al índice

CAPÍTULO 6

Componentes, JSX y props

↑ Volver al índice

EN ESTE CAPÍTULO

6.1 Qué es React
6.2 Componentes: funciones que devuelven interfaz
6.3 JSX en detalle
6.4 Props: los parámetros de un componente
6.5 Renderizado condicional
6.6 Listas y claves
6.7 Eventos
6.8 Componentes como valores
6.9 Accesibilidad en JSX
6.10 Cómo se organizan los componentes en el proyecto
6.11 Errores típicos y depuración
6.12 Ejercicios
6.13 Autoevaluación

React se reduce a una idea: la interfaz es el resultado de una función. Este capítulo presenta las piezas con las que se construye todo lo demás —componentes, JSX y *props*— y los patrones de uso diario: renderizado condicional, listas con claves, eventos y composición. Todos los ejemplos son componentes reales del proyecto.

6.1 Qué es React

↑ Índice

React es una librería para construir interfaces de usuario. Su modelo mental cabe en una línea:

interfaz = f(estado)

Tú escribes funciones que, a partir de unos datos, **describen** cómo debe verse la pantalla. No le dices a React «añade una fila a la tabla» o «cambia el texto de este botón»: le das la descripción completa para los datos actuales y React se encarga de modificar el DOM para que coincida (capítulo 7).

Este enfoque se llama **declarativo**, en contraste con el **imperativo** del ejemplo del capítulo 1, en el que creábamos un botón con `document.createElement` y lo añadíamos a mano. En una interfaz con decenas de estados posibles (cargando, con error, vacía, con datos, guardando…), describir el resultado es mucho más fácil que programar cada transición.

React es una **librería**, no un *framework* completo: se ocupa de los componentes y de su renderizado. La navegación, la obtención de datos, los formularios o los estilos los aportan otras librerías que eliges tú. Es lo que explica el número de dependencias del capítulo 3.

Si vienes de Java…

Si has trabajado con JSF, la idea de un árbol de componentes que se renderiza te resultará familiar. La diferencia es que en JSF el árbol vive en el servidor y cada interacción viaja a él; en React el árbol vive en el navegador y se recalcula allí, sin ida y vuelta. Más cerca aún: una plantilla de Thymeleaf es también una función de un modelo a HTML; React hace lo mismo, pero se re-evalúa cada vez que los datos cambian.

6.2 Componentes: funciones que devuelven interfaz

↑ Índice

Un **componente** es una función de JavaScript que devuelve una descripción de la interfaz. Su nombre empieza siempre por **mayúscula**. Este es uno de los más sencillos del proyecto:

`src/components/common/page-header.tsx`

```tsx
interface PageHeaderProps {
  title: string
  description?: ReactNode
  /** Migas de pan / contexto sobre el título. */
  eyebrow?: ReactNode
  actions?: ReactNode
}

export function PageHeader({ title, description, eyebrow, actions }: PageHeaderProps) {
  return (
    <div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
      <div className="min-w-0 space-y-1">
        {eyebrow && (
          <p className="text-xs font-medium tracking-wide text-brand-strong uppercase">{eyebrow}</p>
        )}
        <h1 className="text-2xl font-semibold tracking-tight">{title}</h1>
        {description && <p className="text-sm text-muted-foreground">{description}</p>}
      </div>
      {actions && <div className="flex shrink-0 flex-wrap items-center gap-2">{actions}</div>}
    </div>
  )
}
```

Y se usa como si fuera una etiqueta HTML más:

`src/features/applications/pages/applications-list-page.tsx`

```tsx
<PageHeader
  eyebrow="Aplicaciones"
  title="Consultar aplicaciones"
  description="Catálogo de aplicaciones gestionadas, con su criticidad y estado."
  actions={<Button asChild variant="brand">…</Button>}
/>
```

Tres observaciones:

- El componente **no tiene estado ni efectos**: dados los mismos datos, devuelve siempre lo mismo. Es una función pura, y la mayoría de los componentes del proyecto lo son.
- Encapsula una decisión de diseño (cómo es la cabecera de cualquier pantalla) en un único lugar. Todas las páginas la usan, y cambiarla aquí las cambia todas.
- La mayúscula no es estética: en JSX, `<pageHeader>` en minúscula se interpretaría como una etiqueta HTML desconocida.

> **Si vienes de Java…**
>
> Un componente es lo más parecido a un método `render(Model model)` que devuelve un fragmento de vista, con los parámetros agrupados en un objeto. No hay clases que extender ni interfaces que implementar. En versiones antiguas de React existían componentes de clase (`class X extends React.Component`); hoy se consideran heredados y el proyecto no tiene ninguno.

6.3 JSX en detalle

↑ Índice

La sintaxis parecida a HTML dentro del código es **JSX**. No es HTML ni una cadena de texto: es azúcar sintáctico que Vite transforma en llamadas a funciones.

```
// Lo que escribes
<h1 className="text-2xl">{title}</h1>

// Lo que genera la transformación (simplificado)
jsx('h1', { className: 'text-2xl', children: title })
```

La llamada `jsx(...)` devuelve un **elemento de React**: un objeto ligero del tipo `{ type: 'h1', props: { className: 'text-2xl', children: 'Consultar aplicaciones' } }`. Un componente, por tanto, no toca el DOM: devuelve un árbol de objetos que describe lo que se quiere ver. Es React quien, después, compara y aplica.

6.3.1 Las reglas de JSX

- **Expresiones entre llaves.** Cualquier expresión de JavaScript puede ir entre `{ }`: variables, llamadas, operaciones, plantillas de texto. Sentencias como `if` o `for`, no (sí sus equivalentes en forma de expresión: `? :`, `&&`, `map`).
- **Atributos con nombre de JavaScript.** `class` es una palabra reservada, así que se escribe `className`; `for` se escribe `htmlFor`. Los eventos van en *camelCase*: `onClick`, `onChange`, `onSubmit`.
- **El atributo `style` recibe un objeto**, no una cadena: `style={{ height: 4 + n * 2 }}` (en `LevelBadge`). Las dobles llaves son una expresión `{ }` que contiene un objeto literal `{ height: … }`. Los números se interpretan como píxeles.
- **Todas las etiquetas se cierran**, incluidas las vacías: `<Separator />`, `<input />`.
- **Un único elemento raíz.** Un componente devuelve un solo elemento. Si necesitas devolver varios hermanos sin envolverlos en un `div`, usa un **fragmento**: `<> … </>`. Las páginas del proyecto lo hacen constantemente, porque devuelven la cabecera y el contenido como hermanos.
- **Texto y caracteres especiales.** El texto se escribe tal cual. Para espacios entre expresiones se usa `{' '}`, como en `PaginationBar`. Las entidades HTML como `&` funcionan en el texto.

> **Cuidado**
>
> **Seguridad por defecto.** Todo valor que insertas con `{ }` se **escapa**: si una aplicación se llama `<script>alert(1)</script>`, se mostrará ese texto literal, no se ejecutará. React te protege de la inyección de código (XSS) sin que tengas que hacer nada. La única forma de saltarse esta protección es la propiedad `dangerouslySetInnerHTML`, cuyo nombre ya avisa; el proyecto no la usa.

6.4 Props: los parámetros de un componente

↑ Índice

Las **props** (propiedades) son los datos que un componente recibe de su padre. React las agrupa en un único objeto, que se desestructura en la firma.

6.4.1 Tipar las props

El contrato de un componente se define con una `interface` que se suele llamar `NombreDelComponenteProps`:

src/features/applications/components/applications-table.tsx

```tsx
interface ApplicationsTableProps {
  applications: Application[]
  sortField: ApplicationSortField
  sortDirection: SortDirection
  onSortChange: (field: ApplicationSortField) => void
  /** "manage": acciones de detalle y modificación; "select": botón de selección. */
  mode?: 'manage' | 'select'
  onSelect?: (application: Application) => void
  /** Atenúa la tabla mientras se carga otra página. */
  isFetching?: boolean
}
```

Este tipo es documentación ejecutable: el editor autocompleta las props al usar el componente, el compilador rechaza las que faltan o sobran, y los comentarios `/** … */` aparecen al pasar el ratón.

6.4.2 Valores por defecto

Se declaran en la desestructuración:

src/features/applications/components/applications-table.tsx

```tsx
export function ApplicationsTable({
  applications,
  sortField,
  sortDirection,
  onSortChange,
  mode = 'manage',
  onSelect,
  isFetching,
}: ApplicationsTableProps) {
```

6.4.3 Las props son de solo lectura

Un componente **nunca modifica sus props**. Si la tabla necesita cambiar el orden, no toca `sortField`: llama a `onSortChange(field)` para **avisar al padre**, que es el dueño de ese dato, y el padre decide. Este flujo es el corazón de React:

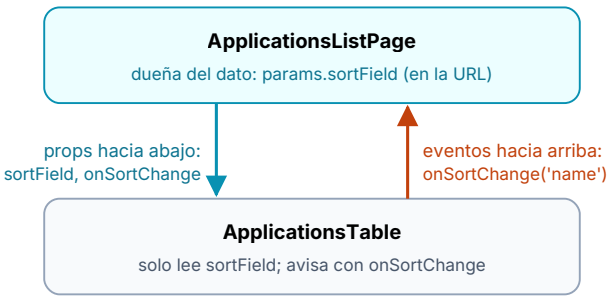


Figura 6.1. Flujo de datos unidireccional: los datos bajan por props y los cambios suben como llamadas a funciones.

Este **flujo de datos unidireccional** hace que siempre sepas quién es el dueño de un dato: el componente que lo guarda. Cuando algo se ve mal, sigues los datos hacia arriba hasta su origen.

6.4.4 `children` y las props como «huecos»

Lo que pones **entre** la etiqueta de apertura y la de cierre de un componente llega en una prop especial llamada `children`:

src/features/applications/components/applications-table.tsx

```
<RowAction to={`/aplicaciones/${app.id}`} label={`Ver detalle de ${app.name}`}>
  <Eye />
</RowAction>
```

`RowAction` recibe `<Eye />` (el icono) como `children` y decide dónde colocarlo. Y cualquier prop puede recibir JSX, no solo `children`: `PageHeader` tiene `actions` y `description` de tipo `ReactNode`, y cada página decide qué poner en esos huecos. El tipo `ReactNode` admite cualquier cosa que React sepa pintar: elementos, textos, números, arrays, `null`, `undefined` o `false` (estos tres últimos no pintan nada).

> **Si vienes de Java…**
>
> Las props de tipo `ReactNode` son como el patrón *template method* o los fragmentos parametrizados de Thymeleaf (`th:fragment` con parámetros): el componente define la estructura y deja huecos que rellena quien lo usa.

6.4.5 Propagar props

Cuando se pasan las mismas props a varios hijos, se pueden agrupar y propagar con `{...objeto}`:

src/features/applications/components/applications-table.tsx

```
const sortProps = { sortField, sortDirection, onSortChange }
// …
<SortableTableHead field="id" label="ID" className="w-20 pl-4" {...sortProps} />
<SortableTableHead field="name" label="Nombre" {...sortProps} />
```

6.5 Renderizado condicional

↑ Índice

Como en JSX no hay `if`, se usan sus equivalentes en forma de expresión. El proyecto usa tres patrones.

1. `&&` para «mostrar o no mostrar»:

```
{eyebrow && <p className="…">{eyebrow}</p>}
```

Si `eyebrow` es falso, la expresión vale `undefined` (o el valor falso) y React no pinta nada; si es verdadero, vale el elemento.

2. El operador ternario para «esto o aquello»:

src/features/applications/components/applications-table.tsx

```
{mode === 'select' ? (
  <Button size="sm" variant="outline" onClick={() => onSelect?.(app)}>Seleccionar</Button>
) : (
  <div className="flex justify-end gap-1">…</div>
)}
```

3. Retorno anticipado, cuando todo el componente depende de una condición:

src/features/applications/pages/application-edit-page.tsx

```
if (id === undefined) return (
  <ResourceNotFound message={`No existe ninguna aplicación con el identificador «${rawId}»`} … />
)
```

Para más de dos casos, el listado de aplicaciones encadena ternarios que cubren **todos los estados posibles** de una consulta. Es un patrón que repetiremos en todo el proyecto:

src/features/applications/pages/applications-list-page.tsx (estructura)

```
{query.isPending ? (
  <TableSkeleton … />                       // 1. primera carga
) : query.isError && !data ? (
  <ErrorState error={query.error} … />      // 2. error y nada que mostrar
) : data && data.content.length === 0 ? (
  <EmptyState … />                          // 3. sin resultados
) : data ? (
  <> <ApplicationsTable … /> <PaginationBar … /> </>   // 4. datos
) : null}
```

Decisión de diseño: Diseñar todos los estados de cada pantalla

Cada pantalla que carga datos contempla explícitamente **cargando, error, vacío y con datos**, y cada uno tiene su componente común (`TableSkeleton` , `ErrorState` , `EmptyState`). Una interfaz que solo contempla el caso feliz muestra pantallas en blanco o tablas vacías sin explicación en cuanto algo falla. El coste es pequeño porque los componentes de estado se reutilizan, y el usuario siempre sabe qué está pasando.

Cuidado

Cuidado con `&&` y los números. `{items.length && <Lista />}` pinta un **0** en pantalla cuando la lista está vacía, porque `0` es falso, pero React sí pinta los números. Usa una comparación explícita: `{items.length > 0 && <Lista />}` .

6.6 Listas y claves

↑ Índice

Para pintar una colección se transforma el array de datos en un array de elementos con `map` :

src/components/layout/sidebar-nav.tsx

```
<ul className="space-y-1">
  {NAV_MODULES.map((module) => (
    <li key={module.id}>
      {module.comingSoon ? <ComingSoonItem module={module} /> : <ModuleGroup module={module} … />}
    </li>
  ))}
</ul>
```

Cada elemento de una lista necesita una prop `key` única entre sus hermanos. React la usa para saber qué elemento es cuál entre un renderizado y el siguiente: si la lista se reordena o se inserta un elemento al principio, React puede mover los nodos del DOM existentes en lugar de destruirlos y recrearlos, y conservar el estado de cada uno.

La clave debe ser un **identificador estable del dato**: `module.id`, `app.id`, `item.to`. Evita usar el índice del array (`key={index}`) en listas que pueden reordenarse, filtrarse o recibir inserciones: el índice describe la posición, no la identidad, y React acabaría asociando el estado de una fila a otra.

> **En el proyecto**
>
> En la tabla, cada fila usa `key={app.id}`: el identificador de la base de datos es la clave natural. En las opciones de los filtros, `key={option.value}`. En el menú, `key={item.to}`, porque la ruta identifica de forma única cada enlace. Fíjate también en el uso de la prop `key` fuera de las listas: `<ApplicationEditForm key={query.data.id} … />` en la página de edición. Lo explicaremos en el capítulo 7.

6.7 Eventos

↑ Índice

Los elementos HTML emiten eventos (`onClick`, `onChange`, `onSubmit`…) y se les pasa una **función**, no una llamada:

```
<Button onClick={() => setMobileOpen(true)}>…</Button>    // correcto: se pasa una función
<Button onClick={setMobileOpen(true)}>…</Button>          // incorrecto: se ejecuta al renderizar
```

La función recibe un objeto de evento. En el buscador por identificador de *Modificar aplicación* se usan los dos casos típicos:

`src/features/applications/pages/application-edit-selector-page.tsx`

```
const onSubmit = (event: FormEvent) => {
  event.preventDefault()          // evita que el navegador envíe el formulario y recargue
  const id = parseEntityId(value.trim())
  setInvalid(id === undefined)
  setSearchedId(id)
}
// …
<form onSubmit={onSubmit} noValidate>
  <Input value={value} onChange={(e) => setValue(e.target.value)} … />
</form>
```

- `event.preventDefault()` cancela el comportamiento por defecto del navegador, que al enviar un formulario haría una petición y recargaría la página: lo último que quieres en una SPA.
- `e.target.value` es el texto actual del campo.

Los eventos que recibes son **eventos sintéticos** de React: envoltorios con la misma interfaz en todos los navegadores. Para el uso diario, se comportan como los nativos.

6.8 Componentes como valores

↑ Índice

Como los componentes son funciones, pueden guardarse en variables y en datos. El menú lo aprovecha para guardar el icono de cada módulo en su configuración:

`src/components/layout/navigation.ts`

```
{ id: 'applications', label: 'Aplicaciones', icon: AppWindow, basePath: '/aplicaciones', items: [
… ] }
```

src/components/layout/sidebar-nav.tsx

```
const Icon = module.icon
// …
<Icon className={cn('size-4 shrink-0', isInModule ? 'text-cyan-300' : 'text-sidebar-muted')} />
```

La variable se llama `Icon`, con mayúscula, para que JSX la trate como componente. Lo mismo ocurre en `ErrorState`, que elige `WifiOff` o `AlertTriangle` según el tipo de error.

En el proyecto

Esto convierte el menú en **configuración declarativa**: `navigation.ts` es una lista de datos y `sidebar-nav.tsx` sabe pintar cualquier lista con esa forma. Añadir el módulo de Incidencias será añadir datos, no escribir más componentes (Parte IV).

6.9 Accesibilidad en JSX

↑ Índice

Una interfaz profesional debe poder usarse con teclado y con lectores de pantalla. JSX admite todos los atributos de accesibilidad de HTML, y el proyecto los usa de forma sistemática:

Atributo	Uso en el proyecto
`aria-label`	Nombre accesible de botones que solo tienen icono: `aria-label="Página siguiente"` en `PaginationBar`.
`role="status"`, `aria-live="polite"`	El indicador de carga anuncia su texto al lector de pantalla sin interrumpir.
`role="alert"`	Los mensajes de error se anuncian inmediatamente.
`aria-hidden="true"`	Oculta elementos decorativos (los puntos de color de los estados, las barras de nivel).
`aria-sort`	Las cabeceras ordenables indican el orden actual.
`aria-invalid`	Marca los campos con error.
`className="sr-only"`	Texto visible solo para lectores de pantalla (el título del menú móvil).

La mayor parte del trabajo pesado de accesibilidad —foco, navegación con teclado en desplegables y diálogos— lo resuelven los componentes de Radix sobre los que se construye shadcn/ui (Parte IV).

6.10 Cómo se organizan los componentes en el proyecto

↑ Índice

Decisión de diseño: Convenciones para escribir componentes

- **Solo componentes de función**, con `function` y exportación con nombre: `export function PageHeader(...)`. Aparecen con nombre propio en las trazas de error y en React DevTools.
- **Un componente público por fichero**, con nombre de fichero en *kebab-case* (`page-header.tsx`). Los subcomponentes que solo usa ese fichero se declaran en él sin exportar: `sidebar-nav.tsx` contiene `ModuleGroup` y `ComingSoonItem`; `application-edit-selector-page.tsx` contiene `FindById` y `PickFromCatalog`.
- **Las props se tipan con una `interface`** junto al componente. Para componentes muy pequeños y privados se acepta el tipo en línea: `function CreatedResult({ application, onCreateAnother }: { application: Application; onCreateAnother: () => void })`.
- **Componentes pequeños y con una responsabilidad**: una página orquesta (lee parámetros, pide datos, decide qué estado mostrar) y delega la presentación en componentes como la tabla, los filtros o la paginación, que no saben de dónde vienen los datos.

Si vienes de Java...

La última regla es la separación de responsabilidades de siempre. Una página es como un controlador que reúne datos y elige vista; los componentes de presentación son las vistas, que reciben todo lo que necesitan y no acceden a servicios. Esto los hace reutilizables: `ApplicationsTable` sirve en el listado y en el selector de *Modificar aplicación* con solo cambiar `mode`.

6.11 Errores típicos y depuración

↑ Índice

- **`Each child in a list should have a unique "key" prop`.** Falta `key` en el elemento que devuelve el `map`. Ojo: la clave va en el elemento **más externo** que devuelve el `map`, no en uno interior.
- **`Objects are not valid as a React child`.** Intentas pintar un objeto entero (`{application}`) en lugar de una propiedad (`{application.name}`).
- **El componente no aparece y no hay error.** Revisa que el nombre empiece por mayúscula y que lo estés devolviendo: una función flecha con llaves necesita `return`.
- **Un manejador se ejecuta al cargar la página.** Escribiste `onClick={hacerAlgo()}` en lugar de `onClick={() => hacerAlgo()}` o `onClick={hacerAlgo}`.
- **Aparece un `0` suelto en la pantalla.** Una condición con `&&` sobre un número (sección 6.5).

Con **React DevTools**, pestaña *Components*, puedes seleccionar cualquier componente del árbol y ver sus props actuales, su estado y qué componente lo ha renderizado (*rendered by*). El icono del ojo te lleva al elemento del DOM correspondiente, y el selector (la flecha) te permite hacer clic en la página para encontrar el componente que la pinta.

6.12 Ejercicios

↑ Índice

Ejercicio 6.1 · Explora el árbol de componentes Ver solución →

Con React DevTools, abre el listado de aplicaciones y localiza en la pestaña *Components* la cadena `App → … → AppLayout → … → ApplicationsListPage → ApplicationsTable → SortableTableHead`. Selecciona `ApplicationsTable` y observa sus props. Cambia el orden de una columna en la página y comprueba qué props cambian.

Ejercicio 6.2 · Crea un componente reutilizable Ver solución →

Crea en `src/components/common/` un componente `CountBadge` que reciba `count: number` y `label: string` y pinte, por ejemplo, «12 aplicaciones». Si `count` es 0, debe mostrar «Sin aplicaciones»; si es 1, el singular. Añade una prop opcional `tone` que acepte solo `'neutral'` o `'brand'`, con `'neutral'` por defecto. Úsalo en la cabecera del listado de aplicaciones, en la prop `description` de `PageHeader`, con `data.totalElements`.

Ejercicio 6.3 · Lee un componente con criterio Ver solución →

Lee `src/components/common/pagination-bar.tsx` completo y responde: ¿qué props son datos y cuáles son eventos?, ¿por qué el componente no guarda la página actual en ningún sitio?, ¿qué ocurre con los botones cuando `disabled` es verdadero? y ¿por qué la página se muestra como `page + 1`?

6.13 Autoevaluación

↑ Índice

Autoevaluación del capítulo 6 Ver respuestas →

1. ¿Qué significa que React sea declarativo? Pon un ejemplo del proyecto.
2. ¿En qué se convierte el JSX al compilar y qué devuelve un componente?
3. ¿Por qué los nombres de los componentes empiezan por mayúscula?
4. ¿Qué hace un componente cuando necesita cambiar un dato que recibe por props?
5. ¿Qué es `children` y qué tipo tienen las props que reciben fragmentos de interfaz?
6. ¿Qué cuatro estados contempla el listado de aplicaciones y por qué?
7. ¿Para qué sirve `key` en una lista y por qué no conviene usar el índice?
8. ¿Qué diferencia hay entre `onClick={guardar}` y `onClick={guardar()}`?
9. ¿Por qué `{count && <Lista />}` puede pintar un 0?

CAPÍTULO 7

Estado, renderizado y reconciliación

↑ Volver al índice

EN ESTE CAPÍTULO

7.1 Props frente a estado
7.2 `useState`: la memoria de un componente
7.3 Qué provoca un renderizado
7.4 Del renderizado al DOM: la reconciliación
7.5 Inmutabilidad: por qué no se modifica el estado
7.6 Dónde vive cada estado
7.7 Identidad del estado y la prop `key`
7.8 Componentes controlados
7.9 Rendimiento: cuándo preocuparse
7.10 Errores típicos y depuración
7.11 Ejercicios
7.12 Autoevaluación

Los componentes del capítulo anterior solo transformaban props en interfaz. Una aplicación real tiene datos que cambian: un menú que se abre, un campo que se escribe, una página de resultados. Este capítulo explica cómo guarda React ese estado, qué ocurre exactamente cuando cambia y cómo decide qué tocar en el DOM. Es el capítulo más importante para dejar de «programar a ver si funciona».

7.1 Props frente a estado

↑ Índice

Un componente trabaja con dos tipos de datos:

	Props	Estado
¿Quién lo define?	El componente padre	El propio componente
¿Puede cambiarlo el componente?	No	Sí, con su función de actualización
¿Sobrevive entre renderizados?	Llega de nuevo en cada uno	Sí: React lo guarda
Ejemplo	`title` de `PageHeader`	Si el menú móvil está abierto

El **estado** es la memoria de un componente: datos que deben recordarse entre un renderizado y el siguiente y cuyo cambio debe reflejarse en pantalla.

7.2 `useState`: la memoria de un componente

↑ Índice

La cabecera de la aplicación necesita recordar si el menú lateral del móvil está abierto:

src/components/layout/app-header.tsx

```tsx
export function AppHeader() {
  const [mobileOpen, setMobileOpen] = useState(false)

  return (
    <header className="…">
      <Button variant="ghost" size="icon" className="lg:hidden"
              onClick={() => setMobileOpen(true)} aria-label="Abrir menú">
```

```
          <Menu />
        </Button>
        <Sheet open={mobileOpen} onOpenChange={setMobileOpen}>
          <SheetContent side="left" className="…">
            <SidebarNav onNavigate={() => setMobileOpen(false)} />
          </SheetContent>
        </Sheet>
        {/* … */}
      </header>
    )
}
```

`useState(false)` devuelve un par que se desestructura:

- **`mobileOpen`**: el valor actual. La primera vez es el valor inicial, `false`.
- **`setMobileOpen`**: la función para cambiarlo. Al llamarla, React guarda el nuevo valor y **vuelve a ejecutar el componente** para obtener la nueva interfaz.

El flujo completo al pulsar el botón de menú es: `onClick` → `setMobileOpen(true)` → React programa un nuevo renderizado de `AppHeader` → la función se ejecuta otra vez y ahora `mobileOpen` vale `true` → el `Sheet` recibe `open={true}` → React aplica al DOM los cambios necesarios para mostrar el panel. Al elegir una opción del menú, `SidebarNav` llama a `onNavigate`, que cierra el panel.

El valor inicial puede ser de cualquier tipo, y TypeScript lo infiere. Cuando no basta con inferirlo, se indica entre `< >`:

src/features/applications/pages/application-edit-selector-page.tsx

```
const [value, setValue] = useState('')                          // string
const [searchedId, setSearchedId] = useState<number | undefined>()      // number | undefined
const [invalid, setInvalid] = useState(false)                   // boolean
```

> **Si vienes de Java…**
>
> No hay nada parecido a un campo de instancia, porque un componente es una función y sus variables locales desaparecen al terminar. `useState` le pide a React que **guarde el valor fuera de la función**, asociado a esa instancia concreta del componente en el árbol, y que se lo devuelva en cada ejecución. Es como si React mantuviera un `Map<posiciónEnElÁrbol, Object[]>` con el estado de cada componente.

7.3 Qué provoca un renderizado

↑ Índice

Renderizar significa, en React, **llamar a la función del componente** para obtener su descripción de la interfaz. No implica necesariamente tocar el DOM. Un componente se vuelve a renderizar cuando:

1. **Cambia su estado** (se llama a su función `set…` con un valor distinto).
2. **Se renderiza su padre**: por defecto, renderizar un componente renderiza también todos sus descendientes.
3. **Cambia un contexto que consume** (capítulo 9).

Que cambien las props no es una causa independiente: las props solo cambian porque el padre se ha renderizado.

7.3.1 El estado es una instantánea

Dentro de un renderizado, el estado **no cambia**, aunque llames a `set…`. La función de actualización no modifica la variable: programa un renderizado **nuevo** en el que la variable tendrá el valor nuevo.

```
const [count, setCount] = useState(0)

function handleClick() {
  setCount(count + 1)
  console.log(count)        // sigue imprimiendo 0: es el valor de ESTE renderizado
  setCount(count + 1)       // vuelve a poner 0 + 1: el resultado final es 1, no 2
}
```

Cada renderizado es una fotografía con sus propios valores de props y de estado, y las funciones creadas en ese renderizado (como `handleClick`) «ven» los valores de esa fotografía.

7.3.2 Actualizaciones funcionales

Cuando el nuevo valor depende del anterior, pasa a `set…` **una función** que recibe el valor previo y devuelve el nuevo. React las encola y las aplica en orden, cada una sobre el resultado de la anterior:

`src/features/applications/pages/application-edit-selector-page.tsx`

```
const [params, setParams] = useState<ApplicationListParams>({ ...DEFAULT_LIST_PARAMS, size: 5 })

onPageChange={(page) => setParams((prev) => ({ ...prev, page }))}
onSizeChange={(size) => setParams((prev) => ({ ...prev, size, page: 0 }))}
```

El selector del catálogo en *Modificar aplicación* usa esta forma en todas sus actualizaciones: así, aunque se produjeran dos cambios seguidos antes de renderizar, ninguno se perdería.

7.3.3 Agrupación de actualizaciones

React **agrupa** (*batching*) todas las actualizaciones de estado que ocurren en un mismo evento y renderiza una sola vez al final. En `FindById`, el envío del formulario llama a `setInvalid` y a `setSearchedId`, pero el componente se renderiza una única vez, con ambos valores nuevos.

7.4 Del renderizado al DOM: la reconciliación

↑ Índice

Si cada cambio de estado vuelve a ejecutar el componente y todos sus hijos, ¿no es carísimo? No, porque **renderizar no es pintar**. El trabajo se divide en dos fases:

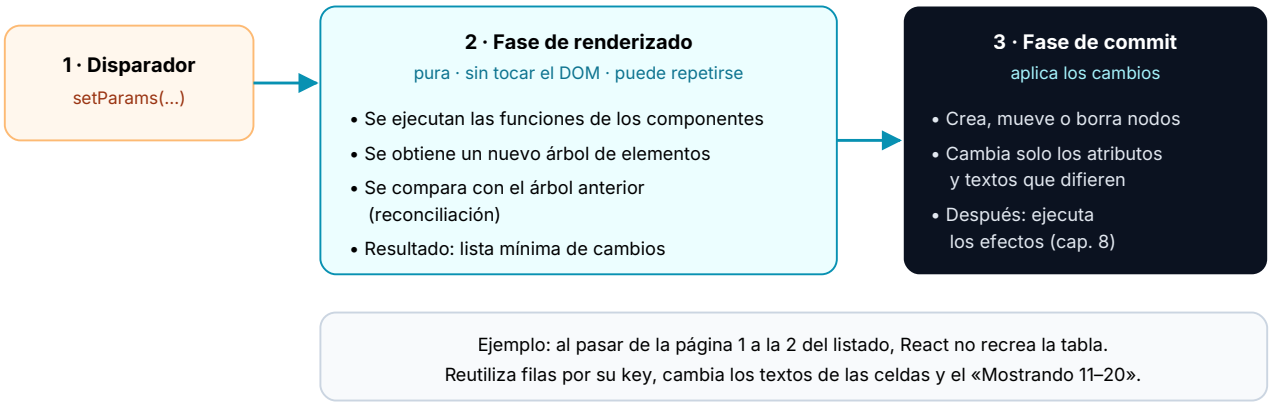


Figura 7.1. Las fases de una actualización. Ejecutar componentes es barato; tocar el DOM es lo caro, y React lo minimiza.

1. **Renderizado.** React llama a los componentes afectados y obtiene un nuevo árbol de elementos (los objetos ligeros que devuelve JSX). Lo **compara** con el árbol del renderizado anterior. A esta comparación se le llama **reconciliación**, y a la representación en memoria, a veces, *DOM virtual*.
2. **Commit.** React aplica al DOM real **solo las diferencias**: si en la tabla solo han cambiado los textos de algunas celdas, solo se modifican esos textos.

7.4.1 Cómo compara React dos árboles

Comparar dos árboles arbitrarios es un problema costoso, así que React usa dos reglas sencillas:

- **Mismo tipo en la misma posición → se actualiza.** Si antes había un `<ApplicationsTable>` y ahora también, React conserva la instancia (y su estado) y le pasa las props nuevas.
- **Tipo distinto → se destruye y se crea de nuevo.** Si en una posición había un `<TableSkeleton>` y ahora hay una `<ApplicationsTable>`, el esqueleto se elimina con todo su subárbol y su estado, y la tabla se crea desde cero.

Y en las listas, la `key` indica qué elemento corresponde a cuál (capítulo 6).

> **Si vienes de Java…**
>
> La analogía más cercana es Hibernate y su comprobación de cambios (*dirty checking*): tú modificas el modelo de objetos libremente y, al hacer *flush*, Hibernate compara con la instantánea y emite **solo** las sentencias `UPDATE` necesarias. React hace lo mismo con el DOM: tú describes el estado deseado, y en el *commit* emite solo las operaciones necesarias.

7.4.2 Por qué los componentes deben ser puros

Como React puede llamar a un componente muchas veces —y, en modo estricto, lo llama **dos veces** a propósito en desarrollo (capítulo 8)—, el cuerpo de un componente debe ser **puro**: a partir de sus props y su estado calcula y devuelve JSX, sin efectos secundarios. Nada de peticiones HTTP, ni de modificar variables externas, ni de tocar el DOM directamente durante el renderizado. Esas cosas van en manejadores de eventos o, si deben sincronizarse con el renderizado, en efectos (capítulo 8).

7.5 Inmutabilidad: por qué no se modifica el estado

↑ Índice

Para decidir si un cambio de estado requiere renderizar, React compara el valor anterior y el nuevo con `Object.is`, que para objetos es **igualdad de referencia**. Si modificas un objeto y lo vuelves a pasar, la referencia es la misma y React concluye que nada ha cambiado:

```
// Incorrecto: muta el objeto existente. Misma referencia → React no renderiza
params.page = 2
setParams(params)

// Correcto: crea un objeto nuevo
setParams({ ...params, page: 2 })
setParams((prev) => ({ ...prev, page: 2 }))
```

Lo mismo con arrays: en lugar de `push`, `splice` o `sort` sobre el array del estado, se crea uno nuevo con `[...lista, nuevo]`, `lista.filter(…)`, `lista.map(…)` o `[...lista].sort(…)`.

Esta regla no es un capricho de React: tratar los datos como inmutables hace que la comparación sea instantánea (basta con comparar referencias), permite conservar el valor anterior para comparaciones y evita una clase entera de errores en los que un cambio en un sitio aparece por sorpresa en otro.

> **Si vienes de Java...**
>
> Piensa en el estado como si fueran `record`s de Java: nunca se modifican, se sustituyen por una copia con cambios. Si vienes de JPA, donde lo natural es modificar la entidad gestionada y dejar que el ORM detecte el cambio, este es el ajuste mental más importante de la Parte III.

7.6 Dónde vive cada estado

↑ Índice

No todo el estado es igual, y una de las decisiones de arquitectura más importantes de una aplicación React es **dónde guardar cada dato**. El proyecto distingue cinco lugares:

Tipo de estado	Dónde vive en el proyecto	Ejemplos
Estado local de interfaz	`useState` en el componente	Menú móvil abierto, grupo del menú desplegado, texto del buscador por ID
Estado de navegación	**La URL** (React Router)	Filtros, página, tamaño y orden del listado; el `id` de la aplicación que se edita
Estado del servidor	**Caché de TanStack Query**	Páginas de aplicaciones, detalle de una demanda
Estado de formulario	**React Hook Form**	Valores, errores y si el formulario tiene cambios sin guardar
Estado global de interfaz	Contexto o librerías específicas	Notificaciones (`sonner`), tooltips

Decisión de diseño: El estado del servidor no se copia en `useState`

Un error muy común es pedir los datos con `fetch` en un efecto y guardarlos en un `useState`. Eso obliga a gestionar a mano la carga, los errores, la cancelación, la caché y la actualización tras modificar. En el proyecto, los datos del servidor **nunca** se copian a estado local: se leen siempre de TanStack Query, que es su única fuente de verdad (Parte VI). La consecuencia práctica es que la mayoría de las páginas apenas tienen `useState`.

7.6.1 Estado en la URL frente a estado local: un ejemplo real

El proyecto tiene **dos listados de aplicaciones** con filtros y paginación, y guardan su estado en sitios distintos a propósito:

- `ApplicationsListPage` (`/aplicaciones`) guarda filtros, página y orden **en la URL**. Así el usuario puede recargar, compartir el enlace o usar atrás y adelante. Cuando cambia un filtro, la página no llama a `useState`: llama a `setSearchParams`, la URL cambia, el router vuelve a renderizar la página, y los parámetros se recalculan a partir de la URL.
- `PickFromCatalog`, dentro de *Modificar aplicación*, guarda los mismos parámetros en `useState`. Es un selector auxiliar dentro de otra pantalla: que sus filtros aparezcan en la URL o en el historial no aportaría nada.

Es un buen ejemplo de que la pregunta no es «¿cómo guardo esto?», sino «¿quién necesita este dato y durante cuánto tiempo?».

7.6.2 Estado mínimo y valores derivados

Guarda en el estado **lo mínimo**, y **calcula** todo lo demás durante el renderizado. Si un valor se puede obtener a partir de props, de otro estado o de los datos de una consulta, no es estado:

`src/features/applications/pages/applications-list-page.tsx`

```
const hasFilters = !!params.status || !!params.criticality
```

`src/features/applications/pages/application-edit-page.tsx`

```
const { isDirty } = form.formState
// Al empezar a editar de nuevo, se oculta la confirmación anterior.
const showUpdated = !isDirty
```

`showUpdated` podría haberse guardado con `useState` y actualizado al editar. Calcularlo elimina la posibilidad de que se desincronice: siempre es coherente con `isDirty`.

7.7 Identidad del estado y la prop key

↑ Índice

El estado de un componente está ligado a su **posición en el árbol**, no a la variable ni a los datos. Mientras React vea un componente del mismo tipo en la misma posición, conserva su estado aunque las props cambien por completo.

Normalmente es lo que quieres, pero hay una excepción importante en el proyecto. La página de edición de una aplicación contiene un formulario cuyos valores iniciales salen de los datos cargados:

src/features/applications/pages/application-edit-page.tsx

```
// key: se reinicia el formulario si se cambia de aplicación.
<ApplicationEditForm key={query.data.id} application={query.data} />
```

Supón que el usuario pasa de editar la aplicación 3 a editar la 7 navegando dentro de la aplicación, sin salir de la ruta `:id/editar`, y que los datos de la 7 ya están en la caché porque los consultó hace un momento. La consulta no pasa por el estado de carga, así que en ningún momento desaparece el formulario: React ve el mismo `ApplicationEditForm` en la misma posición, le pasa la nueva `application` como prop y **conserva su estado**. Y los valores de un formulario de React Hook Form forman parte de ese estado, inicializados con `defaultValues` solo al crearse: ¡el formulario seguiría mostrando los datos de la 3! Al darle `key={query.data.id}`, cuando la clave cambia React considera que es **otro componente**: destruye el anterior y crea uno nuevo con su estado inicial.

Usar `key` para **reiniciar** un componente es una técnica estándar y mucho más fiable que intentar sincronizar el estado con un efecto.

7.8 Componentes controlados

↑ Índice

Un campo de formulario puede funcionar de dos formas:

- **Controlado**: su valor lo decide React. Recibe `value` y, en cada pulsación, `onChange` actualiza el estado, que vuelve a fijar el `value`. La fuente de verdad es el estado.
- **No controlado**: el valor lo guarda el propio elemento del DOM, y se lee cuando hace falta (por ejemplo, al enviar).

El buscador por identificador es un campo controlado clásico:

src/features/applications/pages/application-edit-selector-page.tsx

```
<Input value={value} onChange={(e) => setValue(e.target.value)} … />
<Button type="submit" variant="outline" disabled={!value.trim() || query.isFetching}>
```

Al estar en el estado, el valor puede usarse para otras cosas durante el renderizado, como desactivar el botón *Buscar* si el campo está vacío. React Hook Form, en cambio, trabaja principalmente con campos no controlados y referencias para evitar renderizar el formulario entero en cada pulsación (Parte VII).

7.9 Rendimiento: cuándo preocuparse

↑ Índice

Renderizar es barato: ejecutar unas funciones y comparar objetos cuesta microsegundos. En una aplicación de gestión como esta, **casi nunca** es un problema. Antes de optimizar, mide:

- En React DevTools, la pestaña *Profiler* graba una interacción y muestra qué componentes se renderizaron, cuántas veces y cuánto tardaron.
- En la configuración de *Components* puedes activar *Highlight updates when components render* para ver en pantalla, con un recuadro de color, qué se renderiza en cada interacción.

Las herramientas de optimización (`useMemo`, `useCallback`, `memo`) se ven en el capítulo 8, junto con su coste. El proyecto las usa de forma muy puntual y justificada.

> **Nota**
>
> **React Compiler** es una herramienta oficial que analiza los componentes al construir y añade automáticamente la memorización necesaria, de modo que `useMemo` y `useCallback` se vuelven casi innecesarios. El proyecto no lo tiene activado, pero sus reglas ya se aplican a través del plugin de ESLint (capítulo 5): escribir código que las respete deja el proyecto preparado para activarlo.

7.10 Errores típicos y depuración

↑ Índice

- **«He cambiado el estado y no se actualiza».** Probablemente has mutado un objeto o un array en lugar de crear uno nuevo (sección 7.5).
- **«Leo el estado justo después de cambiarlo y tiene el valor antiguo».** Es la instantánea: el nuevo valor estará en el siguiente renderizado (sección 7.3).
- **`Too many re-renders. React limits the number of renders to prevent an infinite loop`.** Llamas a una función `set…` directamente en el cuerpo del componente (o en `onClick={setX(…)}` sin función flecha): cada renderizado cambia el estado, lo que provoca otro renderizado.
- **El formulario muestra datos de otro registro.** Falta una `key` que reinicie el componente al cambiar de registro (sección 7.7).
- **Se pierde lo escrito al cambiar algo aparentemente irrelevante.** Un componente ha cambiado de tipo o de posición (por ejemplo, porque un ternario lo envuelve en un `div` distinto en cada rama) y React lo ha recreado.

En la pestaña *Components* de React DevTools puedes ver y **editar** el estado de cualquier componente (aparece en el panel derecho como *hooks* `State`), lo que permite probar cómo se ve un estado sin reproducirlo a mano.

7.11 Ejercicios

↑ Índice

Ejercicio 7.1 · Observa la instantánea del estado Ver solución →

En `FindById` (`application-edit-selector-page.tsx`), añade temporalmente `console.log('antes', searchedId)` dentro de `onSubmit`, justo después de `setSearchedId(id)`, y `console.log('render', searchedId)` en el cuerpo del componente. Busca una aplicación y explica el orden y los valores que aparecen en consola. ¿Por qué el `render` aparece dos veces en desarrollo? (Lo explicaremos a fondo en el capítulo 8.)

Ejercicio 7.2 · Rompe la inmutabilidad Ver solución →

En `PickFromCatalog`, cambia `onPageChange` por una versión que mute el estado: `(page) => { params.page = page; setParams(params) }`. Prueba a cambiar de página. ¿Qué ocurre y por qué? Restaura el código original.

Ejercicio 7.3 · Quita la key del formulario Ver solución →

Para reproducir el problema de la sección 7.7 necesitas navegar entre dos ediciones sin salir de la ruta. Añade temporalmente, en las acciones de `PageHeader` de `application-edit-page.tsx`, un enlace `<Link`

`to={` /aplicaciones/${id + 1}/editar `}>Siguiente</Link>` . Elimina también `key={query.data.id}` del formulario. Después:

1. Abre el detalle de la aplicación 2 para que quede en caché.
2. Ve a `/aplicaciones/1/editar` , cambia el nombre sin guardar y pulsa *Siguiente*.
3. Observa el título de la página y los valores del formulario.

Restaura la `key` y repite. ¿Qué ocurre si en el paso 1 no visitas antes la aplicación 2? Explícalo con lo que sabes de los cambios de tipo en una posición del árbol. Deshaz todos los cambios.

Ejercicio 7.4 · Elige dónde vive el estado Ver solución →

Para cada uno de estos datos futuros, decide dónde debería vivir (estado local, URL, caché del servidor, formulario o contexto) y justifícalo: (a) la pestaña seleccionada en el detalle de una demanda (*Datos*, *Historial*, *Comentarios*); (b) el texto de un buscador por nombre en el listado de aplicaciones; (c) si el menú lateral está contraído en escritorio, de forma que se recuerde al navegar; (d) la lista de equipos para un desplegable; (e) un campo «motivo de cancelación» que solo aparece al elegir el estado `CANCELLED` .

7.12 Autoevaluación

↑ Índice

Autoevaluación del capítulo 7 Ver respuestas →

1. ¿Qué diferencia hay entre props y estado?
2. ¿Qué significa renderizar en React? ¿Implica modificar el DOM?
3. Si llamas dos veces seguidas a `setCount(count + 1)` en el mismo evento, ¿cuánto vale `count` en el siguiente renderizado? ¿Cómo lo arreglarías?
4. Describe las dos fases de una actualización y qué ocurre en cada una.
5. ¿Qué hace React si en la misma posición del árbol aparece un componente de tipo distinto?
6. ¿Por qué mutar un objeto del estado no provoca un renderizado?
7. ¿Por qué el listado principal guarda los filtros en la URL y el selector del catálogo en `useState` ?
8. ¿Qué es un valor derivado y por qué es preferible a guardarlo en el estado?
9. ¿Para qué sirve `key={query.data.id}` en la página de edición?

CAPÍTULO 8

Hooks: efectos, memorización y lógica reutilizable

↑ Volver al índice

EN ESTE CAPÍTULO

8.1 Qué es un hook y por qué tiene reglas
8.2 `useEffect`: sincronizar con el exterior
8.3 `StrictMode` y la doble ejecución en desarrollo
8.4 `useMemo` y `useCallback`: estabilidad de referencias
8.5 Hooks personalizados
8.6 Los hooks de las librerías
8.7 Otros hooks de React
8.8 Errores típicos y depuración
8.9 Ejercicios
8.10 Autoevaluación

Los *hooks* son las funciones con las que un componente accede a las capacidades de React: estado, efectos, contexto, referencias. Las librerías del proyecto exponen también su funcionalidad como hooks (`useQuery`, `useForm`, `useSearchParams`...). Este capítulo explica sus reglas, los tres hooks que más malentendidos generan —`useEffect`, `useMemo` y `useCallback`— y cómo escribir hooks propios.

8.1 Qué es un hook y por qué tiene reglas

↑ Índice

Un **hook** es una función cuyo nombre empieza por `use` y que solo puede llamarse durante el renderizado de un componente (o desde otro hook). Ya conoces uno: `useState`. Las dos reglas de los hooks, que ESLint vigila (capítulo 5), son:

1. **Llámalos siempre en el nivel superior** del componente: nunca dentro de un `if`, un bucle, una función anidada o después de un `return` anticipado.
2. **Llámalos solo desde componentes o desde otros hooks**, nunca desde funciones normales.

¿Por qué? Porque React **identifica cada hook por su orden de llamada**. Recuerda que el componente es una función sin memoria propia; React guarda, para cada instancia, una lista con el estado de sus hooks: «el primer hook de `FindById` es un `useState` con valor `'42'`, el segundo es un `useState` con valor `42`...». En cada renderizado, React recorre esa lista en el mismo orden en que se hacen las llamadas.

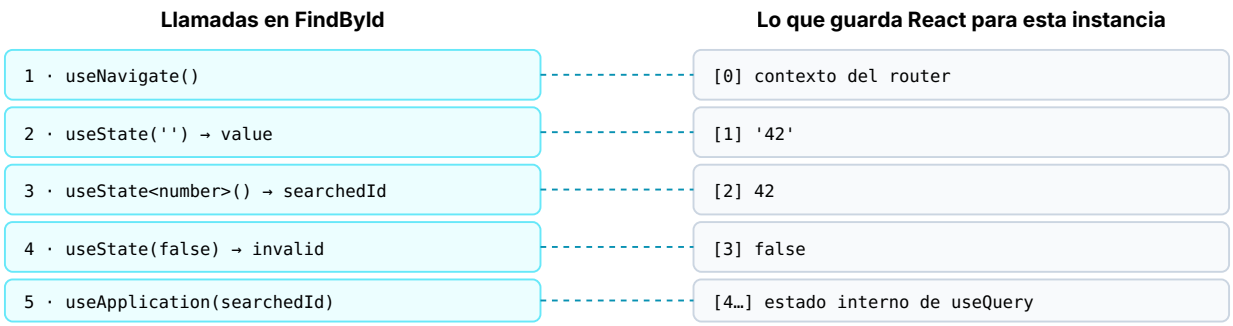


Figura 8.1. React asocia cada llamada a un hook con una posición de la lista de esa instancia. Si una llamada se salta en un renderizado, todas las posiciones siguientes se desplazan.

Si un hook estuviera dentro de un `if` y en un renderizado no se llamara, todas las posiciones siguientes se desplazarían y cada hook recibiría el estado de otro. Por eso la regla es estricta.

> **En el proyecto**
>
> Fíjate en cómo lo resuelve `ApplicationEditPage`: tiene un retorno anticipado cuando el `id` no es válido, pero **antes** de ese `return` ya ha llamado a todos sus hooks (`useParams`, `useApplication`). El formulario, que usa más hooks (`useForm`, `useUpdateApplication`, `useState`), está en un componente aparte, `ApplicationEditForm`, que solo se monta cuando hay datos. Separar en componentes es la forma natural de «llamar a hooks condicionalmente».

8.2 `useEffect`: sincronizar con el exterior

↑ Índice

Los componentes deben ser puros, pero a veces hay que actuar **fuera** de React: cambiar el título de la pestaña, suscribirse a un evento del navegador, iniciar un temporizador, manejar un elemento del DOM que no controla React. Para eso existe `useEffect`:

```
useEffect(() => {
  // código del efecto: se ejecuta DESPUÉS de que React haya aplicado los cambios al DOM
  return () => {
    // limpieza opcional: se ejecuta antes del siguiente efecto y al desmontar
  }
}, [dependencias])
```

La **lista de dependencias** decide cuándo se vuelve a ejecutar:

Dependencias	Cuándo se ejecuta el efecto
`[a, b]`	Tras el primer renderizado y cada vez que `a` o `b` cambian (comparados con `Object.is`)
`[]`	Solo tras el primer renderizado (al montar)
Sin lista	Tras **cada** renderizado (casi nunca es lo que quieres)

La forma correcta de pensar un efecto no es «se ejecuta al montar o al actualizar», sino **«mantén sincronizado este sistema externo con estos valores»**.

8.2.1 Ejemplo 1: el título de la pestaña

src/components/layout/app-layout.tsx

```tsx
/** Título de la pestaña a partir del `handle.title` de la ruta activa. */
function useDocumentTitle() {
  const matches = useMatches()
  const title = [...matches]
    .reverse()
    .map((m) => (m.handle as { title?: string } | undefined)?.title)
    .find(Boolean)
  useEffect(() => {
    document.title = title ? `${title} · IT Service Manager` : 'IT Service Manager'
  }, [title])
}
```

`document.title` está fuera de React: es un sistema externo. El título se **calcula** durante el renderizado (es un valor derivado de la ruta activa) y el efecto solo **sincroniza** ese valor con el exterior cada vez que cambia. Cada ruta declara su título en `handle: { title: 'Nueva aplicación' }` (Parte VII).

8.2.2 Ejemplo 2: corregir una página fuera de rango

src/features/applications/pages/applications-list-page.tsx

```tsx
// Si la página solicitada queda fuera de rango (p. ej. tras cambiar datos), ir a la última.
const totalPages = query.data?.totalPages
useEffect(() => {
  if (totalPages !== undefined && totalPages > 0 && params.page > totalPages - 1) {
    updateParams({ page: totalPages - 1 }, { replace: true })
  }
}, [totalPages, params.page, updateParams])
```

Si el usuario tiene guardado un enlace a `?pagina=9` y ahora solo hay 4 páginas, el backend devuelve una página vacía. Cuando llegan los datos y se conoce `totalPages`, el efecto **sincroniza la URL** (un sistema externo, gestionado por el router) llevándola a la última página. `replace: true` sustituye la entrada del historial en lugar de añadir una, para que *atrás* no devuelva al usuario a la página inexistente.

8.2.3 Cuándo NO usar un efecto

`useEffect` es el hook más usado de más. La documentación de React tiene un artículo entero titulado *You Might Not Need an Effect*. Las situaciones típicas en que **no** hace falta:

En lugar de un efecto para…	Haz esto
Calcular un valor a partir de props o estado	Calcúlalo durante el renderizado (valor derivado, capítulo 7)
Responder a una acción del usuario (guardar, navegar)	Hazlo en el manejador del evento (`onSubmit` , `onClick`)
Reiniciar el estado cuando cambia una prop	Usa `key` (capítulo 7)
Pedir datos al servidor	Usa TanStack Query (Parte VI)

Decisión de diseño: Ningún efecto para pedir datos

El patrón clásico `useEffect(() => { fetch(…).then(setData) }, [id])` tiene problemas sutiles: condiciones de carrera si `id` cambia antes de que llegue la respuesta, ninguna caché, peticiones duplicadas, estados de carga y error gestionados a mano. En el proyecto no hay **ni un solo** `fetch` dentro de un efecto: todas las lecturas pasan por `useQuery`. Los únicos efectos del proyecto sincronizan el título del documento y la URL de los listados.

8.2.4 La limpieza

Si un efecto se suscribe a algo, debe darse de baja en la función de limpieza. Un ejemplo genérico (el proyecto no lo necesita):

```
useEffect(() => {
  const onKey = (e: KeyboardEvent) => { if (e.key === 'Escape') setOpen(false) }
  window.addEventListener('keydown', onKey)
  return () => window.removeEventListener('keydown', onKey)
}, [])
```

Sin la limpieza, cada vez que el componente se montara se añadiría un oyente más, y seguirían activos tras desmontarse.

Si vienes de Java…

La limpieza es el `@PreDestroy` del efecto, o el `close()` de un `try-with-resources`: todo lo que el efecto abre, la limpieza lo cierra. La diferencia es que un efecto puede limpiarse y volver a ejecutarse muchas veces a lo largo de la vida del componente, cada vez que cambian sus dependencias.

8.3 `StrictMode` y la doble ejecución en desarrollo

↑ Índice

`main.tsx` envuelve la aplicación en `<StrictMode>`. En **desarrollo** (nunca en producción), este modo hace dos cosas deliberadamente incómodas:

1. **Renderiza cada componente dos veces** y descarta uno de los resultados. Si un componente no es puro (por ejemplo, incrementa un contador global al renderizar), el resultado diferirá y el error aflorará pronto.
2. **Monta, desmonta y vuelve a montar** cada componente la primera vez, ejecutando los efectos, sus limpiezas y otra vez los efectos. Si un efecto no tiene la limpieza correcta, verás el síntoma de inmediato (dos oyentes, dos suscripciones…).

Por eso, en desarrollo, un `console.log` en el cuerpo de un componente aparece dos veces. No es un error: es React comprobando que tu código aguanta lo que en producción puede ocurrir de verdad (por ejemplo, con funcionalidades que conservan el estado de componentes ocultos y los vuelven a montar).

8.4 `useMemo` y `useCallback`: estabilidad de referencias

↑ Índice

Cada renderizado vuelve a ejecutar todo el cuerpo del componente. Eso significa que:

- Los cálculos se repiten.

- Los objetos, arrays y funciones creados en el cuerpo son **nuevos** en cada renderizado: `{ a: 1 } !== { a: 1 }`.

Lo primero rara vez importa. Lo segundo importa cuando ese objeto o función se usa como **dependencia** de un efecto o de otro hook: si cambia en cada renderizado, el efecto se ejecuta en cada renderizado.

- `useMemo(fn, deps)` ejecuta `fn` y guarda el resultado; en los siguientes renderizados devuelve **el mismo valor** (la misma referencia) mientras las dependencias no cambien.
- `useCallback(fn, deps)` hace lo mismo con una función: devuelve la **misma función** mientras las dependencias no cambien. Equivale a `useMemo(() => fn, deps)`.

8.4.1 La cadena del listado de aplicaciones

El listado los usa juntos, y es un ejemplo muy bueno de por qué:

src/features/applications/pages/applications-list-page.tsx

```tsx
const [searchParams, setSearchParams] = useSearchParams()
const params = useMemo(() => parseListParams(searchParams), [searchParams])
const query = useApplications(params)

const updateParams = useCallback(
  (patch: Partial<ApplicationListParams>, options?: { replace?: boolean }) => {
    setSearchParams(serializeListParams({ ...params, ...patch }), options)
  },
  [params, setSearchParams],
)

useEffect(() => {
  if (totalPages !== undefined && totalPages > 0 && params.page > totalPages - 1) {
    updateParams({ page: totalPages - 1 }, { replace: true })
  }
}, [totalPages, params.page, updateParams])
```

Sigue la cadena de dependencias:

1. El efecto depende de `updateParams`. Si fuera una función nueva en cada renderizado, el efecto se ejecutaría en cada renderizado.
2. Por eso `updateParams` se envuelve en `useCallback`, que depende de `params`.
3. Si `params` fuera un objeto nuevo en cada renderizado (`parseListParams` crea uno cada vez que se llama), `updateParams` cambiaría siempre y el `useCallback` no serviría de nada.
4. Por eso `params` se envuelve en `useMemo`, que solo recalcula cuando cambia `searchParams`, es decir, cuando cambia la URL.

Resultado: mientras la URL no cambie, `params`, `updateParams` y el efecto permanecen estables.

8.4.2 Memorizar un cálculo derivado

src/features/demands/components/application-options.ts

```tsx
export function useApplicationSelectOptions(ensure?: { id: number; name: string }) {
  const query = useApplicationOptions()
  const ensureId = ensure?.id
  const ensureName = ensure?.name

  const options = useMemo<ApplicationOption[]>(() => {
    const list = (query.data?.content ?? []).map((app) => ({
      value: String(app.id), name: app.name, inactive: app.status === 'INACTIVE',
```

```
    }))
    if (ensureId !== undefined && ensureName && !list.some((o) => o.value === String(ensureId))) {
      list.unshift({ value: String(ensureId), name: ensureName, inactive: false })
    }
    return list
  }, [query.data, ensureId, ensureName])
  // …
}
```

Aquí `useMemo` evita transformar hasta mil aplicaciones en cada renderizado del formulario de demandas y, sobre todo, entrega al desplegable **la misma lista** mientras los datos no cambien. Fíjate en un detalle fino: las dependencias son `ensureId` y `ensureName` (valores primitivos), no el objeto `ensure`. Quien llama al hook suele escribir `useApplicationSelectOptions({ id, name })`, que crea un objeto nuevo en cada renderizado; si ese objeto fuera la dependencia, la memorización no serviría de nada.

> **Cuidado**
>
> `useMemo` y `useCallback` **no son gratis**: añaden código, una comparación de dependencias en cada renderizado y la posibilidad de olvidar una dependencia. Úsalos cuando haya un motivo concreto: un cálculo caro, o un valor que es dependencia de un efecto, de otro hook o de un componente memorizado. Envolver por defecto cada función en `useCallback` es un antipatrón frecuente. El proyecto los usa en media docena de sitios, todos justificados.

8.5 Hooks personalizados

↑ Índice

Un **hook personalizado** es una función propia cuyo nombre empieza por `use` y que llama a otros hooks. Sirve para **extraer y reutilizar lógica con estado**, del mismo modo que un componente extrae y reutiliza interfaz. Ya has visto varios:

Hook	Dónde	Qué encapsula
`useDocumentTitle()`	`app-layout.tsx`	Leer el título de la ruta activa y sincronizarlo con la pestaña
`useApplications(params)`	`applications-queries.ts`	La consulta paginada: clave de caché, función de petición, mantener la página anterior
`useApplication(id)`	`applications-queries.ts`	El detalle, sin reintentar los 404
`useCreateApplication()`	`applications-queries.ts`	La mutación de alta y la actualización de la caché
`useApplicationSelectOptions()`	`application-options.ts`	El catálogo de aplicaciones listo para un desplegable

Gracias a ellos, una página no sabe nada de claves de caché, de URLs de la API ni de reintentos: llama a `useApplications(params)` y recibe datos, estado de carga y errores.

`src/features/applications/api/applications-queries.ts`

```
export function useApplications(params: ApplicationListParams) {
  return useQuery({
    queryKey: applicationKeys.list(params),
    queryFn: ({ signal }) => applicationsApi.list(params, signal),
```

```
    // Mantiene la página anterior visible mientras se carga la siguiente.
    placeholderData: keepPreviousData,
  })
}
```

Dos cosas importantes sobre los hooks personalizados:

- **Comparten lógica, no estado.** Si dos componentes llaman a `useDocumentTitle()`, cada uno tiene su propio efecto. (Los hooks de TanStack Query sí comparten datos entre componentes, pero porque la caché vive fuera, en el `QueryClient`, no por ser hooks.)
- **Siguen las reglas de los hooks**, y el prefijo `use` es lo que permite a ESLint comprobarlas.

> **Si vienes de Java…**
>
> Un hook personalizado se parece a un servicio de Spring que encapsula una operación, con una diferencia: no es un *singleton* compartido, sino lógica que se «instala» en cada componente que la llama, con su propio estado. Si necesitas algo compartido de verdad, ese algo vive fuera (una caché, un contexto) y el hook solo da acceso a él.

8.6 Los hooks de las librerías

↑ Índice

Casi toda la funcionalidad de las librerías se consume mediante hooks. Esta tabla resume los que verás en el proyecto; cada uno tiene su explicación detallada en su parte.

Hook	Librería	Qué devuelve
`useParams()`	React Router	Los parámetros de la ruta (`:id`)
`useSearchParams()`	React Router	Los parámetros de la URL y la función para cambiarlos
`useNavigate()`	React Router	Una función para navegar por código
`useLocation()`, `useMatches()`	React Router	La ruta actual y las rutas que encajan
`useQuery()`, `useMutation()`	TanStack Query	Lecturas y escrituras contra la API con su estado
`useQueryClient()`	TanStack Query	Acceso a la caché para invalidar o actualizar
`useForm()`	React Hook Form	El estado y las operaciones de un formulario
`useFormContext()`, `useWatch()`	React Hook Form	Acceso al formulario desde componentes hijos; observar un campo
`useId()`	React	Un identificador único y estable para enlazar etiquetas y campos

8.7 Otros hooks de React

↑ Índice

React incluye más hooks, menos frecuentes en una aplicación de gestión:

- `useRef`: guarda un valor mutable que **no** provoca renderizados al cambiar, o una referencia a un elemento del DOM (`ref={inputRef}` para darle el foco). Es la vía de escape para lo que no encaja en el modelo declarativo.
- `useReducer`: alternativa a `useState` para estados complejos con muchas transiciones, al estilo de una máquina de estados con acciones.
- `useContext`: lee un contexto (capítulo 9).
- `useTransition`, `useDeferredValue`: marcan actualizaciones como no urgentes para mantener la interfaz fluida.
- `use`, `useActionState`, `useOptimistic`: novedades de React 19 para leer promesas y contextos, gestionar el estado de acciones de formularios y mostrar resultados optimistas. En el proyecto, esas necesidades las cubren TanStack Query y React Hook Form.

8.8 Errores típicos y depuración

↑ Índice

- `React Hook "useX" is called conditionally`**.** Has llamado a un hook dentro de un `if` o después de un `return`. Muévelo arriba o extrae un componente.
- **Bucle infinito de peticiones o renderizados con un efecto.** Una dependencia es un objeto o función que cambia en cada renderizado, o el efecto actualiza un estado que es su propia dependencia. Revisa la cadena de dependencias como en la sección 8.4.
- **Datos antiguos dentro de un efecto o un *callback*.** Falta una dependencia, y la función «ve» los valores de un renderizado antiguo (un *closure* obsoleto). No silencies el aviso de `exhaustive-deps`: añade la dependencia.
- **Todo aparece dos veces en consola.** Es `StrictMode` en desarrollo (sección 8.3).
- **El efecto se ejecuta antes de que exista el elemento.** No ocurre: los efectos se ejecutan después del *commit*. Si necesitas medir el DOM antes de que el navegador pinte, existe `useLayoutEffect`, pero es raro necesitarlo.

En la pestaña *Components* de React DevTools, los hooks de cada componente aparecen numerados en el panel derecho, en el orden de llamada: es exactamente la lista de la figura 8.1.

8.9 Ejercicios

↑ Índice

Ejercicio 8.1 · Escribe un hook personalizado Ver solución →

Crea en `src/lib/` (o en `src/hooks/`, que el proyecto tiene previsto como alias en `components.json`) un hook `useMediaQuery(query: string): boolean` que devuelva si una consulta de medios CSS se cumple, por ejemplo `'(min-width: 1024px)'`. Usa `window.matchMedia`, un `useState` y un `useEffect` que se suscriba al evento `change` y se dé de baja en la limpieza. Pruébalo mostrando temporalmente «Escritorio» o «Móvil» en la cabecera, y redimensiona la ventana.

Ejercicio 8.2 · Provoca un bucle con dependencias inestables Ver solución →

En `applications-list-page.tsx`, sustituye temporalmente `useMemo(() => parseListParams(searchParams), [searchParams])` por `parseListParams(searchParams)` a secas. Ejecuta `npm run lint`: ¿qué te dice ESLint? Abre el listado con la pestaña *Profiler* grabando y cambia de página. ¿Se producen renderizados de más? Razona por qué el efecto no provoca un bucle infinito en este caso concreto (pista: mira la condición dentro del efecto). Restaura el código.

Ejercicio 8.3 · Elimina un efecto innecesario Ver solución →

Este componente hipotético usa un efecto que sobra. Reescríbelo sin `useEffect` y explica por qué tu versión es mejor:

```tsx
function DemandCount({ demands }: { demands: Demand[] }) {
  const [openCount, setOpenCount] = useState(0)
  useEffect(() => {
    setOpenCount(demands.filter((d) => d.status === 'OPEN').length)
  }, [demands])
  return <span>{openCount} abiertas</span>
}
```

8.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 8 Ver respuestas →

1. ¿Por qué no se puede llamar a un hook dentro de un `if`?
2. ¿Cuál es la forma correcta de pensar en `useEffect`?
3. ¿Qué tres situaciones típicas no necesitan un efecto y qué se hace en su lugar?
4. ¿Qué hace la función de limpieza y cuándo se ejecuta?
5. ¿Qué dos cosas hace `StrictMode` en desarrollo y para qué?
6. ¿Por qué el listado de aplicaciones necesita `useMemo` en `params` para que el `useCallback` de `updateParams` sirva de algo?
7. ¿Por qué `useApplicationSelectOptions` usa `ensureId` y `ensureName` como dependencias en lugar de `ensure`?
8. ¿Qué comparten dos componentes que llaman al mismo hook personalizado?

CAPÍTULO 9

Composición, contexto y límites de error

↑ Volver al índice

EN ESTE CAPÍTULO

9.1 Composición en lugar de herencia
9.2 El problema de pasar props de mano en mano
9.3 Contexto: datos disponibles para todo un subárbol
9.4 Los proveedores globales de `app.tsx`
9.5 Cuándo usar contexto y cuándo no
9.6 Más patrones de composición del proyecto
9.7 Límites de error
9.8 Carga diferida y espera
9.9 Errores típicos y depuración
9.10 Ejercicios
9.11 Autoevaluación

Con componentes, estado y hooks ya se puede construir cualquier pantalla. Este último capítulo de la Parte III trata de cómo se *combinan* esas piezas en una aplicación de verdad: cómo se reutiliza sin herencia, cómo llegan los servicios globales a cualquier componente sin pasarlos de mano en mano, y cómo se evita que un error en una pantalla deje la aplicación en blanco.

9.1 Composición en lugar de herencia

↑ Índice

En React no se reutiliza mediante herencia (no verás nunca `class TablaDeAplicaciones extends Tabla`), sino mediante **composición**: componentes que contienen otros componentes y que dejan huecos para que quien los usa decida qué poner.

9.1.1 Componentes compuestos

Muchos componentes de interfaz no son una pieza, sino una **familia** de piezas pensadas para usarse juntas. La tarjeta es el ejemplo más claro:

src/features/applications/pages/application-create-page.tsx (simplificado)

```
<Card className="max-w-3xl">
  <CardHeader className="border-b">
    <CardTitle>Datos de la aplicación</CardTitle>
    <CardDescription><RequiredFieldsNote /></CardDescription>
  </CardHeader>
  <CardContent className="grid gap-6">…campos…</CardContent>
  <CardFooter className="justify-end gap-2 border-t">…botones…</CardFooter>
</Card>
```

`Card` no sabe que contendrá un formulario; `CardHeader`, `CardContent` y `CardFooter` aportan la estructura y los estilos de cada zona, y la página decide qué va dentro y en qué orden. Lo mismo ocurre con la tabla (`Table`, `TableHeader`, `TableRow`, `TableCell`…) o con el desplegable (`Select`, `SelectTrigger`, `SelectContent`, `SelectItem`). Es mucho más flexible que un único componente con veinte props de configuración.

9.1.2 Especialización por composición

Para crear una variante de un componente, se **envuelve** en otro que fija algunas props. Los campos del formulario de aplicaciones son un ejemplo de dos niveles:

src/features/applications/components/application-form-fields.tsx

```
export function NameField() {
  return (
    <TextInputField name="name" label="Nombre" required maxLength={NAME_MAX_LENGTH} placeholder="Ej.
: SIGMA" />
  )
}
```

- `components/ui/input.tsx` es un campo de texto genérico con los estilos del sistema.
- `components/common/form-fields.tsx` define `TextInputField`: un campo con etiqueta, contador de caracteres, mensaje de error y conexión con el formulario.
- `features/applications/components/application-form-fields.tsx` define `NameField`: un `TextInputField` configurado con el nombre, la etiqueta y la longitud máxima del dominio.

Las páginas de alta y de modificación usan `<NameField />` sin repetir ninguna de esas decisiones.

> **Si vienes de Java...**
>
> Es el principio «favorece la composición sobre la herencia» llevado al extremo. Si en Java resolverías la variación con una jerarquía de clases y métodos `protected` que se sobrescriben, en React se resuelve con componentes que reciben otros componentes o los envuelven. La ventaja es la misma que en Java: sin acoplamiento a una clase base frágil.

9.2 El problema de pasar props de mano en mano

↑ Índice

`NameField` no recibe ninguna prop relacionada con el formulario. Y, sin embargo, necesita acceder a él: registrarse, leer su valor, mostrar su error. ¿De dónde lo saca?

La opción ingenua sería pasar el objeto `form` como prop desde la página a cada campo, y de cada campo a sus piezas internas (etiqueta, control, mensaje). Cuando un dato tiene que atravesar varios niveles de componentes que no lo usan solo para llegar a uno que sí, se habla de ***prop drilling*** (perforación de props). Es tedioso, acopla los componentes intermedios a datos que no les interesan y complica cualquier cambio.

9.3 Contexto: datos disponibles para todo un subárbol

↑ Índice

El **contexto** de React resuelve ese problema: un componente **proveedor** ofrece un valor, y cualquier descendiente, a cualquier profundidad, puede leerlo directamente.

Tiene tres piezas:

```
// 1. Crear el contexto (normalmente a nivel de módulo), con un valor por defecto
const ThemeContext = createContext<'light' | 'dark'>('light')

// 2. Proveer un valor a un subárbol
<ThemeContext.Provider value="dark">
  <Pagina />
</ThemeContext.Provider>

// 3. Leerlo desde cualquier descendiente
const theme = useContext(ThemeContext)
```

`useContext` busca hacia arriba en el árbol el proveedor más cercano de ese contexto y devuelve su valor. Si el valor cambia, React vuelve a renderizar todos los componentes que lo leen.

9.3.1 El contexto en los formularios del proyecto

Los formularios encadenan **tres contextos**, y es la mejor forma de verlos en acción:

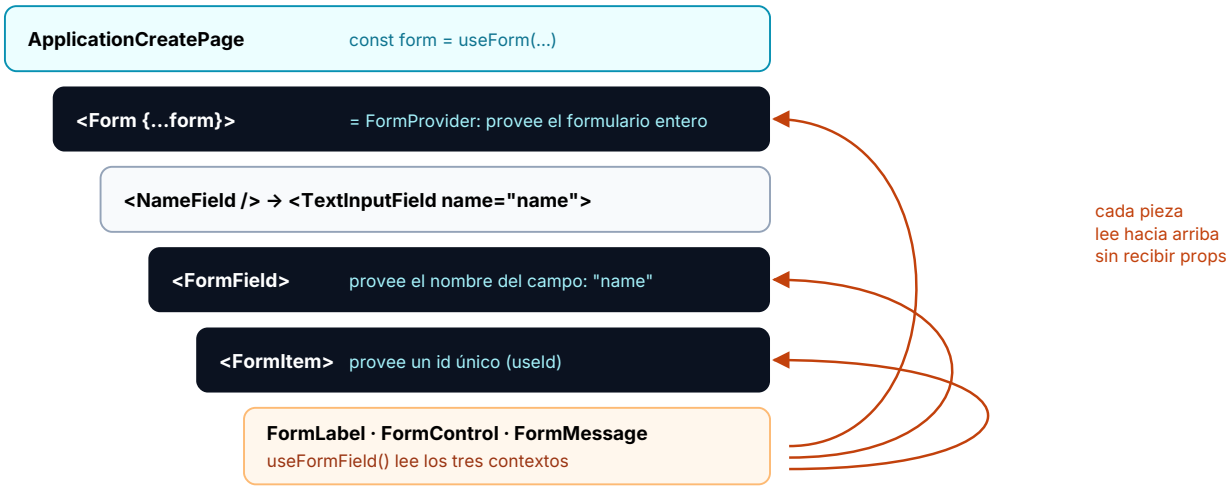


Figura 9.1. Los proveedores de contexto de un formulario. Las piezas del campo encuentran el formulario, su nombre y su identificador sin que nadie se los pase.

1. `<Form {...form}>` es el `FormProvider` de React Hook Form: pone el formulario completo a disposición del subárbol. Por eso `TextInputField` puede hacer `const { control } = useFormContext()` y solo necesita el `name`.
2. `<FormField>`, de shadcn/ui, crea un contexto propio con el nombre del campo:

src/components/ui/form.tsx

```tsx
const FormFieldContext = React.createContext<FormFieldContextValue>({} as FormFieldContextValue)

const FormField = ({ ...props }: ControllerProps<TFieldValues, TName>) => {
  return (
    <FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  )
}
```

1. `<FormItem>` genera un identificador único con `useId()` y lo provee en otro contexto.

Por último, un hook personalizado reúne los tres:

src/components/ui/form.tsx (simplificado)

```tsx
const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState } = useFormContext()
  const fieldState = getFieldState(fieldContext.name, /* … */)
  const { id } = itemContext
  return { id, name: fieldContext.name, formItemId: `${id}-form-item`, /* … */ ...fieldState }
}
```

Con eso, `FormLabel` puede apuntar su `htmlFor` al identificador del control, `FormControl` puede marcar `aria-invalid` si hay error, y `FormMessage` puede mostrar el mensaje de error del campo. Ninguno recibe una sola prop relacionada con el formulario.

> **Si vienes de Java...**
>
> El contexto es la forma que tiene React de hacer **inyección de dependencias**, pero por posición en el árbol en lugar de por tipo en un contenedor global. Un proveedor es como un `@Bean` declarado en un ámbito concreto (*request scope*, o el de una sección de la interfaz), y `useContext` es como un `@Autowired` que busca la definición más cercana hacia arriba. Igual que en Spring, pueden coexistir varias instancias del mismo tipo en ámbitos distintos: cada campo del formulario tiene su propio `FormFieldContext`.

9.4 Los proveedores globales de `app.tsx`

↑ Índice

En el capítulo 1 vimos el componente `App`. Ahora puede leerse con otros ojos:

`src/app/app.tsx`

```tsx
export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <TooltipProvider>
        <RouterProvider router={router} />
        <Toaster position="top-right" />
      </TooltipProvider>
    </QueryClientProvider>
  )
}
```

Proveedor	Qué pone a disposición del árbol	Quién lo consume
`QueryClientProvider`	La instancia de `QueryClient`, que contiene la caché de datos del servidor	`useQuery`, `useMutation`, `useQueryClient`
`TooltipProvider`	La configuración compartida de los *tooltips* (retardos, coordinación entre varios)	Los componentes `Tooltip`
`RouterProvider`	El estado de navegación: URL actual, parámetros, funciones de navegación	`useParams`, `useNavigate`, `Link`...
`Toaster`	No es un proveedor, sino el contenedor donde aparecen las notificaciones	`toast.success(…)` desde cualquier sitio

El **orden importa**: un componente solo puede leer los contextos de sus antepasados. Como `RouterProvider` está **dentro** de `QueryClientProvider`, todas las páginas que pinta el router pueden usar consultas. Si estuviera fuera, `useQuery` fallaría con el error *No QueryClient set, use QueryClientProvider to set one*.

> **En el proyecto**
>
> `Toaster` merece un comentario: la función `toast` de `sonner` se importa y se llama desde cualquier sitio, incluso desde fuera de un componente, sin contexto de React. La librería usa su propio almacén interno y `Toaster` se suscribe a él. Es un patrón habitual para servicios globales muy simples: un único contenedor montado en la raíz y una función importable.

9.5 Cuándo usar contexto y cuándo no

↑ Índice

El contexto es una herramienta potente, pero no es un almacén de estado global para todo:

- **Cada cambio del valor renderiza a todos sus consumidores.** Un contexto con datos que cambian a menudo (por ejemplo, el valor de un campo en cada pulsación) puede provocar muchos renderizados innecesarios. Por eso React Hook Form no guarda los valores en un contexto que se renderiza en cada pulsación, sino en un almacén propio con suscripciones finas (`useWatch` se suscribe solo al campo que observa).
- **No es para datos del servidor.** Para eso está TanStack Query, que ya usa un contexto para dar acceso a su caché y añade todo lo demás (Parte VI).
- **Úsalo para:** servicios e instancias (el `QueryClient`), configuración de un subárbol (el tema, el idioma, el usuario autenticado) y familias de componentes que comparten información interna (formularios, menús, pestañas).

Decisión de diseño: Sin librería de estado global

Contexto. Muchas aplicaciones React incorporan Redux, Zustand o similares para el estado global.

Decisión. No usar ninguna. El estado del proyecto se reparte así: datos del servidor en TanStack Query, navegación y filtros en la URL, formularios en React Hook Form, y lo puramente local en `useState`. Lo que queda para un estado global de interfaz es mínimo (notificaciones y *tooltips*) y lo resuelven sus propias librerías.

Cuándo reconsiderarlo. Si aparece estado de cliente compartido por muchas pantallas y que cambia con frecuencia (por ejemplo, preferencias de usuario editables, un carrito, un asistente de varios pasos), Zustand es una opción ligera. Un contexto propio basta para datos que cambian poco, como el usuario autenticado cuando se añada la seguridad.

9.6 Más patrones de composición del proyecto

↑ Índice

9.6.1 `asChild`: prestar el comportamiento a otro elemento

El botón del sistema de diseño pinta un `<button>`. Pero a veces queremos el **aspecto** de un botón con el **comportamiento** de un enlace de React Router:

`src/features/applications/pages/applications-list-page.tsx`

```
<Button asChild variant="brand">
  <Link to="/aplicaciones/nueva">
    <Plus /> Nueva aplicación
  </Link>
</Button>
```

Con `asChild`, `Button` no pinta su propio elemento: usa el componente `Slot` de Radix, que **fusiona** las props del botón (clases, estilos de la variante, atributos) en su único hijo, el `Link`. El resultado en el DOM es un único `<a>` con aspecto de botón y navegación de React Router. Sin este patrón habría que anidar un enlace dentro de un botón (HTML no válido) o duplicar los estilos del botón en un enlace.

`src/components/ui/button.tsx (extracto)`

```
const Comp = asChild ? Slot.Root : 'button'
```

`FormControl` usa el mismo mecanismo para añadir `id`, `aria-describedby` y `aria-invalid` al campo que envuelve, sea un `Input`, un `Textarea` o el disparador de un `Select`.

9.6.2 *Render props*: pasar una función que pinta

`FormField` no recibe hijos, sino una prop `render` que es **una función que devuelve JSX**:

`src/components/common/form-fields.tsx (simplificado)`

```
<FormField
  control={control}
  name={name}
  render={({ field }) => (
    <FormItem>
      <FormLabel required={required}>{label}</FormLabel>
      <FormControl>
        <Input placeholder={placeholder} {...field} />
      </FormControl>
      <FormMessage />
    </FormItem>
  )}
/>
```

`FormField` (en realidad el `Controller` de React Hook Form que lleva dentro) llama a esa función con el objeto `field`, que contiene `value`, `onChange`, `onBlur`, `name` y `ref`, y pinta lo que devuelve. `{...field}` conecta el campo con el formulario en una línea. Este patrón permite que el componente controle los **datos** y quien lo usa controle la **presentación**.

9.7 Límites de error

↑ Índice

Si un componente lanza una excepción durante el renderizado (por ejemplo, accede a una propiedad de `undefined`), React no puede saber qué pintar en esa parte del árbol. Por defecto, **desmonta toda la aplicación** y deja la página en blanco.

Un **límite de error** (*error boundary*) es un componente que captura los errores de renderizado de sus descendientes y muestra una interfaz alternativa en su lugar, igual que un `try/catch` para una parte del árbol. En el proyecto, el router se encarga de ello mediante `errorElement`:

`src/app/router.tsx (extracto)`

```
{
  path: '/',
  element: <AppLayout />,
  children: [
    {
      errorElement: <RouteErrorPage />,
      children: [ /* rutas de aplicaciones, demandas, 404 */ ],
    },
  ],
}
```

La posición del `errorElement` está elegida con cuidado: es una ruta **sin ruta propia** (no tiene `path`), situada **dentro** de `AppLayout`. Si una página falla, se sustituye solo el contenido por `RouteErrorPage`, y **el menú lateral y la cabecera siguen funcionando**: el usuario puede irse a otra sección.

src/pages/route-error-page.tsx

```tsx
/** Captura errores inesperados de renderizado para no dejar la pantalla en blanco. */
export function RouteErrorPage() {
  const error = useRouteError()
  // … construye un mensaje legible a partir del error …
  return (
    <div className="…">
      <h1 className="text-xl font-semibold">Se ha producido un error inesperado</h1>
      <p className="…">{message}</p>
      <Button variant="outline" onClick={() => window.location.assign('/aplicaciones')}>
        Volver al inicio
      </Button>
    </div>
  )
}
```

Fíjate en que *Volver al inicio* usa `window.location.assign` en lugar de `navigate`: provoca una **recarga completa**. Tras un error inesperado, el estado en memoria puede estar corrupto; recargar es la forma más segura de empezar limpio.

> **Cuidado**
>
> Los límites de error capturan errores **durante el renderizado** (y en efectos y constructores). **No** capturan errores en manejadores de eventos ni en código asíncrono: un fallo dentro de `onClick` o de una promesa no llega a `RouteErrorPage`. Por eso los errores de la API no se gestionan con límites de error, sino de forma explícita: TanStack Query los expone como `query.error` o `mutation.error` y cada pantalla los muestra con `ErrorState` o con una alerta.

9.8 Carga diferida y espera

↑ Índice

El capítulo 1 explicó que cada página se descarga bajo demanda mediante `lazy()`. Mientras se descarga, algo hay que mostrar. React tiene un mecanismo general para ello, `Suspense`: un componente que muestra una interfaz de espera mientras alguno de sus descendientes «suspende» porque espera algo (código o datos).

En el proyecto, la espera de las páginas la gestiona el router:

- Durante la **carga inicial** (abrir directamente `/aplicaciones`), muestra `hydrateFallbackElement`, el `LoadingState` a pantalla completa.
- Durante una **navegación posterior**, el router mantiene la pantalla anterior hasta que la nueva está lista, de modo que no hay parpadeos.

La espera de los **datos** no usa `Suspense`, sino los estados explícitos de cada consulta (`isPending`, `isError`), con esqueletos de carga adaptados a cada pantalla. Es una decisión deliberada: hace visibles en el código todos los estados posibles, que es lo que vimos en el capítulo 6.

9.9 Errores típicos y depuración

↑ Índice

- **`No QueryClient set, use QueryClientProvider to set one`**, **`useNavigate() may be used only in the context of a <Router> component`** o **`useFormContext` devuelve `null`.** El componente se está

renderizando fuera de su proveedor. Comprueba la jerarquía en React DevTools; ocurre a menudo en pruebas o al mover un componente.

- **`useFormField should be used within <FormField>`.** Has usado `FormLabel`, `FormControl` o `FormMessage` fuera de un `FormField`.
- **Un enlace con aspecto de botón no navega, o el HTML tiene un `<a>` dentro de un `<button>`.** Falta `asChild`, o el hijo no es un único elemento.
- **La aplicación se queda en blanco tras un error.** El error se ha producido fuera de la zona cubierta por `errorElement` (por ejemplo, en `AppLayout` o en `App`). Revisa la consola.
- **Muchos renderizados al cambiar un dato de un contexto.** Todos los consumidores se renderizan; considera dividir el contexto o usar un almacén con suscripciones.

En React DevTools, los proveedores aparecen en el árbol como `Context.Provider` (o con el nombre del contexto) y, al seleccionar un componente, el panel derecho muestra los contextos que consume.

9.10 Ejercicios

↑ Índice

Ejercicio 9.1 · Sigue los contextos de un campo Ver solución →

Con React DevTools, selecciona en la página *Nueva aplicación* el componente `FormLabel` del campo *Nombre*. Recorre sus antepasados hasta encontrar `FormItem`, `FormField` y el `FormProvider`. ¿Qué valor provee cada uno? Después, en el código, sigue cómo llega a `FormLabel` el `id` que usa en `htmlFor` y comprueba en la pestaña *Elements* que coincide con el `id` del `<input>`.

Ejercicio 9.2 · Crea un contexto propio Ver solución →

Crea un contexto `DensityContext` con los valores `'comfortable' | 'compact'` y un proveedor que se coloque en `AppLayout`. Añade en la cabecera un botón que alterne la densidad y haz que `ApplicationsTable` la lea con `useContext` para reducir el relleno de las celdas en modo compacto. Reflexiona: ¿debería este dato vivir en un contexto, en la URL o en el almacenamiento del navegador? ¿Qué ventajas e inconvenientes tiene cada opción?

Ejercicio 9.3 · Provoca un error de renderizado Ver solución →

En `applications-table.tsx`, añade temporalmente dentro del `map` una línea que lance un error si el identificador de la aplicación es 1: `if (app.id === 1) throw new Error('Fallo de prueba')`. Abre el listado. ¿Qué ves? ¿Sigue funcionando el menú lateral? Ahora mueve el `throw` a un manejador `onClick` de un botón y pulsa el botón. ¿Lo captura `RouteErrorPage`? ¿Por qué? Deshaz los cambios.

9.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 9 Ver respuestas →

1. ¿Cómo se reutiliza y especializa un componente en React si no hay herencia?
2. ¿Qué es el *prop drilling* y qué problema causa?
3. ¿Cuáles son las tres piezas del contexto de React?
4. ¿Cómo consigue `NameField` acceder al formulario sin recibirlo por props?

5. ¿Por qué `RouterProvider` debe estar dentro de `QueryClientProvider`?
6. ¿Qué inconveniente tiene guardar en un contexto un valor que cambia muy a menudo?
7. ¿Qué hace la prop `asChild` de `Button`?
8. ¿Por qué el `errorElement` está dentro de `AppLayout` y no por encima?
9. ¿Captura un límite de error un fallo dentro de un `onClick`? ¿Y un error de la API?

PARTE IV

Interfaz y diseño

10	Tailwind CSS 4: estilos con clases de utilidad	pág. 100
11	shadcn/ui y Radix: componentes accesibles que son tuyos	pág. 111
12	El layout y el sistema visual	pág. 120

↑ Volver al índice

CAPÍTULO 10

Tailwind CSS 4: estilos con clases de utilidad

↑ Volver al índice

EN ESTE CAPÍTULO

10.1 El problema que resuelve
10.2 Cómo leer las clases de utilidad
10.3 Diseño *responsive*: primero el móvil
10.4 Estados y variantes
10.5 Cómo funciona Tailwind 4 por dentro
10.6 El tema: tokens de diseño en `index.css`
10.7 Componer clases: `cn()`
10.8 Animaciones y transiciones
10.9 Decisión y alternativas
10.10 Errores típicos y depuración
10.11 Ejercicios
10.12 Autoevaluación

Todo el aspecto visual del proyecto —colores, espaciados, tipografía, disposición responsive, estados de foco o animaciones— se expresa con clases de Tailwind CSS escritas directamente en el JSX. A primera vista parece una sopa de letras. Este capítulo te enseña a leerla, explica cómo genera Tailwind el CSS y recorre el sistema de tokens de diseño definido en `index.css`.

10.1 El problema que resuelve

↑ Índice

CSS es un lenguaje **global**: cualquier regla puede afectar a cualquier elemento de la página. En aplicaciones grandes esto produce problemas conocidos:

- **Colisiones de nombres.** Dos equipos crean una clase `.card` con estilos distintos y una pisa a la otra según el orden de carga.
- **Miedo a borrar.** Nadie sabe si una regla de la hoja de estilos se usa todavía, así que el CSS solo crece.
- **Inconsistencia.** Cada pantalla acaba con su propio gris, su propio margen de 13 píxeles y su propio tamaño de letra.
- **Saltar entre ficheros.** Para entender cómo se ve un componente hay que leer su JSX y, en otro fichero, sus estilos.

Las soluciones clásicas son convenciones de nombres (BEM: `.card__title--highlighted`), CSS Modules (clases con ámbito local) o CSS-in-JS (estilos escritos en JavaScript). **Tailwind** propone otra cosa: un catálogo enorme de **clases de utilidad** pequeñas, cada una con una sola propiedad, que se combinan directamente en el marcado.

```
<!-- CSS tradicional: una clase semántica y una regla en otro fichero -->
<p class="page-eyebrow">Aplicaciones</p>
.page-eyebrow { font-size: .75rem; font-weight: 500; letter-spacing: .025em; text-transform:
uppercase; }

<!-- Tailwind: las propiedades directamente en el elemento -->
<p class="text-xs font-medium tracking-wide uppercase">Aplicaciones</p>
```

El resultado: no hay nombres que inventar ni que colisionen, el estilo de un componente está **en el propio componente** (si borras el componente, borras su estilo), y todas las pantallas comparten la misma escala de espaciados, colores y tamaños.

Si vienes de Java...

La comparación justa es la de las anotaciones frente a los ficheros XML de configuración. Antes de Spring con anotaciones, el comportamiento de una clase estaba repartido entre el código y un `applicationContext.xml`; con `@Service` o `@Transactional`, todo está junto a lo que describe. Tailwind hace lo mismo con los estilos: los coloca junto al elemento al que se aplican. Y despierta las mismas objeciones iniciales («ensucia el código»), que suelen desaparecer con el uso.

10.2 Cómo leer las clases de utilidad

↑ Índice

Cada clase sigue el patrón **propiedad-valor**, con abreviaturas sistemáticas. Con una decena de familias se lee el 90 % del proyecto.

Familia	Ejemplos	CSS que genera
Relleno y margen	`p-4`, `px-3`, `py-2`, `mt-1`, `mb-6`, `ml-auto`	`padding: 1rem`; `padding-inline: .75rem`; `margin-left: auto`
Separación entre hijos	`gap-3`, `space-y-1`	`gap: .75rem`; margen vertical entre hijos
Tamaño	`w-64`, `h-16`, `size-4`, `max-w-7xl`, `min-h-svh`, `w-full`	`width: 16rem`; `width` y `height: 1rem`
Disposición	`flex`, `grid`, `flex-col`, `items-center`, `justify-between`, `grid-cols-[1fr_16rem]`	`display: flex`; `align-items: center` …
Tipografía	`text-sm`, `text-2xl`, `font-semibold`, `tracking-tight`, `uppercase`, `truncate`	`font-size`, `font-weight`, `letter-spacing` …
Color	`bg-card`, `text-muted-foreground`, `border-sidebar-border`, `bg-red-50`	`background-color`, `color`, `border-color`
Bordes y radios	`border`, `border-b`, `rounded-md`, `rounded-full`, `ring-1`	`border-width`, `border-radius`, sombra de anillo
Posición	`fixed`, `sticky`, `inset-y-0`, `left-0`, `top-0`, `z-40`	`position`, desplazamientos, `z-index`
Visibilidad	`hidden`, `block`, `sr-only`, `opacity-60`	`display: none` …
Efectos	`shadow-sm`, `backdrop-blur`, `transition-colors`, `animate-spin`	sombras, filtros, transiciones

10.2.1 La escala de espaciado

Los números no son píxeles: son **pasos de una escala**. En Tailwind 4, cada paso vale `0.25rem` (4 píxeles con el tamaño de letra por defecto): `p-1` son 4 px, `p-4` son 16 px, `w-64` son 256 px. Trabajar siempre sobre la misma escala es lo que hace que los espaciados de todas las pantallas encajen sin esfuerzo.

10.2.2 Un ejemplo real, clase a clase

src/components/common/page-header.tsx

```
<div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
```

- `mb-6`: margen inferior de 1,5 rem, que separa la cabecera del contenido.
- `flex flex-col`: contenedor *flexbox* en columna: título arriba, acciones debajo.
- `gap-4`: 1 rem entre ambos bloques.
- `sm:flex-row`: a partir del punto de ruptura `sm` (640 px), en fila: título a la izquierda, acciones a la derecha.
- `sm:items-end sm:justify-between`: alineados por abajo y separados al máximo.

En una frase: «en móvil, título y botones apilados; en pantallas medianas o mayores, en la misma línea con los botones a la derecha».

10.2.3 Valores arbitrarios

Cuando un valor no está en la escala, se escribe entre corchetes: `w-[72px]`, `text-[11px]`, `max-w-[22rem]`, `grid-cols-[1fr_16rem]` (el guion bajo representa un espacio). También se pueden usar variables CSS con paréntesis: `max-h-(--radix-select-content-available-height)`. Úsalos con moderación: cada valor arbitrario es una pequeña excepción al sistema de diseño.

10.3 Diseño *responsive*: primero el móvil

↑ Índice

Tailwind sigue el enfoque *mobile first*: las clases sin prefijo se aplican **a todos los tamaños**, y los prefijos de punto de ruptura añaden estilos **a partir de** un ancho mínimo.

Prefijo	Ancho mínimo	Uso típico
(ninguno)	0	Móvil
`sm:`	40 rem (640 px)	Móvil grande, tableta vertical
`md:`	48 rem (768 px)	Tableta
`lg:`	64 rem (1024 px)	Portátil: aquí aparece el menú lateral fijo
`xl:`	80 rem (1280 px)	Escritorio amplio
`2xl:`	96 rem (1536 px)	Monitores grandes

El layout de la aplicación muestra el patrón completo:

src/components/layout/app-layout.tsx

```
<div className="min-h-svh">
  <aside className="fixed inset-y-0 left-0 z-40 hidden w-64 lg:block">
    <SidebarNav />
  </aside>
  <div className="lg:pl-64">
    <AppHeader />
    <main className="mx-auto w-full max-w-7xl px-4 py-6 lg:px-8 lg:py-8">
      <Outlet />
    </main>
```

```
  </div>
</div>
```

- El `aside` es `hidden` por defecto y `lg:block` desde 1024 px: el menú lateral fijo solo existe en pantallas grandes.
- El contenido tiene `lg:pl-64`: un relleno izquierdo igual al ancho del menú, para no quedar debajo de él.
- En móvil, la cabecera muestra un botón de menú (`lg:hidden`) que abre el mismo `SidebarNav` dentro de un panel deslizante (capítulo 12).
- El `main` limita su ancho (`max-w-7xl`) y se centra (`mx-auto`), con más margen en pantallas grandes (`lg:px-8`).

> **Cuidado**
>
> Un error frecuente es pensar que `sm:` significa «en pantallas pequeñas». Significa «**desde** pantallas pequeñas en adelante». Para algo que solo debe verse en móvil, se usa la clase sin prefijo y se anula en el punto de ruptura: `block lg:hidden`.

10.4 Estados y variantes

↑ Índice

Los prefijos no solo expresan anchos de pantalla. Cualquier clase puede condicionarse a un **estado** o una **situación**, y es una de las mayores ventajas de Tailwind frente a escribir CSS a mano.

Variante	Se aplica cuando…	Ejemplo en el proyecto
`hover:`	El puntero está encima	`hover:bg-sidebar-accent` en los enlaces del menú
`focus-visible:`	El elemento tiene el foco por teclado	`focus-visible:ring-[3px]` en el botón
`disabled:`	El elemento está desactivado	`disabled:opacity-50`
`aria-invalid:`	Tiene `aria-invalid="true"`	`aria-invalid:border-destructive` en los campos con error
`data-[state=open]:`	Tiene `data-state="open"`	Animaciones de apertura de Radix
`group-data-[state=open]:`	Un antepasado con clase `group` tiene ese atributo	La flecha del menú que gira al desplegar
`has-[>svg]:`	Contiene un hijo directo `svg`	Ajuste de relleno de los botones con icono
`before:`	Pseudo-elemento `::before`	La barra cian del enlace activo del menú
`supports-[…]:`	El navegador soporta una propiedad	Fondo translúcido de la cabecera
`[&_svg]:`	Selector arbitrario sobre descendientes	Tamaño por defecto de los iconos dentro de un botón

Veamos dos casos con detalle.

La flecha que gira al abrir un grupo del menú:

src/components/layout/sidebar-nav.tsx

```
<CollapsibleTrigger className={cn('group flex w-full … ', isInModule && 'text-white')}>
  <Icon className="…" />
  <span className="flex-1 text-left">{module.label}</span>
  <ChevronRight className="size-4 text-sidebar-muted transition-transform group-data-
[state=open]:rotate-90" />
</CollapsibleTrigger>
```

Radix pone `data-state="open"` o `"closed"` en el disparador. La clase `group` marca el disparador como grupo, y `group-data-[state=open]:rotate-90` rota la flecha cuando el grupo está abierto. `transition-transform` anima el giro. Ni una línea de JavaScript para el efecto visual.

El enlace activo del menú, con un indicador lateral:

src/components/layout/sidebar-nav.tsx

```
className={({ isActive }) =>
  cn(
    'relative block rounded-md px-3 py-1.5 text-[13px] transition-colors hover:bg-sidebar-accent
hover:text-white',
    isActive &&
      'bg-sidebar-accent font-medium text-white before:absolute before:top-1.5 before:bottom-1.5
before:-left-[13px] before:w-0.5 before:rounded-full before:bg-cyan-400',
  )
}
```

Las clases `before:*` dibujan un pseudo-elemento: una barra de 2 px de ancho, posicionada a la izquierda del enlace, sobre la línea vertical del submenú. `NavLink` de React Router indica si la ruta está activa, y solo entonces se añaden esas clases.

10.5 Cómo funciona Tailwind 4 por dentro

↑ Índice

Tailwind **no es una hoja de estilos gigante** con todas las clases posibles: sería de cientos de megas. Es un **generador**:

1. Al arrancar Vite, el plugin `@tailwindcss/vite` **analiza los ficheros fuente** del proyecto en busca de cadenas que parezcan clases.
2. Para cada clase encontrada que reconoce, genera su regla CSS.
3. Escribe el CSS resultante en lugar de la línea `@import 'tailwindcss'` de `index.css`.
4. En desarrollo, al guardar un fichero con clases nuevas, regenera lo necesario en milisegundos; en producción, el CSS final solo contiene lo que se usa (el del proyecto ocupa unas decenas de kilobytes).

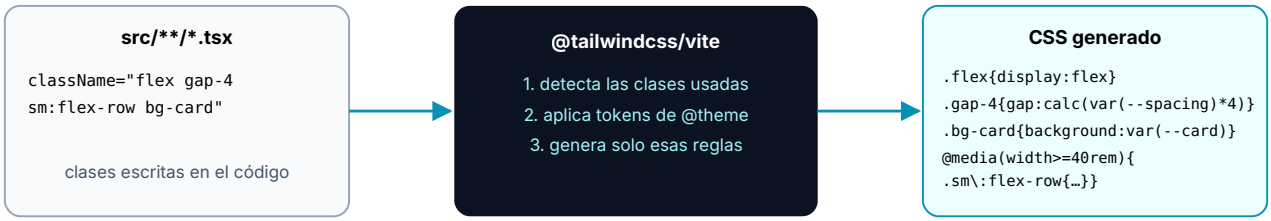


Figura 10.1. Tailwind genera el CSS a partir de las clases que encuentra en el código.

10.5.1 Configuración en CSS, no en JavaScript

Hasta la versión 3, Tailwind se configuraba con un fichero `tailwind.config.js`. **Tailwind 4 se configura directamente en CSS**, con directivas como `@theme`. Por eso el proyecto no tiene `tailwind.config.js`, y en `components.json` (la configuración de shadcn/ui) la entrada `tailwind.config` está vacía. Toda la configuración vive en `src/index.css`.

10.5.2 La consecuencia más importante: clases completas

Como Tailwind busca las clases **como texto** en el código fuente, **no puede ver clases construidas dinámicamente**:

```
// MAL: Tailwind no encuentra «bg-emerald-50» en el código; la clase no se genera
const color = 'emerald'
<span className={`bg-${color}-50`} />

// BIEN: las clases aparecen completas en el código
const PILL_TONES: Record<Tone, { pill: string; dot: string }> = {
  success: { pill: 'bg-emerald-50 text-emerald-700 ring-emerald-600/20', dot: 'bg-emerald-500' },
  neutral: { pill: 'bg-slate-100 text-slate-600 ring-slate-500/20', dot: 'bg-slate-400' },
  // …
}
```

El segundo fragmento es real (`src/components/common/badges.tsx`). Por eso los tonos de los indicadores se guardan en objetos con las cadenas de clases completas, y el componente elige la entrada según la prop `tone`.

> **En el proyecto**
>
> El mismo principio explica por qué `LevelBadge` usa la propiedad `style` para la altura de cada barra: `style={{ height: 4 + n * 2 }}`. Es un valor **calculado** en ejecución, y para eso las clases de Tailwind no sirven. Regla práctica: clases de Tailwind para todo lo que se conoce al escribir el código; `style` solo para valores que dependen de datos.

10.6 El tema: tokens de diseño en `index.css`

↑ Índice

Las clases como `bg-card`, `text-muted-foreground` o `bg-sidebar` no existen en Tailwind por defecto: las define el proyecto. Es el **sistema de tokens de diseño**, el corazón visual de la aplicación, y está en `src/index.css`.

10.6.1 Capa 1: las variables CSS

`src/index.css (extracto)`

```
:root {
  --radius: 0.625rem;

  --background: oklch(0.985 0.003 247);
  --foreground: oklch(0.21 0.035 262);

  /* Primario: azul oscuro corporativo */
  --primary: oklch(0.32 0.075 258);
  --primary-foreground: oklch(0.985 0.003 247);

  /* Acento tecnológico: cian */
```

```
  --brand: oklch(0.78 0.13 210);
  --brand-strong: oklch(0.62 0.12 220);

  --destructive: oklch(0.58 0.22 27);
  --border: oklch(0.915 0.01 252);

  /* Navegación lateral */
  --sidebar: oklch(0.2 0.04 262);
  --sidebar-foreground: oklch(0.86 0.02 250);
  /* … */
}
```

Son **variables CSS** (*custom properties*), definidas en `:root` para que estén disponibles en toda la página. Los nombres siguen la convención de shadcn/ui: cada color de fondo tiene su pareja `-foreground`, el color del texto que va encima (`--primary` y `--primary-foreground`). Así siempre se combinan colores con contraste suficiente.

Los colores están en **OKLCH**, un espacio de color moderno con tres componentes: **L**uminosidad (de 0 a 1), **C**roma (intensidad del color) y **H**ue (tono, en grados). Su ventaja sobre el hexadecimal es que es **perceptualmente uniforme**: dos colores con la misma L se perciben igual de claros, lo que facilita crear variantes coherentes. Fíjate en que casi todos los grises del proyecto tienen un tono de 247 a 262: son grises ligeramente azulados, a juego con el azul corporativo.

10.6.2 Capa 2: el puente con Tailwind

`src/index.css (extracto)`

```
@theme inline {
  --font-sans: 'Inter Variable', ui-sans-serif, system-ui, -apple-system, 'Segoe UI', sans-serif;

  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);

  --color-background: var(--background);
  --color-primary: var(--primary);
  --color-brand-strong: var(--brand-strong);
  --color-sidebar: var(--sidebar);
  /* … */
}
```

La directiva `@theme` declara **tokens de Tailwind**. Cada variable con prefijo `--color-` crea toda una familia de utilidades: `--color-sidebar` habilita `bg-sidebar`, `text-sidebar`, `border-sidebar`, `ring-sidebar`… y con opacidad, `bg-sidebar/80`. Igual con `--radius-*` (`rounded-lg`) y `--font-*` (`font-sans`). La palabra `inline` indica que las utilidades usen directamente el valor de la variable (`var(--sidebar)`), de modo que si una variable de `:root` cambia en ejecución, todo lo que la usa cambia con ella.



Figura 10.2. La cadena de un token: variable CSS → token del tema → clases de utilidad.

10.6.3 Capa 3: los estilos base

src/index.css (extracto)

```
@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground antialiased;
  }
}
```

Estilos por defecto para toda la página: color de borde, color del contorno de foco, fondo y color de texto del `body`. La directiva `@apply` permite usar clases de Tailwind dentro de CSS. `@layer base` coloca estas reglas en la capa de menor prioridad, de modo que cualquier utilidad en el JSX las sobrescribe.

10.6.4 Las importaciones y el modo oscuro

src/index.css (extracto)

```
@import 'tailwindcss';
@import 'tw-animate-css';
@import '@fontsource-variable/inter';

@custom-variant dark (&:is(.dark *));
```

La primera línea trae Tailwind; la segunda, las animaciones de entrada y salida que usan los componentes de Radix (sección 10.8); la tercera, la fuente Inter servida desde el propio proyecto, sin depender de Google Fonts. La última define la variante `dark:` para que se active cuando un antepasado tenga la clase `dark`.

En el proyecto

El proyecto tiene preparada la variante `dark:`, pero **no define todavía los valores oscuros** de los tokens. Añadir el modo oscuro consistiría en redefinir las variables dentro de un selector `.dark { --background: …; --foreground: …; }` y añadir un conmutador que ponga o quite la clase `dark` en `<html>`. Gracias a `@theme inline`, ningún componente tendría que cambiar. Es el ejercicio 10.3.

Si vienes de Java…

Los tokens son el equivalente de externalizar la configuración en `application.properties` en lugar de dejar literales repartidos por el código: el componente dice `bg-primary` («el color primario»), no `bg-[#1e3a5f]`. Cambiar la marca es cambiar una variable.

10.7 Componer clases: `cn()`

↑ Índice

Casi todos los componentes aceptan una prop `className` para que quien los usa pueda añadir o ajustar estilos. Eso plantea un problema: ¿qué pasa si el componente tiene `px-4` y quien lo usa pasa `px-2`? En CSS, ambas clases tienen la misma especificidad y gana **la que aparece después en la hoja de estilos**, no la que escribiste después en el atributo. El resultado sería impredecible.

La función `cn` resuelve esto:

src/lib/utils.ts

```ts
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

- `clsx` construye una cadena a partir de valores variados, descartando los falsos: `clsx('a', false && 'b', isActive && 'c', undefined)` devuelve `'a c'` si `isActive` es verdadero. Permite escribir clases condicionales con `&&`.
- `twMerge` entiende Tailwind y **resuelve conflictos**: si dos clases afectan a la misma propiedad, se queda con la última. `twMerge('px-4 py-2', 'px-2')` devuelve `'py-2 px-2'`.

El patrón aparece en todos los componentes:

src/components/common/state-views.tsx

```tsx
<div className={cn('flex flex-col items-center justify-center gap-3 py-16 text-muted-foreground', className)}>
```

Las clases del componente van primero y el `className` recibido al final, de modo que quien usa el componente siempre puede sobrescribirlas. El router lo aprovecha al usar `<LoadingState className="min-h-svh" />` como pantalla de carga inicial a pantalla completa.

10.8 Animaciones y transiciones

↑ Índice

El proyecto usa tres mecanismos:

- **Transiciones** (`transition-colors`, `transition-transform`, `transition-opacity`): animan el cambio de una propiedad cuando cambia una clase, como el color al pasar el ratón o la opacidad de la tabla mientras carga otra página (`isFetching && 'opacity-60'`).
- **Animaciones de Tailwind** (`animate-spin`, `animate-pulse`): el icono de carga que gira y los esqueletos que palpitan.
- **Animaciones de entrada y salida** de `tw-animate-css`: clases como `animate-in fade-in-0 zoom-in-95` o `slide-in-from-left`, que los componentes de shadcn/ui combinan con `data-[state=open]:` y `data-[state=closed]:` para animar desplegables, *tooltips* y el panel lateral móvil.

10.9 Decisión y alternativas

↑ Índice

Decisión de diseño: Tailwind CSS 4 con tokens propios

Decisión. Estilos con Tailwind CSS 4, integrado mediante su plugin de Vite, con un tema de tokens propio definido en CSS y siguiendo la convención de nombres de shadcn/ui.

Motivos.

- Estilos junto al componente: nada de hojas de estilos globales que crecen sin control.
- Un sistema de diseño **impuesto por la herramienta**: escala de espaciado, tipografía y colores comunes.
- CSS final pequeño, porque solo incluye lo usado.
- Es la base de shadcn/ui (capítulo 11), que da componentes accesibles listos para personalizar.
- Las variantes (`hover:` , `aria-invalid:` , `data-[state=open]:`) resuelven en una clase lo que en CSS requiere selectores complejos.

Inconvenientes asumidos. Cadenas de clases largas en algunos componentes y una curva inicial para memorizar las abreviaturas, mitigada con la extensión del editor (autocompletado y vista previa del CSS al pasar el ratón).

Alternativas. CSS Modules (estilos locales, pero sin sistema de diseño); CSS-in-JS como styled-components o Emotion (coste en ejecución y peor encaje con React moderno); librerías de componentes con estilos propios como MUI o Ant Design (rápidas para empezar, difíciles de adaptar a una identidad visual propia).

10.10 Errores típicos y depuración

↑ Índice

- **Una clase no tiene efecto.** Comprueba que existe (la extensión del editor la colorea y muestra su CSS) y que no la construyes dinámicamente (sección 10.5). En la pestaña *Elements* de las DevTools verás si la clase está en el elemento y si su regla CSS existe.
- **Un color propio no existe** (`bg-brand-soft`). Falta declarar `--color-brand-soft` en `@theme inline` y su variable en `:root` .
- **Dos clases en conflicto.** Si combinas clases fuera de `cn()` , gana la que Tailwind generó después, no la que escribiste después. Usa siempre `cn()` para mezclar clases de distinto origen.
- **El diseño se rompe en móvil.** Prueba con el modo de dispositivo de las DevTools (Ctrl+Shift+M) y revisa qué clases carecen de versión sin prefijo.
- **Contenido desbordado en una celda o un título largo.** Faltan `min-w-0` en el contenedor *flex* o `truncate` en el texto: los hijos de un contenedor *flex* no encogen por debajo de su contenido sin `min-w-0` .

10.11 Ejercicios

↑ Índice

Ejercicio 10.1 · Traduce clases a CSS Ver solución →

Lee la cabecera de la aplicación (`src/components/layout/app-header.tsx`) y explica con palabras qué hace cada clase de este fragmento: `sticky top-0 z-30 flex h-16 items-center gap-3 border-b bg-`

`card/85 px-4 backdrop-blur supports-[backdrop-filter]:bg-card/70 lg:px-8`. Comprueba tu interpretación en las DevTools, seleccionando el elemento y desplazando la página.

Ejercicio 10.2 · Añade un tono a los indicadores Ver solución →

Añade un tono `'violet'` al tipo `Tone` y a `PILL_TONES` en `src/components/common/badges.tsx`, con fondo violeta claro y texto violeta oscuro. Úsalo temporalmente en `STATUS_TONES` para el estado `INACTIVE`. Después prueba a construir la clase de forma dinámica (`` `bg-${color}-50` ``) y explica por qué no funciona.

Ejercicio 10.3 · Prepara el modo oscuro Ver solución →

En `index.css`, añade un bloque `.dark { … }` que redefina al menos `--background`, `--foreground`, `--card`, `--card-foreground`, `--muted`, `--muted-foreground` y `--border` con colores oscuros en OKLCH (luminosidades entre 0,15 y 0,3 para los fondos). Añade la clase `dark` a `<html>` desde las DevTools y observa el resultado. ¿Qué partes de la interfaz no cambian y por qué? (Pista: busca colores escritos directamente, como `bg-emerald-50` o `text-white`.)

10.12 Autoevaluación

↑ Índice

Autoevaluación del capítulo 10 Ver respuestas →

1. ¿Qué problemas de CSS tradicional intenta resolver el enfoque de clases de utilidad?
2. ¿Cuánto mide `p-4` y por qué conviene trabajar con una escala?
3. ¿Qué significa `sm:flex-row`? ¿Cómo harías que algo solo se vea en móvil?
4. ¿Cómo consigue la flecha del menú girar al abrir un grupo sin JavaScript?
5. ¿Por qué Tailwind no puede generar `` `bg-${color}-50` ``?
6. Describe las tres capas del sistema de tokens de `index.css`.
7. ¿Qué ventaja tiene OKLCH frente al hexadecimal para definir una paleta?
8. ¿Qué problema resuelve `twMerge` que no resuelve `clsx`?
9. ¿Cuándo es correcto usar la propiedad `style` en lugar de clases?

CAPÍTULO 11

shadcn/ui y Radix: componentes accesibles que son tuyos

↑ Volver al índice

EN ESTE CAPÍTULO

11.1 El problema de las librerías de componentes clásicas
11.2 Radix UI: comportamiento y accesibilidad sin estilos
11.3 shadcn/ui: un catálogo que se copia
11.4 Anatomía de un componente sencillo: `Button`
11.5 Anatomía de un componente compuesto: `Select`
11.6 Los componentes del proyecto
11.7 Iconos, notificaciones y fuente
11.8 Mantener componentes copiados
11.9 Errores típicos y depuración
11.10 Ejercicios
11.11 Autoevaluación

Botones, desplegables, tarjetas, paneles deslizantes o *tooltips* no se han escrito desde cero ni vienen de una librería cerrada. Vienen de shadcn/ui, un catálogo de componentes que se **copian** al proyecto, construidos sobre los primitivos accesibles de Radix y estilados con Tailwind. Este capítulo explica ese modelo, la anatomía de sus componentes y cómo se mantienen.

11.1 El problema de las librerías de componentes clásicas

↑ Índice

Las librerías tradicionales como MUI, Ant Design o PrimeReact ofrecen cientos de componentes listos. Son muy productivas al principio, pero tienen un coste que aparece con el tiempo:

- **Estilos difíciles de cambiar.** El componente trae su propio diseño y su propio sistema de estilos. Adaptarlo a una identidad visual distinta exige sobrescribir estilos internos, a menudo con selectores frágiles.
- **Caja negra.** Si un comportamiento no encaja, no puedes cambiarlo: dependes de las opciones que la librería haya previsto.
- **Actualizaciones mayores costosas.** Una versión nueva puede cambiar APIs y estilos de todos los componentes a la vez.

Al mismo tiempo, escribir **desde cero** un desplegable accesible es muchísimo trabajo: navegación con flechas y búsqueda por teclado, gestión del foco, cierre con *Escape* o al hacer clic fuera, posicionamiento cuando no cabe en pantalla, atributos ARIA para lectores de pantalla… Es la parte difícil, y casi nadie la hace bien a la primera.

La combinación de **Radix** y **shadcn/ui** separa ambas cosas: el **comportamiento** difícil lo aporta una librería especializada, y la **apariencia** es código tuyo.

11.2 Radix UI: comportamiento y accesibilidad sin estilos

↑ Índice

Radix UI es una librería de componentes ***headless*** (sin cabeza, es decir, sin aspecto): implementan el comportamiento, la accesibilidad y la gestión del estado de los patrones de interfaz complejos, pero **no traen ningún estilo**.

Cada componente de Radix es una familia de piezas que se componen (capítulo 9):

```
<Select.Root value={…} onValueChange={…}>
  <Select.Trigger>          {/* botón que abre */}
    <Select.Value />        {/* texto del valor seleccionado */}
  </Select.Trigger>
  <Select.Portal>           {/* renderiza el contenido al final del <body> */}
    <Select.Content>
      <Select.Viewport>
        <Select.Item value="ACTIVE"><Select.ItemText>Activa</Select.ItemText></Select.Item>
      </Select.Viewport>
    </Select.Content>
  </Select.Portal>
</Select.Root>
```

Lo que Radix resuelve por ti en este desplegable:

- Apertura con clic, con *Intro*, con *Espacio* o con las flechas; cierre con *Escape* o al hacer clic fuera.
- Navegación con flechas y selección por la primera letra al teclear.
- Devolver el foco al botón al cerrar.
- Roles y atributos ARIA (`role="combobox"`, `aria-expanded`, `aria-controls`…).
- Posicionar la lista donde quepa y desplazarla si es larga.

Además, Radix publica su estado en **atributos `data-*`** de los elementos que pinta: `data-state="open"`, `data-side="bottom"`, `data-placeholder`, `data-disabled`. Son el puente con Tailwind: las variantes `data-[state=open]:` del capítulo 10 los usan para estilar y animar cada estado.

11.2.1 Los portales

`Select.Portal` merece una explicación. El contenido de un desplegable, un *tooltip* o un panel lateral se renderiza **fuera** de su posición en el árbol del DOM, al final del `<body>`, aunque en el árbol de React siga siendo hijo de su componente. Así no lo recorta un contenedor con `overflow: hidden` (como la tarjeta del listado, con `overflow-hidden`) ni queda debajo de otro elemento por culpa de los `z-index`. Los eventos y el contexto de React siguen funcionando como si estuviera en su sitio.

> **En el proyecto**
>
> El proyecto usa el paquete unificado `radix-ui`, que agrupa todos los primitivos y se importa por nombre: `import { Select as SelectPrimitive } from 'radix-ui'`. Es lo que vimos en el capítulo 3 con `npm explain`: `@radix-ui/react-select` llega como dependencia de `radix-ui`.

11.3 shadcn/ui: un catálogo que se copia

↑ Índice

shadcn/ui no es una librería que se instala. **No aparece en `package.json`.** Es un catálogo de componentes, construidos con Radix y Tailwind, cuyo **código fuente se copia en tu proyecto** con una herramienta de línea de comandos:

```
npx shadcn@latest add dialog
```

El comando descarga `dialog.tsx`, lo adapta a la configuración del proyecto (alias, estilo, iconos), lo escribe en `src/components/ui/` e instala las dependencias que necesite. A partir de ese momento, **el componente es código del proyecto**: lo lees, lo modificas y lo versionas como cualquier otro fichero.

La configuración de la herramienta está en `components.json`:

components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/index.css",
    "baseColor": "slate",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "iconLibrary": "lucide"
}
```

Clave	Significado
`style: "new-york"`	Variante visual del catálogo: más compacta y definida que la original.
`rsc: false`	El proyecto no usa componentes de servidor de React (es una SPA).
`tsx: true`	Genera TypeScript.
`tailwind.css`	Dónde están los tokens y los estilos globales.
`tailwind.config: ""`	Vacío: Tailwind 4 se configura en CSS (capítulo 10).
`baseColor`, `cssVariables`	Paleta de partida y uso de variables CSS para los colores (los tokens de `index.css`).
`aliases`	Dónde escribir componentes, utilidades y hooks, usando el alias `@/`.
`iconLibrary: "lucide"`	Los componentes usan los iconos de `lucide-react`.

> **Si vienes de Java...**
>
> Piensa en un arquetipo de Maven o en el código que genera Spring Initializr: te da un punto de partida bien hecho y, a partir de ahí, el código es tuyo. Lo opuesto sería una dependencia binaria que solo puedes configurar por las opciones que expone. shadcn/ui aplica el modelo del arquetipo a cada componente de interfaz.

11.4 Anatomía de un componente sencillo: `Button`

↑ Índice

`src/components/ui/button.tsx`

```
const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium
transition-all cursor-pointer disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-
events-none [&_svg:not([class*='size-'])]:size-4 shrink-0 [&_svg]:shrink-0 outline-none focus-
visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px] aria-invalid:ring-
destructive/20 aria-invalid:border-destructive",
  {
    variants: {
      variant: {
        default: 'bg-primary text-primary-foreground shadow-xs hover:bg-primary/90',
        brand: 'bg-brand-strong text-white shadow-xs hover:bg-brand-strong/90',
        destructive: 'bg-destructive text-white shadow-xs hover:bg-destructive/90 focus-
visible:ring-destructive/20',
        outline: 'border bg-background shadow-xs hover:bg-accent hover:text-accent-foreground',
        secondary: 'bg-secondary text-secondary-foreground shadow-xs hover:bg-secondary/80',
        ghost: 'hover:bg-accent hover:text-accent-foreground',
        link: 'text-primary underline-offset-4 hover:underline',
      },
      size: {
        default: 'h-9 px-4 py-2 has-[>svg]:px-3',
        sm: 'h-8 rounded-md gap-1.5 px-3 has-[>svg]:px-2.5',
        lg: 'h-10 rounded-md px-6 has-[>svg]:px-4',
        icon: 'size-9',
        'icon-sm': 'size-8',
      },
    },
    defaultVariants: { variant: 'default', size: 'default' },
  },
)

function Button({
  className, variant, size, asChild = false, ...props
}: React.ComponentProps<'button'> & VariantProps<typeof buttonVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot.Root : 'button'
  return (
    <Comp data-slot="button" className={cn(buttonVariants({ variant, size, className }))}
{...props} />
  )
}

export { Button, buttonVariants }
```

Cinco ideas en treinta líneas:

1. `cva` (***class-variance-authority***) define un catálogo de variantes. El primer argumento son las clases comunes; `variants` asocia cada valor de cada variante con sus clases; `defaultVariants` fija los

valores por defecto. `buttonVariants({ variant: 'outline', size: 'sm' })` devuelve la cadena de clases combinada.

2. **`VariantProps<typeof buttonVariants>`** deriva los tipos de las props de las variantes: `variant` solo admite `'default' | 'brand' | 'destructive' | …`. Si escribes `variant="primario"`, no compila.
3. **`React.ComponentProps<'button'>`** hace que `Button` acepte **todas** las props de un `<button>` nativo (`type`, `disabled`, `onClick`, `aria-label`…), que se propagan con `{...props}`. El componente se usa exactamente como un botón HTML con superpoderes.
4. **`asChild`** con `Slot` presta todo lo anterior a otro elemento, como un `Link` (capítulo 9).
5. **`data-slot="button"`** marca el elemento para que otros componentes puedan estilarlo desde fuera con selectores como `*:data-[slot=…]`.

> **En el proyecto**
>
> La variante `brand` y el tamaño `icon-sm` **no existen en el catálogo original**: los añadió el proyecto. `brand` es el botón cian de acción principal (*Nueva aplicación*) e `icon-sm` es el tamaño de los botones de la paginación y de las acciones de fila. Del mismo modo, `Alert` tiene variantes `success` e `info` propias, usadas en la confirmación de alta. Es exactamente la ventaja del modelo: el componente se adapta al proyecto, no al revés.

11.5 Anatomía de un componente compuesto: `Select`

↑ Índice

Los componentes interactivos de shadcn/ui son **envoltorios finos** sobre las piezas de Radix, con estilos de Tailwind:

`src/components/ui/select.tsx (extracto)`

```
function SelectTrigger({ className, size = 'default', children, ...props }:
  React.ComponentProps<typeof SelectPrimitive.Trigger> & { size?: 'sm' | 'default' }) {
  return (
    <SelectPrimitive.Trigger
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        'border-input data-[placeholder]:text-muted-foreground … aria-invalid:border-destructive
flex w-fit … rounded-md border bg-card px-3 py-2 text-sm … data-[size=default]:h-9 data-
[size=sm]:h-8 …',
        className,
      )}
      {...props}
    >
      {children}
      <SelectPrimitive.Icon asChild>
        <ChevronDownIcon className="size-4 opacity-50" />
      </SelectPrimitive.Icon>
    </SelectPrimitive.Trigger>
  )
}
```

- El tipo de las props es el de la pieza de Radix (`React.ComponentProps<typeof SelectPrimitive.Trigger>`) más lo que añade el envoltorio (`size`).
- Las clases usan los atributos que publica Radix: `data-[placeholder]:text-muted-foreground` pone el texto en gris cuando aún no hay valor.

- `SelectContent` envuelve el contenido en `SelectPrimitive.Portal` y lo anima con `data-[state=open]:animate-in data-[state=open]:fade-in-0 data-[state=open]:zoom-in-95`.

El resultado se usa en el proyecto sin pensar en nada de esto, por ejemplo en el selector de filas por página de `PaginationBar`:

`src/components/common/pagination-bar.tsx`

```
<Select value={String(size)} onValueChange={(value) => onSizeChange(Number(value))}
disabled={disabled}>
  <SelectTrigger size="sm" className="w-[72px]" aria-label="Filas por página">
    <SelectValue />
  </SelectTrigger>
  <SelectContent>
    {pageSizeOptions.map((option) => (
      <SelectItem key={option} value={String(option)}>{option}</SelectItem>
    ))}
  </SelectContent>
</Select>
```

Fíjate en las conversiones `String(size)` y `Number(value)`: el `Select` de Radix trabaja siempre con **cadenas**, así que los números se convierten al entrar y al salir. Por la misma razón, `FilterSelect` usa el valor especial `'__ALL__'` para la opción *Todos*: Radix no admite un ítem con valor vacío.

11.6 Los componentes del proyecto

↑ Índice

Componente (`src/components/ui`)	Base	Dónde se usa
`button`	HTML + `Slot` de Radix	Todas las acciones
`badge`	HTML	Disponible; los indicadores propios están en `common/badges`
`card`	HTML	Contenedor de listados, formularios y fichas
`table`	HTML	Listados de aplicaciones y demandas
`input`, `textarea`, `label`	HTML / `Label` de Radix	Formularios y buscadores
`select`	`Select` de Radix	Filtros, paginación, campos de criticidad, estado, prioridad y aplicación
`form`	React Hook Form + contexto (capítulo 9)	Todos los formularios
`alert`	HTML	Errores al guardar y confirmaciones
`separator`	`Separator` de Radix	Separación entre filtros y tabla
`skeleton`	HTML	Esqueletos de carga
`collapsible`	`Collapsible` de Radix	Grupos del menú lateral
`sheet`	`Dialog` de Radix	Menú lateral en móvil
`tooltip`	`Tooltip` de Radix	Acciones de fila con icono

Componente (`src/components/ui`)	Base	Dónde se usa
`sonner`	Librería `sonner`	Notificaciones emergentes

Dos curiosidades: `Sheet` es en realidad un **diálogo modal** de Radix (`Dialog`) con estilos de panel lateral, así que hereda gratis la captura del foco dentro del panel, el cierre con *Escape* y el bloqueo del desplazamiento de la página. Y `Badge` está instalado pero apenas se usa, porque los indicadores de estado y nivel del proyecto (`StatusPill`, `LevelBadge`) tienen un diseño propio en `components/common/badges.tsx`.

11.7 Iconos, notificaciones y fuente

↑ Índice

Iconos. `lucide-react` ofrece más de mil iconos como componentes de React: `<Plus />`, `<Pencil />`, `<Loader2 className="animate-spin" />`. Son SVG que heredan el color del texto (`currentColor`), se dimensionan con clases (`size-4`) y, al importarse por nombre, solo los usados acaban en el *bundle*. Los botones de shadcn/ui ya dan tamaño por defecto a los iconos que contienen (`[&_svg:not([class*='size-'])]:size-4`).

Notificaciones. La librería `sonner` muestra los mensajes emergentes. El contenedor `<Toaster>` está en `App` y se configura en `components/ui/sonner.tsx` con colores ricos (`richColors`), botón de cierre y los tokens del tema. Desde cualquier sitio:

`src/features/applications/pages/application-edit-page.tsx`

```
toast.success('Cambios guardados', { description: `«${result.name}» se ha actualizado.` })
```

Fuente. `@fontsource-variable/inter` incluye la fuente Inter en el propio *bundle*, como fichero `woff2`, y la declara con `@font-face`. Es una fuente **variable**: un único fichero contiene todos los grosores. Servirla desde el propio proyecto evita depender de un servicio externo (algo relevante en redes corporativas) y no envía datos de los usuarios a terceros. En `index.css`, `font-feature-settings: 'cv11', 'ss01'` activa variantes tipográficas de Inter más legibles en interfaces.

11.8 Mantener componentes copiados

↑ Índice

El modelo de shadcn/ui tiene una contrapartida: como el código es tuyo, **las mejoras del catálogo no llegan solas**. Algunas pautas para convivir con ello:

- **No metas lógica de negocio en `components/ui`.** Esa carpeta contiene piezas genéricas de interfaz. Lo que sabe de aplicaciones o demandas va en `components/common` o en el módulo correspondiente (capítulo 13).
- **Personaliza con intención y documenta.** Añadir variantes (como `brand`) es lo esperado. Cambios profundos de comportamiento conviene hacerlos en un componente propio que envuelva al de `ui`.
- **Para actualizar un componente**, ejecuta de nuevo `npx shadcn@latest add <componente>` en una rama, compara las diferencias con Git y vuelve a aplicar tus personalizaciones.
- **Las dependencias reales** (`radix-ui`, `cva`, `lucide-react`) sí se actualizan con npm como cualquier otra.

Decisión de diseño: shadcn/ui sobre Radix en lugar de una librería de componentes completa

Decisión. Componentes de interfaz de shadcn/ui, copiados al proyecto, sobre los primitivos de Radix y estilados con Tailwind y los tokens propios.

Motivos. Accesibilidad y comportamiento de nivel profesional sin escribirlos; control total sobre el aspecto, coherente con la identidad visual (azul oscuro, pizarra y cian); sin dependencia de un sistema de estilos ajeno; y solo se incluye lo que se usa.

Inconvenientes asumidos. Hay que mantener el código copiado y no existe un componente para todo: tablas avanzadas (con columnas redimensionables, agrupación…) o selectores de fecha requieren combinar piezas o librerías específicas, como TanStack Table.

Alternativas. MUI o Ant Design (catálogo enorme, difícil de personalizar a fondo); Mantine o Chakra UI (más personalizables, pero con su propio sistema de estilos); escribir los componentes desde cero (control total, coste y riesgo de accesibilidad altísimos).

11.9 Errores típicos y depuración

↑ Índice

- **Un desplegable o *tooltip* aparece recortado o detrás de otro elemento.** Comprueba que el contenido está dentro de su `Portal`. En *Elements* deberías verlo al final del `<body>`.
- **`A <Select.Item /> must have a value prop that is not an empty string`.** Radix no admite valores vacíos. Usa un valor especial, como hace `FilterSelect` con `'__ALL__'`.
- **El valor del `Select` llega como texto.** Siempre es una cadena: convierte con `Number()` cuando el dato sea numérico.
- **Aviso en consola de `DialogContent` sin título.** Radix exige un título accesible en los diálogos. Si no debe verse, se oculta visualmente con `sr-only`, como hace la cabecera con `SheetTitle` y `SheetDescription`.
- **`asChild` no funciona.** El hijo debe ser un único elemento capaz de recibir props y una `ref` (un componente de función normal de React 19 la recibe como prop).

Para inspeccionar el estado de Radix, mira en *Elements* los atributos `data-state`, `aria-expanded` y `aria-controls` de los disparadores: es la forma más rápida de saber si el componente cree estar abierto o cerrado.

11.10 Ejercicios

↑ Índice

Ejercicio 11.1 · Añade un componente del catálogo Ver solución →

Añade el componente `dialog` con `npx shadcn@latest add dialog` y revisa el fichero generado en `src/components/ui/dialog.tsx`: ¿qué primitivo de Radix envuelve y qué piezas exporta? Úsalo para mostrar un diálogo de confirmación al pulsar *Descartar* en la página de modificación de una aplicación cuando hay cambios sin guardar. Pista: la propiedad `open` del diálogo puede venir de un `useState`.

Ejercicio 11.2 · Crea una variante de botón Ver solución →

Añade una variante `success` al botón (fondo verde con el token `--success`, texto blanco) y úsala temporalmente en *Guardar cambios*. ¿Qué tipo infiere TypeScript ahora para la prop `variant`? Comprueba que el editor te la ofrece al autocompletar.

Ejercicio 11.3 · Comprueba la accesibilidad con el teclado Ver solución →

Sin usar el ratón, recorre el listado de aplicaciones con *Tab*: abre el filtro de estado con *Intro*, muévete con las flechas, elige una opción y continúa hasta la paginación. Después abre el menú móvil (reduce la ventana) y comprueba que el foco queda atrapado dentro del panel y que *Escape* lo cierra. Inspecciona los atributos `aria-*` del disparador del filtro antes y después de abrirlo.

11.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 11 Ver respuestas →

1. ¿Qué significa que Radix sea una librería *headless*?
2. Enumera tres comportamientos de un desplegable que Radix resuelve por ti.
3. ¿Para qué sirve un portal y por qué lo necesitan los desplegables?
4. ¿Por qué shadcn/ui no aparece en `package.json`?
5. ¿Qué hacen `cva` y `VariantProps` en `button.tsx`?
6. ¿Por qué `Button` acepta `type`, `disabled` u `onClick` sin declararlos uno a uno?
7. ¿Por qué el `Select` de la paginación convierte el tamaño con `String()` y `Number()`?
8. ¿Qué componente de Radix hay detrás de `Sheet` y qué ventajas hereda?
9. ¿Qué contrapartida tiene copiar los componentes en lugar de instalarlos?

CAPÍTULO 12

El layout y el sistema visual

↑ Volver al índice

EN ESTE CAPÍTULO

12.1 La estructura de la pantalla
12.2 `AppLayout`: el armazón
12.3 El menú lateral declarativo
12.4 La cabecera y el menú móvil
12.5 La marca
12.6 El sistema visual
12.7 Los componentes de estado
12.8 Errores típicos y depuración
12.9 Ejercicios
12.10 Autoevaluación

Con Tailwind y los componentes de shadcn/ui como materiales, este capítulo recorre cómo está construida la «carcasa» de la aplicación —menú lateral, cabecera, área de contenido y su versión móvil— y qué patrones visuales comparten todas las pantallas. Es el capítulo que explica por qué la aplicación se ve coherente aunque cada pantalla se haya escrito por separado.

12.1 La estructura de la pantalla

↑ Índice

Todas las pantallas comparten el mismo armazón, definido una sola vez en `AppLayout`:

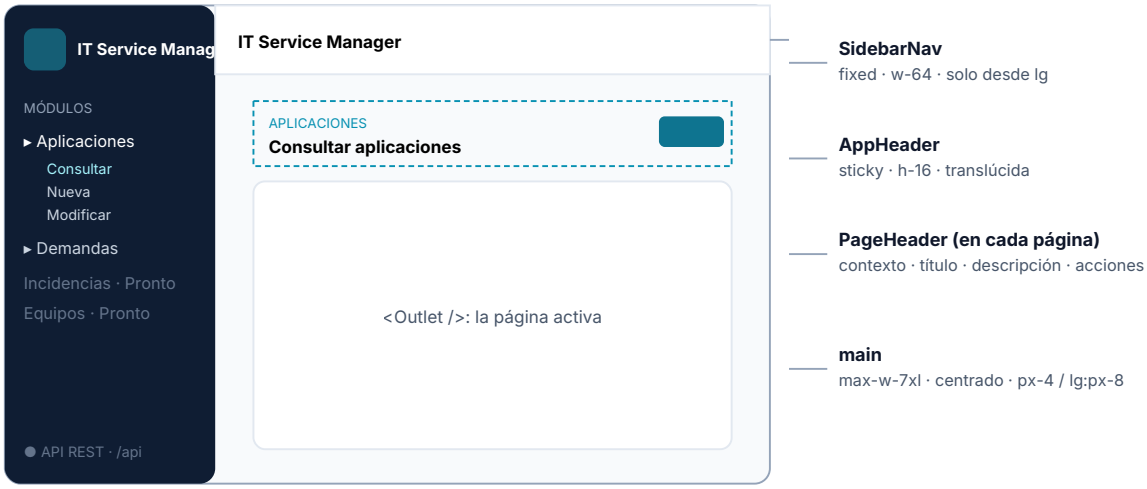


Figura 12.1. Las zonas fijas del layout y la zona variable donde React Router pinta cada página.

12.2 `AppLayout`: el armazón

↑ Índice

`src/components/layout/app-layout.tsx`

```tsx
export function AppLayout() {
  useDocumentTitle()
  return (
    <div className="min-h-svh">
      <aside className="fixed inset-y-0 left-0 z-40 hidden w-64 lg:block">
        <SidebarNav />
```

```
      </aside>
      <div className="lg:pl-64">
        <AppHeader />
        <main className="mx-auto w-full max-w-7xl px-4 py-6 lg:px-8 lg:py-8">
          <Outlet />
        </main>
      </div>
    </div>
  )
}
```

Ya conoces cada pieza: las clases responsive (capítulo 10), el hook `useDocumentTitle` (capítulo 8) y el `Outlet` de React Router, el hueco donde se pinta la ruta hija activa (Parte VII). Tres detalles de diseño:

- **El menú es `fixed`**: no se desplaza con el contenido. En una aplicación de gestión, la navegación siempre visible ahorra movimientos.
- **`min-h-svh`** (*small viewport height*) usa la altura visible real en móviles, donde la barra del navegador aparece y desaparece; `100vh` provocaría saltos.
- **El ancho del contenido está limitado** a `max-w-7xl` (80 rem). En monitores muy anchos, las tablas y formularios de ancho completo son difíciles de leer.

12.3 El menú lateral declarativo

↑ Índice

El menú está dividido en **datos** y **presentación**.

12.3.1 Los datos: `navigation.ts`

`src/components/layout/navigation.ts`

```
export interface NavLeaf {
  label: string
  to: string
  /** Si es true, solo se marca activa con coincidencia exacta de ruta. */
  end?: boolean
}

export interface NavModule {
  id: string
  label: string
  icon: LucideIcon
  /** Rutas cuyo prefijo pertenece al módulo (para abrir el grupo automáticamente). */
  basePath?: string
  items?: NavLeaf[]
  /** Módulos previstos pero todavía no implementados. */
  comingSoon?: boolean
}

export const NAV_MODULES: NavModule[] = [
  {
    id: 'applications', label: 'Aplicaciones', icon: AppWindow, basePath: '/aplicaciones',
    items: [
      { label: 'Consultar aplicaciones', to: '/aplicaciones', end: true },
      { label: 'Nueva aplicación', to: '/aplicaciones/nueva', end: true },
      { label: 'Modificar aplicación', to: '/aplicaciones/modificar', end: true },
    ],
  },
  { id: 'demands', label: 'Demandas', icon: ClipboardList, basePath: '/demandas', items: [ /* …
*/ ] },
```

```
  { id: 'incidents', label: 'Incidencias', icon: TriangleAlert, comingSoon: true },
  { id: 'teams', label: 'Equipos', icon: Users, comingSoon: true },
  { id: 'settings', label: 'Configuración', icon: Settings, comingSoon: true },
]
```

Es una lista de objetos tipados. No hay JSX ni lógica: solo la **estructura de navegación** de la aplicación, con los módulos previstos marcados como `comingSoon`.

12.3.2 La presentación: `SidebarNav`

`SidebarNav` recorre esa lista y decide, para cada módulo, si pinta un grupo desplegable o un elemento «Pronto»:

`src/components/layout/sidebar-nav.tsx (extracto)`

```
<nav className="flex-1 overflow-y-auto px-3 py-5" aria-label="Navegación principal">
  <ul className="space-y-1">
    {NAV_MODULES.map((module) => (
      <li key={module.id}>
        {module.comingSoon ? (
          <ComingSoonItem module={module} />
        ) : (
          <ModuleGroup module={module} onNavigate={onNavigate} />
        )}
      </li>
    ))}
  </ul>
</nav>
```

Cada grupo es un `Collapsible` de Radix con su propio estado de apertura:

`src/components/layout/sidebar-nav.tsx (extracto)`

```
function ModuleGroup({ module, onNavigate }: { module: NavModule; onNavigate?: () => void }) {
  const location = useLocation()
  const isInModule = !!module.basePath && location.pathname.startsWith(module.basePath)
  const [open, setOpen] = useState(true)
  const Icon = module.icon

  return (
    <Collapsible open={open} onOpenChange={setOpen}>
      <CollapsibleTrigger className={cn('group flex w-full …', isInModule && 'text-white')}>
        <Icon className={cn('size-4 shrink-0', isInModule ? 'text-cyan-300' : 'text-sidebar-
muted')} />
        <span className="flex-1 text-left">{module.label}</span>
        <ChevronRight className="… group-data-[state=open]:rotate-90" />
      </CollapsibleTrigger>
      <CollapsibleContent className="data-[state=closed]:animate-collapsible-up data-
[state=open]:animate-collapsible-down overflow-hidden">
        <ul className="mt-1 ml-5 space-y-0.5 border-l border-sidebar-border pl-3">
          {module.items?.map((item) => (
            <li key={item.to}>
              <NavLink to={item.to} end={item.end} onClick={onNavigate} className={({ isActive })
=> cn(/* … */)}>
                {item.label}
              </NavLink>
            </li>
          ))}
        </ul>
      </CollapsibleContent>
    </Collapsible>
```

```
  )
}
```

- `isInModule` es un valor derivado de la URL actual: resalta el icono y el nombre del módulo en el que está el usuario, aunque esté en una ruta que no aparece en el menú, como `/aplicaciones/7/editar`.
- `useState(true)`: los grupos empiezan abiertos y el usuario puede plegarlos. Es estado local de interfaz, sin más trascendencia (capítulo 7).
- `NavLink` de React Router es un `Link` que sabe si su ruta está activa y pasa `isActive` a su función de clases. La propiedad `end` exige coincidencia exacta: sin ella, *Consultar aplicaciones* (`/aplicaciones`) aparecería activo también en `/aplicaciones/nueva`, porque es su prefijo.
- `onNavigate` es opcional: en escritorio no hace nada; en móvil cierra el panel al elegir una opción.

Los elementos previstos se pintan desactivados y accesibles:

src/components/layout/sidebar-nav.tsx (extracto)

```
<div className="flex cursor-not-allowed items-center gap-3 … text-sidebar-muted/80"
     aria-disabled="true" title="Módulo previsto, pendiente de implementar">
  <Icon className="size-4 shrink-0" />
  <span className="flex-1 truncate">{module.label}</span>
  <span className="… uppercase">Pronto</span>
</div>
```

Decisión de diseño: Navegación por módulos con acciones explícitas

Decisión. Un menú lateral con un grupo por módulo de negocio y, dentro, las **acciones** del módulo (*Consultar*, *Nueva*, *Modificar*), en lugar de un único enlace por módulo.

Motivos. Refleja la forma de trabajar de una aplicación de gestión de servicios: el usuario va a «dar de alta una demanda» o a «modificar una aplicación», y el menú le lleva directamente. Los módulos futuros se muestran como «Pronto» para comunicar la hoja de ruta sin enlaces rotos. Y al ser declarativo, añadir un módulo es añadir datos.

Alternativas. Navegación superior horizontal (se queda corta con más de cinco o seis módulos); un enlace por módulo con las acciones dentro de cada pantalla (menos clics en el menú, pero las acciones quedan menos visibles).

12.4 La cabecera y el menú móvil

↑ Índice

En pantallas pequeñas no hay sitio para un menú fijo de 16 rem. La cabecera ofrece un botón que abre el mismo menú en un panel deslizante:

src/components/layout/app-header.tsx

```
export function AppHeader() {
  const [mobileOpen, setMobileOpen] = useState(false)

  return (
    <header className="sticky top-0 z-30 flex h-16 items-center gap-3 border-b bg-card/85 px-4
backdrop-blur supports-[backdrop-filter]:bg-card/70 lg:px-8">
      <Button variant="ghost" size="icon" className="lg:hidden" onClick={() => setMobileOpen(true)}
aria-label="Abrir menú">
        <Menu />
      </Button>
      <Sheet open={mobileOpen} onOpenChange={setMobileOpen}>
```

```
        <SheetContent side="left" className="w-72 border-none p-0 text-sidebar-foreground
[&>button]:text-white">
          <SheetTitle className="sr-only">Menú de navegación</SheetTitle>
          <SheetDescription className="sr-only">Módulos de IT Service Manager</SheetDescription>
          <SidebarNav onNavigate={() => setMobileOpen(false)} />
        </SheetContent>
      </Sheet>
      {/* marca en móvil y título de la aplicación */}
    </header>
  )
}
```

Lo más importante es lo que **no** hay: un segundo menú. El panel móvil reutiliza `SidebarNav` tal cual, y la única diferencia es la prop `onNavigate`, que cierra el panel. Cualquier cambio en `navigation.ts` aparece en ambos.

Sobre el aspecto: la cabecera es `sticky` (se queda arriba al desplazar) y semitransparente con desenfoque del fondo (`bg-card/85 backdrop-blur`), un recurso habitual en interfaces modernas que mantiene el contexto del contenido que pasa por debajo. La variante `supports-[backdrop-filter]:bg-card/70` solo aumenta la transparencia si el navegador soporta el desenfoque; si no, se queda más opaca para que el texto siga legible.

12.5 La marca

↑ Índice

El logotipo es un pequeño componente con un SVG en línea:

`src/components/layout/brand.tsx`

```
export function BrandMark({ className }: { className?: string }) {
  return (
    <div className={cn('flex size-9 items-center justify-center rounded-lg bg-gradient-to-br from-
cyan-400/25 to-sky-500/10 ring-1 ring-cyan-400/40', className)}>
      <svg viewBox="0 0 24 24" className="size-5 text-cyan-300" fill="none" aria-hidden="true">
        <path d="M5 7h14M5 12h14M5 17h9" stroke="currentColor" strokeWidth="2.2" strokeLinecap="roun
d" />
        <circle cx="18.5" cy="17" r="1.6" fill="currentColor" />
      </svg>
    </div>
  )
}
```

Al estar en línea y usar `currentColor`, el color del dibujo lo decide la clase `text-cyan-300`, sin ficheros de imagen. Es decorativo (`aria-hidden="true"`): el nombre de la aplicación ya aparece como texto al lado. La cabecera lo reutiliza en móvil con otro fondo: `<BrandMark className="size-8 bg-sidebar" />`.

12.6 El sistema visual

↑ Índice

La coherencia visual no sale de los componentes por sí solos, sino de **un conjunto pequeño de decisiones repetidas** en todas las pantallas.

12.6.1 Paleta

Papel	Token o clase	Uso
Navegación	`bg-sidebar` (azul noche)	Menú lateral: ancla visual oscura de la aplicación
Fondo de página	`bg-background` (pizarra muy claro)	Detrás de las tarjetas
Superficies	`bg-card` (blanco)	Tarjetas, cabecera, desplegables
Texto secundario	`text-muted-foreground`	Descripciones, etiquetas, ayudas
Acción principal	`variant="brand"` (cian)	Un único botón destacado por pantalla: *Nueva aplicación*
Acción por defecto	`variant="default"` (azul oscuro)	Enviar formularios, *Modificar*
Acciones secundarias	`outline`, `ghost`	Volver, ver detalle, limpiar
Estados	verde, gris, ámbar, rojo…	`StatusPill` y `LevelBadge`

El cian se reserva para **lo importante**: la acción principal, el enlace activo del menú, el contexto de la página (*eyebrow*) y el foco. Usado con moderación, guía la vista.

12.6.2 Tipografía

Una única familia (Inter) y una escala corta: títulos de página en `text-2xl font-semibold tracking-tight`; títulos de tarjeta en `font-semibold`; texto general en `text-sm`; etiquetas y contexto en `text-xs uppercase tracking-wide`; identificadores en `font-mono`. Pocas combinaciones, siempre las mismas.

12.6.3 Patrones de página

Las pantallas se construyen con **cuatro plantillas** que se repiten en los dos módulos:

Plantilla	Estructura	Ejemplos
Listado	`PageHeader` con acción principal → `Card` con filtros, `Separator`, tabla y `PaginationBar`	Consultar aplicaciones y demandas
Formulario	`PageHeader` con «Volver» → `Card` con cabecera, campos en rejilla y pie con acciones → resultado del servidor	Alta y modificación
Ficha	`PageHeader` con acciones → `Card` con una lista de definiciones (`<dl>`)	Detalle
Selector	`PageHeader` → tarjeta de búsqueda por ID → tarjeta de catálogo	Modificar (entrada desde el menú)

12.6.4 Indicadores de estado y nivel

Dos componentes comunes resumen visualmente los enumerados del dominio:

- **`StatusPill`**: una píldora con un punto de color para **estados** (activa, abierta, completada…). El tono (`success`, `neutral`, `info`, `brand`, `warning`, `danger`) lo decide cada módulo.

- `LevelBadge`: una etiqueta con **cuatro barras** de altura creciente para **niveles** de 1 a 4 (criticidad, prioridad). Se lee de un vistazo, incluso sin color, lo que lo hace accesible a personas con dificultades para distinguir colores.

Cada módulo traduce sus valores a estos componentes:

`src/features/applications/components/application-badges.tsx`

```
const STATUS_TONES: Record<ApplicationStatus, Tone> = { ACTIVE: 'success', INACTIVE: 'neutral' }

const CRITICALITY_LEVELS: Record<ApplicationCriticality, Level> = { LOW: 1, MEDIUM: 2, HIGH: 3, CRIT
ICAL: 4 }
```

La apariencia vive en `components/common`; el significado de cada valor, en el módulo.

12.7 Los componentes de estado

↑ Índice

El capítulo 6 explicó por qué cada pantalla contempla todos sus estados. Estos son los componentes que lo hacen barato:

Componente	Cuándo	Detalles de diseño
`TableSkeleton`	Primera carga de un listado	Imita la forma de la tabla para que la página no «salte» al llegar los datos; `role="status"`
`LoadingState`	Carga de una ficha o formulario	Icono giratorio y texto; `aria-live="polite"`
`EmptyState`	Consulta correcta sin resultados	Distingue «no hay datos» de «ningún resultado con estos filtros» y ofrece la acción adecuada en cada caso
`ErrorState`	Fallo al cargar	Distingue error de red (icono de wifi tachado) de error del servidor; muestra el mensaje y *Reintentar*; `role="alert"`
`ResourceNotFound`	ID no válido en la URL o 404	Explica la causa probable y ofrece volver al listado

En el proyecto

Fíjate en un detalle del listado: si falla una actualización pero ya había datos (por ejemplo, al cambiar de página), **no** se sustituye la tabla por el error. Se muestra una franja de aviso sobre los datos anteriores: «No se ha podido actualizar el listado. Se muestran los últimos datos obtenidos». Es la condición `query.isError && !data` frente a `query.isError` con datos. Perder de vista la información por un fallo momentáneo es peor que mostrarla con un aviso.

12.8 Errores típicos y depuración

↑ Índice

- **El contenido queda debajo del menú lateral.** Falta el `lg:pl-64` del contenedor, o se ha cambiado el ancho del menú (`w-64`) sin cambiar el relleno.
- **Un enlace del menú aparece activo en varias rutas.** Le falta `end: true` en `navigation.ts`.

- **El panel móvil no se cierra al navegar.** No se está pasando `onNavigate` a `SidebarNav`, o el elemento no llama a `onClick={onNavigate}`.
- **Un módulo nuevo no aparece en el menú.** Hay que añadirlo en `navigation.ts` **y** registrar sus rutas en el router: son dos pasos separados.
- **Aviso de accesibilidad de Radix sobre el título del diálogo.** El `SheetContent` necesita `SheetTitle` (y una descripción), aunque estén ocultos con `sr-only`.

12.9 Ejercicios

↑ Índice

Ejercicio 12.1 · Activa un módulo en el menú Ver solución →

Sin crear ninguna página, convierte *Incidencias* en un módulo con `basePath: '/incidencias'` y dos acciones (*Consultar incidencias* y *Nueva incidencia*). Comprueba qué ocurre al pulsarlas y por qué (pista: la ruta comodín del router). ¿Qué falta para que funcionen? Deshaz el cambio: este módulo lo construiremos en la Parte IX.

Ejercicio 12.2 · Recuerda los grupos plegados Ver solución →

Haz que el estado abierto o plegado de cada grupo del menú se recuerde al recargar la página. Usa `localStorage` con una clave por módulo, inicializando `useState` con una función que lea el valor guardado. ¿Dónde guardarías el valor: en un efecto o en el manejador `onOpenChange`? Justifícalo con lo visto en el capítulo 8.

Ejercicio 12.3 · Elimina una duplicación Ver solución →

`ApplicationSummary` define un subcomponente privado `Field` que hace exactamente lo mismo que `DetailField` de `src/components/common/detail-field.tsx`, y repite la lógica de `LongText` para la descripción. Refactorízalo para usar los componentes comunes y comprueba con React DevTools que la ficha se pinta igual. ¿Qué ganas y qué riesgo tiene este tipo de cambio?

12.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 12 Ver respuestas →

1. ¿Qué tres zonas fijas tiene el layout y cuál es la zona variable?
2. ¿Por qué se usa `min-h-svh` en lugar de una altura del 100 % de la ventana?
3. ¿Qué ventaja tiene separar `navigation.ts` de `sidebar-nav.tsx`?
4. ¿Para qué sirve la propiedad `end` de los enlaces del menú?
5. ¿Cómo se consigue el menú móvil sin duplicar el menú lateral?
6. ¿Por qué el color cian se reserva para unos pocos elementos?
7. ¿Qué ventaja de accesibilidad tiene `LevelBadge` frente a un simple color?
8. ¿Qué hace el listado si falla la carga de una página nueva pero ya tenía datos?

PARTE V

Arquitectura

13 Organización por funcionalidad y reglas de dependencia pág. 129

14 Anatomía de un módulo pág. 136

↑ Volver al índice

CAPÍTULO 13

Organización por funcionalidad y reglas de dependencia

↑ Volver al índice

EN ESTE CAPÍTULO

13.1 Dos formas de organizar un proyecto
13.2 El árbol de `src/`
13.3 Reglas de dependencia
13.4 Qué va en `common` y qué va en un módulo
13.5 `lib/` : utilidades sin interfaz
13.6 Convenciones de nombres
13.7 Decisión y evolución
13.8 Errores típicos y depuración
13.9 Ejercicios
13.10 Autoevaluación

Un proyecto pequeño funciona con cualquier estructura de carpetas. Uno que va a crecer módulo a módulo —aplicaciones, demandas, incidencias, equipos, SLA— necesita una estructura que haga obvio dónde va cada cosa y que impida que todo acabe dependiendo de todo. Este capítulo explica la organización de `src/`, las reglas que gobiernan qué puede importar qué y los criterios para decidir si algo es común o pertenece a un módulo.

13.1 Dos formas de organizar un proyecto

↑ Índice

Hay dos grandes enfoques para repartir el código en carpetas:

Por tipo técnico. Una carpeta por clase de artefacto: `components/`, `hooks/`, `services/`, `types/`, `utils/`, `pages/`. Es la estructura de muchos tutoriales y funciona bien con diez ficheros. Con doscientos, el código de una funcionalidad queda repartido por seis carpetas, cada una con piezas de todos los módulos mezcladas.

Por funcionalidad (*feature-based*). Una carpeta por módulo de negocio, que contiene **todo** lo de ese módulo: acceso a la API, tipos, componentes y páginas. Lo que es realmente transversal vive fuera, en carpetas compartidas.

```
Por tipo técnico                    Por funcionalidad (este proyecto)
src/                                src/
├── components/                     ├── features/
│   ├── ApplicationsTable.tsx       │   ├── applications/
│   ├── DemandsTable.tsx            │   │   ├── api/  model/  components/  pages/
│   └── …                           │   └── demands/
├── hooks/                          │       ├── api/  model/  components/  pages/
│   ├── useApplications.ts          ├── components/   ← solo lo compartido
│   └── useDemands.ts               └── lib/          ← solo lo compartido
├── services/
└── types/
```

Si vienes de Java...

Es exactamente el debate entre **paquetes por capa** (`controller`, `service`, `repository`, `dto`) y **paquetes por funcionalidad** (`application`, `demand`, cada uno con su controlador, servicio, repositorio y DTO). El backend del proyecto ya está organizado por funcionalidad: `com.albertoderafael.itmanager.demand` contiene `controller`, `service`, `dto`, `entity`, `repository`... El frontend aplica el mismo criterio, de modo que a la carpeta `features/demands` le corresponde el paquete `demand` del backend.

13.2 El árbol de `src/`

↑ Índice

```
src/
├── main.tsx                 Punto de entrada: monta React en #root
├── index.css                Tailwind, tema y estilos base
├── app/                     Arranque de la aplicación
│   ├── app.tsx                Proveedores globales
│   ├── router.tsx             Tabla de rutas: monta las rutas de cada módulo
│   └── query-client.ts        Configuración de la caché de datos
├── components/
│   ├── ui/                  Piezas de interfaz genéricas (shadcn/ui)
│   ├── layout/              Armazón: menú lateral, cabecera, navegación
│   └── common/              Componentes compartidos con algo de criterio propio:
│                              cabecera de página, paginación, estados, indicadores,
│                              filtros, campos de formulario, cabecera ordenable…
├── features/
│   ├── applications/        Módulo de aplicaciones
│   └── demands/             Módulo de demandas
├── lib/                     Utilidades sin interfaz: cliente HTTP, parámetros de
│                              listado, rutas, fechas, errores de formulario, cn()
└── pages/                   Páginas globales que no son de ningún módulo (404, error)
```

Cada carpeta responde a una pregunta distinta:

Carpeta	Pregunta que responde	¿Conoce el negocio?
`app/`	¿Cómo se arranca y se ensambla la aplicación?	Solo para registrar los módulos
`components/ui/`	¿Cómo es un botón, un desplegable, una tarjeta?	No
`components/layout/`	¿Cómo es la carcasa de la aplicación?	Solo la lista de módulos del menú
`components/common/`	¿Qué patrones de interfaz repiten varios módulos?	No: reciben todo por props
`features/<módulo>/`	¿Cómo funciona este módulo?	Sí, y solo el suyo
`lib/`	¿Qué utilidades sin interfaz usa todo el proyecto?	No
`pages/`	¿Qué pantallas no pertenecen a ningún módulo?	No

13.3 Reglas de dependencia

↑ Índice

Una estructura de carpetas solo sirve si se respeta **en qué dirección** se importa. Las reglas del proyecto forman un grafo sin ciclos:

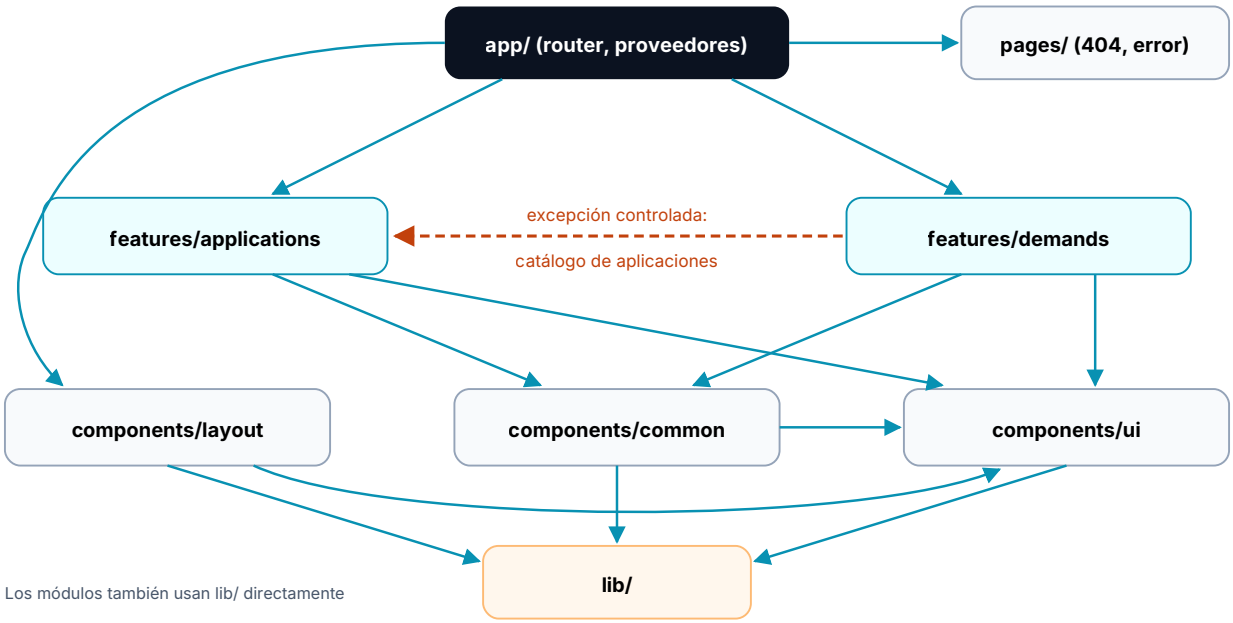


Figura 13.1. Dirección permitida de las importaciones. Las flechas van de quien importa a lo importado; nunca en sentido contrario.

En palabras:

1. `lib/` **no importa nada del proyecto** salvo otros ficheros de `lib/`. Son funciones puras o casi puras: la base de todo.
2. `components/ui/` **solo importa** `lib/utils` (para `cn`) y librerías externas.
3. `components/common/` **importa** `ui/` **y** `lib/`, nunca un módulo. Si un componente común necesitara saber qué es una demanda, ya no sería común.
4. **Un módulo importa de** `components/` **y de** `lib/`, pero **no de otros módulos** (con una excepción explicada abajo).
5. `app/` **es el único que conoce todos los módulos**: los ensambla en el router.

Puedes comprobarlo en el código: ningún fichero de `components/` ni de `lib/` importa nada de `@/features`, y el único fichero fuera de `features/` que lo hace es `app/router.tsx`.

> **Si vienes de Java…**
>
> Son las reglas de una arquitectura por capas bien hecha: el dominio no conoce la web, la persistencia no conoce los controladores. En Java se pueden **verificar con pruebas** usando ArchUnit (`noClasses().that().resideInAPackage("..demand..").should().dependOnClassesThat().resideInAPackage("..application.entity..")`). En el frontend existe lo mismo con ESLint, como verás en el ejercicio 13.2.

13.3.1 La excepción: el catálogo de aplicaciones en las demandas

El módulo de demandas necesita la lista de aplicaciones para el filtro y para el campo *Aplicación* del formulario. Hay **una** importación entre módulos:

`src/features/demands/components/application-options.ts`

```
import { APPLICATION_OPTIONS_LIMIT, useApplicationOptions } from '@/features/applications/api/
applications-queries'
```

¿Es un error? Es una **dependencia de dominio real**: una demanda pertenece a una aplicación, y en el backend ocurre lo mismo (el servicio de demandas consulta el catálogo de aplicaciones). Lo importante es que sea:

- **Unidireccional**: demandas depende de aplicaciones; aplicaciones no sabe nada de demandas. Si ambos se importaran mutuamente, ya no serían módulos separados.
- **Estrecha y explícita**: se usa un único hook pensado para ello (`useApplicationOptions`, documentado como «catálogo de aplicaciones para desplegables de otros módulos») y una constante. Las demandas no tocan las páginas, los componentes ni los esquemas de aplicaciones.
- **Concentrada en un único fichero**: `application-options.ts` es el único punto de contacto. Si mañana cambia la forma de obtener el catálogo, solo cambia ese fichero.

Además, como el hook comparte las claves de caché de las aplicaciones, cuando alguien crea o modifica una aplicación, los desplegables de demandas se actualizan solos (capítulo 17).

> **Nota**
>
> Una mejora habitual cuando hay más dependencias entre módulos es que cada módulo publique una **API pública** en un fichero `index.ts` (por ejemplo, `features/applications/index.ts` que exporta solo `useApplicationOptions` y los tipos necesarios) y prohibir con ESLint importar cualquier otra ruta interna de otro módulo. Con una única dependencia, el proyecto no lo necesita todavía.

13.4 Qué va en `common` y qué va en un módulo

↑ Índice

La pregunta aparece cada vez que se escribe un componente. Los criterios del proyecto:

1. **¿Necesita saber algo del dominio?** Si conoce los estados de una aplicación o las prioridades de una demanda, va en el módulo. Si recibe todo por props, puede ser común.
2. **¿Lo usa más de un módulo, o es un patrón claro de la aplicación?** Si solo lo usa un módulo y no hay motivo para pensar que otro lo necesitará, se queda en el módulo. Mover algo a común «por si acaso» crea abstracciones prematuras.
3. **Separa la apariencia del significado.** A menudo un mismo concepto tiene una parte común y otra de dominio.

El tercer criterio se ve muy bien en tres pares del proyecto:

Común (apariencia, mecánica)	Del módulo (significado)
`components/common/badges.tsx`: `StatusPill` y `LevelBadge` con sus tonos y niveles	`application-badges.tsx`: `ACTIVE` es `success`; `CRITICAL` es nivel 4
`components/common/filter-select.tsx`: desplegable de filtro con opción «Todos»	`application-filters.tsx`: filtros de estado y criticidad con sus etiquetas
`lib/list-params.ts`: leer y escribir página, tamaño y orden en la URL; validar enumerados	`model/list-params.ts`: qué filtros tiene el listado de aplicaciones y cómo se llaman en la URL

Decisión de diseño: Infraestructura común sin duplicar

Cuando se construyó el módulo de demandas, todo lo que resultó idéntico a aplicaciones (indicadores, cabecera ordenable, filtros, campos de formulario, aviso de recurso inexistente, utilidades de URL y paginación) se movió a `components/common` y `lib`. Resultado: **un módulo nuevo solo aporta su dominio**. Es la aplicación práctica de la regla de tres: no se generaliza con el primer caso, sino cuando aparece el segundo y se ve qué es realmente común.

13.5 `lib/`: utilidades sin interfaz

↑ Índice

Fichero	Qué contiene
`api-client.ts`	Cliente HTTP sobre `fetch`, `ApiError` y su interpretación (capítulo 15)
`list-params.ts`	Lectura, validación y escritura de paginación, orden y enumerados en la URL
`route-params.ts`	Conversión segura de `:id` de la ruta en un número
`form-errors.ts`	Traslado de los errores de validación del servidor a los campos del formulario
`format.ts`	Formato de fechas en hora local
`utils.ts`	`cn()`

Que `lib/` no contenga componentes ni hooks de React no es casual: son **funciones normales**, que se prueban con Vitest sin montar nada (`api-client.test.ts`, `list-params.test.ts`). Cuanta más lógica pueda vivir como función pura, más fácil es de probar y de reutilizar.

13.6 Convenciones de nombres

↑ Índice

Convención	Ejemplo
Ficheros y carpetas en *kebab-case*	`application-edit-page.tsx`, `list-params.ts`
Sufijos que dicen qué es el fichero	`-page.tsx`, `-api.ts`, `-queries.ts`, `-table.tsx`, `-filters.tsx`
Componentes y tipos en *PascalCase*	`ApplicationsTable`, `ApplicationListParams`
Constantes de módulo en mayúsculas	`APPLICATION_STATUSES`, `DEFAULT_LIST_PARAMS`
Pruebas junto al código, con `.test.ts`	`model/schemas.ts` y `model/schemas.test.ts`
`.tsx` solo si el fichero contiene JSX	`navigation.ts` (datos) frente a `sidebar-nav.tsx`

Con estas convenciones, la búsqueda de ficheros del editor (Ctrl+P) se convierte en navegación: escribir «demand page» lista todas las páginas de demandas.

13.7 Decisión y evolución

↑ Índice

Decisión de diseño: Organización por funcionalidad con capas compartidas

Decisión. Carpetas por módulo de negocio en `features/`, con una estructura interna idéntica en todos los módulos, y capas compartidas (`components/ui`, `components/layout`, `components/common`, `lib`) con dependencias en una sola dirección.

Motivos. El trabajo diario es por funcionalidad (añadir un campo a las demandas, un filtro a las aplicaciones): todo lo que hay que tocar está en una carpeta. Los módulos se pueden entender, probar y, llegado el caso, eliminar de forma independiente. Y la correspondencia con los paquetes del backend facilita moverse entre ambos proyectos.

Cómo escala. Con muchos módulos, el siguiente paso sería hacer cumplir las reglas con ESLint y publicar una API por módulo. Con varios equipos y aplicaciones, un monorepo con paquetes compartidos (por ejemplo, el sistema de diseño). Arquitecturas más complejas, como los *microfrontends*, solo se justifican con equipos independientes que despliegan por separado, que no es el caso.

13.8 Errores típicos y depuración

↑ Índice

- **Dependencias circulares.** Si `a.ts` importa `b.ts` y `b.ts` importa `a.ts`, un valor puede llegar como `undefined` en ejecución, con errores desconcertantes del tipo *Cannot access 'X' before initialization*. Las reglas de dependencia en una sola dirección evitan casi todos los casos.
- **Un componente «común» que importa algo de un módulo.** Señal de que no es común: o se queda en el módulo, o se separa la parte de dominio (como en los indicadores).
- **Ficheros índice (`index.ts`) que reexportan todo.** Son cómodos, pero dificultan la eliminación de código muerto y favorecen los ciclos. El proyecto importa siempre desde el fichero concreto.
- **Lógica de dominio en una página.** Si una regla de negocio (por ejemplo, qué estados son finales) está escrita dentro de un componente de página, no se puede probar ni reutilizar. Llévala a `model/` (capítulo 14).

13.9 Ejercicios

↑ Índice

Ejercicio 13.1 · Dibuja el grafo de un fichero Ver solución →

Elige `src/features/demands/pages/demand-edit-page.tsx` y clasifica cada una de sus importaciones según su carpeta de origen (`react`/librerías, `components/ui`, `components/common`, `lib`, mismo módulo). ¿Cumple todas las reglas? ¿Cuántas importaciones son del propio módulo y qué subcarpetas toca?

Ejercicio 13.2 · Haz cumplir las reglas con ESLint Ver solución →

Añade a `eslint.config.js` un bloque para `src/components/**` y `src/lib/**` con la regla `no-restricted-imports` que prohíba importar `@/features/*`, con un mensaje explicativo. Después, añade un bloque para `src/features/applications/**` que prohíba `@/features/demands/*`. Comprueba con `npm run lint` que el proyecto actual cumple ambas reglas y provoca una infracción a propósito para ver el mensaje.

Ejercicio 13.3 · Decide dónde va Ver solución →

Para cada pieza hipotética, indica en qué carpeta la pondrías y por qué: (a) un componente que muestra una fecha relativa («hace 3 horas»); (b) el tipo `Incident` alineado con el DTO del backend; (c) un indicador del nivel de SLA con un semáforo de colores; (d) una función que calcula si una incidencia ha incumplido su SLA a partir de sus fechas; (e) un hook que obtiene los equipos para un desplegable de incidencias.

13.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 13 Ver respuestas →

1. ¿Qué diferencia hay entre organizar por tipo técnico y por funcionalidad?
2. ¿Qué carpeta es la única que conoce todos los módulos y para qué?
3. ¿Por qué `components/common` no puede importar de `features/` ?
4. ¿Qué características hacen aceptable la dependencia de demandas hacia aplicaciones?
5. ¿Qué tres criterios usa el proyecto para decidir si algo es común?
6. Explica, con el ejemplo de los indicadores, cómo se separa la apariencia del significado.
7. ¿Por qué conviene que `lib/` contenga funciones normales y no hooks?
8. ¿Qué problema pueden causar las dependencias circulares?

CAPÍTULO 14

Anatomía de un módulo

↑ Volver al índice

EN ESTE CAPÍTULO

14.1 Vista general
14.2 `model/` : el dominio sin dependencias
14.3 `api/` : acceso a los datos
14.4 `components/` : presentación del módulo
14.5 `pages/` : la orquestación
14.6 `routes.ts` : el punto de montaje
14.7 Dos módulos lado a lado
14.8 Receta: crear un módulo nuevo
14.9 Errores típicos y depuración
14.10 Ejercicios
14.11 Autoevaluación

Los dos módulos del proyecto, aplicaciones y demandas, tienen exactamente la misma estructura interna: `api`, `model`, `components`, `pages` y `routes.ts`. Este capítulo disecciona el de aplicaciones carpeta por carpeta, explica qué responsabilidad tiene cada una y cómo fluyen los datos entre ellas, y termina con la receta para crear un módulo nuevo.

14.1 Vista general

↑ Índice

```
features/applications/
├── model/                      Dominio: sin React, sin HTTP
│   ├── types.ts                  Tipos alineados con los DTO del backend y enumerados
│   ├── constants.ts              Etiquetas, valores por defecto y límites
│   ├── schemas.ts                Validación de formularios (Zod)
│   ├── schemas.test.ts
│   ├── list-params.ts            Estado del listado: URL ↔ parámetros ↔ consulta a la API
│   └── list-params.test.ts
├── api/                        Acceso a datos
│   ├── applications-api.ts       Un método por endpoint del controlador
│   └── applications-queries.ts   Hooks de TanStack Query y claves de caché
├── components/                 Presentación específica del módulo
│   ├── applications-table.tsx
│   ├── application-filters.tsx
│   ├── application-badges.tsx
│   ├── application-form-fields.tsx
│   └── application-summary.tsx
├── pages/                      Una pantalla por ruta: orquestan todo lo anterior
│   ├── applications-list-page.tsx
│   ├── application-create-page.tsx
│   ├── application-detail-page.tsx
│   ├── application-edit-page.tsx
│   └── application-edit-selector-page.tsx
└── routes.ts                   Rutas del módulo con carga diferida
```

Las cuatro carpetas son **capas** con una dirección de dependencia clara:

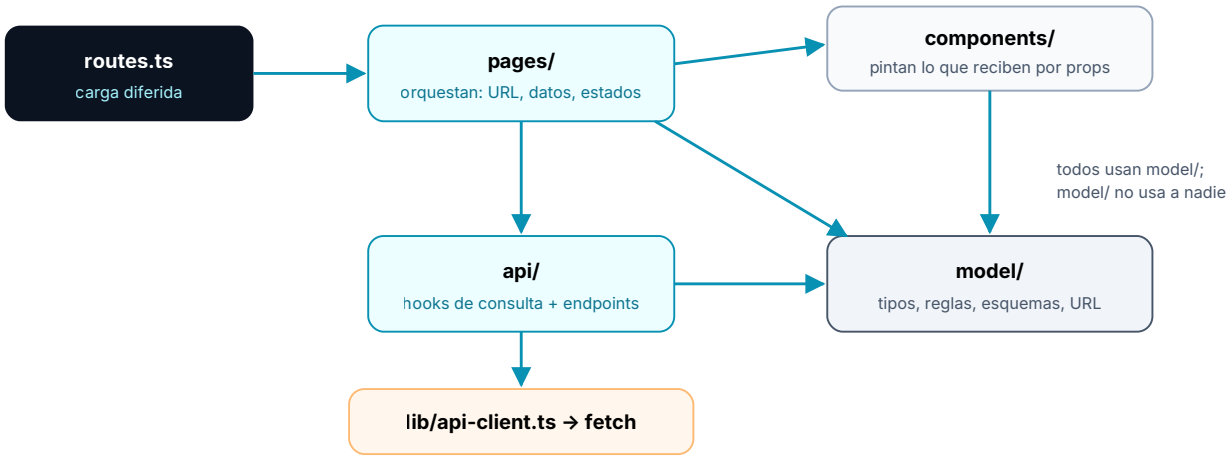


Figura 14.1. Capas de un módulo. Las páginas orquestan; `model/` es la base que no depende de ninguna otra capa del módulo.

> **Si vienes de Java...**
>
> La correspondencia con un módulo de Spring es casi directa: `model/types.ts` son los DTO; `model/` en general, el dominio y sus reglas; `api/` es el cliente de la API (como un `RestClient` o un cliente de Feign, más una caché); `pages/` hace el papel de los controladores, que reciben la petición (la URL), llaman a los servicios y eligen la vista; y `components/` son las vistas. `routes.ts` sería el `@RequestMapping` del módulo.

14.2 `model/`: el dominio sin dependencias

↑ Índice

`model/` es la capa más importante y la más sencilla: **no importa React ni hace peticiones**. Solo contiene tipos, datos y funciones puras. Por eso es la que tiene pruebas unitarias.

14.2.1 `types.ts`: el contrato con el backend

`src/features/applications/model/types.ts (extracto)`

```ts
/**
 * Tipos del dominio "Aplicaciones", alineados con los DTO del backend
 * (paquete com.albertoderafael.itmanager.application.dto).
 */
export const APPLICATION_STATUSES = ['ACTIVE', 'INACTIVE'] as const
export type ApplicationStatus = (typeof APPLICATION_STATUSES)[number]

/** ApplicationResponse */
export interface Application { id: number; name: string; description: string | null; /* … */ }

/** ApplicationCreateRequest */
export interface ApplicationCreateRequest { name: string; description: string | null; criticality: A
pplicationCriticality }

/** ApplicationPageResponse (paginación 0-based de Spring Data). */
export interface ApplicationPage { content: Application[]; page: number; size: number;
totalElements: number; totalPages: number }

/** Campos por los que se permite ordenar (propiedades de la entidad Application). */
export const APPLICATION_SORT_FIELDS = ['id', 'name', 'criticality', 'status'] as const

/** Criterios de consulta del listado. */
export interface ApplicationListParams {
  status?: ApplicationStatus
```

```
  criticality?: ApplicationCriticality
  page: number
  size: number
  sortField: ApplicationSortField
  sortDirection: SortDirection
}
```

Dos clases de tipos conviven aquí: los que **reflejan el backend** (cada uno con el nombre del DTO en su comentario) y los que son **propios del frontend**, como `ApplicationListParams`, que describe los criterios de consulta tal como los maneja la interfaz. La lista de campos de ordenación también es un contrato: son propiedades de la entidad JPA que Spring Data acepta en `sort`.

14.2.2 `constants.ts`: etiquetas y valores alineados

`src/features/applications/model/constants.ts (extracto)`

```
export const STATUS_LABELS: Record<ApplicationStatus, string> = { ACTIVE: 'Activa', INACTIVE:
'Inactiva' }

/** Valores por defecto: coinciden con @PageableDefault(size = 10, sort = "id", ASC) del backend. */
export const DEFAULT_LIST_PARAMS: ApplicationListParams = { page: 0, size: 10, sortField: 'id', sort
Direction: 'asc' }

export const NAME_MAX_LENGTH = 100
export const DESCRIPTION_MAX_LENGTH = 500
```

La API habla en códigos (`ACTIVE`); la interfaz, en español (*Activa*). La traducción vive en un único sitio. Los valores por defecto del listado coinciden con los del controlador de Spring, y las longitudes máximas, con las anotaciones `@Size` de los DTO: esos números los usan tanto los esquemas de validación como los contadores de caracteres de los campos.

14.2.3 `list-params.ts`: tres representaciones del mismo estado

El estado de un listado existe en **tres formas**, y este fichero traduce entre ellas:

Representación	Ejemplo	Quién la usa
URL del navegador	`?estado=ACTIVE&pagina=2&orden=name,desc`	El usuario: la ve, la comparte, la guarda
Objeto de la interfaz	`{ status: 'ACTIVE', page: 1, sortField: 'name', sortDirection: 'desc', … }`	Los componentes
Consulta a la API	`?status=ACTIVE&page=1&size=10&sort=name,desc`	Spring Data (`Pageable`)

`src/features/applications/model/list-params.ts (extracto)`

```
export function parseListParams(search: URLSearchParams): ApplicationListParams { /* URL → objeto,
validando */ }
export function serializeListParams(params: ApplicationListParams): URLSearchParams { /* objeto →
URL */ }
export function toApiQuery(params: ApplicationListParams) { /* objeto → consulta de Spring */ }
```

Fíjate en las diferencias deliberadas: la URL usa nombres en español y páginas desde 1 (es lo que ve el usuario), mientras que la API usa los nombres del backend y páginas desde 0. La validación al leer la URL es estricta: un `?estado=FOO` o un `?pagina=-3` se descartan y se usan los valores por defecto, porque la

URL la puede escribir cualquiera. Lo genérico (paginación, orden, enumerados) está en `lib/list-params.ts`; aquí solo queda lo propio del listado. Lo estudiaremos a fondo en la Parte VII.

14.2.4 `schemas.ts` y las reglas de dominio

`schemas.ts` contiene los esquemas de Zod que validan los formularios, alineados con Bean Validation (Parte VII). Y en el módulo de demandas, `model/` incluye además **reglas de negocio**:

`src/features/demands/model/status-rules.ts (extracto)`

```ts
/*
 * Reglas de transición de estado. Reflejan Demand.changeStatus() del backend.
 * El backend sigue siendo la autoridad: si rechaza un cambio, responde 409.
 */
const NEXT_STATUSES: Record<DemandStatus, readonly DemandStatus[]> = {
  OPEN: ['IN_PROGRESS', 'CANCELLED'],
  IN_PROGRESS: ['COMPLETED', 'CANCELLED'],
  COMPLETED: [],
  CANCELLED: [],
}
export function canTransition(from: DemandStatus, to: DemandStatus): boolean { /* … */ }
```

Que estas reglas vivan en `model/` y no dentro del formulario permite probarlas de forma aislada (`status-rules.test.ts`) y usarlas en varios sitios: el desplegable de estado, el esquema de validación y el diagrama del ciclo de vida (`StatusFlow`). Lo veremos en el capítulo 22.

14.3 `api/`: acceso a los datos

↑ Índice

Dos ficheros con responsabilidades separadas.

`applications-api.ts` es una traducción directa del controlador REST: un método por endpoint, sin lógica.

`src/features/applications/api/applications-api.ts`

```ts
const BASE_PATH = '/applications'

/** Acceso a los endpoints de ApplicationController (/api/applications). */
export const applicationsApi = {
  list: (params: ApplicationListParams, signal?: AbortSignal) =>
    apiClient.get<ApplicationPage>(BASE_PATH, toApiQuery(params), signal),
  getById: (id: number, signal?: AbortSignal) =>
    apiClient.get<Application>(`${BASE_PATH}/${id}`, undefined, signal),
  create: (request: ApplicationCreateRequest) => apiClient.post<Application>(BASE_PATH, request),
  update: (id: number, request: ApplicationUpdateRequest) =>
    apiClient.put<Application>(`${BASE_PATH}/${id}`, request),
}
```

`applications-queries.ts` envuelve esos métodos en hooks de TanStack Query (`useApplications`, `useApplication`, `useCreateApplication`, `useUpdateApplication`, `useApplicationOptions`) y define las claves de caché. Es la única puerta de entrada a los datos para el resto del módulo: **ninguna página llama directamente a `applicationsApi`**. El capítulo 17 está dedicado a este fichero.

Decisión de diseño: Separar los endpoints de la caché

Separar `-api.ts` (qué peticiones existen) de `-queries.ts` (cómo se cachean y se invalidan) mantiene cada fichero con una sola razón para cambiar: un endpoint nuevo en el backend toca el primero; una nueva estrategia de caché, el segundo. Además, los métodos de `-api.ts` son funciones normales que se pueden usar fuera de React, por ejemplo en una prueba o en un cargador de rutas.

14.4 `components/`: presentación del módulo

↑ Índice

Los componentes del módulo **saben del dominio, pero no de dónde vienen los datos**. Reciben objetos ya cargados y funciones de aviso por props:

Componente	Recibe	Hace
`ApplicationsTable`	Lista de aplicaciones, orden actual, `onSortChange`, modo	Pinta la tabla con cabeceras ordenables y acciones por fila
`ApplicationFilters`	Filtros actuales, `onChange`	Pinta los dos filtros y «Limpiar filtros»
`StatusBadge`, `CriticalityBadge`	Un estado o una criticidad	Traducen el valor a `StatusPill` o `LevelBadge`
`NameField`, `CriticalityField` …	Nada (leen el formulario del contexto)	Campos configurados con etiquetas y límites del dominio
`ApplicationSummary`	Una aplicación	Ficha de solo lectura

Al no hacer peticiones, son reutilizables: la tabla y los filtros sirven igual en el listado principal (con estado en la URL) y en el selector del catálogo (con estado local), como vimos en el capítulo 7.

14.5 `pages/`: la orquestación

↑ Índice

Cada página corresponde a **una ruta** y es el único lugar donde se juntan todas las piezas. Una página típica:

1. Lee lo que necesita de la URL (`useParams`, `useSearchParams`) y lo valida con funciones de `model/` o `lib/`.
2. Pide los datos con los hooks de `api/`.
3. Decide qué estado mostrar (cargando, error, vacío, datos).
4. Compone los componentes comunes y del módulo, pasándoles datos y manejadores.
5. Gestiona las acciones del usuario: navegar, enviar formularios, mostrar notificaciones.

El listado de aplicaciones, que ya conoces bien, sigue exactamente este guion. Las páginas son los ficheros más largos del módulo, y es normal: es donde vive la lógica de la pantalla. Lo que no es normal, y conviene evitar, es que contengan reglas de negocio o lógica reutilizable: eso baja a `model/` o a un hook.

14.6 `routes.ts`: el punto de montaje

↑ Índice

`src/features/applications/routes.ts (extracto)`

```
export const applicationRoutes: RouteObject = {
  path: 'aplicaciones',
  children: [
    {
      index: true,
      handle: { title: 'Consultar aplicaciones' },
      lazy: () =>
        import('./pages/applications-list-page').then((m) => ({ Component: m.ApplicationsListPage })
),
    },
    { path: 'nueva', handle: { title: 'Nueva aplicación' }, lazy: () => /* … */ },
    { path: 'modificar', handle: { title: 'Modificar aplicación' }, lazy: () => /* … */ },
    { path: ':id', handle: { title: 'Detalle de aplicación' }, lazy: () => /* … */ },
    { path: ':id/editar', handle: { title: 'Modificar aplicación' }, lazy: () => /* … */ },
  ],
}
```

El módulo **declara** sus rutas y el router de la aplicación las **monta** con una línea (`applicationRoutes` en `app/router.tsx`). Cada página se carga bajo demanda, y `handle.title` da el título de la pestaña (capítulo 8). Las rutas tienen nombres en español porque forman parte de la interfaz que ve el usuario, mientras que el código usa el inglés del dominio del backend. Lo veremos con detalle en el capítulo 18.

14.7 Dos módulos lado a lado

↑ Índice

La mejor prueba de que la estructura funciona es comparar ambos módulos:

Pieza	Aplicaciones	Demandas
Tipos y enumerados	`ApplicationStatus`, `ApplicationCriticality`	`DemandStatus`, `DemandPriority`
Ordenación	`id`, `name`, `criticality`, `status`	además `application.name`, `createdAt`, `updatedAt`
Filtros del listado	estado, criticidad	aplicación, estado, prioridad
Endpoints	`applications-api.ts`	`demands-api.ts`
Hooks y claves de caché	`applicationKeys`, `useApplications`…	`demandKeys`, `useDemands`…
Reglas de dominio	—	`status-rules.ts` y el componente `StatusFlow`
Dependencias de otros módulos	ninguna	catálogo de aplicaciones (`application-options.ts`)
Páginas	listado, alta, detalle, modificación, selector	las mismas cinco

Si sabes dónde está algo en un módulo, sabes dónde está en el otro. Las diferencias son exactamente las del dominio: las demandas tienen un ciclo de vida y pertenecen a una aplicación.

Cuidado

La simetría tiene un pequeño desajuste que conviene conocer: en aplicaciones, las funciones de la URL se llaman `parseListParams` y `serializeListParams`; en demandas, `parseDemandListParams` y `serializeDemandListParams`. Ambas convenciones funcionan, pero un proyecto debería elegir una. Son detalles así los que una revisión de código debe detectar antes de que se conviertan en costumbre.

14.8 Receta: crear un módulo nuevo

↑ Índice

Con todo lo anterior, añadir un módulo (por ejemplo, Incidencias) es seguir una lista de pasos conocida. La desarrollaremos completa en el capítulo 26; este es el esquema:

1. **Leer el contrato del backend**: controlador, DTO, validaciones, enumerados, paginación por defecto y errores posibles.
2. `model/types.ts`: enumerados con `as const`, interfaces de respuesta, creación, modificación y página; campos de ordenación; parámetros del listado.
3. `model/constants.ts`: etiquetas, valores por defecto alineados con `@PageableDefault`, longitudes máximas.
4. `model/list-params.ts` y `model/schemas.ts`, con sus pruebas.
5. `api/<módulo>-api.ts` y `api/<módulo>-queries.ts` con claves jerárquicas.
6. `components/`: indicadores (tonos y niveles), filtros, tabla, campos de formulario y ficha.
7. `pages/`: listado, alta, detalle, modificación y selector.
8. `routes.ts` y registro en `app/router.tsx`.
9. **Menú**: entrada en `components/layout/navigation.ts` (quitar `comingSoon`).
10. **Proxy**: si lo sirve otro servicio, regla en `vite.config.ts`.
11. **Verificación**: `typecheck`, `lint`, `test` y prueba manual con el backend real.

14.9 Errores típicos y depuración

↑ Índice

- **Una página llama directamente a `fetch` o a `applicationsApi`.** Se salta la caché y sus invalidaciones; usa siempre el hook de `-queries.ts`.
- **Etiquetas en español repartidas por los componentes.** Llévalas a `constants.ts` con un `Record` exhaustivo.
- **Un tipo del frontend que ya no coincide con el DTO.** Revisa los tipos cada vez que cambie el backend; el comentario con el nombre del DTO ayuda a localizarlos.
- **Componentes del módulo que hacen peticiones.** Pierden la reutilización. Pásales los datos desde la página.

14.10 Ejercicios

↑ Índice

Ejercicio 14.1 · Compara dos ficheros paralelos Ver solución →

Abre en dos paneles `features/applications/pages/applications-list-page.tsx` y `features/demands/pages/demands-list-page.tsx`. Anota todas las diferencias. ¿Cuántas se deben al dominio (filtros distintos) y cuántas son accidentales? ¿Merecería la pena extraer más código común? Razona los pros y contras.

Ejercicio 14.2 · Verifica el contrato de ordenación Ver solución →

El frontend ofrece ordenar las demandas por los campos de `DEMAND_SORT_FIELDS` (`model/types.ts`). En el backend, `DemandService` valida el parámetro `sort` contra su propia lista, `VALID_SORT_FIELDS`, y responde `400 Invalid demand sort field: …` si no lo reconoce. Compara ambas listas (solo lectura: no modifiques el backend). ¿Qué columnas de la tabla de demandas producirían un error? Con el backend arrancado, pulsa esas cabeceras y observa *Network* y lo que ve el usuario. Propón dos soluciones, una en cada lado, y razona cuál preferirías.

Ejercicio 14.3 · Diseña el `model/` de Incidencias Ver solución →

Sin escribir todavía componentes, escribe el borrador de `features/incidents/model/types.ts` y `constants.ts` para un módulo de incidencias con: identificador, título, descripción opcional, severidad (cuatro niveles), estado (`OPEN`, `IN_PROGRESS`, `RESOLVED`, `CLOSED`), aplicación afectada y fechas de creación y resolución. Incluye los parámetros del listado y las etiquetas en español. ¿Qué decisiones tomas sin conocer todavía el backend y cómo las marcarías como pendientes?

14.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 14 Ver respuestas →

1. ¿Qué contiene cada una de las cuatro carpetas de un módulo y cuál es la dirección de dependencia entre ellas?
2. ¿Por qué `model/` no importa React ni hace peticiones?
3. ¿Qué dos clases de tipos conviven en `types.ts`?
4. ¿Cuáles son las tres representaciones del estado de un listado y quién usa cada una?
5. ¿Qué diferencia hay entre `applications-api.ts` y `applications-queries.ts`?
6. ¿Por qué los componentes del módulo no hacen peticiones?
7. ¿Qué cinco cosas hace una página típica?
8. ¿Qué pasos son necesarios para que un módulo nuevo aparezca y funcione en la aplicación?

PARTE VI

Datos y backend

15 El cliente HTTP y la gestión de errores pág. 145

16 CORS, el proxy de desarrollo y el despliegue pág. 154

17 TanStack Query: el estado del servidor pág. 161

↑ Volver al índice

CAPÍTULO 15

El cliente HTTP y la gestión de errores

↑ Volver al índice

EN ESTE CAPÍTULO

15.1 Por qué un cliente propio
15.2 `fetch` en detalle
15.3 `apiRequest`, línea a línea
15.4 La cadena de consulta
15.5 El modelo de errores
15.6 Los formatos de error del backend
15.7 Cómo llegan los errores a la pantalla
15.8 Decisión y alternativas
15.9 Errores típicos y depuración
15.10 Ejercicios
15.11 Autoevaluación

Toda comunicación con el backend pasa por un único fichero de unas doscientas líneas: `src/lib/api-client.ts`. Este capítulo lo recorre línea a línea, explica cómo funciona `fetch`, cómo se construyen las URL y, sobre todo, cómo se transforma cualquier fallo —de red, de validación, de negocio o del servidor— en un error uniforme que la interfaz sabe presentar.

15.1 Por qué un cliente propio

↑ Índice

Llamar a `fetch` directamente desde cada sitio funcionaría, pero repetiría en cada llamada las mismas decisiones: la URL base, las cabeceras, la conversión a JSON, la comprobación de `response.ok`, la interpretación del cuerpo de error, el tratamiento de los fallos de red. Y cada repetición sería una ocasión de hacerlo de forma distinta.

El cliente centraliza esas decisiones en un único sitio:

- **La URL base** de la API, configurable por entorno.
- **La serialización**: cuerpos en JSON, parámetros de consulta sin valores vacíos.
- **El modelo de errores**: cualquier fallo se convierte en un `ApiError` con un tipo, un código de estado, un mensaje comprensible y, si los hay, los errores por campo.
- **La cancelación**: las peticiones aceptan una señal para abortarlas.

> **Si vienes de Java…**
>
> Es el equivalente de configurar una única vez un `RestClient` de Spring con su `baseUrl`, sus convertidores de mensajes y un `ResponseErrorHandler` que traduce las respuestas de error a excepciones propias. El servicio de demandas del backend hace exactamente eso con su `RestApplicationCatalog`: construye un `RestClient` con la URL del catálogo y traduce los fallos a un `ResponseStatusException` 503.

15.2 `fetch` en detalle

↑ Índice

`fetch` es la API estándar del navegador para hacer peticiones HTTP. Recibe una URL y unas opciones, y devuelve una promesa con un objeto `Response`:

```
const response = await fetch('/api/applications?page=0', {
  method: 'GET',                                  // GET por defecto
  headers: { Accept: 'application/json' },
  signal,                                         // para cancelar
  // body: JSON.stringify(datos)                  // en POST y PUT
})

response.ok        // true si el estado está entre 200 y 299
response.status    // 200, 404, 500…
await response.json()   // lee y parsea el cuerpo (se puede leer UNA sola vez)
await response.text()   // o lo lee como texto
```

Tres particularidades que el cliente tiene en cuenta:

1. **Solo falla con errores de red.** Un 404 o un 500 no rechazan la promesa: llegan como respuestas con `ok` a `false` (capítulo 2).
2. **El cuerpo es un flujo que se lee una vez.** Por eso el cliente lo lee con `text()` y después decide si lo parsea, en lugar de llamar a `json()` y arriesgarse a un error con cuerpos vacíos o que no son JSON.
3. **No sabe nada de JSON al enviar.** Hay que serializar el cuerpo con `JSON.stringify` y declarar `Content-Type: application/json`.

15.3 `apiRequest`, línea a línea

↑ Índice

`src/lib/api-client.ts`

```
export const API_BASE_URL = (import.meta.env.VITE_API_BASE_URL as string | undefined) ?? '/api'

interface RequestOptions {
  method?: 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'
  query?: QueryParams
  body?: unknown
  signal?: AbortSignal
}

export async function apiRequest<T>(path: string, options: RequestOptions = {}): Promise<T> {
  const { method = 'GET', query, body, signal } = options
  const url = `${API_BASE_URL}${path}${buildQueryString(query)}`

  let response: Response
  try {
    response = await fetch(url, {
      method,
      signal,
      headers: {
        Accept: 'application/json',
        ...(body !== undefined ? { 'Content-Type': 'application/json' } : {}),
      },
      body: body !== undefined ? JSON.stringify(body) : undefined,
    })
  } catch (error) {
    if (error instanceof DOMException && error.name === 'AbortError') throw error
    throw new ApiError({
      message: 'No se puede conectar con el servidor. Comprueba que el backend está arrancado.',
      status: 0,
      kind: 'network',
    })
  }
```

```
  if (!response.ok) {
    throw await toApiError(response)
  }

  if (response.status === 204) {
    return undefined as T
  }
  const text = await response.text()
  return (text ? JSON.parse(text) : undefined) as T
}
```

Paso a paso:

1. **URL.** Base (`/api` por defecto) + ruta del recurso (`/applications/7`) + parámetros. En desarrollo, `/api/...` es una ruta relativa al propio Vite, que la reenvía por el proxy (capítulo 16).
2. **Cabeceras.** Siempre `Accept: application/json`; `Content-Type` solo si hay cuerpo. El *spread* condicional `...(cond ? {…} : {})` añade la cabecera solo cuando procede.
3. **Fallo de red.** Si `fetch` lanza, o es una cancelación (que se deja pasar tal cual, porque no es un error que mostrar), o es un problema de conexión, que se convierte en un `ApiError` de tipo `network` con estado 0.
4. **Respuesta de error.** Si `ok` es falso, `toApiError` interpreta el cuerpo y lanza el error resultante.
5. **Respuesta correcta.** Un `204 No Content` no tiene cuerpo; en otro caso se lee como texto y, si no está vacío, se parsea. El `as T` es el punto de confianza en el contrato con el backend (capítulo 2).

Sobre esta función se construye un pequeño objeto de conveniencia, que es lo que usan los ficheros `-api.ts` de los módulos:

`src/lib/api-client.ts`

```
export const apiClient = {
  get: <T>(path: string, query?: QueryParams, signal?: AbortSignal) =>
    apiRequest<T>(path, { method: 'GET', query, signal }),
  post: <T>(path: string, body: unknown) => apiRequest<T>(path, { method: 'POST', body }),
  put: <T>(path: string, body: unknown) => apiRequest<T>(path, { method: 'PUT', body }),
}
```

Solo existen los métodos que el proyecto usa hoy. Cuando haga falta un `delete` o un `patch`, se añaden en una línea: `apiRequest` ya los admite.

15.4 La cadena de consulta

↑ Índice

`src/lib/api-client.ts`

```
type QueryValue = string | number | boolean | null | undefined
export type QueryParams = Record<string, QueryValue | QueryValue[]>

/** Construye la query string ignorando valores vacíos. Los arrays repiten la clave. */
export function buildQueryString(params?: QueryParams): string {
  if (!params) return ''
  const search = new URLSearchParams()
  for (const [key, raw] of Object.entries(params)) {
    const values = Array.isArray(raw) ? raw : [raw]
    for (const value of values) {
      if (value === undefined || value === null || value === '') continue
      search.append(key, String(value))
    }
```

```
  }
  const qs = search.toString()
  return qs ? `?${qs}` : ''
}
```

- `URLSearchParams` es la API estándar para construir y leer cadenas de consulta; se encarga de codificar los caracteres especiales (la coma de `sort=id,asc` se envía como `%2C`).
- **Los valores vacíos se omiten.** Así, un filtro sin seleccionar (`status: undefined`) simplemente no aparece en la URL, que es lo que espera Spring: un parámetro ausente es «sin filtro».
- **Los arrays repiten la clave** (`?status=OPEN&status=IN_PROGRESS`), el formato que Spring convierte en una `List`. El proyecto aún no lo usa, pero el cliente ya lo admite.

`Object.entries(objeto)` devuelve un array de pares `[clave, valor]`, la forma habitual de recorrer un objeto.

15.5 El modelo de errores

↑ Índice

15.5.1 La clase `ApiError`

src/lib/api-client.ts

```ts
export type ApiErrorKind = 'network' | 'validation' | 'not-found' | 'client' | 'server'

export class ApiError extends Error {
  readonly status: number
  readonly kind: ApiErrorKind
  readonly code?: string
  readonly fieldErrors: Record<string, string>
  // constructor que asigna los campos…
}
```

El campo clave es `kind`: una clasificación pensada para la interfaz, que no necesita conocer todos los códigos HTTP para decidir qué mostrar.

kind	Cuándo	Cómo lo trata la interfaz
`network`	`fetch` no pudo conectar (estado 0)	Icono de sin conexión; pide comprobar el backend
`validation`	400	En formularios, los `fieldErrors` se colocan en sus campos
`not-found`	404	Aviso de recurso inexistente con enlace al listado; no se reintenta
`client`	Otros 4xx (409, 403…)	Mensaje del servidor; en demandas, el 409 tiene un tratamiento propio
`server`	5xx	Mensaje genérico o del servidor; se reintenta

`code` conserva el código de error del backend (`VALIDATION_ERROR`, `RESOURCE_ERROR`), y `status`, el código HTTP exacto, por si una pantalla necesita afinar más.

15.5.2 Interpretar la respuesta de error

src/lib/api-client.ts

```
export async function toApiError(response: Response): Promise<ApiError> {
  const status = response.status
  let body: Partial<ApiErrorBody> | null = null
  try {
    const text = await response.text()
    body = text ? (JSON.parse(text) as Partial<ApiErrorBody>) : null
  } catch {
    // Cuerpo no JSON (p. ej. HTML de un proxy): se usa el mensaje por defecto.
  }

  // Formato propio del backend (ApiErrorResponse).
  const hasBackendFormat = body && typeof body.message === 'string' && body.message.length > 0
  // Formato por defecto de Spring Boot ({timestamp, status, error, path}) o cuerpo vacío.
  const message = hasBackendFormat ? (body!.message as string) : defaultMessage(status)

  return new ApiError({
    message,
    status,
    kind: kindFromStatus(status),
    code: typeof body?.error === 'string' ? body.error : undefined,
    fieldErrors: body?.fieldErrors && typeof body.fieldErrors === 'object' ? body.fieldErrors : unde
fined,
  })
}
```

La función es **defensiva**: no presupone nada del cuerpo. Puede recibir JSON del backend, JSON con otro formato, HTML de un proxy o nada en absoluto, y en todos los casos devuelve un `ApiError` válido. Si el cuerpo trae un mensaje, se usa; si no, `defaultMessage(status)` aporta uno en español según el código: «El recurso solicitado no existe» para el 404, «La operación entra en conflicto con el estado actual del recurso» para el 409, «El servidor no está disponible. Comprueba que el backend está arrancado» para 502, 503 y 504…

15.6 Los formatos de error del backend

↑ Índice

Para interpretar bien los errores hay que saber qué envía el backend. Su `GlobalExceptionHandler` (un `@RestControllerAdvice`) traduce las excepciones a un formato común:

it-service-manager · exception/ApiErrorResponse.java

```
public record ApiErrorResponse(int status, String error, String message, Map<String, String>
fieldErrors) { }
```

Excepción en el backend	Respuesta	`kind` en el frontend
`MethodArgumentNotValidException` (Bean Validation en un `@RequestBody`)	400, `VALIDATION_ERROR`, «La petición contiene campos inválidos», `fieldErrors` con un mensaje por campo	`validation`
`MethodArgumentTypeMismatchException` (por ejemplo, un ID no numérico)	400, `VALIDATION_ERROR`, «Invalid value for parameter: id»	`validation`

Excepción en el backend	Respuesta	`kind` en el frontend
`HttpMessageNotReadableException` (JSON mal formado)	400, `VALIDATION_ERROR`, «Malformed request body»	`validation`
`ResponseStatusException(NOT_FOUND)`	404, `RESOURCE_ERROR`, «Demand not found: 7»	`not-found`
`ResponseStatusException(CONFLICT)` (transición de estado no válida)	409, `RESOURCE_ERROR`, mensaje de la regla	`client`
`ResponseStatusException(BAD_REQUEST)` (estado, prioridad u orden no válidos)	400, `RESOURCE_ERROR`, «Invalid demand sort field: …»	`validation` (sin `fieldErrors`)
`ResponseStatusException(SERVICE_UNAVAILABLE)` (el catálogo de aplicaciones no responde)	503, `RESOURCE_ERROR`, «Application Catalog is unavailable»	`server`
Cualquier otra excepción no controlada	500 con el formato por defecto de Spring: `{ timestamp, status, error, path }`	`server` (mensaje por defecto)

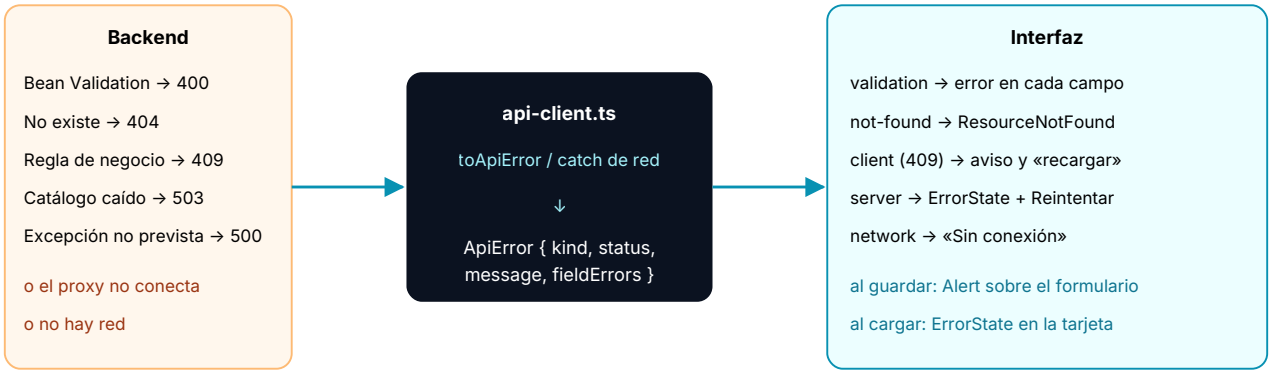


Figura 15.1. Todos los fallos convergen en un `ApiError`, y la interfaz decide qué mostrar según su `kind`.

En el proyecto

Las pruebas de `src/lib/api-client.test.ts` documentan estos formatos con ejemplos: un 400 con `fieldErrors`, un 404 con `RESOURCE_ERROR`, un 500 con el formato por defecto de Spring (el mensaje debe contener «HTTP 500») y un 502 con el cuerpo vacío (el mensaje debe pedir comprobar que el backend está arrancado). Si el backend cambiara su formato de error, estas pruebas indican exactamente qué se espera.

> **Cuidado**
>
> **Los mensajes del backend mezclan idiomas.** El mensaje general de validación está en español («La petición contiene campos inválidos»), pero los de recurso inexistente o de servicio no disponible están en inglés («Application not found: 99», «Application Catalog is unavailable»), y los `fieldErrors` llevan los mensajes por defecto de Bean Validation, que dependen de la configuración regional del servidor. El frontend muestra el mensaje del servidor tal cual cuando existe. Para una aplicación de cara al usuario, lo ideal es que el backend devuelva un **código estable** por error y que el frontend traduzca; o, al menos, que el backend unifique el idioma. Es una nota de integración que retomaremos en el capítulo 25.

15.7 Cómo llegan los errores a la pantalla

↑ Índice

El cliente lanza; TanStack Query captura y expone el error como `query.error` o `mutation.error` (capítulo 17); y cada pantalla decide la presentación. Estos son los caminos:

1. Al cargar datos: `ErrorState`.

src/components/common/state-views.tsx (extracto)

```
export function ErrorState({ className, error, onRetry, title, action }: ErrorStateProps) {
  const isNetwork = isApiError(error) && error.kind === 'network'
  const Icon = isNetwork ? WifiOff : AlertTriangle
  // … título según el tipo, getErrorMessage(error) y botón «Reintentar» si hay onRetry
}
```

2. Recurso inexistente: `ResourceNotFound`. Las páginas de detalle y edición comprueban `query.error.kind === 'not-found'` para mostrar un aviso específico en lugar de un error genérico.

3. Al guardar: una alerta sobre el formulario con `getErrorMessage(mutation.error)`.

4. Errores por campo: `applyServerFieldErrors`. Si el backend rechaza el formulario con `fieldErrors`, cada mensaje se coloca en su campo, como si lo hubiera detectado la validación del cliente:

src/lib/form-errors.ts

```
export function applyServerFieldErrors<T extends FieldValues>(
  error: unknown,
  setError: UseFormSetError<T>,
  knownFields: readonly Path<T>[],
): boolean {
  if (!isApiError(error) || error.kind !== 'validation') return false
  let applied = false
  for (const [field, message] of Object.entries(error.fieldErrors)) {
    if ((knownFields as readonly string[]).includes(field)) {
      setError(field as Path<T>, { type: 'server', message })
      applied = true
    }
  }
  return applied
}
```

Solo se asignan los campos que el formulario conoce (`knownFields`): si el backend devolviera un error para un campo que no existe en la pantalla, se ignora aquí y sigue visible en la alerta general. Es el capítulo 20 el que entra en los formularios.

5. Mensajes legibles en cualquier caso: `getErrorMessage`.

src/lib/api-client.ts

```
export function getErrorMessage(error: unknown): string {
  if (isApiError(error)) return error.message
  if (error instanceof Error) return error.message
  return 'Se ha producido un error inesperado.'
}
```

6. Reintentos automáticos. La configuración global de TanStack Query no reintenta los errores 4xx («repetirlos no cambia el resultado») y reintenta hasta dos veces los demás, incluidos los de red y los 5xx. Lo vemos en el capítulo 17.

15.8 Decisión y alternativas

↑ Índice

Decisión de diseño: Cliente propio sobre `fetch`, sin axios

Decisión. Un cliente mínimo sobre `fetch`, con un modelo de errores propio (`ApiError` con `kind`) que reconoce el formato del backend y el formato por defecto de Spring.

Motivos. `fetch` es estándar y no añade peso al *bundle*. Las funciones que hacen popular a axios (interceptores, transformación de respuestas, cancelación, errores en 4xx/5xx) se resuelven aquí en unas pocas líneas y de forma explícita. El modelo de errores es lo verdaderamente valioso, y sería necesario igual con axios.

Cómo evolucionará. Cuando se añada la autenticación, este es el único sitio donde añadir la cabecera `Authorization` o el envío de *cookies* (`credentials: 'include'`), y donde tratar el 401 de forma global (por ejemplo, redirigiendo al inicio de sesión).

Alternativas. axios (muy extendido; aporta interceptores y una API cómoda, pero es una dependencia más para lo que `fetch` ya cubre); ky (envoltorio ligero sobre `fetch`); clientes generados a partir de OpenAPI (generan los tipos y las funciones; muy recomendables si la API crece).

15.9 Errores típicos y depuración

↑ Índice

- **«No se puede conectar con el servidor».** `fetch` ha fallado a nivel de red: el servidor de Vite no responde o no hay red. Si es un 5xx del proxy, el mensaje será otro (capítulo 16).
- **`SyntaxError: Unexpected token '<'`** en código propio. Se ha intentado parsear como JSON una respuesta HTML (típicamente el `index.html` que devuelve Vite para una ruta de API sin regla de proxy). El cliente del proyecto lo evita leyendo como texto y capturando el error, pero en llamadas escritas a mano es habitual.
- **El backend recibe `null` en todos los campos.** Falta la cabecera `Content-Type: application/json` o el cuerpo no se ha serializado con `JSON.stringify`.
- **Un filtro vacío llega como `status=`.** Hay que omitir los parámetros vacíos, como hace `buildQueryString`.
- **Un error 400 sin errores por campo.** Puede venir de un `ResponseStatusException(BAD_REQUEST)` del servicio (estado, prioridad u orden no válidos), que no rellena `fieldErrors`. Mira el mensaje en *Network → Response*.

En la pestaña *Network*, al seleccionar una petición, *Headers* muestra la URL final, el método, el estado y las cabeceras; *Payload*, el cuerpo y los parámetros enviados; y *Response*, lo que devolvió el servidor. Es la herramienta imprescindible para cualquier problema de integración: te dice si el fallo está en lo que envía el frontend o en lo que responde el backend.

15.10 Ejercicios

↑ Índice

Ejercicio 15.1 · Recorre los formatos de error Ver solución →

Con el backend arrancado, provoca cada uno de estos errores y anota en *Network* el estado, el cuerpo de la respuesta y lo que ve el usuario: (a) visita `/aplicaciones/999999`; (b) sáltate la validación del formulario enviando la petición desde la consola del navegador: `await fetch('/api/applications', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ name: '', description: null, criticality: 'LOW' }) }).then((r) => r.json())`; (c) detén solo el servicio de aplicaciones y abre *Nueva demanda*. ¿Qué `kind` tiene cada `ApiError`?

Ejercicio 15.2 · Añade el método `delete` Ver solución →

Añade `delete` a `apiClient` y escribe una prueba en `api-client.test.ts` que compruebe que una respuesta `204` devuelve `undefined`. Para simular `fetch` en la prueba, usa `vi.stubGlobal('fetch', vi.fn().mockResolvedValue(new Response(null, { status: 204 })))` y restaura con `vi.unstubAllGlobals()`.

Ejercicio 15.3 · Prepara la autenticación Ver solución →

Imagina que el backend pasa a exigir un *token* JWT en la cabecera `Authorization: Bearer …`. Diseña, sin implementarlo del todo, cómo modificarías `apiRequest` para añadir la cabecera a todas las peticiones y para que un 401 provoque un único tratamiento global. ¿Dónde guardarías el *token* y qué riesgos tiene cada opción (memoria, `localStorage`, *cookie* `HttpOnly`)?

15.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 15 Ver respuestas →

1. ¿Qué decisiones centraliza el cliente HTTP que de otro modo se repetirían?
2. ¿Por qué el cliente lee el cuerpo con `text()` en lugar de `json()`?
3. ¿Qué ocurre con un parámetro de consulta `undefined` y por qué es lo correcto para Spring?
4. ¿Qué cinco valores puede tomar `kind` y qué decide la interfaz con cada uno?
5. ¿Por qué `toApiError` no presupone el formato del cuerpo?
6. ¿Qué devuelve el backend ante un error de Bean Validation y qué hace el frontend con `fieldErrors`?
7. ¿Por qué no se reintentan los errores 4xx?
8. ¿Dónde se añadiría la autenticación cuando llegue y por qué ahí?

CAPÍTULO 16

CORS, el proxy de desarrollo y el despliegue

↑ Volver al índice

EN ESTE CAPÍTULO

16.1 La política del mismo origen
16.2 CORS: cómo autoriza un servidor a otro origen
16.3 El proyecto en desarrollo: tres procesos y un proxy
16.4 Opciones para producción
16.5 Un *build* para todos los entornos
16.6 Seguridad básica de una SPA desplegada
16.7 Errores típicos y depuración
16.8 Ejercicios
16.9 Autoevaluación

El navegador aplica una regla de seguridad que condiciona cómo se comunica una SPA con su API: la política del mismo origen. Este capítulo explica esa regla y su mecanismo de excepción, CORS; por qué el proyecto no lo necesita en desarrollo; y cuáles son las opciones para llevar la aplicación a producción, con configuraciones concretas.

16.1 La política del mismo origen

↑ Índice

Un **origen** es la combinación de **esquema, host y puerto**: `http://localhost:5173` y `http://localhost:8080` son orígenes distintos, aunque estén en la misma máquina, porque cambia el puerto. También lo son `http://` y `https://` del mismo dominio.

La **política del mismo origen** (*same-origin policy*) es una regla del navegador: el código JavaScript de una página **no puede leer las respuestas** de peticiones dirigidas a otro origen, salvo que ese otro origen lo autorice expresamente.

¿Por qué existe? Imagina que estás autenticado en la intranet de tu empresa y visitas una web maliciosa. Sin esta regla, el JavaScript de esa web podría hacer peticiones a la intranet —el navegador enviaría tus *cookies* de sesión automáticamente— y leer las respuestas. La política del mismo origen impide que una web lea datos de otra en nombre del usuario.

Cuidado

La política del mismo origen es una protección **del navegador hacia el usuario**, no del servidor. Un `curl`, Postman, un test de integración o el `RestClient` de un servicio Java no la aplican: pueden llamar a cualquier API. Por eso CORS no sustituye a la autenticación ni a la autorización del backend.

16.2 CORS: cómo autoriza un servidor a otro origen

↑ Índice

CORS (*Cross-Origin Resource Sharing*) es el mecanismo estándar por el que un servidor declara, mediante cabeceras HTTP, qué orígenes pueden leer sus respuestas.

16.2.1 Peticiones con verificación previa

Para muchas peticiones, el navegador pregunta antes de enviarlas. Es la **petición de verificación previa** (*preflight*): un `OPTIONS` automático que el código JavaScript no ve. Ocurre, entre otros casos, cuando el método no es `GET`, `HEAD` o `POST`, o cuando se envía un `Content-Type` distinto de los formularios clásicos. Un `PUT` con `Content-Type: application/json`, como el que hace el proyecto al modificar una aplicación, siempre la provoca.

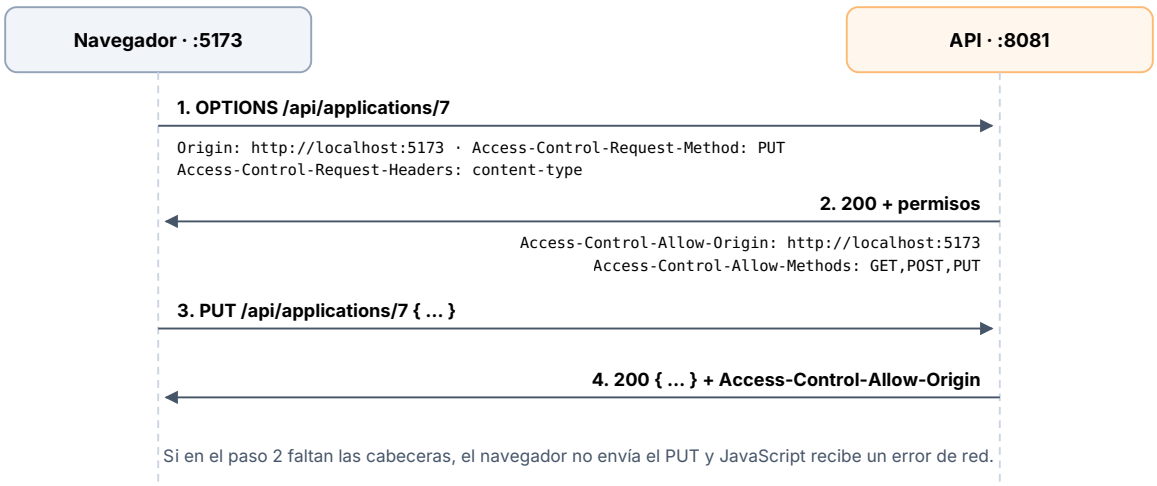


Figura 16.1. Una petición entre orígenes con verificación previa. Sin las cabeceras adecuadas en la respuesta al `OPTIONS`, la petición real nunca se envía.

Las cabeceras principales que devuelve el servidor:

Cabecera	Significado
`Access-Control-Allow-Origin`	Qué origen puede leer la respuesta (uno concreto, o `*` para cualquiera sin credenciales)
`Access-Control-Allow-Methods`	Métodos permitidos
`Access-Control-Allow-Headers`	Cabeceras que puede enviar el cliente (`Content-Type`, `Authorization`…)
`Access-Control-Allow-Credentials`	Si se admiten *cookies* o autenticación HTTP
`Access-Control-Max-Age`	Cuántos segundos puede cachear el navegador la respuesta al `OPTIONS`

Cuando falla, el navegador muestra en consola un mensaje como *Access to fetch at '…' from origin '…' has been blocked by CORS policy*, y para el JavaScript la petición simplemente ha fallado por red: en el proyecto, un `ApiError` de tipo `network`. La pista está siempre en la consola, no en el código.

16.3 El proyecto en desarrollo: tres procesos y un proxy

↑ Índice

Recordemos la topología actual (capítulo 4):

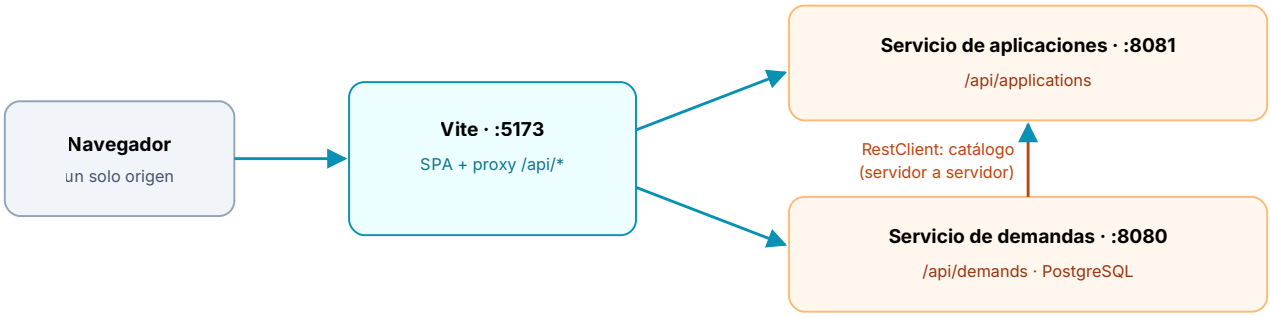


Figura 16.2. En desarrollo, el navegador solo habla con Vite. Los servicios se comunican entre sí sin pasar por el navegador.

- El navegador carga la SPA y llama a `/api/...` en **el mismo origen** (`localhost:5173`): no hay petición entre orígenes y, por tanto, no hay CORS.
- Vite reenvía cada prefijo a su servicio. Entre servidores no existe la política del mismo origen.
- El servicio de demandas consulta el catálogo de aplicaciones directamente con un `RestClient` (`application-catalog.base-url=http://localhost:8081`), también sin CORS.

Por eso el backend **no tiene CORS configurado** y no le hace falta para desarrollar. La cuestión es qué pasa en producción, donde no hay servidor de Vite.

16.4 Opciones para producción

↑ Índice

Hay tres formas de servir la SPA y la API en producción.

16.4.1 Opción A: mismo origen detrás de un proxy inverso (recomendada)

Un servidor web (Nginx, Apache, un balanceador o una pasarela de API) sirve los ficheros de `dist/` y reenvía `/api/...` a los servicios. Es **la misma topología que en desarrollo**, con otro servidor haciendo de Vite:

nginx.conf (ejemplo)

```
server {
  listen 80;
  root /usr/share/nginx/html;          # contenido de dist/

  # API: cada prefijo a su servicio
  location /api/applications { proxy_pass http://applications-service:8081; }
  location /api/demands      { proxy_pass http://demands-service:8080; }

  # Ficheros con huella: caché de larga duración
  location /assets/ {
    add_header Cache-Control "public, max-age=31536000, immutable";
  }

  # SPA: cualquier otra ruta devuelve index.html (fallback del capítulo 1)
  location / {
    try_files $uri /index.html;
    add_header Cache-Control "no-cache";
  }
}
```

Tres reglas concentran todo lo aprendido: el proxy de la API, la caché de los ficheros con huella y el *fallback* a `index.html` para las rutas de la SPA. Como el navegador solo ve un origen, **no hace falta CORS**, y las *cookies* de sesión, si las hubiera, funcionan sin configuración adicional.

16.4.2 Opción B: orígenes distintos con CORS en el backend

La SPA se publica en su propio dominio (por ejemplo, un almacenamiento estático con CDN en `https://itsm.empresa.com`) y la API en otro (`https://api.itsm.empresa.com`). Entonces cada servicio debe declarar CORS. En Spring Boot sería algo así:

Ejemplo de configuración CORS en Spring (no está en el backend)

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("https://itsm.empresa.com")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("Content-Type", "Authorization")
                .maxAge(3600);
    }
}
```

Además, el frontend se construiría con `VITE_API_BASE_URL=https://api.itsm.empresa.com/api`. Con dos servicios hay que configurar CORS en ambos, o poner delante una pasarela (Spring Cloud Gateway, por ejemplo) que lo centralice.

> **Cuidado**
>
> Nunca uses `allowedOrigins("*")` junto con credenciales, y no copies a producción una configuración de desarrollo que permite cualquier origen. CORS bien configurado es una lista corta y explícita de orígenes de confianza.

16.4.3 Opción C: la SPA dentro del propio backend

Spring Boot sirve automáticamente los ficheros de `src/main/resources/static`. Se puede copiar `dist/` ahí durante la construcción y desplegar un único JAR. Es sencillo para una aplicación monolítica, pero exige añadir una regla de *fallback* a `index.html` en Spring y acopla los ciclos de despliegue de frontend y backend. Con la API repartida en dos servicios, deja de ser natural.

	A · Proxy inverso	B · CORS	C · Dentro del backend
CORS	No necesario	Necesario en cada servicio	No necesario
Topología igual a desarrollo	Sí	No	Parcialmente
Despliegue independiente del frontend	Sí	Sí	No
Encaje con varios servicios	Muy bueno (o pasarela de API)	Requiere configurar cada uno	Malo
Mismo *build* para todos los entornos	Sí, con `/api` relativo	No: la URL de la API va en el *build*	Sí

Decisión de diseño: Mismo origen con proxy inverso en producción

Decisión. Mantener en producción la topología de desarrollo: la SPA y la API bajo el mismo origen, con un proxy inverso (o una pasarela de API) que reparte `/api/*` entre los servicios.

Motivos. Sin CORS que mantener; sin diferencias entre entornos; las *cookies* de una futura autenticación funcionan sin configuración especial; y, sobre todo, la URL base de la API sigue siendo la ruta relativa `/api`, de modo que **el mismo *build* sirve para todos los entornos**.

Cuándo cambiaría. Si el frontend tuviera que publicarse en una CDN con dominio propio, independiente de la infraestructura del backend, la opción B sería la adecuada.

16.5 Un *build* para todos los entornos

↑ Índice

Recuerda del capítulo 4 que las variables `VITE_*` se **incrustan en el código al construir**. Si la URL de la API fuera absoluta y distinta en cada entorno (`https://api-pre…`, `https://api…`), habría que construir una versión distinta para desarrollo, preproducción y producción, y lo que se prueba no sería exactamente lo que se despliega.

Con una URL **relativa** (`/api`), el JavaScript no sabe ni le importa en qué dominio está: llama a `/api` de su propio origen, y es el proxy de cada entorno el que sabe dónde están los servicios. El mismo `dist/` (o la misma imagen de contenedor) se promociona de un entorno a otro sin reconstruir. Es un principio clásico de entrega continua: **construir una vez, desplegar muchas**.

16.5.1 Un ejemplo de imagen de contenedor

`Dockerfile (ejemplo)`

```
# 1. Construcción con Node
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# 2. Servidor web con el resultado
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
```

La construcción en dos etapas deja Node fuera de la imagen final: lo que se despliega es un Nginx con ficheros estáticos de unos pocos megas. `npm ci` garantiza que se instalan exactamente las versiones del *lockfile* (capítulo 3).

Si vienes de Java…

Es el mismo patrón que una imagen de Spring Boot construida en dos etapas: una con Maven y la JDK para compilar y otra solo con un JRE y el JAR. Aquí la segunda etapa ni siquiera necesita un entorno de ejecución del lenguaje: solo un servidor web.

16.6 Seguridad básica de una SPA desplegada

↑ Índice

- **Ningún secreto en el *bundle*.** Todo lo que hay en `dist/` es público (capítulo 4). Claves de servicios, contraseñas o *tokens* de sistemas van en el backend.
- **HTTPS siempre**, con redirección desde HTTP.
- **Cabeceras de seguridad** en el servidor web: `Content-Security-Policy` (qué orígenes pueden cargar scripts, estilos o hacer peticiones), `X-Content-Type-Options: nosniff` y `Referrer-Policy`. Una SPA sin scripts externos ni estilos en línea puede tener una política muy estricta.
- **La autorización es del backend.** Ocultar un botón en la interfaz no impide llamar al endpoint (capítulo 1).

16.7 Errores típicos y depuración

↑ Índice

- **`blocked by CORS policy` en consola.** La página y la API están en orígenes distintos y la API no lo permite. O se sirve todo bajo el mismo origen, o se configura CORS en el servicio para ese origen exacto (esquema, host y puerto).
- **La petición `OPTIONS` devuelve 401 o 403.** La seguridad del backend está bloqueando la verificación previa; debe permitir los `OPTIONS` sin autenticación (Spring Security lo hace si se configura CORS en su cadena de filtros).
- **Todo funciona en local y en producción la API devuelve el `index.html`.** El servidor web no tiene la regla del proxy para ese prefijo y está aplicando el *fallback* de la SPA.
- **404 al recargar en una ruta de la SPA.** Falta el *fallback* `try_files $uri /index.html`.
- **Los usuarios ven una versión antigua tras desplegar.** `index.html` se está cacheando. Debe servirse con `Cache-Control: no-cache`; los ficheros de `assets/`, con huella, sí pueden cachearse para siempre.

16.8 Ejercicios

↑ Índice

Ejercicio 16.1 · Provoca un error de CORS Ver solución →

Crea un `.env.local` con `VITE_API_BASE_URL=http://localhost:8081/api` y reinicia Vite. Abre el listado de aplicaciones. ¿Qué error aparece en la consola? ¿Y en *Network*? ¿Qué `kind` tiene el `ApiError` que ve la interfaz? Explica por qué ya no interviene el proxy. Borra el `.env.local` al terminar.

Ejercicio 16.2 · Sirve el build como en producción Ver solución →

Ejecuta `npm run build` y sirve `dist/` con un Nginx en Docker usando la configuración de la sección 16.4, ajustando los destinos del proxy a `host.docker.internal:8080` y `:8081` para llegar a los servicios de tu máquina. Comprueba: que la aplicación funciona, que al recargar en `/demandas/1` no hay 404 y que en *Network* los ficheros de `/assets/` llevan la cabecera de caché de larga duración.

Ejercicio 16.3 · Elige una opción de despliegue Ver solución →

Tu empresa te pide desplegar la aplicación en un entorno donde el equipo de infraestructura ofrece: un balanceador que puede enrutar por prefijo de ruta, un almacenamiento de ficheros estáticos con CDN y un

clúster de contenedores. Escribe una breve propuesta (media página) con la opción elegida, la configuración necesaria y los riesgos. Usa la tabla de la sección 16.4 como guía.

16.9 Autoevaluación

↑ Índice

Autoevaluación del capítulo 16 Ver respuestas →

1. ¿Qué tres elementos definen un origen? ¿Son el mismo origen `http://localhost:5173` y `http://localhost:8080` ?
2. ¿A quién protege la política del mismo origen y de qué ataque?
3. ¿Qué es una petición de verificación previa y cuándo se produce?
4. ¿Por qué el proyecto no necesita CORS en desarrollo?
5. ¿Cuáles son las tres opciones de despliegue y cuál recomienda el proyecto?
6. ¿Por qué una URL de API relativa permite usar el mismo *build* en todos los entornos?
7. ¿Qué cabeceras de caché deben llevar `index.html` y los ficheros de `assets/` ?
8. ¿Sustituye CORS a la autenticación del backend? ¿Por qué?

CAPÍTULO 17

TanStack Query: el estado del servidor

↑ Volver al índice

EN ESTE CAPÍTULO

17.1 Estado del servidor frente a estado del cliente
17.2 El `QueryClient` y la caché
17.3 Anatomía de `useQuery`
17.4 El ciclo de vida de los datos
17.5 La configuración global del proyecto
17.6 Claves jerárquicas
17.7 Paginación sin parpadeos
17.8 Mutaciones: escribir y mantener la caché coherente
17.9 Consultas dependientes y desactivadas
17.10 Cancelación y condiciones de carrera
17.11 Decisión y alternativas
17.12 Errores típicos y depuración
17.13 Ejercicios
17.14 Autoevaluación

Mostrar datos de una API parece sencillo hasta que aparecen los detalles: no repetir peticiones, saber cuándo los datos están desactualizados, mantener la página anterior mientras llega la siguiente, refrescar los listados al guardar, cancelar lo que ya no interesa, reintentar lo que falló. TanStack Query se ocupa de todo eso. Es la librería que más trabajo ahorra en el proyecto, y este capítulo explica su modelo mental y cada decisión de configuración.

17.1 Estado del servidor frente a estado del cliente

↑ Índice

El capítulo 7 distinguía varios tipos de estado. El **estado del servidor** tiene características propias que lo hacen muy distinto del resto:

- **No es tuyo.** Lo tiene el backend; en el navegador solo hay una **copia**, que puede quedar desactualizada en cualquier momento porque otro usuario la cambie.
- **Es asíncrono.** Hay que pedirlo y esperar, con estados de carga y de error.
- **Se comparte.** El mismo dato (el catálogo de aplicaciones) puede necesitarse en varias pantallas a la vez.
- **Se invalida.** Cuando alguien crea una aplicación, todos los listados que la incluirían quedan obsoletos.

Gestionarlo con `useState` y `useEffect` obliga a resolver a mano: carga, errores, caché, deduplicación de peticiones simultáneas, cancelación, condiciones de carrera, reintentos y actualización tras modificar. **TanStack Query** (antes React Query) es una **caché de estado del servidor**: tú le dices *qué* datos necesitas y *cómo* obtenerlos, y ella decide *cuándo* pedirlos, los guarda y los comparte.

Si vienes de Java...

La analogía más útil es la **caché de segundo nivel de Hibernate**, o una caché de Spring (`@Cacheable`) con política de expiración y evicción: lecturas que se sirven de la caché si el dato es reciente, entradas identificadas por una clave, y un `@CacheEvict` tras cada escritura. La diferencia es que TanStack Query además **notifica** a los componentes que usan un dato cuando este cambia, para que se vuelvan a renderizar, y refresca en segundo plano.

17.2 El `QueryClient` y la caché

↑ Índice

En el centro hay un objeto, el `QueryClient`, creado una vez y puesto a disposición de toda la aplicación por `QueryClientProvider` (capítulo 9). Contiene la **caché de consultas**: un mapa de **claves** a **entradas**.

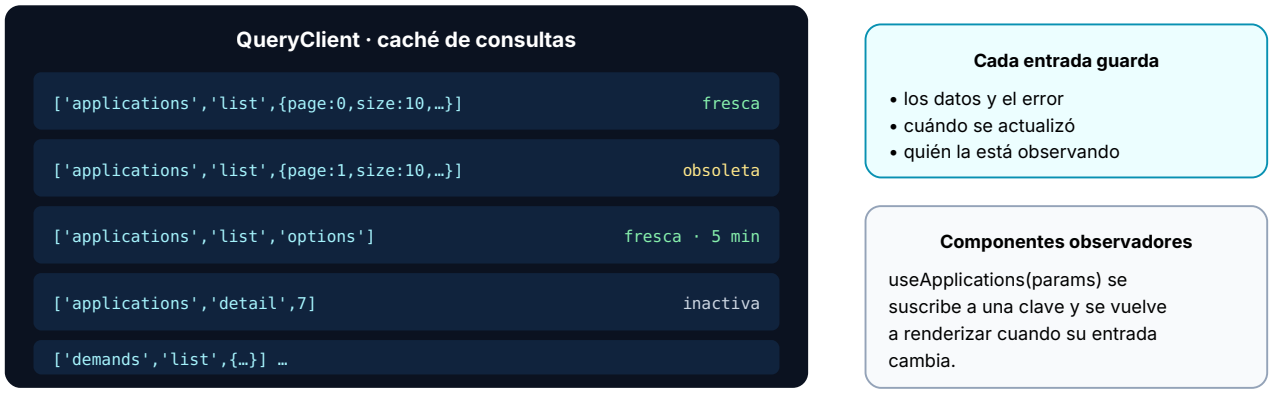


Figura 17.1. La caché de TanStack Query: una entrada por clave, observada por los componentes que la usan.

Si dos componentes piden la misma clave a la vez, TanStack Query hace **una sola petición** y ambos reciben el resultado. Si un componente pide una clave que ya está en caché, recibe los datos **al instante**, y la librería decide si además hay que refrescarlos.

17.3 Anatomía de `useQuery`

↑ Índice

`src/features/applications/api/applications-queries.ts`

```ts
export function useApplication(id: number | undefined) {
  return useQuery({
    queryKey: applicationKeys.detail(id ?? -1),
    queryFn: ({ signal }) => applicationsApi.getById(id!, signal),
    enabled: id !== undefined,
    // Un 404 no se reintenta: la aplicación no existe.
    retry: (failureCount, error) =>
      !(isApiError(error) && error.kind === 'not-found') && failureCount < 2,
  })
}
```

Opción	Qué hace
`queryKey`	Identifica la entrada de la caché. Si cambia, es otra consulta (sección 17.6).
`queryFn`	Cómo obtener los datos. Recibe, entre otras cosas, una `signal` para cancelar la petición. Debe devolver una promesa o lanzar un error.
`enabled`	Si es `false`, la consulta no se ejecuta. Aquí, mientras no haya un `id` válido.
`retry`	Cuántas veces reintentar, o una función que lo decide según el error.
`staleTime`	Cuánto tiempo se consideran frescos los datos (sección 17.4).
`placeholderData`	Qué mostrar mientras se carga la clave nueva (sección 17.7).

El hook devuelve un objeto con los datos y el estado:

Propiedad	Significado
`data`	Los datos, o `undefined` si aún no hay
`error`	El error (en el proyecto, normalmente un `ApiError`)
`isPending`	Todavía no hay datos (primera carga, o consulta desactivada)
`isError`, `isSuccess`	El último resultado fue un error o un éxito
`isFetching`	Hay una petición en curso, sea la primera o un refresco en segundo plano
`isPlaceholderData`	Lo que se muestra son datos provisionales de otra clave
`refetch()`	Vuelve a pedir los datos (el botón *Reintentar*)

La distinción entre **`isPending`** («no tengo nada que enseñar») e **`isFetching`** («estoy pidiendo algo») es clave: el listado muestra el esqueleto con `isPending`, pero con `isFetching` solo atenúa la tabla y desactiva la paginación, porque ya hay datos que mostrar.

> **Si vienes de Java…**
>
> `isPending`, `isError` e `isSuccess` son excluyentes, como los estados de un `CompletableFuture` (sin completar, completado con excepción, completado normalmente). `isFetching` es independiente: una entrada puede estar en `success` con datos antiguos y, a la vez, refrescándose.

17.4 El ciclo de vida de los datos

↑ Índice

Cada entrada de la caché pasa por estas fases:

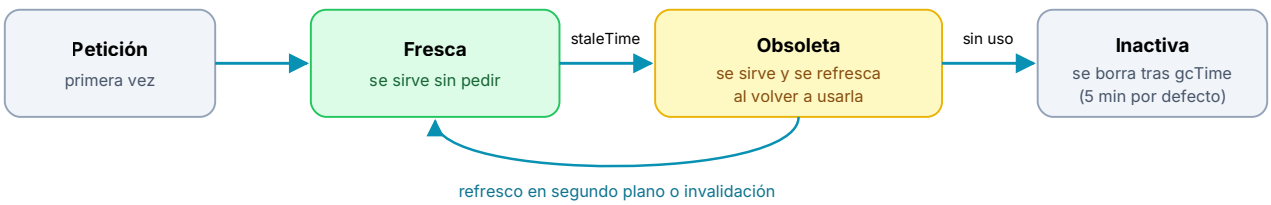


Figura 17.2. Fresca, obsoleta, inactiva. Los datos obsoletos no se tiran: se siguen mostrando mientras se refrescan.

- **Fresca.** Durante `staleTime` desde la última actualización, la entrada se sirve **sin hacer ninguna petición**.
- **Obsoleta.** Pasado ese tiempo, se sigue mostrando, pero se refresca en segundo plano cuando un componente vuelve a montarse con esa clave, cuando la ventana recupera el foco o cuando se recupera la red (según la configuración).
- **Inactiva.** Cuando ningún componente la observa, la entrada permanece en memoria durante `gcTime` (5 minutos por defecto) por si se vuelve a necesitar, y luego se elimina.

Esta estrategia se llama ***stale-while-revalidate***: mostrar enseguida lo que se tiene y actualizarlo en segundo plano. Es la razón de que volver a un listado ya visitado sea instantáneo.

17.5 La configuración global del proyecto

↑ Índice

src/app/query-client.ts

```ts
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30_000,
      refetchOnWindowFocus: false,
      // No se reintentan errores de cliente (4xx): repetirlos no cambia el resultado.
      retry: (failureCount, error) => {
        if (isApiError(error) && error.status >= 400 && error.status < 500) return false
        return failureCount < 2
      },
    },
    mutations: {
      retry: false,
    },
  },
})
```

Cada valor es una decisión frente a los valores por defecto de la librería:

Opción	Por defecto	En el proyecto	Por qué
`staleTime`	0 (siempre obsoleto)	30 segundos	Navegar entre listado, detalle y vuelta no dispara peticiones repetidas; medio minuto es un retraso aceptable para datos de gestión
`refetchOnWindowFocus`	Activado	Desactivado	Al volver a la pestaña no se refresca todo: evita peticiones y parpadeos inesperados mientras se edita
`retry` (consultas)	3 reintentos	2, y nunca en 4xx	Un 404 o un 400 no van a cambiar por insistir; un fallo de red o un 503 momentáneo, sí
`retry` (mutaciones)	0	0 (explícito)	Repetir automáticamente un alta podría crear duplicados: las escrituras no se reintentan

El número `30_000` usa separadores numéricos de JavaScript: el guion bajo solo mejora la legibilidad, como en Java.

Las consultas pueden sobrescribir estos valores. El catálogo de aplicaciones para los desplegables, que cambia poco, usa `staleTime: 5 * 60_000`; el detalle cambia la política de reintentos para no insistir con los 404.

17.6 Claves jerárquicas

↑ Índice

La clave es lo más importante de una consulta: identifica la entrada de la caché **y** permite invalidarla. El proyecto las genera con una factoría por módulo:

src/features/applications/api/applications-queries.ts

```ts
/** Claves de caché jerárquicas para invalidar de forma selectiva. */
export const applicationKeys = {
  all: ['applications'] as const,
  lists: () => [...applicationKeys.all, 'list'] as const,
```

```
  list: (params: ApplicationListParams) => [...applicationKeys.lists(), params] as const,
  details: () => [...applicationKeys.all, 'detail'] as const,
  detail: (id: number) => [...applicationKeys.details(), id] as const,
}
```

Llamada	Clave	Representa
`applicationKeys.all`	`['applications']`	Todo lo del módulo
`applicationKeys.lists()`	`['applications', 'list']`	Todos los listados, con cualquier filtro
`applicationKeys.list(params)`	`['applications', 'list', { status: 'ACTIVE', page: 0, … }]`	Un listado concreto
(catálogo de opciones)	`['applications', 'list', 'options']`	El catálogo para desplegables
`applicationKeys.details()`	`['applications', 'detail']`	Todos los detalles
`applicationKeys.detail(7)`	`['applications', 'detail', 7]`	El detalle de la aplicación 7

Dos propiedades hacen que esto funcione:

- **Las claves se comparan por contenido**, no por referencia. TanStack Query las serializa de forma determinista, incluso ordenando las propiedades de los objetos: `{ page: 0, size: 10 }` y `{ size: 10, page: 0 }` son la misma clave. Por eso se pueden incluir los parámetros del listado tal cual: cada combinación de filtros, página y orden tiene su propia entrada.
- **La invalidación funciona por prefijo.** `invalidateQueries({ queryKey: ['applications', 'list'] })` afecta a **todas** las claves que empiezan así: todos los listados con cualquier filtro, y también el catálogo de opciones. Una sola línea marca como obsoleto todo lo que podría haber cambiado.

> **Decisión de diseño: Factorías de claves por módulo**
>
> Escribir las claves a mano en cada `useQuery` (`['applications', 'list', params]`) funciona, pero basta un error tipográfico para que una invalidación no surta efecto, y nadie lo notaría hasta ver datos antiguos. Centralizar las claves en `applicationKeys` y `demandKeys` garantiza que lectura e invalidación usan exactamente la misma estructura, y documenta la jerarquía de la caché del módulo en cinco líneas.

17.7 Paginación sin parpadeos

↑ Índice

Cuando el usuario pasa de la página 1 a la 2, la clave cambia y, en principio, la nueva entrada no tiene datos: la consulta volvería a `isPending` y la tabla se sustituiría por el esqueleto durante un instante. Para evitarlo, el listado usa `placeholderData`:

`src/features/applications/api/applications-queries.ts`

```
export function useApplications(params: ApplicationListParams) {
  return useQuery({
    queryKey: applicationKeys.list(params),
    queryFn: ({ signal }) => applicationsApi.list(params, signal),
    // Mantiene la página anterior visible mientras se carga la siguiente.
    placeholderData: keepPreviousData,
```

```
  })
}
```

Con `keepPreviousData`, mientras llega la página 2 la consulta sigue devolviendo los datos de la página 1, con `isPlaceholderData` a `true`. La página aprovecha esa señal:

src/features/applications/pages/applications-list-page.tsx

```
<ApplicationsTable … isFetching={query.isFetching && query.isPlaceholderData} />
<PaginationBar … disabled={query.isFetching} />
```

La tabla se atenúa (`opacity-60`) para indicar que su contenido va a cambiar, y la paginación se desactiva para evitar clics acumulados. Cuando llegan los datos nuevos, se sustituyen sin que la página cambie de altura ni pierda la posición del desplazamiento.

17.8 Mutaciones: escribir y mantener la caché coherente

↑ Índice

Las escrituras usan `useMutation`. Este es el alta de aplicaciones:

src/features/applications/api/applications-queries.ts

```
export function useCreateApplication() {
  const queryClient = useQueryClient()
  return useMutation({
    mutationFn: (request: ApplicationCreateRequest) => applicationsApi.create(request),
    onSuccess: (created) => {
      queryClient.setQueryData(applicationKeys.detail(created.id), created)
      return queryClient.invalidateQueries({ queryKey: applicationKeys.lists() })
    },
  })
}
```

Cuando el servidor confirma el alta, `onSuccess` hace dos cosas con la respuesta:

1. **`setQueryData`** escribe la aplicación creada en la entrada de su detalle. Si el usuario pulsa *Ver detalle*, la ficha aparece al instante, sin petición: el servidor ya nos ha dado los datos.
2. **`invalidateQueries`** marca como obsoletos todos los listados (y el catálogo de opciones). Los que estén en pantalla se refrescan en el acto; los demás, cuando se vuelvan a usar. No se intenta insertar la aplicación nueva en cada listado a mano: con filtros, orden y paginación, calcular dónde iría es complejo y propenso a errores. Es más seguro preguntar al servidor.

Al **devolver** la promesa de `invalidateQueries`, la mutación no se considera terminada hasta que los listados visibles se han refrescado, de modo que el estado `isPending` de la mutación cubre todo el proceso.

La mutación de modificación sigue exactamente el mismo patrón. Fíjate en lo que **no** se invalida: los detalles. El detalle de la aplicación modificada ya se ha actualizado con `setQueryData`, y los demás detalles no han cambiado.

17.8.1 Usar una mutación desde una página

src/features/applications/pages/application-create-page.tsx (extracto)

```
const mutation = useCreateApplication()

const onSubmit = form.handleSubmit(async (values) => {
  try {
```

```
    const result = await mutation.mutateAsync({ name: values.name, /* … */ })
    setCreated(result)
    toast.success('Aplicación creada', { description: `«${result.name}» con identificador #$
{result.id}.` })
  } catch (error) {
    applyServerFieldErrors(error, form.setError, ['name', 'description', 'criticality'])
  }
})
```

Propiedad o método	Uso en el proyecto
`mutateAsync(datos)`	Lanza la mutación y devuelve una promesa: permite usar `await` y `try/catch` en el manejador del formulario
`mutate(datos)`	Alternativa sin promesa, con *callbacks*; útil cuando no hay que encadenar nada
`isPending`	Desactiva el botón y muestra «Creando…»
`isError`, `error`	Alerta con el mensaje sobre el formulario
`reset()`	Limpia el estado de la mutación (al pulsar *Crear otra* o *Descartar*)

17.9 Consultas dependientes y desactivadas

↑ Índice

La opción `enabled` permite que una consulta **espere** a tener lo que necesita. El buscador por identificador de *Modificar aplicación* la usa de forma natural:

`src/features/applications/pages/application-edit-selector-page.tsx`

```
const [searchedId, setSearchedId] = useState<number | undefined>()
const query = useApplication(searchedId)   // enabled: searchedId !== undefined
```

Mientras el usuario no busca, `searchedId` es `undefined` y no se hace ninguna petición. Al buscar, la clave pasa a ser `['applications', 'detail', 42]` y la consulta se activa. Si vuelve a buscar el mismo identificador en menos de 30 segundos, la respuesta sale de la caché.

17.10 Cancelación y condiciones de carrera

↑ Índice

Imagina que el usuario pulsa rápidamente la página 2 y la 3. Sin control, la respuesta de la página 2 podría llegar **después** de la de la 3 y mostrarse por error: es una **condición de carrera**. TanStack Query la evita porque cada respuesta se guarda en **su propia clave**: la página 2 se guarda en la entrada de la página 2, y la pantalla solo observa la de la página 3.

Además, la `signal` que recibe `queryFn` y que el proyecto pasa hasta `fetch` permite **cancelar** de verdad la petición cuando deja de interesar (por ejemplo, si el usuario sale de la pantalla antes de que llegue). En *Network* la verás como *(canceled)*. El cliente HTTP deja pasar ese error de cancelación sin convertirlo en un error de red (capítulo 15).

17.11 Decisión y alternativas

↑ Índice

Decisión de diseño: TanStack Query para todo el estado del servidor

Decisión. Todas las lecturas de la API pasan por `useQuery` y todas las escrituras por `useMutation`, con claves jerárquicas por módulo, una configuración global explícita y, tras cada escritura, actualización del detalle e invalidación de los listados.

Motivos. Elimina todo el código de carga, caché, deduplicación, cancelación y reintentos que habría que escribir a mano; da una experiencia más rápida (navegación instantánea a datos ya vistos, paginación sin parpadeos); y hace que las páginas apenas necesiten estado local.

Alternativas.

- `useEffect + useState`: válido para un único caso trivial; en una aplicación real reinventa una caché con errores.
- Redux Toolkit con RTK Query: equivalente en funcionalidad, pero acoplado a Redux, que el proyecto no necesita (capítulo 9).
- SWR: más ligero y con la misma filosofía *stale-while-revalidate*, pero con menos herramientas para mutaciones e invalidación.
- Cargadores de datos de React Router (`loader`): cargan los datos antes de pintar la ruta; útiles, pero sin una caché compartida tan completa. Pueden combinarse con TanStack Query si se quisiera precargar.

17.12 Errores típicos y depuración

↑ Índice

- **Los datos no se actualizan tras guardar.** La mutación no invalida la clave correcta, o la clave de lectura no empieza por el prefijo invalidado. Comprueba que ambas usan la factoría de claves.
- **Peticiones duplicadas o en bucle.** La `queryKey` contiene algo que cambia en cada renderizado (una fecha, un objeto con una función). Las claves deben contener solo datos serializables y estables.
- **`queryFn` devuelve `undefined`.** TanStack Query no admite `undefined` como resultado de una consulta y avisa en consola. Devuelve `null` si no hay datos.
- **La pantalla de carga aparece en cada cambio de página.** Falta `placeholderData: keepPreviousData`.
- **Un error 404 tarda varios segundos en mostrarse.** Se está reintentando: comprueba la política `retry`.
- **Datos de un usuario visibles para otro tras cambiar de sesión** (cuando haya autenticación). Hay que vaciar la caché al cerrar sesión con `queryClient.clear()`.

Para ver la caché por dentro existe **TanStack Query Devtools** (`@tanstack/react-query-devtools`), un panel que lista cada clave, su estado (fresca, obsoleta, inactiva, cargando), sus datos y sus observadores, y permite invalidar o refrescar a mano. El proyecto no lo tiene instalado; añadirlo es el ejercicio 17.1.

17.13 Ejercicios

↑ Índice

Ejercicio 17.1 · Instala las herramientas de TanStack Query Ver solución →

Instala `@tanstack/react-query-devtools` como dependencia de desarrollo y añade `<ReactQueryDevtools initialIsOpen={false} />` dentro de `QueryClientProvider` en `app.tsx` (el componente se excluye automáticamente del *build* de producción). Navega por los listados y detalles y observa

cómo aparecen las claves, cómo pasan de frescas a obsoletas a los 30 segundos y qué ocurre en la caché al crear una aplicación.

Ejercicio 17.2 · Mide el efecto de `staleTime` Ver solución →

Con *Network* abierto, ve del listado de aplicaciones al detalle de una y vuelve al listado, varias veces seguidas. ¿Cuántas peticiones a `/api/applications` ves? Espera 30 segundos y repite. Después cambia temporalmente `staleTime` a `0` y repite el experimento. Resume el compromiso entre frescura de los datos y número de peticiones.

Ejercicio 17.3 · Precarga el detalle al pasar el ratón Ver solución →

Usa `queryClient.prefetchQuery` para precargar el detalle de una aplicación cuando el usuario pase el ratón por su botón *Ver detalle* en la tabla (`onMouseEnter`). Reutiliza `applicationKeys.detail(id)` y `applicationsApi.getById` . Comprueba en *Network* que la petición sale al pasar el ratón y que al hacer clic la ficha aparece sin estado de carga. ¿En qué casos no compensaría esta técnica?

17.14 Autoevaluación

↑ Índice

Autoevaluación del capítulo 17 Ver respuestas →

1. ¿Qué cuatro características distinguen el estado del servidor del estado del cliente?
2. ¿Qué diferencia hay entre `isPending` e `isFetching` ? ¿Qué hace el listado con cada uno?
3. Describe las fases fresca, obsoleta e inactiva de una entrada de la caché.
4. ¿Por qué el proyecto usa `staleTime: 30_000` y desactiva `refetchOnWindowFocus` ?
5. ¿Por qué no se reintentan los 4xx ni las mutaciones?
6. ¿Qué invalida `invalidateQueries({ queryKey: applicationKeys.lists() })` y por qué incluye el catálogo de opciones?
7. ¿Qué hacen `setQueryData` e `invalidateQueries` tras crear una aplicación y por qué no se inserta la aplicación en los listados a mano?
8. ¿Cómo evita TanStack Query que la respuesta tardía de una página se muestre en lugar de la actual?

PARTE VII

Navegación y formularios

18	React Router: rutas, navegación y carga diferida	pág. 171
19	El estado del listado en la URL	pág. 179
20	Formularios con React Hook Form y Zod	pág. 187

↑ Volver al índice

CAPÍTULO 18

React Router: rutas, navegación y carga diferida

↑ Volver al índice

EN ESTE CAPÍTULO

18.1 Qué hace un router en una SPA
18.2 Tres formas de usar React Router
18.3 La tabla de rutas
18.4 Rutas anidadas y `<Outlet />`
18.5 Parámetros de ruta
18.6 Navegar
18.7 Carga diferida de las páginas
18.8 `handle` y `useMatches`: metadatos de las rutas
18.9 Errores y páginas inexistentes
18.10 El diseño de las URL
18.11 Errores típicos y depuración
18.12 Ejercicios
18.13 Autoevaluación

El router decide qué se pinta para cada URL. En el proyecto define el armazón común, monta las rutas de cada módulo, carga cada página bajo demanda, da título a las pestañas y captura los errores inesperados. Este capítulo recorre la tabla de rutas completa y todas las formas de navegar que usa la aplicación.

18.1 Qué hace un router en una SPA

↑ Índice

El capítulo 1 explicó que en una SPA la URL cambia sin recargar la página gracias a la History API. Un **router** convierte esa URL en interfaz:

1. Mantiene sincronizada la URL con el historial del navegador (atrás, adelante, enlaces directos, recargas).
2. Compara la URL con una **tabla de rutas** y decide qué componentes corresponden.
3. Ofrece componentes y hooks para navegar (`Link`, `useNavigate`) y para leer la URL (`useParams`, `useSearchParams`).

React Router es el router más extendido del ecosistema. Desde la versión 7 se importa del paquete `react-router` (antes, `react-router-dom`); el proyecto usa la versión 8.

Si vienes de Java...

La tabla de rutas cumple el papel de los `@RequestMapping` de Spring MVC: asocia patrones de URL (`/aplicaciones/{id}/editar`) con quien los atiende. La diferencia es que aquí el «controlador» es un componente que se ejecuta en el navegador y la «respuesta» es interfaz, no HTML generado en el servidor. Y las rutas se anidan: una ruta padre aporta un armazón común que envuelve a sus hijas, algo parecido a una plantilla de diseño (*layout*) de Thymeleaf aplicada por jerarquía de URL.

18.2 Tres formas de usar React Router

↑ Índice

React Router ofrece tres modos de uso, con capacidades crecientes:

Modo	Cómo se declara	Qué aporta
Declarativo	`<BrowserRouter><Routes><Route … /></Routes></BrowserRouter>` en JSX	Lo básico: emparejar URL y componentes, navegar
De datos	`createBrowserRouter([…])` + `<RouterProvider>`	Además: carga diferida con `lazy`, límites de error por ruta, `handle`, cargadores y acciones de datos
Framework	Plugin de Vite y rutas por ficheros	Además: renderizado en servidor, división automática del código…

El proyecto usa el **modo de datos**, pero **sin cargadores de datos** (`loader`): los datos los gestiona TanStack Query (capítulo 17). Del modo de datos aprovecha la carga diferida, los elementos de error por ruta, el `handle` para los títulos y `useMatches`. El modo *framework* quedó descartado en el capítulo 1 por requerir un servidor Node.

18.3 La tabla de rutas

↑ Índice

`src/app/router.tsx`

```tsx
export const router = createBrowserRouter([
  {
    path: '/',
    element: <AppLayout />,
    // Se muestra mientras se descarga la página inicial (rutas con carga diferida).
    hydrateFallbackElement: <LoadingState className="min-h-svh" />,
    children: [
      {
        errorElement: <RouteErrorPage />,
        children: [
          { index: true, element: <Navigate to="/aplicaciones" replace /> },
          applicationRoutes,
          demandRoutes,
          // Futuros módulos: incidencias, equipos, configuración.
          { path: '*', element: <NotFoundPage /> },
        ],
      },
    ],
  },
])
```

Una vez montadas las rutas de los módulos, el árbol completo es este:

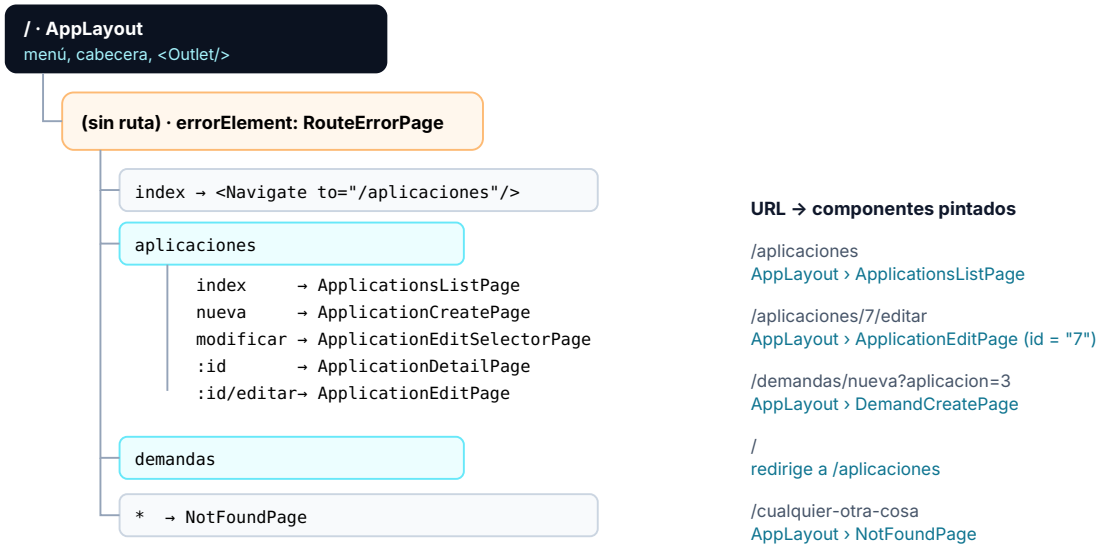


Figura 18.1. Árbol de rutas de la aplicación. Cada URL activa una cadena de rutas, de la raíz a la hoja, y cada una aporta su componente.

Repasemos cada pieza:

- **La ruta raíz** (`path: '/'`) pinta `AppLayout`. Como todas las demás rutas son hijas suyas, el menú y la cabecera están siempre presentes.
- **Una ruta sin `path`** agrupa a todas las hijas para darles un `errorElement` común. Una ruta sin ruta propia no consume ningún segmento de la URL: solo sirve para envolver (capítulo 9).
- **La ruta índice** (`index: true`) es la que se pinta cuando la URL coincide exactamente con la del padre (`/`). Aquí redirige a `/aplicaciones`.
- **Las rutas de los módulos** se importan de cada módulo y se montan como un objeto más del array.
- **La ruta comodín** (`path: '*'`) encaja con cualquier URL que no haya encajado antes: la página 404.

18.4 Rutas anidadas y `<Outlet />`

↑ Índice

Cuando la URL es `/aplicaciones/7/editar`, React Router encuentra la cadena de rutas que encaja: raíz → grupo de error → `aplicaciones` → `:id/editar`. Cada ruta de la cadena pinta su componente, y cada padre indica **dónde** va el hijo con `<Outlet />`:

`src/components/layout/app-layout.tsx (extracto)`

```
<main className="mx-auto w-full max-w-7xl px-4 py-6 lg:px-8 lg:py-8">
  <Outlet />
</main>
```

Las rutas que no tienen `element` propio (como `aplicaciones` o el grupo de error) simplemente dejan pasar a su hijo.

Fíjate en que las rutas hijas se escriben **sin barra inicial**: `'aplicaciones'`, `'nueva'`, `':id/editar'`. Son **relativas** a su padre, así que `nueva` dentro de `aplicaciones` corresponde a `/aplicaciones/nueva`. Esto permite que cada módulo declare sus rutas sin saber dónde se montará.

18.4.1 ¿nueva o :id?

La URL `/aplicaciones/nueva` encajaría en principio con dos rutas: `nueva` y `:id` (con `id = "nueva"`). React Router no elige por orden de declaración, sino por **puntuación**: un segmento estático (`nueva`) vale más que uno dinámico (`:id`). Por eso `/aplicaciones/nueva` y `/aplicaciones/modificar` llegan a sus páginas, y solo lo demás llega a la de detalle.

18.5 Parámetros de ruta

↑ Índice

Los segmentos que empiezan por dos puntos son **parámetros**. Se leen con `useParams`:

src/features/applications/pages/application-detail-page.tsx

```
const { id: rawId } = useParams()
const id = parseEntityId(rawId)
const query = useApplication(id)
```

Dos detalles importantes:

- **Los parámetros siempre son texto** (o `undefined`). `useParams()` devuelve `{ id: '7' }`, no el número 7.
- **La URL la puede escribir cualquiera.** Antes de usar el parámetro hay que validarlo:

src/lib/route-params.ts

```
/** Convierte un parámetro de ruta (:id) en un identificador numérico válido. */
export function parseEntityId(raw: string | null | undefined): number | undefined {
  if (!raw || !/^\d+$/.test(raw)) return undefined
  const id = Number(raw)
  return Number.isSafeInteger(id) && id > 0 ? id : undefined
}
```

Solo dígitos, entero seguro y positivo. `/aplicaciones/abc`, `/aplicaciones/-1` o `/aplicaciones/1.5` producen `undefined`, y la página muestra el aviso de recurso inexistente **sin hacer ninguna petición** al backend. Si el identificador es válido pero no existe, la consulta devuelve un 404 y se muestra el mismo aviso.

> **Si vienes de Java...**
>
> Es la diferencia entre `@PathVariable Long id` (Spring convierte y, si no puede, responde 400) y recibir un `String` y convertirlo a mano. En el frontend no hay conversión automática, así que la validación es responsabilidad de quien lee el parámetro. Que el backend también lo valide (el `MethodArgumentTypeMismatchException` del capítulo 15) es la segunda línea de defensa.

18.6 Navegar

↑ Índice

El proyecto usa cuatro mecanismos, cada uno para una situación.

1. `Link`, para enlaces que el usuario pulsa. Genera un `<a href>` real (accesible, que se puede abrir en otra pestaña) e intercepta el clic para navegar sin recargar:

```
<Link to="/aplicaciones/nueva">…</Link>
<Link to={`/demandas?aplicacion=${id}`}>Ver demandas</Link>
```

2. `NavLink`, para enlaces que deben saber si están activos. Los del menú lateral (capítulo 12). Su `className` puede ser una función que recibe `{ isActive }`, y la prop `end` exige coincidencia exacta.

3. `useNavigate`, para navegar desde código, cuando la navegación es consecuencia de algo que no es un enlace:

`src/features/applications/pages/application-edit-selector-page.tsx`

```tsx
const navigate = useNavigate()
// …
<ApplicationsTable … mode="select" onSelect={(app) => navigate(`/aplicaciones/${app.id}/editar`)} />
```

4. `<Navigate />`, para redirigir al renderizar: la ruta índice redirige a `/aplicaciones` en cuanto se pinta.

18.6.1 Añadir o sustituir en el historial

Por defecto, navegar **añade** una entrada al historial (como `history.pushState`), y el botón *atrás* vuelve a la página anterior. Con `replace`, la navegación **sustituye** la entrada actual (`history.replaceState`):

- `<Navigate to="/aplicaciones" replace />`: si el usuario abre `/` y es redirigido, al pulsar *atrás* no vuelve a `/` (que le redirigiría otra vez, atrapándolo).
- `updateParams({ page: totalPages - 1 }, { replace: true })`: la corrección automática de una página fuera de rango no deja una entrada «rota» en el historial (capítulo 19).

Y una excepción deliberada: `RouteErrorPage` usa `window.location.assign('/aplicaciones')`, una **recarga completa**, para empezar de cero tras un error inesperado (capítulo 9).

18.7 Carga diferida de las páginas

↑ Índice

Cada ruta de módulo declara su página con `lazy`:

`src/features/applications/routes.ts (extracto)`

```ts
{
  path: ':id/editar',
  handle: { title: 'Modificar aplicación' },
  lazy: () =>
    import('./pages/application-edit-page').then((m) => ({ Component: m.ApplicationEditPage })),
},
```

`lazy` es una función que devuelve una promesa con las propiedades de la ruta que faltan; aquí, `Component`. El router la ejecuta **la primera vez** que la ruta se activa, descarga el *chunk* de la página (capítulo 1) y guarda el resultado para las siguientes. El `.then(m => ({ Component: m.ApplicationEditPage }))` adapta la exportación con nombre del módulo al formato que espera el router.

Mientras se descarga:

- En la **carga inicial** (abrir directamente una URL), el router muestra `hydrateFallbackElement`: el `LoadingState` a pantalla completa.
- En una **navegación posterior**, mantiene la pantalla actual hasta que la nueva está lista. Con *chunks* de pocos kilobytes, suele ser imperceptible.

Decisión de diseño: Carga diferida por página

Decisión. Cada página es un *chunk* independiente; el *bundle* inicial contiene React, el router, TanStack Query, el layout y los componentes comunes.

Motivos. El primer pintado es más rápido y el coste de añadir módulos no recae sobre quien solo usa uno. La sintaxis está concentrada en `routes.ts`, así que las páginas son componentes normales que no saben que se cargan bajo demanda.

Coste. Una pequeña espera la primera vez que se visita cada página. Si llegara a notarse, React Router permite precargar el código de una ruta al pasar el ratón por un enlace.

18.8 `handle` y `useMatches`: metadatos de las rutas

↑ Índice

Cada ruta puede llevar un objeto `handle` con datos arbitrarios. El proyecto lo usa para el título de la pestaña:

```
{ path: 'nueva', handle: { title: 'Nueva aplicación' }, lazy: … }
```

`useMatches()` devuelve la cadena de rutas activas, de la raíz a la hoja, con su `handle`. El hook `useDocumentTitle` del layout (capítulo 8) recorre esa cadena desde la hoja y toma el primer título que encuentra:

src/components/layout/app-layout.tsx

```
const matches = useMatches()
const title = [...matches]
  .reverse()
  .map((m) => (m.handle as { title?: string } | undefined)?.title)
  .find(Boolean)
```

La ventaja es que **cada ruta declara su título junto a su definición**, en lugar de que cada página tenga que acordarse de cambiar `document.title`. El mismo mecanismo serviría para construir unas migas de pan automáticas.

18.9 Errores y páginas inexistentes

↑ Índice

Hay tres situaciones distintas, y el proyecto da a cada una su respuesta:

Situación	Ejemplo	Quién responde
La URL no corresponde a ninguna ruta	`/informes`	Ruta comodín `*` → `NotFoundPage`, dentro del layout
La URL encaja, pero el recurso no existe o el `id` no es válido	`/aplicaciones/999999`, `/demandas/abc`	La propia página → `ResourceNotFound` con enlace a su listado
Un componente falla al renderizarse	Un error de programación	`errorElement` → `RouteErrorPage`, dentro del layout

`RouteErrorPage` obtiene el error con `useRouteError()` y distingue con `isRouteErrorResponse()` las respuestas de error que lanza el propio router (con `status` y `statusText`) de los errores normales de JavaScript.

18.10 El diseño de las URL

↑ Índice

Las URL forman parte de la interfaz: el usuario las ve, las guarda y las comparte. El proyecto sigue unos criterios coherentes:

Criterio	Ejemplo
En español, como el resto de la interfaz	`/aplicaciones`, `/demandas/nueva`
Recurso en plural, identificador después	`/demandas/7`
Acciones como segmento final	`/aplicaciones/7/editar`
Filtros y estado del listado en la consulta	`/demandas?aplicacion=3&estado=OPEN&pagina=2`
Enlaces profundos entre módulos	*Ver demandas* → `/demandas?aplicacion=7`; `/demandas/nueva?aplicacion=7` preselecciona la aplicación

Mientras tanto, el código y la API usan los nombres en inglés del dominio del backend (`applications`, `demands`). La traducción ocurre en dos sitios: las rutas y los parámetros de la URL (capítulo 19).

Decisión de diseño: Router de datos sin cargadores

Decisión. `createBrowserRouter` con rutas por módulo, carga diferida, `errorElement` y `handle`, pero sin `loader` ni `action`: los datos se piden desde los componentes con TanStack Query.

Motivos. Un único mecanismo para los datos (con su caché, sus invalidaciones y sus estados explícitos) en lugar de dos. Los cargadores obligan a esperar los datos antes de pintar la ruta; el proyecto prefiere pintar la página enseguida con un esqueleto de carga.

Alternativas. Cargadores de React Router combinados con TanStack Query (`queryClient.ensureQueryData` dentro del `loader`) para empezar a pedir los datos en paralelo a la descarga del código; **TanStack Router**, con rutas y parámetros de búsqueda totalmente tipados, una opción atractiva para proyectos que empiezan desde cero.

18.11 Errores típicos y depuración

↑ Índice

- **`useNavigate() may be used only in the context of a <Router> component`.** El componente se renderiza fuera del `RouterProvider`, típicamente en una prueba o en `App` por encima del router.
- **Una ruta hija no se pinta.** Su padre no tiene `<Outlet />`, o la ruta se declaró con barra inicial (`'/nueva'`), lo que la convierte en absoluta.
- **Al pulsar *atrás* el usuario vuelve a una página que le redirige otra vez.** Falta `replace` en la redirección.
- **Un enlace recarga la página entera.** Se ha usado `<a href>` en lugar de `Link` para una ruta interna.
- **`/aplicaciones/abc` hace una petición al backend.** Falta validar el parámetro antes de usarlo.
- **404 al recargar en producción.** Es el servidor, no el router: falta el *fallback* a `index.html` (capítulo 16).

18.12 Ejercicios

↑ Índice

Ejercicio 18.1 · Añade una ruta de informes Ver solución →

Crea una página `ApplicationsReportPage` sencilla (un `PageHeader` y un texto) y móntala en `/aplicaciones/informe` con carga diferida y título propio. Comprueba: que el título de la pestaña cambia, que se descarga un *chunk* nuevo al visitarla y que no la confunde con `:id`. ¿Qué ocurre si la declaras después de `:id` en el array?

Ejercicio 18.2 · Migas de pan automáticas Ver solución →

Añade a algunas rutas un `handle: { crumb: 'Aplicaciones' }` (y `crumb` en las hijas) y crea un componente `Breadcrumbs` en el layout que use `useMatches()` para pintar la ruta de migas con enlaces a cada nivel (`match.pathname`). ¿Cómo mostrarías en la miga de un detalle el nombre de la aplicación en lugar de «Detalle»? Pista: piensa en qué datos tiene la página y cuáles el layout.

Ejercicio 18.3 · Navega desde código con estado Ver solución →

Al crear una aplicación, en lugar de mostrar la tarjeta de resultado, navega al detalle de la aplicación creada llamando a `navigate` con la ruta `/aplicaciones/` seguida de `result.id`. ¿Aparece la ficha sin estado de carga? ¿Por qué? (Recuerda `setQueryData` del capítulo 17.) Valora qué experiencia es mejor para el usuario y deshaz el cambio si prefieres la original.

18.13 Autoevaluación

↑ Índice

Autoevaluación del capítulo 18 Ver respuestas →

1. ¿Qué tres modos ofrece React Router y cuál usa el proyecto?
2. ¿Para qué sirve una ruta sin `path` como la que contiene el `errorElement`?
3. ¿Qué hace `<Outlet />` y quién lo usa en el proyecto?
4. ¿Por qué `/aplicaciones/nueva` no llega a la página de detalle con `id = "nueva"`?
5. ¿Por qué hay que validar los parámetros de ruta y qué hace `parseEntityId`?
6. ¿Cuándo se usa `Link`, `NavLink`, `useNavigate` y `<Navigate />`?
7. ¿Qué diferencia hay entre navegar con y sin `replace`?
8. ¿Qué muestra el router mientras descarga una página en la carga inicial y en una navegación posterior?
9. ¿Qué tres situaciones de «no encontrado» o error distingue el proyecto y quién responde a cada una?

CAPÍTULO 19

El estado del listado en la URL

↑ Volver al índice

EN ESTE CAPÍTULO

19.1 Por qué en la URL
19.2 El diseño de los parámetros
19.3 El ciclo completo
19.4 Leer y validar
19.5 Escribir
19.6 Actualizar desde la página
19.7 El historial: añadir o sustituir
19.8 La URL y la caché, juntas
19.9 Enlaces profundos entre pantallas
19.10 Pruebas
19.11 Errores típicos y depuración
19.12 Ejercicios
19.13 Autoevaluación

Los filtros, la página, el tamaño y el orden de los listados no viven en el estado de React, sino en la barra de direcciones. Esta decisión, aparentemente pequeña, condiciona el diseño de las páginas de listado, de varias funciones de `lib/` y de sus pruebas. Este capítulo explica el porqué, el diseño de los parámetros y el ciclo completo desde un clic hasta la petición al backend.

19.1 Por qué en la URL

↑ Índice

Compara dos versiones del listado de aplicaciones. En la primera, los filtros se guardan con `useState`; en la segunda, en la URL.

Situación	Con `useState`	Con la URL
El usuario recarga la página (F5)	Pierde filtros, página y orden	Lo conserva todo
Envía el enlace a un compañero («mira estas aplicaciones críticas inactivas»)	El compañero ve el listado sin filtrar	Ve exactamente lo mismo
Guarda la vista en marcadores	Imposible	Funciona
Entra en un detalle y pulsa *atrás*	Vuelve al listado reiniciado (página 1, sin filtros)	Vuelve a la misma página con los mismos filtros
Recorre las páginas y pulsa *atrás*	Sale del listado	Vuelve a la página anterior del listado

Para un listado de gestión, que el usuario consulta, filtra y comparte a diario, la diferencia es enorme. Por eso el proyecto lo trata como **estado de navegación** (capítulo 7). El selector del catálogo de *Modificar aplicación* es la excepción consciente: es un componente auxiliar y usa `useState`.

> **Si vienes de Java...**
> Es lo mismo que en una aplicación JSP o Thymeleaf bien hecha: un formulario de búsqueda con `method="GET"` deja los criterios en la URL (`/aplicaciones?estado=ACTIVE&page=2`), y el controlador los lee con `@RequestParam` . En una SPA es fácil caer en la tentación de guardarlo todo en memoria y perder esas ventajas; el proyecto las recupera de forma deliberada.

19.2 El diseño de los parámetros

↑ Índice

```
/aplicaciones?estado=INACTIVE&criticidad=HIGH&pagina=3&tamano=20&orden=name,desc
/demandas?aplicacion=7&estado=OPEN&prioridad=CRITICAL&orden=updatedAt,desc
```

Decisión	Motivo
Nombres en español (`estado` , `pagina` , `tamano` , `orden`)	La URL forma parte de la interfaz que ve el usuario
Valores con los códigos del backend (`INACTIVE` , `HIGH`)	Son estables e inequívocos; las etiquetas pueden cambiar o traducirse
Páginas desde 1	Es como cuenta un usuario; Spring Data cuenta desde 0 y la conversión se hace en el código
Orden como `campo,dirección`	Mismo formato que el `sort` de Spring Data, fácil de leer
Se omiten los valores por defecto	La URL del listado sin filtros es simplemente `/aplicaciones` , limpia y estable
Sin `ñ` ni tildes (`tamano` , `pagina`)	Evitan la codificación `%C3%B1` en la barra de direcciones

Los nombres de los parámetros de paginación son comunes a todos los listados:

`src/lib/list-params.ts`

```ts
/** Nombres de los parámetros de paginación en la URL (comunes a todos los listados). */
export const PAGINATION_URL_KEYS = { page: 'pagina', size: 'tamano', sort: 'orden' } as const
```

19.3 El ciclo completo

↑ Índice

El capítulo 14 presentó las tres representaciones del estado de un listado. Este es el ciclo que las conecta cuando el usuario cambia algo:

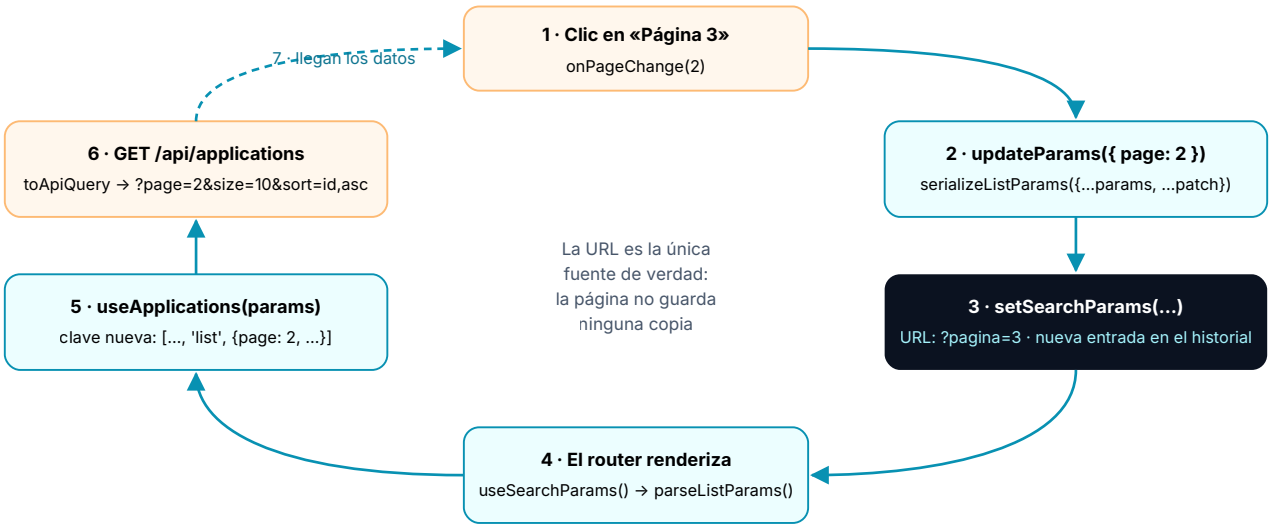


Figura 19.1. Del clic a la petición. La página nunca modifica los parámetros directamente: modifica la URL y vuelve a leerla.

La idea clave es que **la URL es la única fuente de verdad**. La página no tiene un `useState` con los filtros que haya que mantener sincronizado con la URL: los **calcula** a partir de ella en cada renderizado. Cuando el usuario pulsa *atrás*, la URL cambia, la página se vuelve a renderizar con los parámetros anteriores, y TanStack Query sirve esos datos desde la caché. No hay ningún código específico para el botón *atrás*: funciona solo.

19.4 Leer y validar

↑ Índice

`src/features/applications/pages/applications-list-page.tsx`

```
const [searchParams, setSearchParams] = useSearchParams()
const params = useMemo(() => parseListParams(searchParams), [searchParams])
```

`useSearchParams` de React Router devuelve un objeto `URLSearchParams` con la consulta actual y una función para cambiarla. `parseListParams` lo convierte en un `ApplicationListParams` tipado y **válido**:

`src/features/applications/model/list-params.ts`

```
export function parseListParams(search: URLSearchParams): ApplicationListParams {
  const sort = parseSortParam(search.get('orden'), APPLICATION_SORT_FIELDS, {
    field: DEFAULT_LIST_PARAMS.sortField,
    direction: DEFAULT_LIST_PARAMS.sortDirection,
  })
  return {
    status: parseEnumParam(APPLICATION_STATUSES, search.get('estado')),
    criticality: parseEnumParam(APPLICATION_CRITICALITIES, search.get('criticidad')),
    page: parsePageParam(search.get('pagina'), DEFAULT_LIST_PARAMS.page),
    size: parseSizeParam(search.get('tamano'), DEFAULT_LIST_PARAMS.size),
    sortField: sort.field,
    sortDirection: sort.direction,
  }
}
```

Cada parámetro se valida con una función genérica de `lib/list-params.ts`, y **cualquier valor no válido se sustituye por el valor por defecto** en lugar de provocar un error:

Función	Acepta	Ejemplos rechazados
`parseEnumParam(allowed, value)`	Solo valores de la lista	`estado=FOO`, `estado=active` (mayúsculas obligatorias)
`parsePageParam(value)`	Enteros ≥ 1 en la URL; devuelve página 0-based	`pagina=0`, `pagina=-1`, `pagina=abc`
`parseSizeParam(value, fallback)`	Solo las opciones del selector: 5, 10, 20, 50	`tamano=7`, `tamano=10000`
`parseSortParam(value, fields, fallback)`	Campo de la lista y dirección `asc` / `desc`, validados por separado	`orden=password,up` → campo y dirección por defecto
`parseIdParam(value)`	Enteros positivos	`aplicacion=0`, `aplicacion=1.5`

¿Por qué tanto rigor? Porque **la URL es una entrada del usuario**: puede venir de un enlace antiguo, de un marcador de cuando existían otros filtros o de alguien que la edita a mano. Validarla evita enviar al backend valores que no entiende (que responderían con un 400 o, como vimos en el capítulo 14, con un error por campo de ordenación no admitido) y limita lo que se puede pedir: nadie puede solicitar una página de diez mil elementos cambiando `tamano`.

> **Si vienes de Java...**
>
> Es la validación de `@RequestParam` que harías con Bean Validation o con un `Converter` de Spring, pero con una política distinta: en lugar de responder 400, se corrige silenciosamente al valor por defecto. Para una URL que escribe o pega el usuario, «mostrar algo razonable» es mejor experiencia que una pantalla de error.

19.5 Escribir

↑ Índice

El camino inverso convierte los parámetros en URL **omitiendo los valores por defecto**:

`src/features/applications/model/list-params.ts`

```ts
/** Serializa a URL omitiendo los valores por defecto para mantenerla limpia. */
export function serializeListParams(params: ApplicationListParams): URLSearchParams {
  const search = new URLSearchParams()
  if (params.status) search.set('estado', params.status)
  if (params.criticality) search.set('criticidad', params.criticality)
  writePaginationParams(search, params, DEFAULT_LIST_PARAMS)
  return search
}
```

`src/lib/list-params.ts`

```ts
/** Escribe paginación y orden en la URL omitiendo los valores por defecto. */
export function writePaginationParams(search, current, defaults): void {
  if (current.page > 0) search.set(PAGINATION_URL_KEYS.page, String(current.page + 1))
  if (current.size !== defaults.size) search.set(PAGINATION_URL_KEYS.size, String(current.size))
  if (current.sortField !== defaults.sortField || current.sortDirection !== defaults.sortDirection)
{
    search.set(PAGINATION_URL_KEYS.sort, `${current.sortField},${current.sortDirection}`)
  }
}
```

Lectura y escritura son **inversas**: serializar unos parámetros y volver a leerlos debe dar exactamente los mismos parámetros. Es una propiedad que se prueba explícitamente (sección 19.10).

19.6 Actualizar desde la página

↑ Índice

Todos los cambios pasan por una única función de la página:

src/features/applications/pages/applications-list-page.tsx

```
const updateParams = useCallback(
  (patch: Partial<ApplicationListParams>, options?: { replace?: boolean }) => {
    setSearchParams(serializeListParams({ ...params, ...patch }), options)
  },
  [params, setSearchParams],
)
```

Recibe solo lo que cambia (`Partial<…>`), lo combina con los parámetros actuales, serializa y escribe la URL. Los componentes la usan así:

src/features/applications/pages/applications-list-page.tsx

```
<ApplicationFilters … onChange={(filters) => updateParams({ ...filters, page: 0 })} />
<PaginationBar …
  onPageChange={(page) => updateParams({ page })}
  onSizeChange={(size) => updateParams({ size, page: 0 })}
/>
const handleSortChange = (field: ApplicationSortField) => updateParams({ ...nextSort(params,
field), page: 0 })
```

Fíjate en `page: 0` en los filtros, en el tamaño y en el orden. **Cualquier cambio que altere el conjunto de resultados o su orden vuelve a la primera página.** Si el usuario está en la página 5 y filtra por *Crítica*, quizá solo haya una página de resultados; y si cambia el orden, «la página 5» deja de tener sentido. Solo el cambio de página conserva lo demás.

`nextSort` decide la siguiente ordenación al pulsar una cabecera: si es la misma columna, alterna entre ascendente y descendente; si es otra, empieza en ascendente.

19.6.1 Corregir una página fuera de rango

Si el usuario abre un enlace guardado con `?pagina=9` y ahora solo hay 4 páginas, el backend devuelve una página vacía. El efecto que vimos en el capítulo 8 lo corrige cuando llegan los datos:

src/features/applications/pages/applications-list-page.tsx

```
const totalPages = query.data?.totalPages
useEffect(() => {
  if (totalPages !== undefined && totalPages > 0 && params.page > totalPages - 1) {
    updateParams({ page: totalPages - 1 }, { replace: true })
  }
}, [totalPages, params.page, updateParams])
```

19.7 El historial: añadir o sustituir

↑ Índice

Acción	Historial	Motivo
Cambiar de página, filtro, tamaño u orden	Añade una entrada	El usuario espera que *atrás* deshaga su última acción de navegación
Corrección automática de página fuera de rango	Sustituye la entrada	No es una acción del usuario; volver a la página inexistente no tiene sentido

Una consecuencia interesante: como cada cambio crea una entrada, pulsar *atrás* varias veces recorre el historial de filtros y páginas del usuario. Hay equipos que prefieren que los filtros usen `replace` para que *atrás* salga directamente del listado. Ambas opciones son razonables; lo importante es elegirla de forma consciente y aplicarla igual en todos los listados.

19.8 La URL y la caché, juntas

↑ Índice

Cada combinación de parámetros es una clave distinta de TanStack Query (capítulo 17). La URL y la caché forman así un sistema muy eficiente:

- Volver a una combinación ya visitada en los últimos 30 segundos no hace ninguna petición.
- El botón *atrás* muestra al instante el listado anterior, con su página y sus filtros.
- Los parámetros por defecto se omiten de la URL, pero sí forman parte de la clave: `/aplicaciones` y `/aplicaciones?pagina=1` producen los mismos parámetros y, por tanto, la misma entrada de caché.

19.9 Enlaces profundos entre pantallas

↑ Índice

Al vivir en la URL, el estado de un listado se puede **construir desde otra pantalla** con un simple enlace:

src/features/applications/pages/application-detail-page.tsx

```
<Link to={`/demandas?aplicacion=${id}`}>
  <ClipboardList /> Ver demandas
</Link>
```

El detalle de una aplicación lleva al listado de demandas ya filtrado por ella, sin compartir ningún estado entre módulos: el listado de demandas lee `aplicacion` de la URL como si lo hubiera elegido el usuario en el filtro. Del mismo modo, la página de alta de demandas lee `?aplicacion=3` para preseleccionar la aplicación (capítulo 22).

19.10 Pruebas

↑ Índice

Las funciones de lectura y escritura son funciones puras, el candidato perfecto para pruebas unitarias:

src/features/applications/model/list-params.test.ts (extracto)

```
it('descarta valores no válidos en la URL', () => {
  const params = parseListParams(
    new URLSearchParams('estado=FOO&criticidad=x&pagina=-1&tamano=7&orden=password,up'),
  )
```

```
  expect(params).toEqual({ ...DEFAULT_LIST_PARAMS, status: undefined, criticality: undefined })
})

it('serializa omitiendo los valores por defecto y es reversible', () => {
  expect(serializeListParams(DEFAULT_LIST_PARAMS).toString()).toBe('')
  const original = { status: 'ACTIVE' as const, criticality: 'CRITICAL' as const, page: 1, size: 50,
                     sortField: 'status' as const, sortDirection: 'desc' as const }
  expect(parseListParams(serializeListParams(original))).toEqual(original)
})
```

La segunda prueba comprueba las dos propiedades clave del diseño: los valores por defecto producen una URL vacía, y serializar y volver a leer es la identidad. Las funciones genéricas tienen sus propias pruebas en `src/lib/list-params.test.ts`.

Decisión de diseño: El estado de los listados vive en la URL

Decisión. Filtros, página, tamaño y orden de los listados principales se guardan en la URL, con nombres en español, páginas desde 1 y valores por defecto omitidos. La página los deriva de la URL en cada renderizado, los valida con funciones puras y probadas, y los actualiza escribiendo la URL.

Motivos. Recargar, compartir, guardar y navegar con el historial funcionan sin código adicional, y la URL se convierte en la única fuente de verdad, sin copias que sincronizar.

Coste. Más código que un `useState`: funciones de lectura, escritura y validación por módulo, y la disciplina de no guardar copias del estado. Las partes genéricas se escribieron una vez en `lib/list-params.ts`.

Alternativas. Estado local (solo para listados auxiliares, como el selector del catálogo); librerías como `nuqs`, que gestionan parámetros de URL tipados con una API parecida a `useState`; TanStack Router, que valida los parámetros de búsqueda con esquemas.

19.11 Errores típicos y depuración

↑ Índice

- **El listado no se actualiza al cambiar un filtro.** Se está guardando el filtro en un `useState` en lugar de en la URL, o `params` no se recalcula porque falta `searchParams` en las dependencias del `useMemo`.
- **Al filtrar, aparece una página vacía.** No se ha reiniciado `page: 0` al cambiar el filtro.
- **La URL se llena de valores por defecto** (`?pagina=1&tamano=10&orden=id,asc`). Falta omitirlos al serializar.
- **Un valor de la URL produce un error del backend.** No se ha validado al leer: añade el valor a la lista permitida o corrige la función de lectura.
- **El botón *atrás* no vuelve a la página anterior del listado.** Se está usando `replace` donde debería añadirse una entrada, o se navega por otro mecanismo que no pasa por el router.

19.12 Ejercicios

↑ Índice

Ejercicio 19.1 · Añade un filtro de texto a la URL Ver solución →

Imagina que el backend añade un parámetro `name` para buscar aplicaciones por nombre. Añade al listado un campo de búsqueda cuyo valor viva en la URL como `?buscar=…`: amplía `ApplicationListParams`, `parseListParams`, `serializeListParams`, `toApiQuery` y sus pruebas. ¿Debería cada pulsación de tecla crear

una entrada en el historial? ¿Y una petición al backend? Investiga qué es un *debounce* y propón cómo aplicarlo.

Ejercicio 19.2 · Escribe una prueba que falle Ver solución →

En `list-params.test.ts` de demandas, escribe una prueba que compruebe que `parseDemandListParams` descarta `?orden=createdAt,asc`. Ejecútala: ¿pasa o falla con el código actual? Relaciónalo con el desajuste del capítulo 14 y explica qué cambio en `DEMAND_SORT_FIELDS` la haría pasar.

Ejercicio 19.3 · Extrae un hook genérico Ver solución →

Las páginas de listado de aplicaciones y demandas repiten la lectura de la URL con `useMemo`, la función `updateParams` con `useCallback` y el efecto de corrección de página. Escribe en `src/lib/` (o en `src/hooks/`) un hook `useUrlListState<P>(parse, serialize)` que devuelva `{ params, updateParams }` y úsalo en ambas páginas. ¿Dónde dejarías el efecto de corrección de página y por qué?

19.13 Autoevaluación

↑ Índice

Autoevaluación del capítulo 19 Ver respuestas →

1. Enumera cuatro situaciones en las que guardar el estado del listado en la URL mejora la experiencia frente a `useState`.
2. ¿Por qué la URL usa los códigos del backend (`ACTIVE`) y no las etiquetas (*Activa*)?
3. ¿Por qué se omiten los valores por defecto al escribir la URL?
4. ¿Qué significa que la URL sea la única fuente de verdad del listado?
5. ¿Qué ocurre si la URL contiene `?tamano=10000&estado=FOO`?
6. ¿Por qué los cambios de filtro, tamaño y orden reinician la página a 0?
7. ¿Cuándo se usa `replace` al escribir la URL del listado y por qué?
8. ¿Cómo consigue el detalle de una aplicación abrir el listado de demandas ya filtrado?
9. ¿Qué dos propiedades del diseño comprueba la prueba de reversibilidad?

CAPÍTULO 20

Formularios con React Hook Form y Zod

↑ Volver al índice

EN ESTE CAPÍTULO

20.1 El problema de los formularios
20.2 Zod: describir y validar datos
20.3 React Hook Form: el estado del formulario
20.4 La integración con los componentes de shadcn/ui
20.5 De los valores del formulario al DTO
20.6 El ciclo de envío completo
20.7 Esquemas que dependen de los datos
20.8 Accesibilidad de los formularios
20.9 Pruebas de los esquemas
20.10 Errores típicos y depuración
20.11 Ejercicios
20.12 Autoevaluación

Los formularios de alta y modificación son la parte de la aplicación con más reglas: campos obligatorios, longitudes máximas, valores permitidos, transformaciones antes de enviar, errores del servidor que hay que colocar en su campo y estados como «hay cambios sin guardar». El proyecto reparte ese trabajo entre dos librerías: Zod describe y valida los datos, y React Hook Form gestiona el estado del formulario. Este capítulo explica ambas y cómo encajan con los componentes de shadcn/ui.

20.1 El problema de los formularios

↑ Índice

Un formulario aparentemente sencillo, como el alta de una aplicación, obliga a gestionar mucho estado:

- El **valor** de cada campo y si se ha modificado respecto al inicial (*dirty*).
- Los **errores** de cada campo, y cuándo mostrarlos: ¿al escribir, al salir del campo, al enviar?
- La **validación**: reglas por campo y, a veces, reglas entre campos o que dependen de otros datos.
- La **transformación** de lo que hay en pantalla a lo que espera la API: textos recortados, cadenas vacías como `null`, identificadores de texto a número.
- El **envío**: evitar dobles envíos, mostrar «Guardando…», tratar el error del servidor y colocar sus errores por campo.
- El **reinicio**: limpiar el formulario, descartar cambios o cargar los datos devueltos por el servidor.

Hacerlo a mano con un `useState` por campo es tedioso, propenso a errores y lento: cada pulsación de tecla renderizaría el formulario entero. Por eso casi todos los proyectos React usan una librería.

20.2 Zod: describir y validar datos

↑ Índice

Zod es una librería para declarar **esquemas**: descripciones de la forma y las reglas de unos datos, con las que se puede validar cualquier valor en ejecución y, a la vez, derivar sus tipos de TypeScript.

```
import { z } from 'zod'
```

```
const schema = z.object({
  name: z.string().trim().min(1, 'El nombre es obligatorio.').max(100),
  criticality: z.enum(['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']),
})

schema.parse({ name: '  SIGMA ', criticality: 'HIGH' })     // { name: 'SIGMA', criticality:
'HIGH' }
schema.parse({ name: '', criticality: 'URGENT' })           // lanza ZodError con dos «issues»
schema.safeParse({ name: '' })                              // { success: false, error: ZodError }
```

- `parse` devuelve los datos validados (y transformados) o lanza un error; `safeParse` devuelve un objeto con `success` y los datos o el error, sin lanzar.
- Cada fallo es un ***issue*** con la ruta del campo (`path: ['name']`) y el mensaje.
- Los métodos se encadenan: `trim()` **transforma** (recorta espacios) y `min` / `max` **validan**.

Recuerda el capítulo 2: los tipos de TypeScript desaparecen en ejecución y no pueden validar lo que escribe un usuario. Zod cubre exactamente ese hueco.

20.2.1 Los esquemas del proyecto, alineados con Bean Validation

`src/features/applications/model/schemas.ts`

```
/*
 * Esquemas de validación de formularios. Reflejan las restricciones de
 * Bean Validation de los DTO del backend (@NotBlank, @Size, @Pattern), de modo
 * que el usuario recibe los errores antes de enviar la petición.
 */
const nameField = z
  .string()
  .trim()
  .min(1, 'El nombre es obligatorio.')
  .max(NAME_MAX_LENGTH, `El nombre no puede superar los ${NAME_MAX_LENGTH} caracteres.`)

const descriptionField = z
  .string()
  .trim()
  .max(DESCRIPTION_MAX_LENGTH, `La descripción no puede superar los ${DESCRIPTION_MAX_LENGTH}
caracteres.`)

const criticalityField = z.enum(APPLICATION_CRITICALITIES, {
  errorMap: () => ({ message: 'Selecciona una criticidad.' }),
})

const statusField = z.enum(APPLICATION_STATUSES, {
  errorMap: () => ({ message: 'Selecciona un estado.' }),
})

export const applicationCreateSchema = z.object({
  name: nameField,
  description: descriptionField,
  criticality: criticalityField,
})

export const applicationUpdateSchema = applicationCreateSchema.extend({
  status: statusField,
})
```

Bean Validation (backend)	Zod (frontend)
`@NotBlank`	`z.string().trim().min(1, '…')`

Bean Validation (backend)	Zod (frontend)
`@Size(max = 100)`	`.max(100, '…')`, con la constante `NAME_MAX_LENGTH`
`@NotNull` sobre un enumerado	`z.enum(VALORES)`: rechaza `undefined` y cualquier valor fuera de la lista
`@Pattern(regexp = "…")`	`.regex(/…/, '…')`
Herencia o composición de DTO	`.extend({ … })`: la modificación es el alta más el estado

Detalles a observar:

- **Las listas de valores vienen de `types.ts`** (`APPLICATION_CRITICALITIES`): el mismo array de los tipos, los desplegables y la validación de la URL.
- **Las longitudes vienen de `constants.ts`**: los mismos números que muestran los contadores de caracteres.
- **Los mensajes están en español** y se personalizan por campo. Para los enumerados, la opción `errorMap` de Zod 3 sustituye el mensaje genérico («Invalid enum value…») por uno comprensible.

> **Si vienes de Java…**
>
> Es la misma idea que anotar un DTO con Bean Validation, pero con los datos y las reglas en un objeto en lugar de en anotaciones. Igual que en Spring, el esquema es la **fuente única de las reglas** del formulario. Y, como en cualquier arquitectura cliente-servidor, la validación del cliente es una comodidad: el backend vuelve a validarlo todo (capítulo 1).

20.2.2 Tipos de entrada y de salida

Un esquema con transformaciones tiene **dos tipos**: lo que entra y lo que sale. Zod los deriva:

`src/features/applications/model/schemas.ts`

```ts
/** Valores del formulario tal y como los maneja React Hook Form (antes de validar). */
export type ApplicationCreateFormInput = z.input<typeof applicationCreateSchema>
export type ApplicationCreateFormValues = z.output<typeof applicationCreateSchema>
```

En aplicaciones ambos coinciden casi del todo, pero en demandas la diferencia es clara:

`src/features/demands/model/schemas.ts`

```ts
/** El desplegable trabaja con texto; se envía al backend como número. */
const applicationIdField = z
  .string({ required_error: 'Selecciona una aplicación.', invalid_type_error: 'Selecciona una aplicación.' })
  .regex(/^\d+$/, 'Selecciona una aplicación.')
  .transform(Number)
```

La **entrada** es `string` (lo que maneja el `Select` de Radix, capítulo 11) y la **salida** es `number` (lo que espera `DemandCreateRequest.applicationId`). El formulario trabaja con texto y, al validar, recibe el número listo para enviar. No hay conversiones manuales repartidas por la página.

20.3 React Hook Form: el estado del formulario

↑ Índice

React Hook Form gestiona valores, errores, estado de modificación y envío. Su característica distintiva es el **rendimiento**: guarda el estado fuera de React, en un almacén propio, y solo vuelve a renderizar los componentes suscritos a lo que cambia. Escribir en un campo no renderiza el formulario entero.

src/features/applications/pages/application-edit-page.tsx (extracto)

```
const form = useForm<ApplicationUpdateFormInput, unknown, ApplicationUpdateFormValues>({
  resolver: zodResolver(applicationUpdateSchema),
  defaultValues: toFormValues(application),
  mode: 'onSubmit',
})
const { isDirty } = form.formState
```

Pieza	Qué hace
Genéricos `<Input, Context, Output>`	El tipo de los valores del formulario y el de los valores ya validados que recibirá el manejador de envío
`resolver: zodResolver(schema)`	Delega la validación en el esquema de Zod: los *issues* se convierten en errores por campo
`defaultValues`	Valores iniciales; también son la referencia para saber si hay cambios (`isDirty`)
`mode: 'onSubmit'`	La validación se ejecuta al enviar; después del primer intento, se revalida al cambiar cada campo
`form.formState`	Estado: `isDirty`, `errors`, `isSubmitting`… Solo se renderiza lo que se lee
`form.handleSubmit(fn)`	Valida y, solo si es correcto, llama a `fn` con los valores de salida
`form.reset(valores?)`	Vuelve a los valores iniciales o a unos nuevos, que pasan a ser la referencia
`form.setError(campo, error)`	Marca un error en un campo desde fuera (los del servidor)

Decisión de diseño: Validar al enviar y revalidar al corregir

Con `mode: 'onSubmit'`, el usuario no ve errores mientras rellena el formulario por primera vez: un mensaje de «obligatorio» en un campo que aún no ha tenido ocasión de rellenar resulta agresivo. Tras el primer intento de envío, los errores se actualizan al escribir, de modo que desaparecen en cuanto se corrigen. Es el comportamiento que menos interrumpe y que más ayuda a corregir.

20.4 La integración con los componentes de shadcn/ui

↑ Índice

El capítulo 9 explicó los contextos de `components/ui/form.tsx`. Ahora vemos cómo se usan para construir campos reutilizables.

20.4.1 El formulario en la página

src/features/applications/pages/application-edit-page.tsx (extracto)

```
<Form {...form}>
  <form onSubmit={onSubmit} noValidate className="contents">
```

```
    <CardContent className="grid gap-6">
      <div className="grid gap-6 sm:grid-cols-[1fr_16rem]">
        <NameField />
        <CriticalityField />
      </div>
      <DescriptionField />
      <div className="sm:max-w-64"><StatusField /></div>
    </CardContent>
    {/* pie con los botones */}
  </form>
</Form>
```

- `<Form {...form}>` es el `FormProvider` de React Hook Form: los campos encuentran el formulario por contexto.
- `noValidate` desactiva la validación nativa del navegador (los globos de «Rellena este campo»), porque la validación la hace Zod, con mensajes propios y coherentes.
- `className="contents"` hace que el elemento `<form>` no genere una caja propia en el diseño: sus hijos se comportan como hijos directos de la tarjeta.

20.4.2 Los campos genéricos

`src/components/common/form-fields.tsx (extracto)`

```
export function TextInputField({ name, label, required, description, maxLength, placeholder }:
BaseFieldProps & { maxLength: number; placeholder?: string }) {
  const { control } = useFormContext()
  return (
    <FormField
      control={control}
      name={name}
      render={({ field }) => (
        <FormItem>
          <div className="flex items-center justify-between">
            <FormLabel required={required}>{label}</FormLabel>
            <CharCounter name={name} max={maxLength} />
          </div>
          <FormControl>
            <Input placeholder={placeholder} autoComplete="off" aria-required={required ||
undefined} {...field} />
          </FormControl>
          {description && <FormDescription>{description}</FormDescription>}
          <FormMessage />
        </FormItem>
      )}
    />
  )
}
```

`FormField` envuelve el `Controller` de React Hook Form: un componente que conecta un campo con el almacén del formulario y entrega, en la función `render`, el objeto `field` (`value`, `onChange`, `onBlur`, `name`, `ref`). `{...field}` lo conecta todo al `Input`. El campo se renderiza cuando cambia su propio valor, no cuando cambian los demás.

Los tres campos genéricos (`TextInputField`, `TextareaField`, `SelectField`) comparten la misma estructura: etiqueta (con asterisco si es obligatorio), control, descripción opcional y mensaje de error. Los módulos los configuran (capítulo 9): `NameField` es un `TextInputField` con nombre, etiqueta y longitud máxima del dominio.

20.4.3 El contador de caracteres

src/components/common/form-fields.tsx

```
function CharCounter({ name, max }: { name: string; max: number }) {
  const { control } = useFormContext()
  const value = (useWatch({ control, name }) as string | undefined) ?? ''
  const length = value.length
  return (
    <span className={cn('text-xs leading-none tabular-nums text-muted-foreground', length > max && '
font-medium text-destructive')}
          aria-live="polite">
      {length}/{max}
    </span>
  )
}
```

`useWatch` suscribe **solo este componente** a los cambios del campo indicado: al escribir, se renderiza el contador, no el formulario. Observa que el `Input` no lleva atributo `maxLength`: el usuario puede escribir más de la cuenta, el contador se pone en rojo y el error aparece al enviar. Es una decisión de experiencia de uso: cortar en seco lo que se pega desde otro documento hace que el texto se pierda sin aviso.

20.4.4 El desplegable, un campo controlado

src/components/common/form-fields.tsx (extracto)

```
<Select value={(field.value as string | undefined) ?? ''} onValueChange={field.onChange}
disabled={disabled}>
  <FormControl>
    <SelectTrigger className="w-full" onBlur={field.onBlur} ref={field.ref} aria-required={required
|| undefined}>
      <SelectValue placeholder={placeholder} />
    </SelectTrigger>
  </FormControl>
  <SelectContent>{/* opciones, algunas desactivadas */}</SelectContent>
</Select>
```

El `Select` de Radix no es un `<select>` nativo, así que no se le puede pasar `{...field}` tal cual: se conecta pieza a pieza. `value` y `onValueChange` sincronizan el valor; `onBlur` permite saber cuándo se ha tocado el campo; `ref` permite que React Hook Form le dé el foco al primer campo con error. El valor vacío `''` muestra el texto de ayuda (*placeholder*).

20.5 De los valores del formulario al DTO

↑ Índice

Los valores del formulario no son exactamente el cuerpo de la petición. La página los transforma al enviar:

src/features/applications/pages/application-edit-page.tsx (extracto)

```
const result = await mutation.mutateAsync({
  name: values.name,                                    // ya recortado por Zod
  description: normalizeDescription(values.description),  // '' → null
  criticality: values.criticality,
  status: values.status,
})
```

src/features/applications/model/schemas.ts

```ts
/** Una descripción vacía se envía como null para no almacenar cadenas vacías. */
export function normalizeDescription(value: string): string | null {
  const trimmed = value.trim()
  return trimmed.length > 0 ? trimmed : null
}
```

Y al revés, al cargar un registro para editarlo, se convierte el DTO en valores de formulario:

src/features/applications/pages/application-edit-page.tsx

```ts
function toFormValues(app: Application): ApplicationUpdateFormInput {
  return {
    name: app.name,
    description: app.description ?? '',   // null → '' (un textarea no admite null)
    criticality: app.criticality,
    status: app.status,
  }
}
```

Campo	En la API	En el formulario	Conversión
Descripción vacía	`null`	`''`	`?? ''` al cargar; `normalizeDescription` al enviar
Aplicación de una demanda	`number`	`string`	`String()` al cargar; `transform(Number)` en el esquema
Criticidad o prioridad sin elegir (alta)	—	`undefined`	Valor inicial «vacío» que el esquema rechaza al enviar

Sobre la última fila: el tipo de entrada de `z.enum` no admite `undefined`, pero el formulario de alta necesita empezar sin criticidad elegida. El proyecto lo resuelve con una aserción explícita en el valor inicial (`undefined as unknown as …['criticality']`), consciente de que es un valor provisional que el esquema rechazará con el mensaje «Selecciona una criticidad.». Es un compromiso habitual con React Hook Form; la alternativa más limpia sería que el tipo de entrada admitiera `undefined` (con `z.enum(…).optional()` y un `refine` que lo exija).

20.6 El ciclo de envío completo

↑ Índice

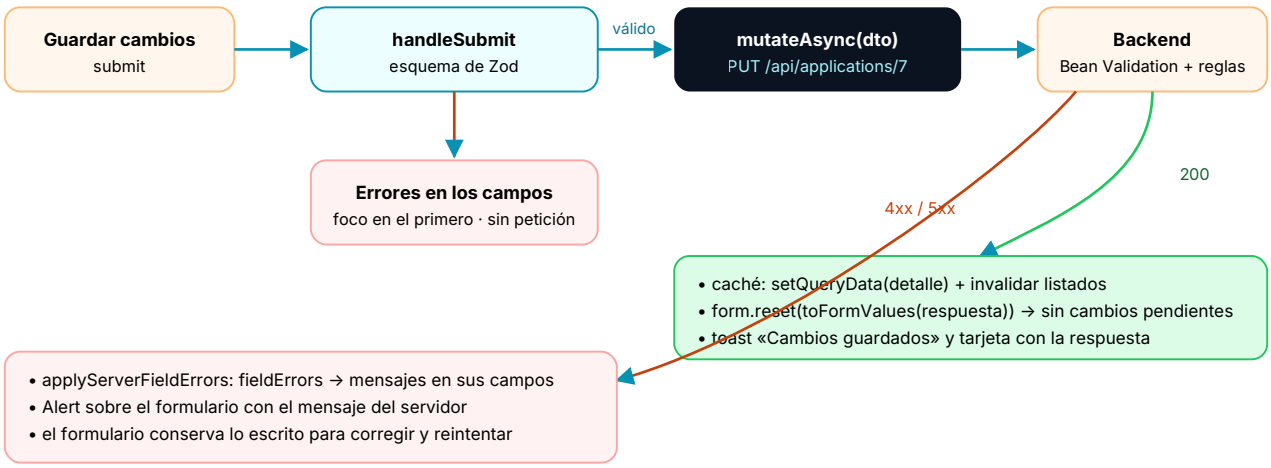


Figura 20.1. El envío de un formulario: primero valida el cliente; si es válido, decide el servidor.

`src/features/applications/pages/application-edit-page.tsx (extracto)`

```
const onSubmit = form.handleSubmit(async (values) => {
  try {
    const result = await mutation.mutateAsync({ /* DTO */ })
    form.reset(toFormValues(result))
    setUpdated(result)
    toast.success('Cambios guardados', { description: `«${result.name}» se ha actualizado.` })
  } catch (error) {
    applyServerFieldErrors(error, form.setError, ['name', 'description', 'criticality', 'status'])
  }
})
```

Tres decisiones de este manejador:

- `form.reset(toFormValues(result))` carga en el formulario **lo que ha guardado el servidor**, no lo que escribió el usuario. Si el backend normaliza algo, el formulario lo refleja. Además, esos valores pasan a ser la nueva referencia, así que `isDirty` vuelve a `false`.
- **Los errores no se relanzan.** Se capturan, se colocan los errores por campo y el estado de error de la mutación muestra la alerta general. El formulario conserva lo escrito.
- **El botón de envío se desactiva** mientras la mutación está pendiente y cuando no hay cambios (`!isDirty`), lo que evita dobles envíos y peticiones inútiles.

20.7 Esquemas que dependen de los datos

↑ Índice

A veces las reglas dependen del registro que se edita. En demandas, los estados permitidos dependen del estado actual: una demanda `OPEN` puede pasar a `IN_PROGRESS` o `CANCELLED`, pero no a `COMPLETED`. El esquema se construye con una función:

`src/features/demands/model/schemas.ts`

```
export function createDemandUpdateSchema(currentStatus: DemandStatus) {
  return demandCreateSchema.extend({ status: statusField }).superRefine((values, ctx) => {
    if (!canTransition(currentStatus, values.status)) {
      ctx.addIssue({
```

```
        code: z.ZodIssueCode.custom,
        path: ['status'],
        message: `No se puede pasar de «${DEMAND_STATUS_LABELS[currentStatus]}» a «${DEMAND_STATUS_L
ABELS[values.status]}».`,
      })
    }
  })
}
```

`superRefine` añade validaciones personalizadas que pueden mirar varios campos y datos externos, y asignar el error al campo que corresponda (`path: ['status']`). La página memoriza el *resolver* según el estado actual:

src/features/demands/pages/demand-edit-page.tsx (extracto)

```
const currentStatus = demand.status
const resolver = useMemo(() => zodResolver(createDemandUpdateSchema(currentStatus)), [currentStatus]
)
```

Lo veremos en contexto en el capítulo 22.

20.8 Accesibilidad de los formularios

↑ Índice

Requisito	Cómo se cumple
Cada campo tiene una etiqueta asociada	`FormLabel` con `htmlFor` igual al `id` del control (generado con `useId`)
Los campos obligatorios se anuncian	Asterisco visual con `aria-hidden` y `aria-required` en el control; nota «Los campos marcados con * son obligatorios»
Los errores se asocian al campo	`aria-invalid` y `aria-describedby` apuntando al mensaje de error (`FormControl`)
El foco va al primer error	React Hook Form llama a `focus()` sobre el `ref` del primer campo inválido
El contador se anuncia sin interrumpir	`aria-live="polite"`

20.9 Pruebas de los esquemas

↑ Índice

Los esquemas son funciones puras de datos a resultado, así que se prueban sin montar ningún componente:

src/features/applications/model/schemas.test.ts (extracto)

```
it('exige nombre no vacío (equivale a @NotBlank)', () => {
  const result = applicationCreateSchema.safeParse({ ...valid, name: '   ' })
  expect(result.success).toBe(false)
  expect(result.error?.issues[0]?.message).toBe('El nombre es obligatorio.')
})

it('limita el nombre a 100 y la descripción a 500 caracteres', () => {
  expect(applicationCreateSchema.safeParse({ ...valid, name: 'a'.repeat(100) }).success).toBe(true)
  expect(applicationCreateSchema.safeParse({ ...valid, name: 'a'.repeat(101) }).success).toBe(false)
```

```
  // …
})
```

Las pruebas de los límites (100 y 101 caracteres) son las que detectan cuándo una constante deja de coincidir con el `@Size` del backend.

Decisión de diseño: React Hook Form + Zod

Decisión. Esquemas de Zod como única fuente de las reglas de cada formulario, alineados con Bean Validation; React Hook Form para el estado, conectado con `zodResolver`; y campos genéricos sobre los componentes `Form` de shadcn/ui.

Motivos. Rendimiento (solo se renderiza lo que cambia), reglas declarativas que se prueban de forma aislada, tipos derivados de los esquemas, transformaciones explícitas de formulario a DTO y una integración accesible ya resuelta por shadcn/ui.

Alternativas. Formik (pionera, pero con peor rendimiento y menos mantenida); TanStack Form (moderna y muy tipada, menos extendida); las acciones de formulario de React 19 (`useActionState`), pensadas sobre todo para envíos al servidor en *frameworks* con renderizado en servidor; y el estado controlado a mano, viable solo para formularios mínimos como el buscador por identificador.

20.10 Errores típicos y depuración

↑ Índice

- **El manejador de envío no se ejecuta y no se ve ningún error.** Hay un error de validación en un campo que no se está pintando (no tiene `FormMessage` o no existe en el formulario). Inspecciona `form.formState.errors` con un `console.log` o con React DevTools.
- **`isDirty` es verdadero sin haber tocado nada.** Los valores iniciales no coinciden con lo que el campo produce (por ejemplo, `null` frente a `''`). Normaliza en `toFormValues`.
- **El `Select` no muestra el valor inicial.** El valor no es una cadena o no coincide exactamente con ningún `value` de las opciones (por ejemplo, el número 7 frente a la cadena `'7'`).
- **Tras guardar, el formulario sigue marcando cambios.** Falta `form.reset(…)` con los datos devueltos.
- **Un error del servidor no aparece en su campo.** El nombre del campo en `fieldErrors` no coincide con el del formulario, o no está en la lista `knownFields` de `applyServerFieldErrors`.
- **Mensajes en inglés de Zod.** Falta el mensaje personalizado en la regla (o el `errorMap` en los enumerados).

20.11 Ejercicios

↑ Índice

Ejercicio 20.1 · Añade un campo con validación Ver solución →

Añade al formulario de aplicaciones un campo opcional «Responsable» con un correo electrónico. En el esquema, usa `z.string().trim().email('…')` combinado con la posibilidad de dejarlo vacío (pista: `.or(z.literal(''))`). Crea `OwnerField` en `application-form-fields.tsx` y añade pruebas al esquema. Como el backend no tiene ese campo, no lo envíes en el DTO: ¿qué pasaría si lo enviaras?

Ejercicio 20.2 · Valida al salir del campo Ver solución →

Cambia temporalmente el formulario de alta a `mode: 'onTouched'` y compara la experiencia con `onSubmit` y con `onChange`. Escribe tres frases con las ventajas e inconvenientes de cada modo para un formulario de gestión como este.

Ejercicio 20.3 · Simula un error de servidor por campo Ver solución →

Sin tocar el backend, comprueba el camino de `applyServerFieldErrors`. Pista: con las DevTools, en *Network*, puedes usar *Override content* sobre una respuesta, o bien, en una rama, modifica temporalmente `useCreateApplication` para que lance un `new ApiError({ message: 'La petición contiene campos inválidos', status: 400, kind: 'validation', fieldErrors: { name: 'ya existe una aplicación con ese nombre' } })`. Comprueba dónde aparece cada mensaje.

20.12 Autoevaluación

↑ Índice

Autoevaluación del capítulo 20 Ver respuestas →

1. ¿Qué responsabilidades tiene Zod y cuáles React Hook Form en el proyecto?
2. ¿Cómo se expresa `@NotBlank` en Zod y por qué hace falta `trim()`?
3. ¿Qué diferencia hay entre `z.input` y `z.output`? Pon el ejemplo de `applicationId`.
4. ¿Por qué React Hook Form no renderiza el formulario entero en cada pulsación?
5. ¿Qué hace `mode: 'onSubmit'` y por qué se eligió?
6. ¿Por qué el `Select` de Radix no recibe `{...field}` directamente?
7. ¿Qué conversiones se hacen entre DTO y formulario con la descripción?
8. ¿Por qué tras guardar se llama a `form.reset` con la respuesta del servidor y no con lo que escribió el usuario?
9. ¿Para qué sirve `superRefine` en el esquema de modificación de demandas?

PARTE VIII

Recorridos completos

21 El módulo de aplicaciones de principio a fin pág. 199

22 El módulo de demandas y sus reglas de negocio pág. 206

↑ Volver al índice

CAPÍTULO 21

El módulo de aplicaciones de principio a fin

↑ Volver al índice

EN ESTE CAPÍTULO

21.1 El mapa del módulo
21.2 Recorrido 1: consultar el catálogo
21.3 Recorrido 2: ver el detalle
21.4 Recorrido 3: dar de alta
21.5 Recorrido 4: modificar
21.6 La caché a lo largo de una sesión
21.7 El contrato con el backend
21.8 Errores típicos y depuración
21.9 Ejercicios
21.10 Autoevaluación

Ya conoces todas las piezas por separado. Este capítulo las recorre juntas, siguiendo al usuario por cada pantalla del módulo de aplicaciones: qué ficheros intervienen, en qué orden, qué viaja por la red y qué ocurre en la caché. Es el capítulo para leer con la aplicación abierta y las DevTools a la vista.

21.1 El mapa del módulo

↑ Índice

Ruta	Pantalla	Página	Llamadas a la API
`/aplicaciones`	Consulta con filtros, paginación y orden	`ApplicationsListPage`	`GET /api/applications?status&criticality&page&size&sort`
`/aplicaciones/nueva`	Alta	`ApplicationCreatePage`	`POST /api/applications`
`/aplicaciones/modificar`	Buscar por ID o elegir del catálogo	`ApplicationEditSelectorPage`	`GET /api/applications/{id}`, `GET /api/applications`
`/aplicaciones/:id`	Detalle	`ApplicationDetailPage`	`GET /api/applications/{id}`
`/aplicaciones/:id/editar`	Modificación	`ApplicationEditPage`	`GET` y `PUT /api/applications/{id}`

En el backend, cada fila corresponde a un método del controlador de aplicaciones, que en desarrollo atiende el servicio del puerto 8081 a través del proxy de Vite.

21.2 Recorrido 1: consultar el catálogo

↑ Índice

El usuario pulsa *Consultar aplicaciones* en el menú lateral.

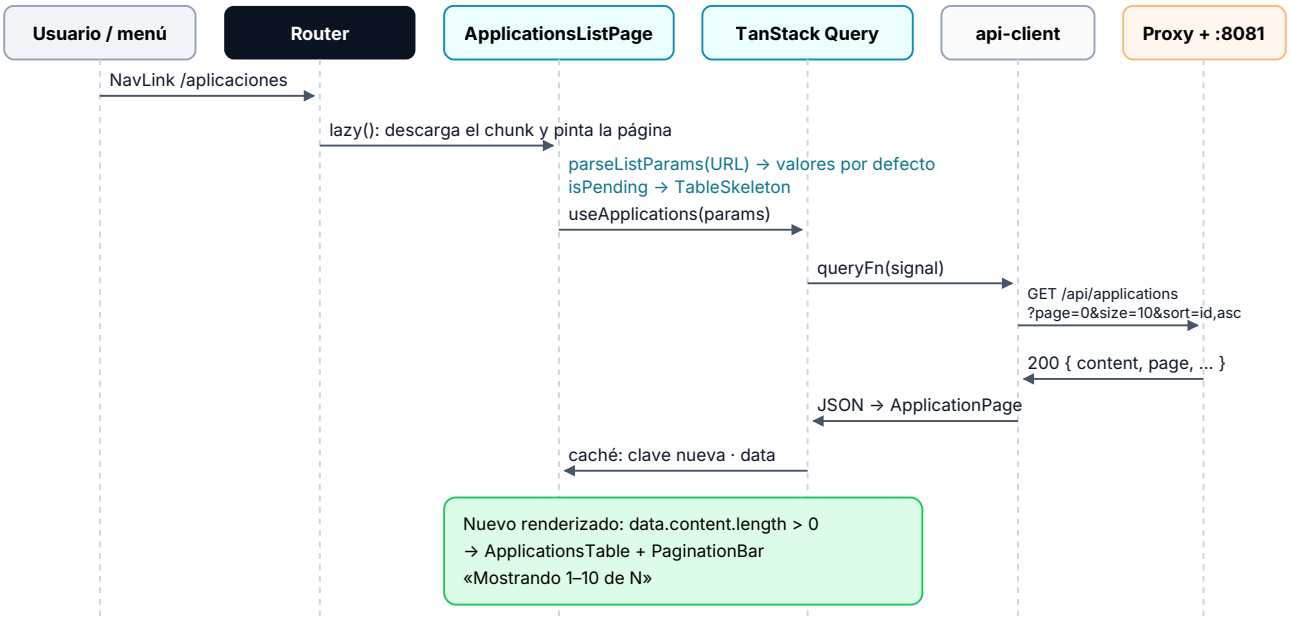


Figura 21.1. Primera visita al listado. Las flechas son llamadas o respuestas; cada columna, una pieza del sistema.

Paso	Fichero	Qué ocurre
1	`components/layout/sidebar-nav.tsx`	`NavLink` navega a `/aplicaciones` sin recargar
2	`features/applications/routes.ts`	La ruta índice ejecuta `lazy()`: se descarga el *chunk* de la página
3	`components/layout/app-layout.tsx`	`useDocumentTitle` lee `handle.title` y pone «Consultar aplicaciones · IT Service Manager»
4	`pages/applications-list-page.tsx`	`parseListParams` convierte la URL vacía en los valores por defecto
5	`api/applications-queries.ts`	`useApplications` busca la clave en la caché; no existe → `isPending`
6	`components/common/table-skeleton.tsx`	Se pinta el esqueleto de la tabla
7	`model/list-params.ts` → `api/applications-api.ts` → `lib/api-client.ts`	`toApiQuery` construye `page=0&size=10&sort=id,asc`; `apiRequest` hace el `fetch`
8	`vite.config.ts`	El proxy reenvía `/api/applications` al servicio de aplicaciones
9	TanStack Query	Guarda la respuesta en la clave y notifica a la página
10	`components/applications-table.tsx`, `pagination-bar.tsx`	Se pinta la tabla con indicadores y la paginación

21.2.1 Interacciones del listado

El usuario…	La página llama a	URL resultante	¿Petición?
Filtra por criticidad *Alta*	`updateParams({ criticality: 'HIGH', page: 0 })`	`?criticidad=HIGH`	Sí, clave nueva
Pulsa la cabecera *Nombre*	`updateParams({ ...nextSort(params, 'name'), page: 0 })`	`?criticidad=HIGH&orden=name,asc`	Sí
Pulsa de nuevo *Nombre*	Alterna la dirección	`…&orden=name,desc`	Sí
Elige 20 filas por página	`updateParams({ size: 20, page: 0 })`	`…&tamano=20`	Sí
Pasa a la página 2	`updateParams({ page: 1 })`	`…&pagina=2`	Sí; la tabla anterior se atenúa mientras tanto
Pulsa *atrás*	(el navegador)	Vuelve a la URL anterior	No, si han pasado menos de 30 s
Pulsa *Limpiar filtros*	`updateParams({ status: undefined, criticality: undefined, page: 0 })`	`/aplicaciones` (más orden y tamaño, si no son los de por defecto)	Probablemente no: está en caché

Y los estados posibles de la tarjeta: esqueleto (primera carga), error con *Reintentar* (sin datos), franja de aviso sobre los datos anteriores (error al actualizar), vacío con «Limpiar filtros» o vacío con «Nueva aplicación» según haya filtros o no, y tabla con paginación.

21.3 Recorrido 2: ver el detalle

↑ Índice

Desde la tabla, el usuario pulsa el icono del ojo de una fila. `RowAction` es un `Link` con *tooltip* a `/aplicaciones/7`.

1. La ruta `:id` carga `ApplicationDetailPage`.
2. `useParams()` devuelve `{ id: '7' }` y `parseEntityId` lo convierte en `7`. Si no fuera válido, se mostraría `ResourceNotFound` sin petición.
3. `useApplication(7)` pide `GET /api/applications/7`. Mientras tanto, la cabecera muestra «Aplicación #7» y la tarjeta, un `LoadingState`.
4. Con los datos, el título pasa a ser el nombre de la aplicación, aparecen las acciones *Ver demandas* y *Modificar*, y `ApplicationSummary` pinta la ficha.
5. Si la respuesta es un 404, la consulta **no reintenta** (política propia de `useApplication`) y se muestra `ResourceNotFound`.

Ver demandas enlaza con `/demandas?aplicacion=7`: el listado de demandas, ya filtrado, sin compartir estado entre módulos (capítulo 19).

21.4 Recorrido 3: dar de alta

↑ Índice

El usuario pulsa *Nueva aplicación*.

Momento	Qué ve el usuario	Qué ocurre por dentro
Al entrar	Formulario vacío, criticidad sin elegir	`useForm` con `EMPTY_FORM` y `zodResolver(applicationCreateSchema)`
Escribe el nombre	El contador avanza (`4/100`)	`useWatch` renderiza solo el contador
Pulsa *Crear* sin criticidad	«Selecciona una criticidad.» bajo el campo, foco en él	`handleSubmit` valida con Zod; no hay petición
Elige criticidad	El error desaparece	Revalidación al cambiar (tras el primer intento)
Pulsa *Crear*	Botón «Creando…» desactivado	`mutateAsync` → `POST /api/applications` con `{ name, description: null, criticality }`
Responde el servidor con la aplicación creada	Notificación «Aplicación creada» y tarjeta con los datos registrados	`setQueryData(detail(id))` + invalidación de listados; `setCreated(result)`
Pulsa *Ver detalle*	La ficha aparece **sin estado de carga**	El detalle ya estaba en caché por `setQueryData`
Pulsa *Crear otra*	Formulario vacío de nuevo	`form.reset(EMPTY_FORM)`, `mutation.reset()`, `setCreated(null)`

Si el servidor rechaza el alta, la alerta muestra su mensaje y los `fieldErrors` aparecen en sus campos (capítulo 20). El identificador y el estado inicial los asigna el servidor, y la página lo explica en su descripción: es información que evita dudas al usuario.

21.5 Recorrido 4: modificar

↑ Índice

Hay dos caminos para llegar a la modificación.

Desde el menú: *Modificar aplicación*. La página de selección ofrece dos tarjetas:

- **Buscar por identificador** (`FindById`): un campo controlado con `useState`. Al enviar, se valida el texto con `parseEntityId` y se guarda en `searchedId`, lo que activa `useApplication(searchedId)` (consulta dependiente, capítulo 17). Si existe, se muestra un resumen y el botón *Modificar*, que navega con `useNavigate`.
- **Seleccionar del catálogo** (`PickFromCatalog`): la misma tabla y los mismos filtros del listado, en modo `select` y con su estado en `useState` (capítulo 7). *Seleccionar* navega a la edición.

Desde el listado o el detalle: el icono del lápiz o el botón *Modificar* llevan directamente a `/aplicaciones/7/editar`.

En la página de edición:

1. Se valida el `id` y se pide el detalle (si se viene del detalle, sale de la caché al instante).
2. Con los datos, se monta `ApplicationEditForm` con `key={application.id}` (capítulo 7) y `defaultValues: toFormValues(application)`.
3. El identificador se muestra en un campo desactivado con un candado: no es modificable.
4. El pie indica «Sin cambios pendientes» o «Hay cambios sin guardar» según `isDirty`, y los botones *Descartar* y *Guardar cambios* solo se activan con cambios.
5. Al guardar: `PUT /api/applications/7` con el DTO completo (nombre, descripción, criticidad y estado). Con éxito, el formulario se reinicia con la respuesta, aparece la tarjeta «Datos actualizados» y la notificación.
6. En cuanto el usuario vuelve a editar, la tarjeta de confirmación desaparece: `showUpdated` es un valor derivado de `!isDirty`.

> **En el proyecto**
>
> La modificación envía **todos** los campos con `PUT`, no solo los que cambian. Es lo que espera el contrato del backend (`ApplicationUpdateRequest` con todos los campos y Bean Validation sobre ellos). Si en el futuro hubiera ediciones parciales, el método adecuado sería `PATCH`, y el cliente HTTP ya lo admite (capítulo 15).

21.6 La caché a lo largo de una sesión

↑ Índice

Sigamos una sesión real y lo que contiene la caché en cada momento:

Acción	Peticiones	Estado relevante de la caché
Abre `/aplicaciones`	`GET list(página 1)`	`list{p0}` fresca
Pasa a la página 2	`GET list(página 2)`	`list{p0}`, `list{p1}` frescas
Abre el detalle de la 7	`GET detail(7)`	+ `detail(7)`
Pulsa *atrás* (antes de 30 s)	Ninguna	`list{p1}` sale de la caché
Modifica la 7 y guarda	`PUT`; después `GET list(página 2)`	`detail(7)` actualizado con la respuesta; listados invalidados, el visible se refresca
Va a *Nueva demanda*	`GET list(options)` y otras de demandas	+ `list(options)`, fresca 5 min
Crea una aplicación desde otra pestaña del navegador	—	Esta pestaña no se entera (cada pestaña tiene su propia caché)
Vuelve al listado pasados 30 s	Se muestra la caché y `GET list(página 2)` en segundo plano	Aparece la aplicación creada en la otra pestaña

La penúltima fila es importante: TanStack Query mantiene coherente lo que hace **este** usuario en **esta** pestaña. Los cambios de otros se ven cuando los datos se consideran obsoletos y se refrescan. Para una aplicación de gestión es suficiente; si se necesitara ver cambios ajenos en tiempo real, habría que añadir notificaciones del servidor (WebSocket o *Server-Sent Events*) que invalidaran las claves afectadas.

21.7 El contrato con el backend

↑ Índice

Aspecto	Contrato	Cómo lo refleja el frontend
Paginación	Spring Data `Pageable`, página 0-based, `@PageableDefault(size = 10, sort = "id", ASC)`	`DEFAULT_LIST_PARAMS` coincide; conversión 1-based ↔ 0-based en la URL
Respuesta paginada	`ApplicationPageResponse { content, page, size, totalElements, totalPages }`	`ApplicationPage`
Filtros	`status`, `criticality` opcionales	Se omiten si están vacíos (`buildQueryString`)
Ordenación	Propiedades de la entidad: `id`, `name`, `criticality`, `status`	`APPLICATION_SORT_FIELDS`; solo se envían esos campos
Validación	`@NotBlank`, `@Size(max = 100)` el nombre, `@Size(max = 500)` la descripción	Esquemas de Zod con las mismas constantes
Errores	`ApiErrorResponse` (400 con `fieldErrors`, 404)	`ApiError` + `applyServerFieldErrors` + `ResourceNotFound`

Dos limitaciones del contrato, detectadas al construir el módulo, condicionan la interfaz:

- **La criticidad se ordena alfabéticamente.** Se guarda como texto, así que ordenar por criticidad da `CRITICAL, HIGH, LOW, MEDIUM`, no el orden de severidad. El frontend no puede corregirlo sin cargar todos los datos; la solución está en el backend (un valor ordinal o una expresión `CASE` en la consulta).
- **No hay búsqueda por nombre.** Por eso *Modificar aplicación* ofrece búsqueda por identificador y selección en el catálogo filtrado, en lugar de un buscador de texto.

Ambas se recogerán en las notas de integración del capítulo 25.

21.8 Errores típicos y depuración

↑ Índice

Ante cualquier comportamiento extraño en este módulo, recorre la cadena de la figura 21.1 de izquierda a derecha:

1. **¿La URL es la esperada?** Mira la barra de direcciones: los parámetros de la URL deciden todo lo demás.
2. **¿La página interpreta bien la URL?** Con React DevTools, inspecciona `ApplicationsListPage` y mira el valor de `params` (en los *hooks*, el `Memo`).
3. **¿Se hace la petición correcta?** En *Network*, revisa la URL final, los parámetros y el cuerpo.
4. **¿Qué responde el backend?** Pestaña *Response*.
5. **¿Qué hay en la caché?** Con TanStack Query Devtools (ejercicio 17.1), busca la clave.
6. **¿Qué estado pinta la página?** `isPending`, `isError`, `data`…

21.9 Ejercicios

↑ Índice

Ejercicio 21.1 · Traza una sesión completa Ver solución →

Con *Network* y React DevTools abiertos, realiza esta secuencia y anota cada petición y cada cambio de URL: listado → filtro *Inactiva* → página 2 (si existe) → detalle de una aplicación → *Modificar* → cambia la criticidad y guarda → *Volver al listado*. ¿Qué peticiones se hicieron tras guardar y por qué? ¿Cuáles no se hicieron gracias a la caché?

Ejercicio 21.2 · Mejora la confirmación del alta Ver solución →

Tras crear una aplicación, la página ofrece *Crear otra*, *Volver al listado* y *Ver detalle*. Añade un cuarto botón, *Crear demanda para esta aplicación*, que lleve a `/demandas/nueva` con la aplicación preseleccionada. ¿Qué parámetro de la URL lo hace posible y qué página lo lee?

Ejercicio 21.3 · Avisa de los cambios sin guardar Ver solución →

Si el usuario tiene cambios sin guardar en la edición y pulsa un enlace del menú, los pierde sin aviso. Investiga el hook `useBlocker` de React Router y úsalo para pedir confirmación cuando `isDirty` sea verdadero. Añade también un aviso al cerrar la pestaña con el evento `beforeunload`. ¿En qué casos no debería bloquearse la navegación (pista: justo después de guardar)?

21.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 21 Ver respuestas →

1. ¿Qué cinco rutas tiene el módulo y qué endpoint usa cada una?
2. ¿En qué orden intervienen el router, la página, TanStack Query y el cliente HTTP en la primera visita al listado?
3. ¿Por qué al pulsar *atrás* desde un detalle el listado aparece al instante?
4. ¿Qué pasa si el usuario pulsa *Crear* sin elegir la criticidad?
5. ¿Por qué la ficha aparece sin carga al pulsar *Ver detalle* tras un alta?
6. ¿Qué dos caminos llevan a la edición desde el menú y qué estado usa cada uno?
7. ¿Por qué la modificación envía todos los campos con `PUT`?
8. ¿Se entera una pestaña de los cambios hechos en otra? ¿Cuándo los verá?
9. ¿Qué dos limitaciones del contrato del backend condicionan la interfaz del módulo?

CAPÍTULO 22

El módulo de demandas y sus reglas de negocio

↑ Volver al índice

EN ESTE CAPÍTULO

22.1 Qué añade el dominio de las demandas
22.2 El ciclo de vida
22.3 Tres capas de defensa
22.4 El selector de aplicación
22.5 Las fechas
22.6 Navegación entre módulos
22.7 Diferencias con el contrato del backend
22.8 Errores típicos y depuración
22.9 Ejercicios
22.10 Autoevaluación

Las demandas comparten estructura con las aplicaciones, pero su dominio es más rico: tienen un ciclo de vida con transiciones restringidas, pertenecen a una aplicación que vive en otro servicio y llevan fechas de creación y modificación. Este capítulo se centra en lo que las hace distintas y en cómo el frontend refleja reglas de negocio que siguen perteneciendo al backend.

22.1 Qué añade el dominio de las demandas

↑ Índice

Aspecto	Aplicaciones	Demandas
Estado	Activa o inactiva; cualquier cambio permitido	Ciclo de vida con transiciones restringidas y estados finales
Relaciones	Ninguna	Pertenece a una aplicación (de otro servicio)
Fechas	No	`createdAt` y `updatedAt`, asignadas por el servidor
Alta	El servidor asigna el estado	El servidor asigna `OPEN` y las fechas
Errores propios	—	409 si la transición no es válida; 404 si la aplicación no existe; 503 si el catálogo no responde

Las páginas siguen el mismo esquema de cinco pantallas, así que aquí nos centramos en las diferencias.

22.2 El ciclo de vida

↑ Índice

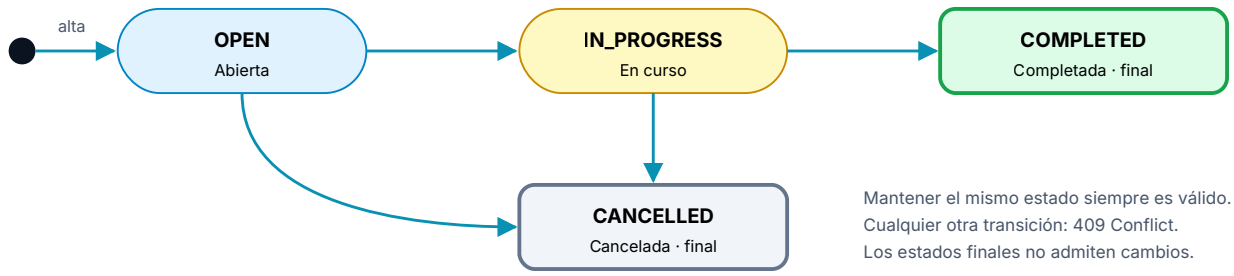


Figura 22.1. Máquina de estados de una demanda. Los estados con borde grueso son finales.

La regla **pertenece al backend**, a la entidad `Demand`:

`it-service-manager · demand/entity/Demand.java (extracto)`

```
public void changeStatus(String newStatus) {
    if (this.status.equals(newStatus)) {
        return;
    }
    boolean validTransition =
            ("OPEN".equals(this.status) && ("IN_PROGRESS".equals(newStatus) || "CANCELLED".equals(ne
wStatus)))
            || ("IN_PROGRESS".equals(this.status) && ("COMPLETED".equals(newStatus) || "CANCELLED".e
quals(newStatus)));
    if (!validTransition) {
        throw new IllegalStateException("Invalid demand status transition: " + this.status + " -> "
+ newStatus);
    }
    this.status = newStatus;
    this.updatedAt = Instant.now();
}
```

`DemandService` captura la `IllegalStateException` y la convierte en un `ResponseStatusException(CONFLICT)`: un **409** con el mensaje de la regla.

El frontend tiene un **reflejo** de esa regla, declarado como datos:

`src/features/demands/model/status-rules.ts`

```
/*
 * Reglas de transición de estado. Reflejan Demand.changeStatus() del backend.
 * El backend sigue siendo la autoridad: si rechaza un cambio, responde 409.
 */
const NEXT_STATUSES: Record<DemandStatus, readonly DemandStatus[]> = {
  OPEN: ['IN_PROGRESS', 'CANCELLED'],
  IN_PROGRESS: ['COMPLETED', 'CANCELLED'],
  COMPLETED: [],
  CANCELLED: [],
}

export function getNextStatuses(from: DemandStatus): readonly DemandStatus[] { return
NEXT_STATUSES[from] ?? [] }
export function canTransition(from: DemandStatus, to: DemandStatus): boolean {
  return from === to || getNextStatuses(from).includes(to)
}
export function getSelectableStatuses(from: DemandStatus): readonly DemandStatus[] {
```

```
  return [from, ...getNextStatuses(from)]
}
export function isFinalStatus(status: DemandStatus): boolean { return
getNextStatuses(status).length === 0 }
```

Expresarla como una **tabla de transiciones** (`Record<estado, estados siguientes>`) en lugar de con condiciones tiene dos ventajas: es exhaustiva (añadir un estado obliga a declarar sus transiciones) y es fácil de comparar con el backend a simple vista. Las funciones derivadas responden a las preguntas que necesita la interfaz.

> **Si vienes de Java…**
>
> Duplicar una regla de negocio en el cliente es una decisión que hay que justificar, igual que duplicar una validación. Aquí se justifica por la experiencia de uso: el usuario debe ver qué puede hacer **antes** de intentarlo. Lo esencial es que la duplicación sea **explícita y localizada** (un único fichero con el comentario que apunta a `Demand.changeStatus()`), cubierta por pruebas y subordinada al backend, que sigue siendo la autoridad. Si las reglas crecieran, una alternativa sería que el backend expusiera las transiciones permitidas en la respuesta (un campo `allowedTransitions` en `DemandResponse`), al estilo de los enlaces de HATEOAS.

22.3 Tres capas de defensa

↑ Índice

El frontend aplica la regla en tres sitios, cada uno con un propósito:

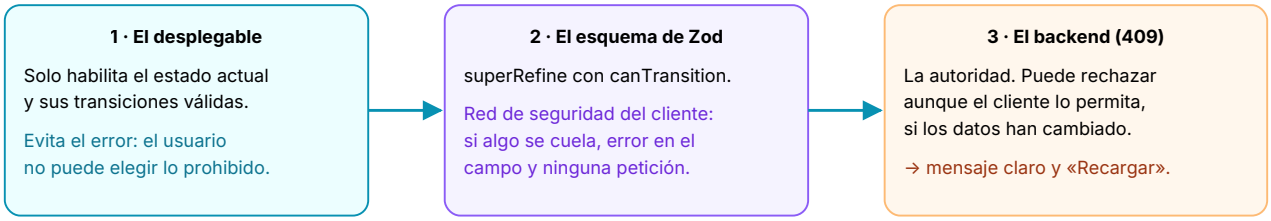


Figura 22.2. La regla de transición aplicada en tres capas: prevenir, comprobar y, si aun así falla, explicar.

22.3.1 Capa 1: el desplegable de estado

`src/features/demands/components/demand-form-fields.tsx (extracto)`

```
export function DemandStatusField({ currentStatus }: { currentStatus: DemandStatus }) {
  const selectable = getSelectableStatuses(currentStatus)
  const final = isFinalStatus(currentStatus)

  const options = DEMAND_STATUSES.map((value) => ({
    value,
    label: DEMAND_STATUS_LABELS[value],
    hint: value === currentStatus ? 'actual' : selectable.includes(value) ? undefined : 'no
permitido',
    disabled: !selectable.includes(value),
  }))

  return (
    <SelectField name="status" label="Estado" required options={options} placeholder="Selecciona el
estado"
      disabled={final}
```

```
        description={final ? 'Estado final: no admite más cambios de estado.' : undefined} />
    )
}
```

El desplegable muestra **todos** los estados, pero desactiva los no permitidos y los marca como «no permitido». Mostrarlos desactivados, en lugar de ocultarlos, enseña al usuario el ciclo completo. En un estado final, el campo entero queda desactivado con una explicación.

Junto al desplegable, el componente `StatusFlow` dibuja el ciclo de vida: el estado actual resaltado, los siguientes posibles con borde discontinuo, el resto atenuado, y una frase como «Desde «Abierta» puede pasar a «En curso» o «Cancelada».». La ficha de detalle también lo muestra.

22.3.2 Capa 2: el esquema de Zod

El esquema de modificación se construye con el estado actual de la demanda y añade la comprobación con `superRefine` (capítulo 20). Como depende de `demand.status`, el *resolver* se memoriza con ese valor:

`src/features/demands/pages/demand-edit-page.tsx (extracto)`

```
// Las transiciones válidas dependen del estado que tiene la demanda en el
// servidor, que cambia tras guardar o recargar. React Hook Form toma el
// resolver actualizado en cada render.
const currentStatus = demand.status
const resolver = useMemo(() => zodResolver(createDemandUpdateSchema(currentStatus)), [currentStatus]
)
```

Tras guardar un cambio de `OPEN` a `IN_PROGRESS`, la caché actualiza `demand.status`, el *resolver* se reconstruye y las transiciones permitidas pasan a ser las de `IN_PROGRESS`.

22.3.3 Capa 3: el 409 del servidor

¿Cómo puede llegar un 409 si el cliente solo permite transiciones válidas? Porque el cliente decide con **los datos que cargó**, y pueden haber cambiado. Ejemplo: un compañero completa la demanda mientras tú la tienes abierta en *En curso* e intentas cancelarla. Tu pantalla cree que `IN_PROGRESS → CANCELLED` es válido; el servidor, que ya la tiene en `COMPLETED`, responde:

```
409 { "status": 409, "error": "RESOURCE_ERROR", "message": "Invalid demand status transition:
COMPLETED -> CANCELLED" }
```

La página lo trata de forma específica:

`src/features/demands/pages/demand-edit-page.tsx (extracto)`

```
/** Traduce un 409 de transición a un mensaje comprensible. */
function describeConflict(error: unknown): string {
  const message = getErrorMessage(error)
  const transition = parseTransitionError(message)
  if (!transition) return message
  return `El servidor no permite pasar de «${DEMAND_STATUS_LABELS[transition.from]}» a «${DEMAND_STA
TUS_LABELS[transition.to]}».`
}
```

`parseTransitionError` extrae los dos estados del mensaje del backend con una expresión regular y comprueba que sean estados conocidos. La alerta resultante dice «Cambio rechazado por el servidor · El servidor no permite pasar de «Completada» a «Cancelada». Puede que la demanda haya cambiado mientras la editabas.» y ofrece un botón **Recargar datos actuales**:

src/features/demands/pages/demand-edit-page.tsx (extracto)

```
const reload = async () => {
  setReloading(true)
  try {
    const fresh = await onReload()          // query.refetch() en la página
    if (fresh) {
      form.reset(toFormValues(fresh))       // el formulario muestra el estado real
      mutation.reset()                      // desaparece la alerta
      setUpdated(null)
    }
  } finally {
    setReloading(false)
  }
}
```

Al recargar, el formulario y las transiciones permitidas se recalculan a partir del estado real del servidor.

> **Cuidado**
>
> `parseTransitionError` depende del **texto exacto** del mensaje del backend («Invalid demand status transition: X → Y»). Si alguien lo traduce o lo reformula, el frontend dejará de reconocerlo y mostrará el mensaje original, sin romperse pero con peor experiencia. Por eso existe una prueba que fija el formato (`status-rules.test.ts`), y por eso lo ideal sería que el backend devolviera un código de error estable y los estados como campos del cuerpo, en lugar de dentro de un texto (capítulo 25).

22.3.4 Una mejora pendiente: la concurrencia optimista

El 409 de transición detecta un caso concreto de modificación concurrente, pero no todos: si dos usuarios editan el título a la vez, gana el último en guardar, sin aviso. La solución estándar es la **concurrencia optimista**: un campo de versión (`@Version` en JPA) que viaja en la respuesta y se devuelve al guardar; si no coincide, el backend responde 409 y el frontend ofrece recargar, exactamente como ya hace para las transiciones. La infraestructura del frontend (alerta de conflicto y recarga) ya estaría preparada.

22.4 El selector de aplicación

↑ Índice

Una demanda pertenece a una aplicación, y el formulario necesita un desplegable con el catálogo. Es el punto de contacto entre módulos del capítulo 13:

src/features/demands/components/application-options.ts (extracto)

```
export function useApplicationSelectOptions(ensure?: { id: number; name: string }) {
  const query = useApplicationOptions()
  const ensureId = ensure?.id
  const ensureName = ensure?.name

  const options = useMemo<ApplicationOption[]>(() => {
    const list = (query.data?.content ?? []).map((app) => ({
      value: String(app.id), name: app.name, inactive: app.status === 'INACTIVE',
    }))
    if (ensureId !== undefined && ensureName && !list.some((o) => o.value === String(ensureId))) {
      list.unshift({ value: String(ensureId), name: ensureName, inactive: false })
    }
    return list
  }, [query.data, ensureId, ensureName])

  const truncated = (query.data?.totalElements ?? 0) > APPLICATION_OPTIONS_LIMIT
```

```
  return { query, options, truncated }
}
```

Las decisiones de este hook:

Decisión	Motivo
Se piden hasta 1000 aplicaciones ordenadas por nombre en una sola página	La API no tiene un endpoint de «todas»; Spring Data limita el tamaño de página a 2000 por defecto
Se incluyen las inactivas, marcadas	El backend acepta demandas sobre aplicaciones inactivas, y una demanda existente puede estar asociada a una
`ensure` añade la aplicación actual si no está en la lista	Si el catálogo supera el límite, la aplicación de la demanda que se edita podría no estar entre las mil primeras, y el desplegable no podría mostrarla
`truncated` avisa si hay más de 1000	El campo muestra «Se muestran las primeras 1000 aplicaciones por orden alfabético.»
Los valores son cadenas	El `Select` de Radix trabaja con texto; el esquema lo convierte a número (capítulo 20)
Caché de 5 minutos y colgada de `lists()`	El catálogo cambia poco, pero se refresca al crear o modificar aplicaciones (capítulo 17)

El campo `ApplicationField` añade los estados de la consulta: «Cargando aplicaciones…» desactivado mientras llega el catálogo; si falla, un mensaje con *Reintentar*; si no hay ninguna aplicación, un enlace a *Crear una aplicación*. El mismo hook alimenta el filtro por aplicación del listado.

> **Nota**
>
> El límite de mil aplicaciones es razonable para el volumen actual, pero no escala indefinidamente. La evolución natural es un **selector con búsqueda** (*combobox*) que consulte al backend mientras el usuario escribe, para lo que haría falta el filtro por nombre que hoy no existe en la API.

22.5 Las fechas

↑ Índice

Las demandas llevan `createdAt` y `updatedAt`, que el backend serializa como `java.time.Instant` en ISO-8601 y UTC: `"2026-10-24T08:15:30.123Z"`. El frontend las muestra en la hora local del navegador:

src/lib/format.ts

```
const dateTimeFormatter = new Intl.DateTimeFormat('es-ES', { dateStyle: 'short', timeStyle:
'short' })
const longDateTimeFormatter = new Intl.DateTimeFormat('es-ES', { dateStyle: 'long', timeStyle: 'medi
um' })

/** Formatea un instante ISO-8601 (p. ej. java.time.Instant) en hora local. */
export function formatDateTime(iso: string | null | undefined, style: 'short' | 'long' = 'short'): s
tring {
  if (!iso) return '—'
  const date = new Date(iso)
  if (Number.isNaN(date.getTime())) return iso
```

```
  return (style === 'long' ? longDateTimeFormatter : dateTimeFormatter).format(date)
}
```

- `Intl.DateTimeFormat` es la API estándar de internacionalización: formatea según el idioma y la zona horaria del usuario, sin librerías. Los formateadores se crean **una vez** a nivel de módulo, porque crearlos es relativamente costoso.
- La tabla muestra el formato corto (`24/10/26, 10:15`) y, al pasar el ratón, el largo (`24 de octubre de 2026, 10:15:30`) en el atributo `title`.
- Si la fecha no es válida, se muestra tal cual en lugar de «Invalid Date».

> **Si vienes de Java...**
>
> `Instant` en el backend y `Date` en el navegador representan lo mismo: un punto en el tiempo, sin zona. La zona solo se aplica al **mostrar**, como harías con un `DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT).withZone(zonaDelUsuario)`. Guardar y transportar en UTC, mostrar en local: la regla de oro de las fechas.

22.6 Navegación entre módulos

↑ Índice

Las demandas se conectan con las aplicaciones en ambos sentidos, siempre a través de la URL:

- Desde el **detalle de una aplicación**, *Ver demandas* abre `/demandas?aplicacion=7`: el listado lee el parámetro como su filtro de aplicación.
- **`/demandas/nueva?aplicacion=7`** preselecciona la aplicación en el alta:

`src/features/demands/pages/demand-create-page.tsx (extracto)`

```
// Permite llegar con la aplicación preseleccionada: /demandas/nueva?aplicacion=3
const [searchParams] = useSearchParams()
const presetApplicationId = parseIdParam(searchParams.get('aplicacion'))
// …
defaultValues: emptyForm(presetApplicationId),
```

- En la **tabla de demandas**, el nombre de la aplicación es un enlace a su detalle.

Si al crear la demanda la aplicación ya no existe (alguien la borró en otro servicio, o el enlace tenía un identificador inválido), el backend responde 404 y la alerta lo explica: «La aplicación seleccionada ya no existe. Recarga la página y elige otra.». Si es el catálogo el que no responde, el servicio de demandas devuelve un 503 y la alerta muestra su mensaje (capítulo 15).

22.7 Diferencias con el contrato del backend

↑ Índice

Aspecto	Backend	Frontend	Comentario
Orden por defecto	`@PageableDefault(sort = "id", direction = ASC)`	`id` descendente (`DEFAULT_DEMAND_LIST_PARAMS`)	Decisión del frontend: las más recientes primero. Como siempre envía `sort`, no hay conflicto

Aspecto	Backend	Frontend	Comentario
Campos de ordenación	`id`, `title`, `priority`, `status`, `updatedAt`	Además `application.name` y `createdAt`	**Desajuste**: esas dos columnas producen un 400 (ejercicio 14.2)
Orden de prioridad y estado	Se guardan como texto	—	Se ordenan alfabéticamente, como la criticidad de las aplicaciones
`updatedAt`	Se actualiza en cada `PUT`, aunque no cambie nada	—	La fecha de modificación puede inducir a error si se guarda sin cambios; el frontend lo mitiga desactivando *Guardar* sin cambios

El desajuste de ordenación es un buen ejemplo de por qué las listas que duplican un contrato (`DEMAND_SORT_FIELDS` frente a `VALID_SORT_FIELDS`) deben revisarse cuando cambia el backend: tras separar las aplicaciones en otro servicio, el backend dejó de poder ordenar por su nombre, y el frontend no se actualizó.

22.8 Errores típicos y depuración

↑ Índice

- **El desplegable de estado está desactivado.** La demanda está en un estado final; es el comportamiento esperado.
- **Aparece «Cambio rechazado por el servidor».** Alguien ha cambiado la demanda; usa *Recargar datos actuales*. Si ocurre siempre con la misma transición, compara `status-rules.ts` con `Demand.changeStatus()`.
- **El desplegable de aplicación no muestra la aplicación de la demanda.** Comprueba que se pasa `current` a `ApplicationField` (lo hace la página de edición).
- **«No se han podido cargar las aplicaciones».** El servicio de aplicaciones (8081) no responde; el resto del módulo de demandas puede seguir funcionando.
- **Las fechas aparecen desplazadas una o dos horas.** Se está formateando la cadena como si fuera hora local o se ha perdido la `Z` final; `new Date(iso)` con el formato ISO completo lo interpreta bien.

22.9 Ejercicios

↑ Índice

Ejercicio 22.1 · Reproduce un conflicto de transición Ver solución →

Abre la misma demanda `OPEN` en modificación en dos pestañas. En la primera, cámbiala a *En curso* y guarda; después, en la otra, a *Completada*... ¿puedes? ¿Por qué no? Ahora, en la primera, pásala a *Completada* y guarda, y en la segunda (que sigue creyendo que está *Abierta*) cámbiala a *Cancelada* y guarda. Describe la respuesta en *Network*, el mensaje que ve el usuario y lo que ocurre al pulsar *Recargar datos actuales*.

Ejercicio 22.2 · Corrige el desajuste de ordenación Ver solución →

Alinea el frontend con el backend: quita `application.name` y `createdAt` de `DEMAND_SORT_FIELDS` y convierte esas columnas de la tabla en cabeceras no ordenables (`TableHead` en lugar de

`SortableTableHead`). Comprueba con `npm run typecheck` qué más hay que tocar, actualiza las pruebas y verifica que una URL antigua con `?orden=createdAt,asc` cae al orden por defecto.

Ejercicio 22.3 · Muestra las transiciones como acciones Ver solución →

En el detalle de una demanda, añade botones de acción rápida para cada transición permitida («Iniciar», «Completar», «Cancelar») usando `getNextStatuses`. Cada botón debe enviar un `PUT` con los datos actuales y el nuevo estado, mediante `useUpdateDemand`. Pide confirmación antes de las transiciones a estados finales. ¿Qué haces si la respuesta es un 409?

22.10 Autoevaluación

↑ Índice

Autoevaluación del capítulo 22 Ver respuestas →

1. ¿Qué tres cosas añade el dominio de las demandas respecto al de aplicaciones?
2. ¿Dónde vive la regla de transición de estados y dónde su reflejo en el frontend?
3. ¿Por qué se expresa la regla como una tabla de transiciones?
4. Describe las tres capas de defensa de la regla y el propósito de cada una.
5. ¿Cómo puede llegar un 409 si el cliente solo permite transiciones válidas?
6. ¿Qué hace el botón *Recargar datos actuales*?
7. ¿Por qué `useApplicationSelectOptions` acepta un parámetro `ensure`?
8. ¿Por qué las fechas viajan en UTC y se muestran en hora local?
9. ¿Qué desajuste hay entre los campos de ordenación del frontend y los del backend, y cuál es su origen?

PARTE IX

Calidad y evolución

23 Pruebas con Vitest pág. 216

24 Registro de decisiones de arquitectura pág. 224

25 Notas de integración con el backend pág. 231

26 Ejercicio guiado: el módulo de Incidencias pág. 238

↑ Volver al índice

CAPÍTULO 23

Pruebas con Vitest

↑ Volver al índice

EN ESTE CAPÍTULO

23.1 Qué merece la pena probar en un frontend
23.2 Vitest
23.3 Anatomía de una prueba
23.4 Inventario de las pruebas
23.5 Patrones que usan las pruebas
23.6 Dobles de prueba
23.7 Ampliar la estrategia
23.8 Cobertura e integración continua
23.9 Errores típicos y depuración
23.10 Ejercicios
23.11 Autoevaluación

El proyecto tiene 38 pruebas unitarias repartidas en siete ficheros, y todas pasan en algo más de un segundo. No prueban componentes ni pantallas: prueban la lógica que más valor tiene y más fácil es de romper sin darse cuenta. Este capítulo explica esa elección, cómo funciona Vitest, qué patrones usan las pruebas existentes y cómo ampliar la estrategia a componentes, hooks y pruebas de extremo a extremo.

23.1 Qué merece la pena probar en un frontend

↑ Índice

En el backend, la estrategia habitual es una **pirámide**: muchas pruebas unitarias del dominio, menos pruebas de integración (repositorios con una base de datos real, controladores con MockMvc) y unas pocas pruebas de extremo a extremo. En el frontend la forma es parecida, con sus propias capas:

Nivel	Qué prueba	Herramientas	En el proyecto
Unitario	Funciones puras: validación, reglas, transformaciones	Vitest	**Sí: 38 pruebas**
Componente	Que un componente pinte lo esperado y responda a eventos	Vitest + Testing Library + un DOM simulado	No, todavía
Integración	Una página completa con su router, su caché y una API simulada	Lo anterior + MSW	No, todavía
Extremo a extremo	La aplicación real en un navegador real contra el backend real	Playwright	No, todavía

El proyecto empezó por donde la relación valor/coste es mayor: **la lógica de `model/` y `lib/`**. Son funciones puras, rápidas de probar, y concentran justo lo que más fácilmente se rompe sin que nadie lo note al probar a mano: un límite de longitud que deja de coincidir con el backend, un valor de la URL que se acepta sin validar, una transición de estado mal copiada, un formato de error que se interpreta mal.

Si vienes de Java...

La correspondencia es directa: Vitest es JUnit 5 (con aserciones al estilo de AssertJ); `vi.fn()` es Mockito; Testing Library cumple el papel de MockMvc para la interfaz; MSW es WireMock; y Playwright con el backend real equivale a tus pruebas con Testcontainers, que el backend del proyecto ya usa (`spring-boot-testcontainers`).

23.2 Vitest

↑ Índice

Vitest es el ejecutor de pruebas del ecosistema Vite. Reutiliza la misma configuración y la misma transformación de TypeScript, así que no hay que configurar nada aparte para que las pruebas entiendan el alias `@/`, los tipos o los módulos ES. Su API es compatible con la de Jest, el ejecutor más extendido durante años, de modo que casi toda la documentación y los ejemplos de Jest sirven.

La configuración está en `vite.config.ts`:

`vite.config.ts (extracto)`

```
test: {
  environment: 'node',
  include: ['src/**/*.test.ts'],
},
```

- **`environment: 'node'`**: las pruebas se ejecutan en Node, sin simular un navegador. Basta para funciones puras, y es lo más rápido. Las pruebas de componentes necesitarán un entorno con DOM simulado (`jsdom` o `happy-dom`).
- **`include`**: solo los ficheros `*.test.ts` dentro de `src`, junto al código que prueban.

Comando	Qué hace
`npm run test`	Ejecuta todas las pruebas una vez (`vitest run`); es lo que usaría la integración continua
`npx vitest`	Modo vigilancia: vuelve a ejecutar las pruebas afectadas al guardar
`npx vitest run list-params`	Solo los ficheros cuyo nombre contiene el texto
`npx vitest run -t "transición"`	Solo las pruebas cuyo nombre contiene el texto
`npx vitest run --reporter=verbose`	Lista cada prueba con su resultado

La salida de una ejecución completa:

```
RUN  v5.0.1 …/it-service-manager-web

Test Files  7 passed (7)
     Tests  38 passed (38)
  Duration  1.25s
```

23.3 Anatomía de una prueba

↑ Índice

`src/features/demands/model/schemas.test.ts (extracto)`

```
import { describe, expect, it } from 'vitest'

import { createDemandUpdateSchema, demandCreateSchema, normalizeDescription } from './schemas'

describe('validación de formularios de demanda', () => {
  const valid = { title: 'Adaptación a la nueva normativa', description: '', priority: 'HIGH', appli
cationId: '1' }

  it('acepta un alta válida, recorta el título y convierte la aplicación a número', () => {
    const result = demandCreateSchema.parse({ ...valid, title: '  Adaptación  ' })
    expect(result.title).toBe('Adaptación')
    expect(result.applicationId).toBe(1)
  })

  it('la modificación rechaza una transición no permitida con un mensaje claro', () => {
    const result = createDemandUpdateSchema('IN_PROGRESS').safeParse({ ...valid, status: 'OPEN' })
    expect(result.success).toBe(false)
    expect(result.error?.issues[0]?.path).toEqual(['status'])
    expect(result.error?.issues[0]?.message).toBe('No se puede pasar de «En curso» a «Abierta».')
  })
})
```

- `describe` agrupa pruebas relacionadas (como una clase de pruebas de JUnit).
- `it` (o su sinónimo `test`) define una prueba con una frase que describe el comportamiento esperado. Las del proyecto están en español y se leen como especificaciones.
- `expect(valor)` seguido de un ***matcher*** expresa la comprobación.
- Un **objeto válido de referencia** (`valid`) se reutiliza en todas las pruebas, cambiando solo lo que cada una quiere comprobar con el operador de propagación. Así cada prueba muestra exactamente qué la hace distinta.

Matcher	Comprueba	Ejemplo
`toBe(x)`	Igualdad estricta (`Object.is`): primitivos o misma referencia	`expect(result.applicationId).toBe(1)`
`toEqual(x)`	Igualdad profunda de objetos y arrays	`expect(params).toEqual({ …DEFAULT_LIST_PARAMS })`
`toBeNull()`, `toBeUndefined()`	Valores vacíos	`expect(normalizeDescription(' ')).toBeNull()`
`toContain(x)`	Que una cadena o un array contenga algo	`expect(empty.message).toContain('backend está arrancado')`
`toHaveProperty(p)`	Que un objeto tenga una propiedad	`expect(result).not.toHaveProperty('status')`
`resolves`, `rejects`	El resultado de una promesa	`await expect(promesa).resolves.toBeUndefined()`
`.not`	Niega cualquier *matcher*	

23.4 Inventario de las pruebas

↑ Índice

Fichero	Pruebas	Qué protege
`lib/api-client.test.ts`	4	Construcción de la consulta y la interpretación de los tres formatos de error (backend, Spring por defecto, cuerpo vacío)
`lib/list-params.test.ts`	4	Conversión de páginas 1-based ↔ 0-based, identificadores, orden y alternancia de dirección
`features/applications/model/list-params.test.ts`	5	URL ↔ parámetros ↔ consulta del listado de aplicaciones
`features/applications/model/schemas.test.ts`	6	Reglas del formulario alineadas con Bean Validation
`features/demands/model/list-params.test.ts`	5	Lo mismo para demandas, con filtro por aplicación
`features/demands/model/schemas.test.ts`	8	Reglas del formulario, conversión de la aplicación y transiciones en la modificación
`features/demands/model/status-rules.test.ts`	6	La máquina de estados y la interpretación del mensaje del 409

23.5 Patrones que usan las pruebas

↑ Índice

1. Valores límite. Para cada longitud máxima, una prueba con el valor exacto y otra con uno más:

`src/features/applications/model/schemas.test.ts`

```
expect(applicationCreateSchema.safeParse({ ...valid, name: 'a'.repeat(100) }).success).toBe(true)
expect(applicationCreateSchema.safeParse({ ...valid, name: 'a'.repeat(101) }).success).toBe(false)
```

2. Propiedades del diseño. En lugar de probar casos sueltos, se prueba una propiedad general: que serializar y volver a leer la URL devuelve lo mismo, o que mantener el estado siempre es una transición válida para **todos** los estados:

`src/features/demands/model/status-rules.test.ts`

```
it('mantener el mismo estado siempre es válido', () => {
  for (const status of DEMAND_STATUSES) expect(canTransition(status, status)).toBe(true)
})
```

3. Pruebas de contrato. Varias pruebas fijan lo que el frontend espera del backend: las transiciones copiadas de `Demand.changeStatus()`, el texto del 409 (`'Invalid demand status transition: IN_PROGRESS -> OPEN'`), los cuerpos de error de `GlobalExceptionHandler`. Si el backend cambia, estas pruebas indican exactamente qué hay que revisar en el frontend.

4. Objetos reales de la plataforma. Las pruebas del cliente HTTP construyen objetos `Response` reales, disponibles en Node 22 igual que en el navegador, en lugar de simularlos:

`src/lib/api-client.test.ts`

```
const json = (status: number, body: unknown) =>
  new Response(JSON.stringify(body), { status, headers: { 'Content-Type': 'application/json' } })
```

```
const error = await toApiError(json(404, { status: 404, error: 'RESOURCE_ERROR', message: 'Applicati
on not found: 99', fieldErrors: {} }))
expect(error.kind).toBe('not-found')
```

Que `toApiError` reciba un `Response` (y no haga la petición) es lo que permite probarlo sin red: una decisión de diseño que facilita las pruebas.

5. Tablas de casos. Cuando una función se prueba con muchas entradas, `it.each` evita repetir código:

```
it.each([
  ['3', 2], ['1', 0], ['0', 0], ['-1', 0], ['abc', 0], [null, 0],
])('parsePageParam(%s) → %i', (input, expected) => {
  expect(parsePageParam(input)).toBe(expected)
})
```

23.6 Dobles de prueba

↑ Índice

Vitest incluye utilidades para sustituir dependencias, a través del objeto `vi`:

Utilidad	Para qué	Equivalente en Java
`vi.fn()`	Función simulada que registra sus llamadas y puede devolver valores	`Mockito.mock()`
`vi.spyOn(obj, 'metodo')`	Observa o sustituye un método existente	`Mockito.spy()`
`vi.stubGlobal('fetch', …)`	Sustituye una variable global, como `fetch`	—
`vi.mock('./modulo')`	Sustituye un módulo importado completo	—
`vi.useFakeTimers()`	Controla el tiempo (`setTimeout` , fechas)	Un `Clock` inyectable

El ejercicio 15.2 usaba `vi.stubGlobal` para probar `apiClient.delete` sin red. Úsalos con moderación: una prueba con muchos dobles prueba sobre todo los dobles. Las pruebas actuales no necesitan ninguno porque prueban funciones puras, y es una buena señal de diseño.

23.7 Ampliar la estrategia

↑ Índice

23.7.1 Pruebas de componentes

Con **Testing Library** se monta un componente en un DOM simulado y se interactúa con él **como lo haría un usuario**: buscando elementos por su texto o su rol accesible, no por clases CSS. Haría falta instalar `@testing-library/react` , `@testing-library/user-event` y `jsdom` , y usar `environment: 'jsdom'` para esos ficheros (por ejemplo, con un comentario `// @vitest-environment jsdom` al principio, o ampliando `include` a `*.test.tsx`).

`src/components/common/pagination-bar.test.tsx (ejemplo, requiere las dependencias)`

```
// @vitest-environment jsdom
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
```

```
import { describe, expect, it, vi } from 'vitest'
import { PaginationBar } from './pagination-bar'

describe('PaginationBar', () => {
  it('muestra el rango y avisa al pasar de página', async () => {
    const onPageChange = vi.fn()
    render(<PaginationBar page={0} size={10} totalElements={25} totalPages={3}
                          pageSizeOptions={[10, 20]} onPageChange={onPageChange}
onSizeChange={vi.fn()} />)

    expect(screen.getByText(/Mostrando/)).toHaveTextContent('Mostrando 1–10 de 25')
    expect(screen.getByRole('button', { name: 'Página anterior' })).toBeDisabled()

    await userEvent.click(screen.getByRole('button', { name: 'Página siguiente' }))
    expect(onPageChange).toHaveBeenCalledWith(1)
  })
})
```

Fíjate en que la prueba encuentra los botones por su `aria-label` («Página siguiente»): la accesibilidad del capítulo 6 hace los componentes más fáciles de probar. (`toHaveTextContent` y `toBeDisabled` vienen de `@testing-library/jest-dom`, otra dependencia habitual.)

23.7.2 Pruebas de hooks y páginas

Los hooks de consulta se prueban con `renderHook`, envolviéndolos en un `QueryClientProvider` con un `QueryClient` nuevo por prueba y los reintentos desactivados. Para que no dependan del backend, **MSW** (*Mock Service Worker*) intercepta las peticiones `fetch` y devuelve respuestas definidas en la prueba, sin tocar el código de la aplicación.

> **Nota**
>
> La norma del proyecto de **no usar datos simulados como sustituto de la API real** se refiere a la aplicación: las pantallas siempre hablan con el backend. En las pruebas automáticas, simular respuestas es legítimo y necesario: permite comprobar casos difíciles de reproducir a mano (un 409, un 503 del catálogo, una página vacía) de forma rápida y repetible. Las pruebas de extremo a extremo, en cambio, sí deben usar el backend real.

23.7.3 Pruebas de extremo a extremo

Playwright controla un navegador real (Chromium, Firefox o WebKit) y ejecuta recorridos completos contra la aplicación y el backend arrancados: crear una aplicación, verla en el listado, crear una demanda para ella y cambiar su estado. Son las más lentas y las más frágiles, así que se reservan para los recorridos críticos. Combinadas con Testcontainers en el backend, permitirían levantar todo el sistema en la integración continua.

23.8 Cobertura e integración continua

↑ Índice

Vitest mide la **cobertura** (qué líneas ejecutan las pruebas) con el paquete `@vitest/coverage-v8`, que el proyecto no tiene instalado: `npm install -D @vitest/coverage-v8` y `npx vitest run --coverage`. Úsala para encontrar lógica importante sin probar, no como objetivo numérico: un 100 % de cobertura con aserciones débiles protege menos que un 60 % bien elegido.

En integración continua, el orden razonable de comprobaciones es del más barato al más caro:

`.github/workflows/ci.yml (ejemplo)`

```yaml
name: ci
on: [push, pull_request]
jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run typecheck
      - run: npm run lint
      - run: npm run test
      - run: npm run build
```

Decisión de diseño: Empezar por las pruebas unitarias de la lógica pura

Decisión. Probar con Vitest la lógica de `model/` y `lib/` (validaciones, reglas de dominio, URL, interpretación de errores) y dejar para más adelante las pruebas de componentes y de extremo a extremo.

Motivos. Es donde están las reglas que más daño hacen al romperse y que menos se notan en una prueba manual; las pruebas son rápidas, estables y no requieren infraestructura. El diseño del proyecto (lógica fuera de los componentes) es lo que lo hace posible.

Siguiente paso recomendado. Una prueba de extremo a extremo por módulo con Playwright sobre el recorrido principal, y pruebas de componente para los componentes comunes con más lógica (`PaginationBar` , `FilterSelect` , `StatusFlow`).

23.9 Errores típicos y depuración

↑ Índice

- **`ReferenceError: document is not defined`.** La prueba usa el DOM con `environment: 'node'`. Usa `jsdom` para ese fichero.
- **Una prueba pasa sola y falla con las demás.** Comparten estado: un objeto global modificado, un `vi.stubGlobal` sin restaurar o un `QueryClient` reutilizado. Restaura en `afterEach`.
- **`toBe` falla con dos objetos iguales.** `toBe` compara referencias; usa `toEqual`.
- **Una prueba asíncrona pasa aunque la aserción sea falsa.** Falta `await` (o `return`) delante de la promesa: la prueba termina antes de comprobar nada.
- **Pruebas frágiles que se rompen con cualquier cambio de diseño.** Buscan elementos por clases CSS o por estructura del DOM. Busca por rol, etiqueta o texto, como haría un usuario.
- **Vitest no arranca en otra máquina o en Docker** con un error de *native binding*. Las dependencias se instalaron en otro sistema operativo (los binarios de Rolldown son específicos de cada plataforma): reinstala con `npm ci` en esa máquina.

Para depurar una prueba, puedes poner un punto de ruptura con `debugger;` y ejecutar Vitest desde el depurador del editor (VS Code e IntelliJ lo integran), o usar `npx vitest --ui` para una interfaz web con el resultado de cada prueba.

23.10 Ejercicios

↑ Índice

Ejercicio 23.1 · Convierte una prueba en tabla Ver solución →

Reescribe la prueba «convierte la página de la URL (1-based) a Spring Data (0-based)» de `src/lib/list-params.test.ts` con `it.each`, añadiendo casos para `'-1'`, `'abc'`, `'2.5'` y `''`. ¿Qué devuelve `parsePageParam('2.5')` y por qué? (Pista: `Number.parseInt`.) ¿Te parece el comportamiento correcto?

Ejercicio 23.2 · Prueba `formatDateTime` Ver solución →

Escribe `src/lib/format.test.ts` con pruebas para `null`, una cadena no válida y un instante válido. El resultado de un instante válido depende de la zona horaria de la máquina: ¿cómo harías la prueba estable? Investiga la variable de entorno `TZ` y la opción `env` de la configuración de Vitest.

Ejercicio 23.3 · Tu primera prueba de componente Ver solución →

Instala `@testing-library/react`, `@testing-library/user-event`, `@testing-library/jest-dom` y `jsdom` como dependencias de desarrollo y amplía `include` a `src/**/*.test.{ts,tsx}`. Escribe una prueba para `StatusFlow` que compruebe que, para una demanda `OPEN`, el texto indica que puede pasar a «En curso» o «Cancelada» y que el estado actual tiene `aria-current="step"`. Ejecuta `npm run typecheck` y `npm run lint` para asegurarte de que todo sigue en orden.

23.11 Autoevaluación

↑ Índice

Autoevaluación del capítulo 23 Ver respuestas →

1. ¿Qué cuatro niveles de pruebas se distinguen en un frontend y cuál cubre hoy el proyecto?
2. ¿Por qué se empezó por las funciones de `model/` y `lib/`?
3. ¿Qué ventaja tiene que Vitest reutilice la configuración de Vite?
4. ¿Qué diferencia hay entre `toBe` y `toEqual`?
5. ¿Qué es una prueba de contrato y qué ejemplos hay en el proyecto?
6. ¿Por qué el diseño de `toApiError` facilita probarlo?
7. ¿Cómo busca los elementos Testing Library y por qué?
8. ¿Es contradictorio usar MSW en pruebas con la norma de no usar datos simulados?
9. ¿En qué orden pondrías `typecheck`, `lint`, `test` y `build` en la integración continua y por qué?

CAPÍTULO 24

Registro de decisiones de arquitectura

↑ Volver al índice

EN ESTE CAPÍTULO

24.1 Qué es un ADR y para qué sirve
24.2 El registro del proyecto
24.3 Decisiones propuestas
24.4 Cómo escribir los siguientes ADR
24.5 Errores típicos
24.6 Ejercicios
24.7 Autoevaluación

A lo largo de la guía han aparecido más de cuarenta recuadros «Decisión de diseño». Este capítulo los reúne en un registro de decisiones de arquitectura (ADR): un formato breve y estándar que explica qué se decidió, por qué y con qué consecuencias. Sirve como resumen de la guía, como documentación para quien se incorpore al proyecto y como plantilla para las decisiones que vendrán.

24.1 Qué es un ADR y para qué sirve

↑ Índice

Un ***Architecture Decision Record*** es un documento corto que recoge **una** decisión significativa. El formato más extendido, propuesto por Michael Nygard, tiene cinco apartados:

Apartado	Contenido
Título y número	`ADR-007: El estado de los listados vive en la URL`
Estado	Propuesta, aceptada, sustituida por ADR-NNN, obsoleta
Contexto	Qué problema o fuerza obliga a decidir
Decisión	Qué se hace, en una o dos frases
Consecuencias	Qué se gana, qué se pierde y qué obliga a hacer a partir de ahora

Las ventajas son prácticas. Dentro de un año, cuando alguien pregunte «¿por qué no usamos Redux?» o «¿por qué los filtros están en la URL?», la respuesta estará escrita con el contexto de entonces. Y cuando el contexto cambie, se escribe un ADR nuevo que **sustituye** al anterior, en lugar de borrar la historia.

Si vienes de Java...

Si has trabajado en un equipo grande con un comité de arquitectura, es la versión ligera de un documento de diseño. La convención habitual es guardarlos en el propio repositorio, en `docs/adr/`, como ficheros Markdown numerados (`0007-estado-listados-en-url.md`), para que viajen con el código y se revisen en las mismas solicitudes de cambio.

24.2 El registro del proyecto

↑ Índice

Las decisiones están agrupadas por área. Cada una enlaza mentalmente con el capítulo donde se explica a fondo.

24.2.1 Plataforma y herramientas

Decisión de diseño: ADR-001 · SPA pura construida con Vite

Contexto. El backend es una API REST independiente en Spring Boot; la aplicación es interna y autenticada. **Decisión.** SPA servida como ficheros estáticos, construida con Vite; sin renderizado en servidor. **Consecuencias.** Un único servidor que mantener (el backend); despliegue estático; primera carga algo mayor, mitigada con división del código. Hay que configurar el *fallback* a `index.html` en producción. *Capítulos 1 y 4.*

Decisión de diseño: ADR-002 · TypeScript en modo estricto

Contexto. Código mantenido a largo plazo por un equipo acostumbrado al tipado estático de Java. **Decisión.** TypeScript con `strict` y comprobaciones de variables y parámetros sin usar; dos subproyectos (navegador y Node). **Consecuencias.** Errores de nulos y de contrato detectados al compilar; algo más de código en los tipos. Vite no comprueba tipos: `npm run typecheck` y el *build* lo hacen. *Capítulos 2 y 5.*

Decisión de diseño: ADR-003 · npm, ESLint con reglas recomendadas y sin formateador (por ahora)

Contexto. Proyecto único, sin necesidades de monorepo. **Decisión.** npm como gestor; ESLint con los conjuntos recomendados de JS, TypeScript, *hooks* y React Refresh. **Consecuencias.** Configuración mínima y estándar. Pendiente: añadir Prettier. *Capítulos 3 y 5.*

24.2.2 Interfaz

Decisión de diseño: ADR-004 · Tailwind CSS 4 con tokens propios

Contexto. Se busca una estética propia (azul oscuro, pizarra, cian) y coherente en todas las pantallas. **Decisión.** Tailwind CSS 4 con un tema de tokens en `index.css` (variables OKLCH + `@theme inline`), siguiendo la convención de shadcn/ui. **Consecuencias.** Estilos junto al componente; CSS final pequeño; clases siempre completas (no dinámicas). Modo oscuro preparado pero sin definir. *Capítulo 10.*

Decisión de diseño: ADR-005 · Componentes de shadcn/ui sobre Radix, copiados al proyecto

Contexto. Se necesitan componentes accesibles (desplegables, paneles, *tooltips*) con aspecto propio. **Decisión.** shadcn/ui copiado en `components/ui`, sobre los primitivos de Radix. **Consecuencias.** Accesibilidad y comportamiento resueltos; control total del aspecto; hay que mantener el código copiado y no meter lógica de negocio en `ui/`. *Capítulo 11.*

Decisión de diseño: ADR-006 · Todas las pantallas contemplan todos sus estados

Contexto. Una API puede tardar, fallar o devolver listas vacías. **Decisión.** Cada pantalla que carga datos pinta explícitamente carga, error, vacío y datos, con componentes comunes. **Consecuencias.** Nunca hay pantallas en blanco sin explicación; las páginas tienen más ramas, pero son uniformes. *Capítulos 6 y 12.*

24.2.3 Arquitectura

Decisión de diseño: ADR-007 · Organización por funcionalidad con capas compartidas

Contexto. La aplicación crecerá módulo a módulo (aplicaciones, demandas, incidencias, equipos, SLA). **Decisión.** `features/<módulo>` con estructura interna fija (`api` , `model` , `components` , `pages` , `routes.ts`) y capas compartidas con dependencias en una sola dirección. **Consecuencias.** Todo lo de un módulo en una carpeta; correspondencia con los paquetes del backend; una dependencia entre módulos permitida y documentada (catálogo de aplicaciones). *Capítulos 13 y 14.*

Decisión de diseño: ADR-008 · Infraestructura común sin duplicar

Contexto. Al construir demandas, gran parte del código de aplicaciones resultó idéntico. **Decisión.** Mover lo común (indicadores, filtros, campos, paginación, utilidades de URL) a `components/common` y `lib` , separando apariencia y significado. **Consecuencias.** Un módulo nuevo solo aporta su dominio. Generalizar con el segundo caso, no con el primero. *Capítulo 13.*

Decisión de diseño: ADR-009 · Menú lateral declarativo por módulos y acciones

Contexto. Los usuarios trabajan por acciones («nueva demanda», «modificar aplicación»). **Decisión.** Un grupo por módulo con sus acciones, definido como datos en `navigation.ts` ; módulos futuros marcados como «Pronto». **Consecuencias.** Añadir un módulo al menú es añadir datos; el mismo menú sirve en escritorio y móvil. *Capítulo 12.*

24.2.4 Datos

Decisión de diseño: ADR-010 · Cliente HTTP propio sobre `fetch` con un modelo de errores uniforme

Contexto. Hay que interpretar varios formatos de error (backend, Spring por defecto, proxy) y mostrarlos de forma comprensible. **Decisión.** Un cliente mínimo sobre `fetch` que convierte cualquier fallo en un `ApiError` con `kind` , `status` , mensaje y errores por campo. **Consecuencias.** Un único sitio para la URL base, la serialización y, en el futuro, la autenticación; sin axios. *Capítulo 15.*

Decisión de diseño: ADR-011 · Mismo origen: proxy en desarrollo y proxy inverso en producción

Contexto. El backend no tiene CORS; la API está repartida en dos servicios. **Decisión.** URL base relativa `/api` ; Vite reparte los prefijos en desarrollo y un proxy inverso o pasarela lo hará en producción. **Consecuencias.** Sin CORS que mantener; el mismo *build* para todos los entornos. *Capítulos 4 y 16.*

Decisión de diseño: ADR-012 · TanStack Query para todo el estado del servidor

Contexto. Los datos de la API se comparten entre pantallas, se cachean y se invalidan al escribir. **Decisión.** `useQuery` / `useMutation` con claves jerárquicas por módulo; `staleTime` de 30 s, sin refresco al recuperar el foco, sin reintentar 4xx ni mutaciones; tras escribir, actualizar el detalle e invalidar los listados. **Consecuencias.** Sin `fetch` en efectos ni datos del servidor en `useState` ; navegación instantánea a datos ya vistos. *Capítulo 17.*

Decisión de diseño: ADR-013 · Sin librería de estado global

Contexto. El estado se reparte entre servidor, URL, formularios e interfaz local. **Decisión.** No usar Redux, Zustand ni similares; contexto solo para servicios y familias de componentes. **Consecuencias.** Menos conceptos y dependencias. Se reconsiderará si aparece estado de cliente compartido y cambiante. *Capítulo 9.*

Decisión de diseño: ADR-014 · Tipos de los DTO escritos a mano; validación en ejecución solo de la entrada del usuario

Contexto. El contrato con el backend está controlado por el mismo equipo. **Decisión.** Tipos en `model/types.ts` alineados a mano con los DTO; Zod solo para formularios. **Consecuencias.** Sincronización manual: cuando cambie el backend, hay que revisar tipos y listas (el desajuste de ordenación de demandas es un ejemplo). Evolución propuesta: generar tipos desde OpenAPI (ver ADR-022). *Capítulos 2 y 25.*

24.2.5 Navegación y formularios

Decisión de diseño: ADR-015 · Router de datos sin cargadores, con carga diferida por página

Contexto. Se quieren límites de error por ruta, títulos por ruta y *chunks* por página, pero los datos ya los gestiona TanStack Query. **Decisión.** `createBrowserRouter` con rutas por módulo, `lazy`, `errorElement` y `handle`; sin `loader` ni `action`. **Consecuencias.** Un único mecanismo de datos; páginas que se pintan enseguida con esqueletos. *Capítulo 18.*

Decisión de diseño: ADR-016 · El estado de los listados vive en la URL

Contexto. Los listados se recargan, se comparten y se navegan con el historial. **Decisión.** Filtros, página, tamaño y orden en la URL (nombres en español, páginas desde 1, valores por defecto omitidos), validados al leer con funciones puras. **Consecuencias.** Recargar, compartir y *atrás* funcionan sin código adicional; más código de lectura y escritura, probado. *Capítulo 19.*

Decisión de diseño: ADR-017 · React Hook Form + Zod, alineados con Bean Validation

Contexto. Formularios con reglas, transformaciones y errores del servidor. **Decisión.** Esquemas de Zod como fuente única de reglas; React Hook Form con `zodResolver`; campos genéricos sobre `Form` de shadcn/ui; validación al enviar. **Consecuencias.** Reglas probadas de forma aislada; conversión explícita formulario ↔ DTO; las constantes deben coincidir con `@Size` del backend. *Capítulo 20.*

Decisión de diseño: ADR-018 · Reglas de negocio reflejadas en el cliente y subordinadas al backend

Contexto. El usuario debe ver qué transiciones de estado puede hacer antes de intentarlo. **Decisión.** Copia explícita de la regla en `status-rules.ts`, aplicada en el desplegable y en el esquema; el backend sigue siendo la autoridad y su 409 se explica con opción de recargar. **Consecuencias.** Mejor experiencia; duplicación localizada y probada que hay que mantener sincronizada. *Capítulo 22.*

24.2.6 Calidad

Decisión de diseño: ADR-019 · Pruebas unitarias de la lógica pura primero

Contexto. Hay que proteger reglas que se rompen sin notarse en pruebas manuales. **Decisión.** Vitest sobre `model/` y `lib/`; componentes y extremo a extremo, más adelante. **Consecuencias.** 38 pruebas rápidas y estables; las pantallas siguen sin pruebas automáticas. *Capítulo 23.*

24.3 Decisiones propuestas

↑ Índice

Estas decisiones no se han tomado todavía. Están redactadas como **propuestas** para discutirlas cuando llegue el momento:

Decisión de diseño: ADR-020 (propuesta) · Autenticación con cookie de sesión gestionada por el backend

Contexto. La aplicación será interna y necesitará identificar al usuario. **Propuesta.** Autenticación en el backend (Spring Security, idealmente contra el proveedor de identidad corporativo) con una *cookie* `HttpOnly` en el mismo origen; el cliente HTTP trata el 401 de forma global y la caché se vacía al cerrar sesión. **Alternativa.** *Tokens* JWT gestionados en el navegador (más expuestos a XSS). *Capítulos 15 y 16.*

Decisión de diseño: ADR-021 (propuesta) · Pruebas de extremo a extremo con Playwright

Propuesta. Un recorrido principal por módulo contra el backend real, ejecutado en integración continua con los servicios levantados con Docker Compose o Testcontainers.

Decisión de diseño: ADR-022 (propuesta) · Contrato generado desde OpenAPI

Contexto. Los tipos y listas del frontend se desincronizan del backend (desajuste de ordenación). **Propuesta.** Añadir springdoc-openapi a los servicios y generar los tipos con `openapi-typescript` en un script de `package.json`; una prueba o un paso de la integración continua falla si el contrato cambia sin regenerar.

Decisión de diseño: ADR-023 (propuesta) · Formateo automático con Prettier

Propuesta. Prettier con `eslint-config-prettier` y el plugin de ordenación de clases de Tailwind, ejecutado al guardar y en la integración continua.

24.4 Cómo escribir los siguientes ADR

↑ Índice

Cuando tengas que tomar una decisión que cumpla alguna de estas condiciones, merece un ADR:

- Es **difícil de revertir** (elegir una librería central, un formato de URL, un modelo de autenticación).
- **Afecta a varios módulos** o a la forma de trabajar del equipo.
- Se descartaron **alternativas razonables** que alguien volverá a proponer.

Plantilla mínima:

docs/adr/0024-titulo-breve.md

```
# ADR-024: Título breve en forma de decisión

- Estado: propuesta | aceptada | sustituida por ADR-NNN
- Fecha: AAAA-MM-DD

## Contexto
Qué problema hay y qué fuerzas intervienen (requisitos, restricciones, equipo).

## Decisión
Qué se hace, en una o dos frases.

## Consecuencias
Qué se gana, qué se pierde, qué hay que hacer a partir de ahora.

## Alternativas consideradas
Cada una con el motivo por el que se descartó.
```

En el proyecto

Un buen primer paso sería crear `docs/adr/` en el proyecto con los ADR 001 a 019 de este capítulo, uno por fichero. Además de documentar el proyecto, es un ejercicio excelente para repasar la guía: escribir cada decisión con tus palabras obliga a entenderla.

24.5 Errores típicos

↑ Índice

- **ADR demasiado largos.** Si ocupa varias páginas, es un documento de diseño. Un ADR se lee en dos minutos.
- **Decisiones sin alternativas.** Sin las opciones descartadas, el ADR no responde a «¿y por qué no X?», que es su principal utilidad.
- **Reescribir ADR antiguos.** Cuando una decisión cambia, se escribe uno nuevo que sustituye al anterior y se marca el antiguo como sustituido.
- **Registrar trivialidades.** El nombre de una variable no es una decisión de arquitectura.

24.6 Ejercicios

↑ Índice

Ejercicio 24.1 · Escribe un ADR sobre la ordenación de demandas Ver solución →

Escribe un ADR completo, en estado «propuesta», que resuelva el desajuste entre los campos de ordenación del frontend y del backend. Incluye el contexto (la separación del servicio de aplicaciones), al menos dos alternativas y las consecuencias para ambos proyectos.

Ejercicio 24.2 · Evalúa una decisión con otro contexto Ver solución →

Imagina que la aplicación pasa a ser pública, para clientes externos, con miles de usuarios concurrentes y necesidad de buen posicionamiento en buscadores. ¿Qué ADR de este capítulo revisarías y en qué sentido? Elige tres y razona brevemente.

Ejercicio 24.3 · Crea el registro en el repositorio Ver solución →

Crea la carpeta `docs/adr/` con un `README.md` que explique el formato y los ADR 001, 012 y 016 escritos con tus palabras siguiendo la plantilla. Añade al `README.md` del proyecto un enlace a la carpeta.

24.7 Autoevaluación

↑ Índice

Autoevaluación del capítulo 24 Ver respuestas →

1. ¿Qué cinco apartados tiene un ADR según el formato de Nygard?
2. ¿Qué se hace con un ADR cuando la decisión cambia?
3. ¿Qué tres condiciones indican que una decisión merece un ADR?
4. ¿Qué ADR explica por qué no hay `fetch` dentro de efectos?
5. ¿Qué consecuencia negativa de ADR-014 se ha materializado ya en el proyecto?
6. ¿Por qué es importante registrar las alternativas descartadas?

CAPÍTULO 25

Notas de integración con el backend

↑ Volver al índice

EN ESTE CAPÍTULO

25.1 La frontera actual
25.2 Inventario priorizado
25.3 Las notas, una a una
25.4 Deudas internas del frontend
25.5 Procedimiento: cuando cambia el backend
25.6 Ejercicios
25.7 Autoevaluación

Escribir esta guía ha obligado a leer con lupa el frontend y, en modo de solo lectura, el backend. De esa lectura han salido desajustes, fragilidades y oportunidades de mejora en la frontera entre ambos. Este capítulo las reúne en un único inventario priorizado, con su causa y una propuesta para cada lado, y termina con un procedimiento para que el contrato no vuelva a desincronizarse.

25.1 La frontera actual

↑ Índice

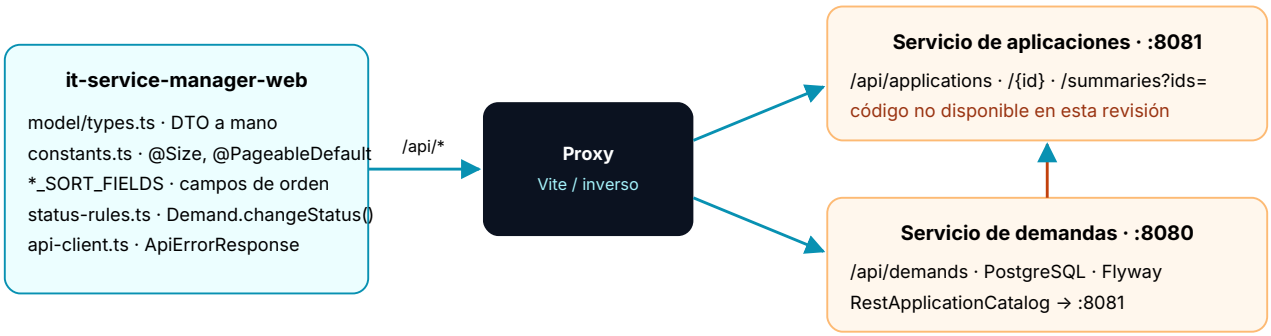


Figura 25.1. Los puntos del frontend que replican el contrato del backend (izquierda) son los que pueden desincronizarse.

> **Nota**
>
> **Alcance de la revisión.** Del backend se ha leído el repositorio `it-service-manager` (el servicio de demandas). El código del servicio de aplicaciones no estaba disponible, así que lo que se dice de él procede de cómo lo consume el frontend, de cómo lo consume el servicio de demandas (`RestApplicationCatalog`) y del análisis inicial del contrato. Ninguna nota ha requerido modificar el backend.

25.2 Inventario priorizado

↑ Índice

La prioridad combina el impacto para el usuario y el riesgo de que el problema pase desapercibido.

#	Nota	Prioridad	Lado que debe actuar	Estado
1	Campos de ordenación de demandas desalineados	**Alta**	Frontend (ya) y backend (decidir)	Abierta
2	El 409 de transición se interpreta leyendo un texto	Media	Backend (código y campos) y frontend	Abierta
3	Sin control de concurrencia en las modificaciones	Media	Backend y frontend	Abierta
4	Mensajes de error en idiomas mezclados	Media	Backend (códigos estables)	Abierta
5	Enumerados ordenados alfabéticamente	Media	Backend	Abierta
6	Sin búsqueda por nombre de aplicación	Media	Backend, después frontend	Abierta
7	Catálogo de desplegables limitado a 1000 elementos	Baja	Backend y frontend	Abierta
8	Contrato replicado a mano en el frontend	Media	Ambos (OpenAPI)	Propuesta
9	Errores no controlados con el formato por defecto de Spring	Baja	Backend	Parcialmente resuelta
10	`updatedAt` cambia en cada `PUT` aunque no haya cambios	Baja	Backend	Abierta
11	Indisponibilidad del catálogo de aplicaciones	Informativa	Frontend (opcional)	Gestionada
12	`.env.example` obsoleto	Baja	Frontend	Abierta
13	Despliegue: sin CORS ni proxy de producción definidos	Informativa	Infraestructura	Pendiente de despliegue
14	Consulta N+1 al resolver el nombre de la aplicación	—	Backend	**Resuelta**

25.3 Las notas, una a una

↑ Índice

25.3.1 1. Campos de ordenación de demandas desalineados

Síntoma. En el listado de demandas, ordenar por *Aplicación* o por *Creación* muestra un error: `400 Invalid demand sort field: application.name` (o `createdAt`). Una URL guardada con ese orden falla siempre.

Causa. El frontend ofrece `DEMAND_SORT_FIELDS = ['id', 'title', 'application.name', 'priority', 'status', 'createdAt', 'updatedAt']`, pero `DemandService` valida contra `VALID_SORT_FIELDS = ['id', 'title', 'priority', 'status', 'updatedAt']`. Al separar las aplicaciones en otro servicio, la entidad `Demand` dejó de tener una relación con `Application` (solo guarda `applicationId`), y ordenar por su nombre ya no es posible en la base de datos de demandas. La lista del frontend no se actualizó.

Propuesta. - *Frontend (inmediato):* alinear `DEMAND_SORT_FIELDS` con el backend y pintar esas columnas como no ordenables (ejercicio 22.2), con la prueba del ejercicio 19.2. - *Backend (decisión):* volver a admitir `createdAt`, que es una columna propia. Para ordenar por nombre de aplicación, o se guarda una copia del

nombre en la demanda (actualizada por eventos o al consultar), o se acepta como limitación del diseño distribuido.

25.3.2 2. El 409 de transición se interpreta leyendo un texto

Síntoma. Ninguno hoy. El riesgo es que, si alguien cambia el mensaje `Invalid demand status transition: X -> Y`, el frontend deje de reconocerlo y muestre un mensaje técnico en inglés.

Causa. `GlobalExceptionHandler` devuelve todos los `ResponseStatusException` con `error: 'RESOURCE_ERROR'` y el motivo en `message`. El frontend extrae los estados con una expresión regular (`parseTransitionError`).

Propuesta. Que el backend devuelva un código específico y los datos estructurados, por ejemplo:

```
{ "status": 409, "error": "INVALID_STATUS_TRANSITION", "message": "…",
  "details": { "from": "COMPLETED", "to": "CANCELLED" } }
```

El frontend comprobaría `error.code === 'INVALID_STATUS_TRANSITION'` y leería los estados de `details`, sin depender del texto.

25.3.3 3. Sin control de concurrencia en las modificaciones

Síntoma. Si dos usuarios editan la misma aplicación o demanda, gana el último en guardar sin que el primero se entere. Solo los cambios de estado de demandas tienen una protección parcial (el 409 de transición).

Propuesta. Concurrencia optimista: un campo `@Version` en las entidades, expuesto en los DTO de respuesta y exigido en los de modificación (o mediante cabeceras `ETag` / `If-Match`). Ante una versión distinta, el backend responde 409; el frontend ya tiene el patrón de alerta con *Recargar datos actuales* (capítulo 22) y solo habría que generalizarlo.

25.3.4 4. Mensajes de error en idiomas mezclados

Síntoma. El usuario ve mensajes en español («La petición contiene campos inválidos») junto a otros en inglés («Application not found: 99», «Application Catalog is unavailable», «Invalid value for parameter: id»), y los `fieldErrors` de Bean Validation dependen de la configuración regional del servidor.

Propuesta. Separar el **código** (estable, para las máquinas) del **mensaje** (para las personas). Con códigos estables (`APPLICATION_NOT_FOUND`, `CATALOG_UNAVAILABLE`…), el frontend puede traducir con un diccionario y usar el mensaje del servidor solo como respaldo. Mientras tanto, configurar los mensajes de validación en español en el backend (`ValidationMessages_es.properties` o mensajes explícitos en las anotaciones) unificaría al menos los errores por campo.

25.3.5 5. Enumerados ordenados alfabéticamente

Síntoma. Ordenar por criticidad, prioridad o estado da `CRITICAL, HIGH, LOW, MEDIUM` en lugar del orden lógico.

Causa. Se guardan como texto (`VARCHAR`), y la base de datos ordena alfabéticamente.

Propuesta. En el backend, ordenar con una expresión `CASE` cuando el campo de orden sea un enumerado, o persistir un valor ordinal adicional. El frontend no puede corregirlo sin descargar todos los datos.

25.3.6 6. Sin búsqueda por nombre de aplicación

Síntoma. *Modificar aplicación* obliga a buscar por identificador o a recorrer el catálogo; el listado no tiene buscador de texto.

Propuesta. Un parámetro opcional `name` (búsqueda parcial, sin distinguir mayúsculas) en `GET /api/applications`. El frontend lo incorporaría como en el ejercicio 19.1, con *debounce*.

25.3.7 7. Catálogo de desplegables limitado a 1000 elementos

Síntoma. Ninguno con el volumen actual. Con más de mil aplicaciones, el desplegable de demandas mostraría solo las primeras y un aviso.

Propuesta. Con la búsqueda por nombre (nota 6), sustituir el desplegable por un selector con búsqueda que consulte al backend mientras se escribe. El servicio de aplicaciones ya expone un endpoint ligero, `GET /api/applications/summaries?ids=…` (lo usa el servicio de demandas), que podría ampliarse con búsqueda por nombre para este fin.

25.3.8 8. Contrato replicado a mano en el frontend

Síntoma. Las notas 1 y 2 son consecuencia de la misma causa: el frontend replica a mano tipos, longitudes, listas de campos y reglas del backend.

Propuesta. Publicar la especificación OpenAPI de cada servicio con `springdoc-openapi` y generar los tipos del frontend con `openapi-typescript` (ADR-022). No resuelve las reglas de negocio (la máquina de estados), pero sí tipos, enumerados y parámetros. Para lo que siga siendo manual, el procedimiento de la última sección.

25.3.9 9. Errores no controlados con el formato por defecto de Spring

Estado. El `GlobalExceptionHandler` actual ya cubre los casos más frecuentes que antes llegaban con el formato por defecto: identificadores no numéricos (`MethodArgumentTypeMismatchException`) y JSON mal formado (`HttpMessageNotReadableException`). Siguen llegando con el formato por defecto los errores 500 no previstos y algunos casos concretos, como una respuesta 4xx del catálogo distinta de 404, que `RestApplicationCatalog` relanza sin traducir. El frontend los muestra con un mensaje genérico, así que el impacto es bajo.

Propuesta. Un manejador genérico de `Exception` que devuelva `ApiErrorResponse` con un 500 y un mensaje neutro (y registre el detalle en el log del servidor).

25.3.10 10. `updatedAt` cambia en cada `PUT`

Causa. `Demand.update()` actualiza la fecha aunque los valores no cambien.

Mitigación en el frontend. El botón *Guardar* está desactivado sin cambios (`isDirty`), así que un uso normal no la provoca. **Propuesta en el backend:** actualizar la fecha solo si algún campo cambia.

25.3.11 11. Indisponibilidad del catálogo de aplicaciones

Comportamiento actual. Si el servicio de aplicaciones no responde, el de demandas devuelve **503 «Application Catalog is unavailable»** al crear, modificar o consultar demandas (necesita resolver el nombre de la aplicación). El frontend lo muestra como error del servidor, con reintento en las consultas. Es coherente, pero significa que el módulo de demandas depende por completo del de aplicaciones.

Posible mejora (backend). Degradar con elegancia en las lecturas: devolver las demandas con `applicationName: null` si el catálogo no responde, y que el frontend muestre «Aplicación #7 (no disponible)». Es una decisión de diseño distribuido que merece su propio ADR.

25.3.12 12. `.env.example` obsoleto

Documenta `VITE_API_PROXY_TARGET`, que la configuración ya no usa; las variables reales son `VITE_DEMAND_API_TARGET` y `VITE_APPLICATION_API_TARGET` (ejercicio 4.2). Es un arreglo de dos líneas en el frontend.

25.3.13 13. Despliegue

No hay todavía configuración de producción. La recomendación (capítulo 16) es un proxy inverso o pasarela que sirva la SPA y reparta `/api/applications` y `/api/demands`, lo que evita configurar CORS en los servicios.

25.3.14 14. Consulta N+1 al resolver el nombre de la aplicación (resuelta)

El análisis inicial detectó que el listado de demandas hacía una consulta adicional por cada aplicación distinta. Tras la separación de servicios, `DemandService` reúne los identificadores de la página y llama **una sola vez** a `ApplicationCatalog.findByIds()`, que se traduce en una petición a `/api/applications/summaries?ids=…`. La documentación del backend (`docs/frontera-catalogo.md`) lo describe y lo cubre con pruebas. Se mantiene en el inventario como ejemplo de nota cerrada.

25.4 Deudas internas del frontend

↑ Índice

Además de la frontera con el backend, la revisión ha dejado algunas mejoras pendientes en el propio frontend, de menor importancia:

Deuda	Propuesta	Capítulo
Sin pruebas de componentes ni de extremo a extremo	Testing Library para componentes comunes; Playwright para los recorridos principales	23
Sin formateador automático	Prettier + plugin de clases de Tailwind	5
Modo oscuro preparado pero no definido	Tokens oscuros y conmutador	10
`ApplicationSummary` duplica `DetailField` y `LongText`	Usar los componentes comunes	12
Nombres distintos para las funciones de URL entre módulos	Unificar convención	14
Lectura de URL y `updateParams` repetidos en las páginas de listado	Hook `useUrlListState`	19
Sin aviso al abandonar un formulario con cambios	`useBlocker` y `beforeunload`	21

25.5 Procedimiento: cuando cambia el backend

↑ Índice

Mientras el contrato se replique a mano, este procedimiento evita la mayoría de los desajustes. Úsalo cada vez que cambie un controlador, un DTO, una validación o una regla de dominio:

1. **Leer el cambio en el backend**: controlador (rutas, parámetros, `@PageableDefault`), DTO (campos, tipos, `@NotBlank`, `@Size`), excepciones nuevas y su traducción en `GlobalExceptionHandler`, listas de validación como `VALID_SORT_FIELDS`, reglas de dominio.
2. **Localizar el reflejo en el frontend**: `model/types.ts` (busca el nombre del DTO en los comentarios), `model/constants.ts`, `*_SORT_FIELDS`, `model/schemas.ts`, `status-rules.ts`, los `-api.ts`.
3. **Actualizar primero las pruebas** de `model/` para que expresen el nuevo contrato y fallen.
4. **Actualizar el código** hasta que pasen, y ejecutar `npm run typecheck`: los `Record` exhaustivos y los tipos derivados señalarán todo lo que falta.
5. **Probar a mano** el recorrido afectado contra el backend real, con *Network* abierto.
6. **Registrar** el cambio: si es una decisión, un ADR; si es una limitación, una nota en este inventario.

Si vienes de Java...

Es la disciplina de los *consumer-driven contracts* (por ejemplo, con Spring Cloud Contract o Pact) aplicada a mano: el consumidor (el frontend) declara en sus pruebas qué espera del proveedor (el backend). El paso natural siguiente, cuando el proyecto crezca, es automatizarla con OpenAPI o con pruebas de contrato compartidas.

25.6 Ejercicios

↑ Índice

Ejercicio 25.1 · Prioriza con tu criterio Ver solución →

Reordena las notas 1 a 13 según tu criterio de jefe de proyecto, considerando esfuerzo e impacto. Agrúpalas en tres iteraciones de trabajo, indicando en cada una qué hace el equipo de frontend y qué el de backend, y qué dependencias hay entre ambas.

Ejercicio 25.2 · Diseña el contrato de errores Ver solución →

Diseña una versión 2 de `ApiErrorResponse` con códigos estables y detalles estructurados que resuelva las notas 2 y 4. Escribe el `record` de Java y el tipo de TypeScript equivalente, y adapta `toApiError` y `parseTransitionError` para que funcionen con ambos formatos (el antiguo y el nuevo) durante la transición.

Ejercicio 25.3 · Prueba el procedimiento Ver solución →

Elige un cambio pequeño del backend (por ejemplo, ampliar la longitud máxima del título de una demanda a 200 caracteres) y aplica el procedimiento completo de la última sección en una rama del frontend: lista los ficheros que tocarías, escribe primero la prueba que falla y después el cambio. ¿Cuántos ficheros has tenido que cambiar? ¿Lo habría detectado el compilador si te hubieras olvidado de alguno?

25.7 Autoevaluación

↑ Índice

Autoevaluación del capítulo 25

Ver respuestas →

1. ¿Qué cinco puntos del frontend replican el contrato del backend?
2. ¿Cuál es la causa de fondo del desajuste de ordenación de demandas?
3. ¿Por qué es frágil la interpretación actual del 409 y cómo se resolvería?
4. ¿Qué es la concurrencia optimista y qué parte del frontend ya está preparada para ella?
5. ¿Por qué conviene separar el código de error del mensaje?
6. ¿Qué ocurre hoy si el servicio de aplicaciones cae mientras se usa el módulo de demandas?
7. ¿Qué nota del análisis inicial está resuelta y cómo?
8. Resume los seis pasos del procedimiento a seguir cuando cambia el backend.

CAPÍTULO 26

Ejercicio guiado: el módulo de Incidencias

↑ Volver al índice

EN ESTE CAPÍTULO

26.1 El objetivo
26.2 Paso 0 · El contrato de la API
26.3 Paso 1 · Conectar el servicio
26.4 Paso 2 · `model/` : tipos, constantes y reglas
26.5 Paso 3 · `api/` : endpoints y caché
26.6 Paso 4 · `components/`
26.7 Paso 5 · `pages/` y `routes.ts`
26.8 Paso 6 · Integrar en la aplicación
26.9 Paso 7 · Verificación final
26.10 Retos adicionales
26.11 Ejercicios
26.12 Autoevaluación

Este último capítulo no explica nada nuevo: te propone construir tú el tercer módulo de la aplicación, Incidencias, aplicando todo lo anterior. Está organizado en pasos con puntos de control, empieza por un contrato de API que puedes implementar en tu proyecto de backend y deja el código de los ficheros más instructivos para que lo compares con el tuyo. Es el examen final de la guía.

26.1 El objetivo

↑ Índice

Al terminar, el menú lateral tendrá el módulo **Incidencias** activo (sin la etiqueta «Pronto») con las mismas cinco pantallas que los otros módulos: consulta con filtros, alta, búsqueda para modificar, detalle y modificación. Una incidencia:

- Pertenece a una aplicación (como las demandas).
- Tiene una **severidad** de cuatro niveles.
- Sigue un **ciclo de vida** con una particularidad nueva: se puede **reabrir**.
- Registra cuándo se creó, cuándo se modificó y **cuándo se resolvió**.

> **Cuidado**
>
> La norma del proyecto se mantiene: **nada de datos simulados en la aplicación**. El frontend de Incidencias solo funcionará cuando exista el endpoint en el backend. Por eso el paso 0 es el contrato: puedes implementarlo en tu proyecto de backend (Java 21, Spring Boot, JPA y PostgreSQL, como el servicio de demandas) y construir el frontend en paralelo, probando cada pantalla en cuanto el endpoint correspondiente esté disponible.

26.2 Paso 0 · El contrato de la API

↑ Índice

Propuesta de contrato, siguiendo las convenciones de los servicios existentes. Si al implementarlo en el backend cambias algo, adapta el frontend: el contrato manda.

Método y ruta	Cuerpo	Respuesta	Errores
`GET /api/incidents?applicationId&status&severity&page&size&sort`	—	`IncidentPageResponse`	400 si el filtro u orden no son válidos
`GET /api/incidents/{id}`	—	`IncidentResponse`	404
`POST /api/incidents`	`IncidentCreateRequest`	`IncidentResponse` (201)	400 con `fieldErrors`; 404 si la aplicación no existe; 503 si el catálogo no responde
`PUT /api/incidents/{id}`	`IncidentUpdateRequest`	`IncidentResponse`	400, 404, **409** si la transición no es válida, 503

DTO propuestos (backend)

```
public record IncidentCreateRequest(
        @NotBlank @Size(max = 150) String title,
        @Size(max = 2000) String description,
        @NotNull @Pattern(regexp = "LOW|MEDIUM|HIGH|CRITICAL") String severity,
        @NotNull Long applicationId) { }

public record IncidentUpdateRequest(
        @NotBlank @Size(max = 150) String title,
        @Size(max = 2000) String description,
        @NotNull @Pattern(regexp = "LOW|MEDIUM|HIGH|CRITICAL") String severity,
        @NotNull @Pattern(regexp = "OPEN|IN_PROGRESS|RESOLVED|CLOSED") String status,
        @NotNull Long applicationId) { }

public record IncidentResponse(
        Long id, String title, String description, String severity, String status,
        Long applicationId, String applicationName,
        Instant createdAt, Instant updatedAt, Instant resolvedAt) { }
```

Ciclo de vida propuesto. `OPEN → IN_PROGRESS | CLOSED`; `IN_PROGRESS → RESOLVED`; `RESOLVED → CLOSED | IN_PROGRESS` (reabrir); `CLOSED` es final. Al pasar a `RESOLVED` se fija `resolvedAt`; al reabrir, se borra. Paginación por defecto: `@PageableDefault(size = 10, sort = "id", direction = DESC)`. Campos de ordenación admitidos: `id`, `title`, `severity`, `status`, `createdAt`, `updatedAt`.

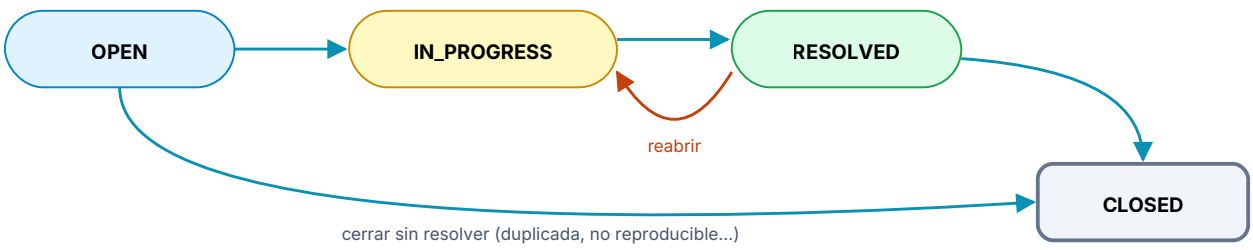


Figura 26.1. Ciclo de vida propuesto para una incidencia. A diferencia de las demandas, un estado no final puede volver atrás (reabrir).

Punto de control 0. Con el backend arrancado, `GET /api/incidents` responde una página vacía y `POST` crea una incidencia (pruébalo con la consola del navegador, como en el ejercicio 15.1, o con un cliente HTTP).

26.3 Paso 1 · Conectar el servicio

↑ Índice

Si las incidencias las sirve el servicio de demandas (8080), no hay que tocar nada más que una regla del proxy; si las sirve un servicio nuevo, usa su puerto y una variable propia.

vite.config.ts (regla añadida)

```
'/api/incidents': {
  target: env.VITE_INCIDENT_API_TARGET || 'http://localhost:8080',
  changeOrigin: true,
},
```

Documenta la variable en `.env.example` (y corrige de paso la nota 12 del capítulo 25). Reinicia `npm run dev`.

26.4 Paso 2 · `model/`: tipos, constantes y reglas

↑ Índice

Crea `src/features/incidents/model/`. Este es el `types.ts` completo, porque fija las bases de todo lo demás:

src/features/incidents/model/types.ts

```
import type { SortDirection } from '@/lib/list-params'

/**
 * Tipos del dominio "Incidencias", alineados con los DTO del backend
 * (IncidentResponse, IncidentCreateRequest, IncidentUpdateRequest).
 */
export const INCIDENT_STATUSES = ['OPEN', 'IN_PROGRESS', 'RESOLVED', 'CLOSED'] as const
export type IncidentStatus = (typeof INCIDENT_STATUSES)[number]

export const INCIDENT_SEVERITIES = ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL'] as const
export type IncidentSeverity = (typeof INCIDENT_SEVERITIES)[number]

/** IncidentResponse. Fechas: java.time.Instant en ISO-8601 (UTC). */
export interface Incident {
  id: number
  title: string
  description: string | null
  severity: IncidentSeverity
  status: IncidentStatus
  applicationId: number
  applicationName: string
  createdAt: string
  updatedAt: string
  resolvedAt: string | null
}

export interface IncidentCreateRequest {
  title: string
  description: string | null
  severity: IncidentSeverity
  applicationId: number
```

```
}

export interface IncidentUpdateRequest extends IncidentCreateRequest {
  status: IncidentStatus
}

export interface IncidentPage {
  content: Incident[]
  page: number
  size: number
  totalElements: number
  totalPages: number
}

/** Debe coincidir con los campos que valida el backend. */
export const INCIDENT_SORT_FIELDS = ['id', 'title', 'severity', 'status', 'createdAt', 'updatedAt']
as const
export type IncidentSortField = (typeof INCIDENT_SORT_FIELDS)[number]

export interface IncidentListParams {
  applicationId?: number
  status?: IncidentStatus
  severity?: IncidentSeverity
  page: number
  size: number
  sortField: IncidentSortField
  sortDirection: SortDirection
}
```

Después, por analogía con demandas:

- **`constants.ts`**: etiquetas (`OPEN: 'Abierta'`, `IN_PROGRESS: 'En curso'`, `RESOLVED: 'Resuelta'`, `CLOSED: 'Cerrada'`; severidades *Baja*, *Media*, *Alta*, *Crítica*), `DEFAULT_INCIDENT_LIST_PARAMS` alineado con `@PageableDefault`, `TITLE_MAX_LENGTH = 150` y `DESCRIPTION_MAX_LENGTH = 2000`.
- **`status-rules.ts`**: la tabla de transiciones de la figura 26.1 y las mismas funciones que en demandas.

`src/features/incidents/model/status-rules.ts (núcleo)`

```
const NEXT_STATUSES: Record<IncidentStatus, readonly IncidentStatus[]> = {
  OPEN: ['IN_PROGRESS', 'CLOSED'],
  IN_PROGRESS: ['RESOLVED'],
  RESOLVED: ['CLOSED', 'IN_PROGRESS'],
  CLOSED: [],
}
```

- **`list-params.ts`**: `parseIncidentListParams`, `serializeIncidentListParams` y `toIncidentApiQuery`, con los parámetros de URL `aplicacion`, `estado` y `severidad`. Elige una convención de nombres y anótala (capítulo 14).
- **`schemas.ts`**: `incidentCreateSchema` y `createIncidentUpdateSchema(currentStatus)`, con `applicationId` de texto a número y el `superRefine` de transiciones.

> **En el proyecto**
>
> Fíjate en lo que **no** hay que escribir: la validación genérica de la URL (`lib/list-params.ts`), los campos de formulario, los indicadores, la paginación, los estados de carga y error, el cliente HTTP. Todo eso es común (ADR-008). El módulo nuevo solo aporta su dominio.

Punto de control 2. Escribe las pruebas de `model/` antes de seguir: transiciones (incluida la reapertura y que `CLOSED` es final), límites de longitud, reversibilidad de la URL y que `?orden=application.name,asc` se descarta. `npm run test` y `npm run typecheck` en verde.

26.5 Paso 3 · `api/`: endpoints y caché

↑ Índice

`src/features/incidents/api/incidents-queries.ts (claves)`

```ts
export const incidentKeys = {
  all: ['incidents'] as const,
  lists: () => [...incidentKeys.all, 'list'] as const,
  list: (params: IncidentListParams) => [...incidentKeys.lists(), params] as const,
  details: () => [...incidentKeys.all, 'detail'] as const,
  detail: (id: number) => [...incidentKeys.details(), id] as const,
}
```

`incidents-api.ts` y los hooks `useIncidents`, `useIncident`, `useCreateIncident` y `useUpdateIncident` son copias adaptadas de los de demandas: `keepPreviousData` en el listado, sin reintentos para el 404 en el detalle, `setQueryData` + invalidación de listados en las mutaciones.

> **Nota**
>
> Piensa en las relaciones de caché entre módulos. Si en el futuro el detalle de una aplicación muestra un contador de incidencias abiertas, crear una incidencia debería invalidar también esa consulta. Hoy no es necesario, pero es el tipo de dependencia que conviene anotar al diseñar las claves.

26.6 Paso 4 · `components/`

↑ Índice

Componente	Base	Particularidad
`incident-badges.tsx`	`StatusPill`, `LevelBadge`	Tonos: `OPEN` → `info`, `IN_PROGRESS` → `warning`, `RESOLVED` → `success`, `CLOSED` → `neutral`; severidad → niveles 1 a 4
`incident-filters.tsx`	`FilterSelect` + `useApplicationSelectOptions`	Mismo patrón que los filtros de demandas
`incident-form-fields.tsx`	Campos genéricos	`ApplicationField` puede reutilizar la misma lógica que demandas: valora si moverlo a un sitio común
`incidents-table.tsx`	`Table`, `SortableTableHead`	Columna de severidad y fecha de creación
`incident-summary.tsx`	`DetailField`, `LongText`, `formatDateTime`	Muestra `resolvedAt` solo si existe
`incident-status-flow.tsx`	Como `StatusFlow`	Debe dibujar la reapertura

Decisión de diseño: ¿Dónde vive el campo de aplicación?

`ApplicationField` y `useApplicationSelectOptions` están hoy en `features/demands/components`. Si incidencias los importa de demandas, se crea una dependencia entre dos módulos que no tienen relación de dominio entre sí. Las dos opciones razonables son moverlos a `features/applications` como parte de su API para otros módulos (junto a `useApplicationOptions`), o a `components/common` si se generalizan. La primera respeta mejor el dominio: el catálogo de aplicaciones es cosa del módulo de aplicaciones. Es una buena ocasión para escribir el ADR correspondiente.

26.7 Paso 5 · `pages/` y `routes.ts`

↑ Índice

Crea las cinco páginas partiendo de las de demandas, con estas diferencias:

- **Listado** (`/incidencias`): filtros de aplicación, estado y severidad; *Nueva incidencia* como acción principal.
- **Alta** (`/incidencias/nueva`): admite `?aplicacion=ID` para preseleccionar; mensaje específico si la aplicación ya no existe (404).
- **Detalle** (`/incidencias/:id`): ficha, ciclo de vida y, si está resuelta, cuánto tardó en resolverse (diferencia entre `createdAt` y `resolvedAt`).
- **Modificación** (`/incidencias/:id/editar`): `key` por identificador, *resolver* memorizado por estado actual, tratamiento del 409 con *Recargar datos actuales*.
- **Selector** (`/incidencias/modificar`): búsqueda por ID y selección del catálogo.

`src/features/incidents/routes.ts (inicio)`

```
export const incidentRoutes: RouteObject = {
  path: 'incidencias',
  children: [
    {
      index: true,
      handle: { title: 'Consultar incidencias' },
      lazy: () => import('./pages/incidents-list-page').then((m) => ({ Component: m.IncidentsListPag
e })),
    },
    // nueva, modificar, :id, :id/editar
  ],
}
```

26.8 Paso 6 · Integrar en la aplicación

↑ Índice

1. **Router**: añade `incidentRoutes` en `app/router.tsx`, antes de la ruta comodín.
2. **Menú**: en `navigation.ts`, sustituye la entrada `comingSoon` de Incidencias por un módulo con `basePath: '/incidencias'` y sus tres acciones.
3. **Enlaces cruzados**: en el detalle de una aplicación, un botón *Ver incidencias* que lleve a `/incidencias?aplicacion=ID`.

Punto de control 6. Recorrido completo contra el backend real: crear una incidencia desde el detalle de una aplicación, verla en el listado filtrado, pasarla a *En curso*, a *Resuelta* (comprueba que aparece la fecha de resolución), reabrirla y cerrarla. Provoca un 409 con dos pestañas, como en el ejercicio 22.1.

26.9 Paso 7 · Verificación final

↑ Índice

Comprobación	Comando o acción
Tipos	`npm run typecheck`
Estilo y reglas de *hooks*	`npm run lint`
Pruebas	`npm run test` (las de `model/` nuevas incluidas)
Build de producción	`npm run build`; comprueba que hay *chunks* de las páginas de incidencias
Estados de cada pantalla	Carga, error (detén el backend), vacío (filtros sin resultados), datos
Accesibilidad	Recorrido completo con teclado; etiquetas de los botones de icono
Móvil	Modo de dispositivo de las DevTools; menú lateral en el panel deslizante
Documentación	ADR de las decisiones nuevas; notas de integración si aparece alguna

26.10 Retos adicionales

↑ Índice

Cuando el módulo funcione, estas ampliaciones ponen a prueba la arquitectura:

- **SLA por severidad.** Un plazo objetivo de resolución por severidad (por ejemplo, 4 horas para *Crítica*) y un indicador en el listado: en plazo, en riesgo, incumplido. ¿Lo calcula el backend o el frontend? ¿Qué pasa con la zona horaria y los fines de semana?
- **Equipos.** Asignar cada incidencia a un equipo, con su propio módulo (`features/teams`) y el patrón del catálogo para el desplegable.
- **Panel de control.** Una página de inicio con indicadores: incidencias abiertas por severidad, demandas por estado, aplicaciones críticas. ¿Qué endpoints de agregación harían falta para no descargar todos los datos?

26.11 Ejercicios

↑ Índice

Ejercicio 26.1 · Pruebas de las reglas de incidencias Ver solución →

Escribe `src/features/incidents/model/status-rules.test.ts` con al menos cinco pruebas: transiciones permitidas, transiciones prohibidas (incluida `IN_PROGRESS → CLOSED`), la reapertura, que `CLOSED` es el único estado final y los estados seleccionables desde `RESOLVED`.

Ejercicio 26.2 · Duración de la resolución Ver solución →

Escribe en `src/lib/format.ts` una función `formatDuration(fromIso: string, toIso: string): string` que devuelva textos como «3 h 25 min» o «2 d 4 h», con sus pruebas. Úsala en el detalle de una incidencia resuelta. ¿Por qué esta función no depende de la zona horaria, a diferencia de `formatDateTime`?

Ejercicio 26.3 · Mueve el catálogo de aplicaciones Ver solución →

Haz la refactorización propuesta en la decisión del paso 4: mueve `useApplicationSelectOptions` y `ApplicationField` (o su lógica) al módulo de aplicaciones como API para otros módulos, actualiza demandas para usarla y comprueba con `npm run typecheck`, `npm run lint` y `npm run test` que nada se ha roto. Actualiza la regla de ESLint del ejercicio 13.2 si la añadiste.

26.12 Autoevaluación

↑ Índice

Autoevaluación del capítulo 26 Ver respuestas →

1. ¿Por qué el primer paso es el contrato de la API y no el código del frontend?
2. ¿Qué diferencia el ciclo de vida de una incidencia del de una demanda y cómo se refleja en `status-rules.ts`?
3. ¿Qué partes del módulo no hay que escribir porque ya son comunes?
4. ¿Por qué no conviene que incidencias importe el campo de aplicación desde demandas?
5. ¿Qué tres cambios fuera de `features/incidents` hacen falta para integrar el módulo?
6. ¿Qué comprobaciones forman la verificación final?

ANEXO A

Soluciones y respuestas

↑ Volver al índice

Cada solución enlaza de vuelta con su ejercicio, y cada bloque de respuestas con las preguntas de su capítulo. Intenta resolverlos antes de mirar: el valor del ejercicio está en el intento.

A.1 Capítulo 1 · Cómo funciona una aplicación web moderna

↑ Índice

Solución 1.1 · Observa el arranque en la pestaña Network

← Volver al ejercicio

1. La primera petición, de tipo *document*, es `GET /aplicaciones`. Vite devuelve el `index.html` (aunque no exista ningún fichero con ese nombre) con una pequeña inyección de su cliente de recarga en caliente.
2. A continuación aparecen `main.tsx`, `index.css`, `app.tsx`, `router.tsx`… En desarrollo Vite **no empaqueta**: sirve cada módulo por separado, ya transformado a JavaScript, y el navegador pide cada `import` (capítulo 4). Verás también módulos de `node_modules` precompilados, en rutas como `/node_modules/.vite/deps/`.
3. `GET /api/applications?page=0&size=10&sort=id,asc` con estado `200` y un JSON con la forma `{ content: [...], page, size, totalElements, totalPages }`.

Al navegar a *Demandas* **no** se vuelve a pedir `index.html`: el router cambia la URL con la History API. Solo aparecen el módulo de la página de demandas (carga diferida) y las peticiones `GET /api/demands…` y, para el filtro, `GET /api/applications…`.

Solución 1.2 · Provoca y diagnostica errores

← Volver al ejercicio

1. Se muestra el estado de error del listado («No se han podido cargar los datos») con el botón *Reintentar*. En *Network*, la petición a `/api/applications` termina con un **500 generado por el propio proxy de Vite**, que no consigue conectar con el backend; en la terminal de `npm run dev` verás un aviso `http proxy error … ECONNREFUSED`. Nuestro cliente HTTP convierte la respuesta en un `ApiError` de tipo `server`. Observa también que la petición aparece **tres veces**: la configuración de TanStack Query reintenta dos veces los errores que no son 4xx (capítulo 17).
2. La genera **React**. En *Network* la petición del documento devuelve `200` con el mismo `index.html` de siempre; después el router no encuentra ninguna ruta que encaje y pinta `NotFoundPage` gracias a la ruta comodín `{ path: '*' }` de `router.tsx`.

Solución 1.3 · Mide el efecto de la división del código

← Volver al ejercicio

En `dist/assets` encontrarás un `index-<hash>.js` (el *bundle* inicial: React, el router, el layout…), un `index-<hash>.css` y varios ficheros con nombres como `applications-list-page-<hash>.js` o `application-edit-page-<hash>.js`. Cada página con `lazy()` produce su propio *chunk*; el código compartido entre varias páginas se extrae a *chunks* comunes.

Con `npm run preview` (que sirve `dist/`) y la aplicación abierta en `/demandas`, el *chunk* `applications-list-page-…js` no aparece en *Network* hasta que pulsas *Consultar aplicaciones* en el menú.

Respuestas · capítulo 1 ← Volver a las preguntas

1. La primera vez, un `index.html` casi vacío junto con el JavaScript y el CSS de la aplicación. Después, solo datos JSON a través de la API.
2. Porque el frontend se ejecuta en una máquina que controla el usuario: puede modificarse o saltarse llamando a la API directamente. La validación del frontend mejora la experiencia de uso; la autoridad es el backend.
3. Es el árbol de objetos que el navegador construye a partir del HTML. Lo que se ve en pantalla es siempre el reflejo del DOM actual; si JavaScript lo modifica, la pantalla cambia.
4. Delegando la operación lenta al navegador y registrando una función que se ejecutará cuando termine. El bucle de eventos ejecuta esa función cuando el hilo queda libre.
5. Crea una raíz de React ligada a `div#root` y le indica que dibuje el componente `App` dentro. Desde ese momento React gestiona todo el contenido de ese elemento.
6. Porque al recargar o abrir un enlace directo, el navegador pide al servidor una ruta como `/demandas/7` que no existe como fichero. El servidor debe devolver `index.html` para que el router de React resuelva la ruta en el navegador.
7. El primero es una importación estática: se resuelve al construir y el módulo forma parte del mismo *bundle*. El segundo es una importación dinámica: devuelve una promesa, se resuelve en ejecución y genera un *chunk* aparte que se descarga bajo demanda.
8. Para poder guardar los ficheros en la caché del navegador de forma indefinida: si el contenido cambia, cambia el nombre, de modo que nunca se sirve una versión antigua por error.

A.2 Capítulo 2 · JavaScript moderno y TypeScript

↑ Índice

Solución 2.1 · Añade un estado a la enumeración ← Volver al ejercicio

Dejan de compilar los objetos `Record<ApplicationStatus, …>` que no tienen la nueva clave: como mínimo `STATUS_LABELS` en `model/constants.ts` y `STATUS_TONES`, que asigna un tono de color a cada estado en `components/application-badges.tsx`. El error es del tipo *Property 'RETIRED' is missing in type … but required in type 'Record'*. Es exactamente la exhaustividad de la sección 2.5.

Una vez añadidas la etiqueta y el tono, el nuevo estado aparece **sin tocar nada más** en: el filtro de estado del listado y del selector (que se construyen con `APPLICATION_STATUSES.map`), el desplegable de estado del formulario de modificación, la validación del parámetro `?estado=` de la URL (`parseEnumParam`) y el esquema de Zod (`z.enum(APPLICATION_STATUSES)`). Todo deriva de la misma constante.

Solución 2.2 · Escribe una función genérica ← Volver al ejercicio

```ts
export function groupBy<T, K extends string>(
  items: readonly T[],
  key: (item: T) => K,
): Partial<Record<K, T[]>> {
  const result: Partial<Record<K, T[]>> = {}
  for (const item of items) {
    const k = key(item)
    ;(result[k] ??= []).push(item)
  }
  return result
}
```

```
const porCriticidad = groupBy(page.content, (app) => app.criticality)
// tipo inferido: Partial<Record<ApplicationCriticality, Application[]>>
```

La firma pedida devolvía `Record<K, T[]>`, pero es más honesto devolver `Partial<Record<K, T[]>>`: si ninguna aplicación es `CRITICAL`, esa clave no existirá, y el tipo debe obligar a quien lo use a contemplarlo (`porCriticidad.CRITICAL ?? []`). Este tipo de matiz es justo lo que el modo estricto te ayuda a ver. TypeScript infiere `T = Application` y `K = ApplicationCriticality` a partir de los argumentos. El operador `??=` asigna solo si el valor es `null` o `undefined`; el punto y coma inicial evita que la línea se interprete como continuación de la anterior.

Solución 2.3 · Estrechamiento con `unknown`

← Volver al ejercicio

```
import { isApiError } from '@/lib/api-client'

export function describeError(error: unknown): string {
  if (isApiError(error)) {
    if (error.kind === 'network') return 'Error de red'
    const campos = Object.keys(error.fieldErrors)
    if (campos.length > 0) return `Datos no válidos: ${campos.join(', ')}`
    return error.message
  }
  if (error instanceof Error) return error.message
  return 'Error desconocido'
}
```

Si escribes `error.kind` antes del `if (isApiError(error))`, el compilador responde *'error' is of type 'unknown'*. Dentro del `if`, la guarda de tipo ha estrechado `error` a `ApiError`. Ten en cuenta que el orden importa: `ApiError` también es un `Error`, así que la comprobación más específica debe ir primero.

Respuestas · capítulo 2

← Volver a las preguntas

1. Nada: se eliminan al compilar. El navegador ejecuta JavaScript sin ninguna información de tipos.
2. `undefined` significa «no asignado» y `null`, «vacío asignado explícitamente». El backend envía `null` en los campos sin valor (`description: null`); el frontend usa `undefined` (propiedades opcionales) para «sin filtro».
3. `??` solo sustituye `null` y `undefined`; `||` sustituye cualquier valor falso. Con `const page = 0`, `page ?? 1` vale `0` y `page || 1` vale `1`.
4. Que un valor es compatible con un tipo si tiene la forma requerida, sin declararlo. No hace falta `implements`, y basta con objetos literales donde en Java usarías clases DTO.
5. `as const` hace que el array tenga tipo de tupla de solo lectura con los literales exactos, en lugar de `string[]`. `(typeof X)[number]` obtiene la unión de los tipos de sus elementos: `'ACTIVE' | 'INACTIVE'`. Así se declara la lista una vez y se deriva el tipo.
6. Porque exige una entrada para cada valor de la unión: si se añade un valor nuevo y falta su entrada, el proyecto no compila.
7. Porque `fetch` solo rechaza la promesa ante errores de red. Una respuesta 4xx o 5xx es una respuesta válida para `fetch`, con `ok` a `false`.
8. `unknown` obliga a comprobar el tipo antes de usar el valor; `any` desactiva la comprobación y se contagia a todo lo que toca.
9. Nada en ejecución: solo le dice al compilador que trate el resultado como `T`. Si el JSON no tiene esa forma, el error aparecerá más adelante, en ejecución.

A.3 Capítulo 3 · Node.js, npm y el ecosistema de paquetes

↑ Índice

Solución 3.1 · Lee el árbol de dependencias

← Volver al ejercicio

`npm ls react` muestra `react@19.3.0` en la raíz y, debajo, todos los paquetes que lo declaran como dependencia (`@tanstack/react-query`, `lucide-react`, `react-hook-form`, `sonner`, cada subpaquete de `radix-ui` …) marcados como `deduped`: todos comparten **la misma copia**. Es importante, porque dos copias de React en un mismo *bundle* producen errores muy confusos con los hooks.

`npm explain @radix-ui/react-select` responde:

```
@radix-ui/react-select@2.3.7
  @radix-ui/react-select@"2.3.7" from radix-ui@1.6.7
    radix-ui@"^1.6.7" from the root project
```

Lo introduce el paquete unificado `radix-ui`, que agrupa todos los primitivos de Radix. El proyecto solo declara `radix-ui`, y es `components/ui/select.tsx` quien usa la parte del desplegable.

Solución 3.2 · Comprueba qué está desactualizado

← Volver al ejercicio

La clasificación depende del momento en que lo ejecutes, pero el criterio es: si la columna *Wanted* es mayor que *Current*, basta `npm update`; si *Latest* es mayor que *Wanted* y comparte versión mayor, el rango del `package.json` lo impide (raro con `^`); si *Latest* tiene otra versión mayor, es un salto mayor que requiere decisión.

Para Zod 3 → 4, los cambios que afectan al proyecto están en los esquemas de los formularios: la opción `errorMap` de `z.enum(…, { errorMap })` se sustituye por la opción `error`. Habría que revisar `features/applications/model/schemas.ts` y `features/demands/model/schemas.ts`, ejecutar sus pruebas (`schemas.test.ts`) y comprobar `@hookform/resolvers`, cuyo `zodResolver` admite ambas versiones en sus versiones recientes.

Solución 3.3 · Rompe el build a propósito

← Volver al ejercicio

`npm run dev` arranca sin quejarse y la aplicación funciona: Vite elimina los tipos sin comprobarlos (sección 4.4). Solo el editor subraya el error. `npm run build`, en cambio, falla en el primer paso, `tsc -b`, con un error del tipo *Type '"ACTIVA"' is not assignable to type 'ApplicationStatus'*, y por culpa del `&&` nunca llega a ejecutar `vite build`: no se genera `dist/`.

Respuestas · capítulo 3

← Volver a las preguntas

1. No. Node solo ejecuta las herramientas de desarrollo y construcción (Vite, TypeScript, ESLint, Vitest). En producción se sirven ficheros estáticos.
2. Si el código acaba en el *bundle* que descarga el usuario (`dependencies`) o solo se usa al desarrollar (`devDependencies`).
3. `^5.103.2` admite cualquier `5.x.y` igual o superior a `5.103.2`. `~5.103.2` admite solo `5.103.x` igual o superior a `5.103.2`.
4. Fija la versión exacta, el origen y la suma de verificación de cada paquete, incluidos los transitivos. Se versiona para que todo el equipo y la integración continua instalen exactamente lo mismo.
5. En integración continua o cuando quieres una instalación limpia y reproducible: `npm ci` instala exactamente el *lockfile* y falla si no coincide con `package.json`.

6. Porque es grande, depende del sistema operativo en algunos paquetes y se regenera de forma determinista con `npm install` a partir del *lockfile*.
7. Que los ficheros `.js` del proyecto son módulos ES (`import` / `export`) y no CommonJS.
8. `tsc -b` falla y, por el `&&`, `vite build` no se ejecuta: no se genera una versión de producción con errores de tipos.

A.4 Capítulo 4 · Vite: el entorno de desarrollo y construcción

↑ Índice

Solución 4.1 · Observa HMR y Fast Refresh

← Volver al ejercicio

En el primer caso el texto escrito se conserva: Fast Refresh sustituye el componente y mantiene su estado (incluido el del formulario). Al añadir `export const PRUEBA = 1`, el fichero deja de exportar solo componentes: Vite no puede aplicar Fast Refresh con seguridad y hace una **recarga completa**, de modo que se pierde lo escrito. ESLint marca la línea con *Fast refresh only works when a file only exports components*. Es la razón de que las constantes vivan en ficheros `.ts` separados.

Solución 4.2 · Actualiza `.env.example`

← Volver al ejercicio

`.env.example`

```
# URL base de la API usada por el navegador. Por defecto /api (proxy de Vite en desarrollo).
VITE_API_BASE_URL=/api
# Destino del proxy para /api/applications (servicio de aplicaciones).
VITE_APPLICATION_API_TARGET=http://localhost:8081
# Destino del proxy para /api/demands (servicio de demandas).
VITE_DEMAND_API_TARGET=http://localhost:8080
```

Con `VITE_APPLICATION_API_TARGET=http://localhost:9999` en `.env.local` (y Vite reiniciado), el listado de aplicaciones muestra el estado de error, la terminal registra `http proxy error: /api/applications?… ECONNREFUSED` y en *Network* las peticiones a `/api/applications` responden con un 500 del proxy. El módulo de demandas sigue funcionando, salvo su filtro por aplicación, que también consulta `/api/applications`.

Solución 4.3 · Inspecciona el resultado del build

← Volver al ejercicio

El *chunk* inicial `index-<hash>.js` suele ser el mayor: contiene React, React DOM, React Router, TanStack Query, el layout y los componentes comunes. En `dist/index.html` aparece como `<script type="module" crossorigin src="/assets/index-<hash>.js">`, junto a un `<link rel="stylesheet">` para el CSS. Al buscar `/api` en ese fichero encontrarás la cadena `"/api"` del cliente HTTP: como no hay `VITE_API_BASE_URL` definida, `import.meta.env.VITE_API_BASE_URL` se ha sustituido por `undefined` y queda el valor por defecto. Es probable que el minificador haya simplificado la expresión `undefined ?? "/api"` hasta dejar solo la cadena: el código de producción no se parece al fuente, y por eso existen los *source maps*.

Respuestas · capítulo 4

← Volver a las preguntas

1. En desarrollo, el navegador entiende módulos ES y transformar ficheros sueltos bajo demanda es instantáneo. En producción, cientos de peticiones serían lentas, y conviene optimizar, minificar y dividir en *chunks*.

2. Vite empaqueta cada dependencia de `node_modules` en un único módulo ES (convirtiendo CommonJS si hace falta) y lo guarda en `node_modules/.vite/deps`.
3. HMR es el mecanismo general de sustituir un módulo en caliente; Fast Refresh es su integración con React, que vuelve a renderizar el componente modificado conservando su estado.
4. Porque Vite elimina los tipos sin comprobarlos. En producción lo impide el `tsc -b` que precede a `vite build` en el script `build`.
5. Porque Vite resuelve los ficheros al construir y TypeScript comprueba los tipos, y cada herramienta tiene su propia configuración.
6. Solo las que empiezan por `VITE_`. Se incrustan como texto en el JavaScript que descarga cualquier usuario.
7. Evita las restricciones de origen cruzado (CORS) haciendo que el navegador hable solo con Vite, que reenvía al backend. En producción no hay servidor de Vite: hace falta un proxy inverso o CORS en el backend.
8. Al servicio de demandas: `GET http://localhost:8080/api/demands/7` (o al destino de `VITE_DEMAND_API_TARGET`).

A.5 Capítulo 5 · Configuración de TypeScript, ESLint y calidad del código

↑ Índice

Solución 5.1 · Desactiva el modo estricto ← Volver al ejercicio

Con `strict: false` desaparecen errores, pero no porque el código sea mejor: el compilador deja de comprobar los nulos. Con el modo estricto, escribir `query.data.name` sin comprobación produce *'query.data' is possibly 'undefined'*. Sin modo estricto compila, y en ejecución, durante la carga (cuando `data` aún es `undefined`), React lanzaría *TypeError: Cannot read properties of undefined (reading 'name')* y la página se sustituiría por `RouteErrorPage`.

Solución 5.2 · Provoca errores de ESLint ← Volver al ejercicio

`npm run lint` informa de *React Hook "useState" is called conditionally. React Hooks must be called in the exact same order in every component render* (`react-hooks/rules-of-hooks`, error) y de *React Hook useEffect has a missing dependency: '...'. Either include it or remove the dependency array* (`react-hooks/exhaustive-deps`, aviso).

La primera regla existe porque React identifica cada hook por su orden de llamada: si un hook se llama en unos renderizados y en otros no, los siguientes reciben el estado de otro (capítulo 8, figura 8.1).

Solución 5.3 · Comprueba la separación de entornos ← Volver al ejercicio

`document.title = 'x'` en `vite.config.ts` falla en el subproyecto **node** (*Cannot find name 'document'*), porque `tsconfig.node.json` solo incluye `lib: ["ES2023"]`, sin DOM. `process.env.HOME` en `main.tsx` falla en el subproyecto **app** (*Cannot find name 'process'*), porque `tsconfig.app.json` limita los tipos globales a `vite/client`, sin los de Node. Cada línea habría fallado en ejecución en su entorno.

Respuestas · capítulo 5 ← Volver a las preguntas

1. Porque hay código para dos entornos distintos (navegador y Node) con APIs globales diferentes, y cada uno necesita sus propias bibliotecas de tipos.

2. Vite (con Oxc) en desarrollo y Rolldown en el *build*. TypeScript solo comprueba.
3. Usar `import type` para las importaciones que solo traen tipos. Permite a Vite eliminar esas importaciones procesando cada fichero de forma aislada.
4. `strictNullChecks`: impide usar valores que pueden ser `null` o `undefined` sin comprobarlos.
5. Patrones válidos pero problemáticos: un hook llamado dentro de un `if`, una dependencia olvidada en un efecto, una variable que nunca se reasigna declarada con `let`, código inalcanzable, `any` explícito.
6. Porque los componentes generados por shadcn/ui exportan sus variantes junto al componente, lo que dispara la regla de Fast Refresh, y no conviene reescribir ese código.
7. Recorre las referencias de proyecto, comprueba cada subproyecto en orden y guarda información incremental para acelerar las siguientes ejecuciones.

A.6 Capítulo 6 · Componentes, JSX y props

↑ Índice

Solución 6.1 · Explora el árbol de componentes

← Volver al ejercicio

La cadena aproximada es `App → QueryClientProvider → TooltipProvider → RouterProvider → … → AppLayout → Outlet → … → ApplicationsListPage → Card → ApplicationsTable → Table → TableHeader → TableRow → SortableTableHead`. Entre medias verás componentes internos de React Router y de Radix. Las props de `ApplicationsTable` son `applications` (array de 10 elementos por defecto), `sortField`, `sortDirection`, `onSortChange`, `isFetching`. Al pulsar la cabecera *Nombre* cambian `sortField` (`'name'`), `sortDirection` y, al llegar los datos, `applications`. `onSortChange` también cambia de referencia, porque `handleSortChange` se crea en cada renderizado de la página.

Solución 6.2 · Crea un componente reutilizable

← Volver al ejercicio

`src/components/common/count-badge.tsx`

```tsx
import { cn } from '@/lib/utils'

interface CountBadgeProps {
  count: number
  /** Nombre en singular y plural, p. ej. { one: 'aplicación', other: 'aplicaciones' }. */
  label: { one: string; other: string }
  tone?: 'neutral' | 'brand'
}

export function CountBadge({ count, label, tone = 'neutral' }: CountBadgeProps) {
  const text = count === 0 ? `Sin ${label.other}` : `${count} ${count === 1 ? label.one : label.other}`
  return (
    <span className={cn(
      'inline-flex rounded-full px-2.5 py-0.5 text-xs font-medium',
      tone === 'brand' ? 'bg-cyan-50 text-cyan-800' : 'bg-slate-100 text-slate-700',
    )}>
      {text}
    </span>
  )
}
```

Con una única cadena `label` no se puede formar bien el plural en español, así que la solución cambia la prop a un objeto con ambas formas: diseñar bien las props es parte del ejercicio. En la página: `descrip-`

`tion={data ? <CountBadge count={data.totalElements} label={{ one: 'aplicación', other: 'aplicaciones' }} /> : '…'}`. Funciona porque `description` es de tipo `ReactNode`.

Solución 6.3 · Lee un componente con criterio ← Volver al ejercicio

Datos: `page`, `size`, `totalElements`, `totalPages`, `pageSizeOptions`, `disabled`. Eventos: `onPageChange`, `onSizeChange`. El componente no guarda la página porque **no es su dueño**: la página actual vive en la URL y la gestiona `ApplicationsListPage`; la barra solo la muestra y avisa de los cambios (flujo unidireccional). Con `disabled`, todos los botones y el selector se desactivan, lo que evita pedir otra página mientras se carga una. La página se muestra como `page + 1` porque Spring Data numera las páginas desde 0 y los usuarios, desde 1.

Respuestas · capítulo 6 ← Volver a las preguntas

1. Que el componente describe cómo debe verse la interfaz para unos datos, y React calcula los cambios del DOM. Por ejemplo, el listado describe qué mostrar en cada estado de la consulta en lugar de mostrar u ocultar elementos a mano.
2. En llamadas a `jsx(tipo, props)` que devuelven elementos de React: objetos ligeros que describen la interfaz. Un componente devuelve un árbol de esos objetos.
3. Porque JSX interpreta los nombres en minúscula como etiquetas HTML y los que empiezan por mayúscula como componentes.
4. Llama a una función que ha recibido por props (por ejemplo, `onSortChange`) para que el padre, dueño del dato, lo cambie.
5. `children` es el contenido entre las etiquetas del componente. Las props que reciben interfaz se tipan como `ReactNode`.
6. Cargando, error sin datos, vacío y con datos. Para que el usuario siempre sepa qué está pasando, incluso cuando algo falla.
7. Para identificar cada elemento entre renderizados y conservar o mover correctamente su DOM y su estado. El índice describe la posición, no la identidad, y falla al reordenar, filtrar o insertar.
8. La primera pasa la función para que se ejecute al hacer clic; la segunda la ejecuta durante el renderizado y pasa su resultado.
9. Porque si `count` es 0, la expresión vale `0`, y React pinta los números (aunque sean falsos).

A.7 Capítulo 7 · Estado, renderizado y reconciliación

↑ Índice

Solución 7.1 · Observa la instantánea del estado ← Volver al ejercicio

Al buscar, la consola muestra primero `antes undefined` (o el identificador de la búsqueda anterior): dentro del manejador, `searchedId` es el valor de la instantánea actual; `setSearchedId` solo programa el siguiente renderizado. Después aparecen dos líneas `render 42` (y más, cuando la consulta cambia de estado). El doble `render` se debe a `StrictMode`, que en desarrollo ejecuta cada renderizado dos veces para detectar componentes impuros (sección 8.3).

Solución 7.2 · Rompe la inmutabilidad ← Volver al ejercicio

Al cambiar de página no ocurre nada: ni cambia la tabla ni el número de página. `setParams(params)` recibe **la misma referencia** que ya tenía el estado; `Object.is` concluye que no ha cambiado y React descarta el

renderizado. Peor aún, el objeto del estado ha quedado modificado en silencio, de modo que el siguiente cambio legítimo (por ejemplo, un filtro) arrastrará esa página «fantasma».

Solución 7.3 · Quita la key del formulario

← Volver al ejercicio

Sin la `key` y con la aplicación 2 en caché, tras pulsar *Siguiente* el título de la página cambia a «Modificar «nombre de la 2»», pero el formulario sigue mostrando el nombre modificado de la 1 y su indicador de cambios sin guardar. Si guardaras, ¡enviarías los datos de la 1 a la 2! Con la `key`, el formulario se recrea y muestra los datos de la 2.

Si no visitas antes la aplicación 2, sus datos no están en caché: la consulta pasa por `isPending`, la página muestra `LoadingState` **en lugar** del formulario (un componente de otro tipo en esa posición), así que React desmonta el formulario y lo vuelve a crear al llegar los datos. El problema queda oculto y solo aparece cuando hay caché: un error de los que se escapan en las pruebas manuales. La `key` lo resuelve en todos los casos.

Solución 7.4 · Elige dónde vive el estado

← Volver al ejercicio

(a) **Estado de URL** si se quiere poder enlazar directamente a una pestaña (`?pestana=historial`); estado local si no importa. (b) **URL**, junto al resto de filtros del listado, por coherencia y para poder compartirlo; además debería enviarse al backend, que hoy no ofrece filtro por nombre. (c) **Almacenamiento del navegador** (`localStorage`) leído por un contexto o un hook en el layout: es una preferencia de usuario que debe sobrevivir a recargas, no un dato de navegación. (d) **Caché del servidor** (TanStack Query), como el catálogo de aplicaciones de los desplegables. (e) **Formulario** (React Hook Form): es un dato más del formulario, que se muestra condicionalmente con `useWatch` sobre el campo de estado y se valida con el esquema.

Respuestas · capítulo 7

← Volver a las preguntas

1. Las props las define el padre y son de solo lectura; el estado lo define y lo cambia el propio componente, y React lo conserva entre renderizados.
2. Ejecutar la función del componente para obtener la descripción de la interfaz. No implica tocar el DOM: eso ocurre en el *commit*, solo con las diferencias.
3. Vale 1, porque ambas llamadas calculan `0 + 1` con la instantánea. Con actualizaciones funcionales, `setCount((c) => c + 1)` dos veces, valdría 2.
4. Renderizado: se ejecutan los componentes, se obtiene el nuevo árbol y se compara con el anterior. *Commit*: se aplican al DOM solo las diferencias y después se ejecutan los efectos.
5. Destruye el componente anterior con todo su subárbol y su estado, y crea el nuevo desde cero.
6. Porque React compara referencias con `Object.is`: el objeto modificado es la misma referencia y React concluye que no ha cambiado.
7. El listado principal necesita que su estado sea recargable, enlazable y navegable con el historial. El selector del catálogo es auxiliar dentro de otra pantalla y eso no aporta nada.
8. Un valor que se calcula a partir de props, estado o datos durante el renderizado. Es preferible porque nunca se desincroniza con su origen.
9. Para que, al cambiar de aplicación, React trate el formulario como un componente nuevo y lo reinicie con los valores de la aplicación correcta.

A.8 Capítulo 8 · Hooks

↑ Índice

Solución 8.1 · Escribe un hook personalizado

← Volver al ejercicio

`src/hooks/use-media-query.ts`

```ts
import { useEffect, useState } from 'react'

export function useMediaQuery(query: string): boolean {
  const [matches, setMatches] = useState(() => window.matchMedia(query).matches)

  useEffect(() => {
    const mql = window.matchMedia(query)
    const onChange = () => setMatches(mql.matches)
    onChange()                          // por si la consulta ha cambiado
    mql.addEventListener('change', onChange)
    return () => mql.removeEventListener('change', onChange)
  }, [query])

  return matches
}
```

El valor inicial se calcula con una función (`useState(() => …)`), que React solo ejecuta en el primer renderizado. El efecto se suscribe al cambio y la limpieza se da de baja; depende de `query` para volver a suscribirse si cambia. La versión 7 del plugin de ESLint puede avisar de la llamada síncrona a `setMatches` dentro del efecto (`set-state-in-effect`); en este caso se puede eliminar esa línea, porque el estado ya se inicializa con el valor correcto. La alternativa más idiomática para suscribirse a fuentes externas es el hook `useSyncExternalStore` de React.

Solución 8.2 · Provoca un bucle con dependencias inestables

← Volver al ejercicio

ESLint avisa en el `useCallback`: *The 'params' object makes the dependencies of useCallback Hook change on every render*. En el *Profiler* verás que el efecto vuelve a ejecutarse en cada renderizado, porque `updateParams` es una función nueva cada vez. No hay bucle infinito porque el cuerpo del efecto solo llama a `updateParams` cuando la página está fuera de rango: en el caso normal, el efecto se ejecuta, no cambia nada y termina. El código es correcto por casualidad; bastaría con que el efecto tuviera otra lógica para provocar un bucle.

Solución 8.3 · Elimina un efecto innecesario

← Volver al ejercicio

```ts
function DemandCount({ demands }: { demands: Demand[] }) {
  const openCount = demands.filter((d) => d.status === 'OPEN').length
  return <span>{openCount} abiertas</span>
}
```

Es un valor derivado. La versión con efecto renderiza dos veces en cada cambio (primero con el recuento antiguo, después con el nuevo), muestra durante un instante un valor incorrecto y añade estado que puede desincronizarse. Si el cálculo fuera caro, se envolvería en `useMemo(…, [demands])`, pero sigue sin hacer falta un efecto.

Respuestas · capítulo 8 ← Volver a las preguntas

1. Porque React identifica cada hook por su orden de llamada; si un hook se salta en un renderizado, los siguientes reciben el estado de otro.
2. Como una sincronización de un sistema externo con ciertos valores, que se repite cuando esos valores cambian; no como un evento del ciclo de vida.
3. Calcular valores derivados (se calculan al renderizar), responder a acciones del usuario (se hace en el manejador) y reiniciar estado al cambiar una prop (se usa `key`). Y pedir datos, para lo que se usa TanStack Query.
4. Deshace lo que hizo el efecto (darse de baja, cancelar un temporizador). Se ejecuta antes de volver a ejecutar el efecto y al desmontar el componente.
5. Renderiza cada componente dos veces para detectar impurezas, y monta, desmonta y vuelve a montar para detectar efectos sin limpieza correcta.
6. Porque `params` es dependencia de `updateParams`: si fuera un objeto nuevo en cada renderizado, `useCallback` devolvería también una función nueva cada vez.
7. Porque quien llama suele crear el objeto `ensure` en cada renderizado; con valores primitivos, la comparación detecta cambios reales y no cambios de referencia.
8. Solo la lógica: cada llamada tiene su propio estado y sus propios efectos. Comparten datos únicamente si el hook lee algo que vive fuera, como la caché de TanStack Query.

A.9 Capítulo 9 · Composición, contexto y límites de error

↑ Índice

Solución 9.1 · Sigue los contextos de un campo ← Volver al ejercicio

`FormProvider` provee el objeto completo devuelto por `useForm` (`control`, `register`, `formState`…); `FormField` provee `{ name: 'name' }`; `FormItem` provee `{ id }` con un identificador generado por `useId()` (una cadena opaca, cuyo formato exacto varía entre versiones de React). `FormLabel` llama a `useFormField()`, que lee `FormItemContext` y construye `formItemId`: ese identificador seguido de `-form-item`. Lo usa en `htmlFor`. `FormControl` usa el mismo `formItemId` como `id` y, mediante `Slot`, lo pone en el `<input>`. En *Elements* verás que el atributo `for` de la etiqueta y el `id` del `<input>` tienen exactamente el mismo valor, terminado en `-form-item`.

Solución 9.2 · Crea un contexto propio ← Volver al ejercicio

`src/components/layout/density.tsx (esbozo)`

```
type Density = 'comfortable' | 'compact'
const DensityContext = createContext<{ density: Density; toggle: () => void } | null>(null)

export function DensityProvider({ children }: { children: ReactNode }) {
  const [density, setDensity] = useState<Density>('comfortable')
  const value = useMemo(
    () => ({ density, toggle: () => setDensity((d) => (d === 'compact' ? 'comfortable' : 'comp
act')) }),
    [density],
  )
  return <DensityContext.Provider value={value}>{children}</DensityContext.Provider>
}

export function useDensity() {
  const ctx = useContext(DensityContext)
  if (!ctx) throw new Error('useDensity debe usarse dentro de DensityProvider')
```

```
  return ctx
}
```

El valor por defecto `null` y la comprobación en `useDensity` detectan el uso fuera del proveedor. El `useMemo` evita que el objeto `value` cambie en cada renderizado del proveedor y renderice a los consumidores sin motivo. Para que React Refresh funcione, este fichero exporta un componente y un hook; ESLint puede pedir separarlos.

Sobre dónde guardarlo: el **contexto** basta para compartirlo entre la cabecera y la tabla durante la sesión; la **URL** no es adecuada, porque no es navegación; el **almacenamiento del navegador** (`localStorage`) añade persistencia entre recargas. Lo más completo es combinar los dos últimos: el proveedor inicializa el estado desde `localStorage` y lo guarda al cambiar.

Solución 9.3 · Provoca un error de renderizado

← Volver al ejercicio

Con el `throw` dentro del `map`, el contenido de la página se sustituye por *Se ha producido un error inesperado — Fallo de prueba*, y **el menú lateral y la cabecera siguen funcionando**, porque el `errorElement` está dentro de `AppLayout`. En la consola verás el error y la pila de componentes.

Con el `throw` en un `onClick`, la página no cambia: el error aparece en la consola como no capturado (*Uncaught Error*), pero `RouteErrorPage` no se muestra. Los límites de error solo capturan errores producidos durante el renderizado, en efectos o en constructores; un manejador de eventos se ejecuta fuera de ese proceso, cuando React ya terminó de renderizar.

Respuestas · capítulo 9

← Volver a las preguntas

1. Por composición: envolviendo el componente en otro que fija algunas props, o dejando huecos (`children`, props de tipo `ReactNode`, componentes compuestos) que rellena quien lo usa.
2. Pasar un dato por varios niveles de componentes que no lo usan solo para que llegue a uno que sí. Acopla los intermedios y complica los cambios.
3. `createContext` para crearlo, un proveedor (`Context.Provider`) para ofrecer un valor a un subárbol y `useContext` para leerlo.
4. Por contexto: `<Form {...form}>` provee el formulario y `TextInputField` lo obtiene con `useFormContext()`.
5. Porque un componente solo ve los contextos de sus antepasados, y las páginas que pinta el router necesitan el `QueryClient` para usar consultas.
6. Que cada cambio renderiza a todos los consumidores, lo que puede provocar muchos renderizados innecesarios.
7. Hace que `Button` no pinte su propio `<button>`, sino que fusione sus props y estilos en su único hijo (por ejemplo, un `Link`).
8. Para que un error en una página sustituya solo el contenido y el menú lateral y la cabecera sigan disponibles.
9. No: un error en un `onClick` no se captura. Los errores de la API tampoco, porque son asíncronos; se gestionan explícitamente con el estado de error de TanStack Query.

A.10 Capítulo 10 · Tailwind CSS 4

↑ Índice

Solución 10.1 · Traduce clases a CSS

← Volver al ejercicio

- `sticky top-0`: la cabecera se desplaza con la página hasta tocar el borde superior y ahí se queda fija.
- `z-30`: queda por encima del contenido (pero por debajo del menú lateral, `z-40`, y de los paneles, `z-50`).
- `flex h-16 items-center gap-3`: fila de 4 rem de alto, con los elementos centrados verticalmente y separados 0,75 rem.
- `border-b`: línea inferior con el color de borde del tema.
- `bg-card/85`: fondo blanco al 85 % de opacidad.
- `px-4` y `lg:px-8`: relleno horizontal de 1 rem, 2 rem desde 1024 px, alineado con el `main`.
- `backdrop-blur`: desenfoca lo que pasa por debajo.
- `supports-[backdrop-filter]:bg-card/70`: si el navegador soporta el desenfoque, el fondo baja al 70 % de opacidad; si no, se queda en el 85 % para que el texto siga siendo legible.

En las DevTools, al desplazar un listado largo, verás la tabla pasar difuminada por debajo de la cabecera.

Solución 10.2 · Añade un tono a los indicadores

← Volver al ejercicio

`src/components/common/badges.tsx`

```tsx
export type Tone = 'success' | 'neutral' | 'info' | 'brand' | 'warning' | 'danger' | 'violet'

const PILL_TONES: Record<Tone, { pill: string; dot: string }> = {
  // …
  violet: { pill: 'bg-violet-50 text-violet-700 ring-violet-600/20', dot: 'bg-violet-500' },
}
```

Gracias al `Record<Tone, …>`, si añades `'violet'` al tipo y olvidas la entrada, el proyecto no compila. Con la versión dinámica (`` `bg-${color}-50` ``), el texto `bg-violet-50` no aparece literalmente en ningún fichero; Tailwind no lo encuentra al analizar el código y no genera su regla: la clase está en el elemento, pero no tiene efecto. Solo funcionaría por casualidad si otra parte del código usara la misma clase completa.

Solución 10.3 · Prepara el modo oscuro

← Volver al ejercicio

`src/index.css (añadido)`

```css
.dark {
  --background: oklch(0.17 0.03 262);
  --foreground: oklch(0.95 0.01 250);
  --card: oklch(0.21 0.035 262);
  --card-foreground: oklch(0.95 0.01 250);
  --muted: oklch(0.26 0.03 260);
  --muted-foreground: oklch(0.72 0.03 255);
  --border: oklch(0.3 0.03 260);
  /* además: --popover, --input, --secondary, --accent… */
}
```

Al añadir la clase `dark` a `<html>`, cambian el fondo, las tarjetas, los textos y los bordes, porque todos usan tokens. No cambian los elementos con **colores escritos directamente**: los indicadores de estado (`bg-emerald-50 text-emerald-700`), la alerta de éxito, los textos `text-white` o el `toaster`, configurado con

`theme="light"`. Para un modo oscuro completo habría que añadir variantes `dark:` en esos componentes o convertir también esos colores en tokens.

Respuestas · capítulo 10 ← Volver a las preguntas

1. Colisiones de nombres, CSS que solo crece por miedo a borrar, inconsistencia visual y tener que saltar entre el componente y su hoja de estilos.
2. 1 rem (16 px): cuatro pasos de 0,25 rem. Una escala común hace que los espaciados de todas las pantallas encajen.
3. «En fila a partir de 640 px». Para algo solo en móvil: clase sin prefijo y anulación en el punto de ruptura, como `block lg:hidden`.
4. Radix pone `data-state="open"` en el disparador, marcado con la clase `group`, y la flecha tiene `group-data-[state=open]:rotate-90` y `transition-transform`.
5. Porque Tailwind genera el CSS buscando las clases como texto en el código fuente, y esa clase solo existe en ejecución.
6. Variables CSS con los valores en `:root`; el puente `@theme inline` que convierte cada variable en un token de Tailwind; y los estilos base en `@layer base`.
7. Es perceptualmente uniforme: la luminosidad se corresponde con la claridad percibida, lo que facilita crear variantes y combinaciones coherentes.
8. `clsx` une clases y descarta las falsas, pero no sabe que `px-2` y `px-4` chocan; `twMerge` resuelve el conflicto quedándose con la última.
9. Cuando el valor se calcula en ejecución a partir de datos, como la altura de cada barra de `LevelBadge`.

A.11 Capítulo 11 · shadcn/ui y Radix

↑ Índice

Solución 11.1 · Añade un componente del catálogo ← Volver al ejercicio

`dialog.tsx` envuelve el primitivo `Dialog` de Radix y exporta `Dialog`, `DialogTrigger`, `DialogPortal`, `DialogOverlay`, `DialogContent`, `DialogHeader`, `DialogFooter`, `DialogTitle`, `DialogDescription` y `DialogClose`. Un esbozo para la página de modificación:

```
const [confirmOpen, setConfirmOpen] = useState(false)
// …
<Button type="button" variant="ghost" onClick={() => setConfirmOpen(true)} disabled={!isDirty
|| mutation.isPending}>
  <RotateCcw /> Descartar
</Button>
<Dialog open={confirmOpen} onOpenChange={setConfirmOpen}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>¿Descartar los cambios?</DialogTitle>
      <DialogDescription>Se perderán las modificaciones que no has guardado.</DialogDescriptio
n>
    </DialogHeader>
    <DialogFooter>
      <Button variant="outline" onClick={() => setConfirmOpen(false)}>Seguir editando</Button>
      <Button variant="destructive" onClick={() => { form.reset(); mutation.reset(); setConfir
mOpen(false) }}>
        Descartar
      </Button>
    </DialogFooter>
```

```
  </DialogContent>
</Dialog>
```

El diálogo se controla con estado local, igual que el `Sheet` de la cabecera. Radix aporta gratis el foco atrapado, el cierre con *Escape* y el bloqueo del desplazamiento.

Solución 11.2 · Crea una variante de botón

← Volver al ejercicio

`src/components/ui/button.tsx (variante añadida)`

```
success: 'bg-success text-white shadow-xs hover:bg-success/90',
```

`--color-success` ya existe en `@theme inline`, así que `bg-success` está disponible. `VariantProps<typeof buttonVariants>` deriva ahora para `variant` el tipo `'default' | 'brand' | 'destructive' | 'outline' | 'secondary' | 'ghost' | 'link' | 'success' | null | undefined`, y el editor ofrece la nueva variante al autocompletar. No hay que tocar ningún otro tipo: está todo derivado de la definición.

Solución 11.3 · Comprueba la accesibilidad con el teclado

← Volver al ejercicio

Con *Tab* el foco recorre: botón del menú (solo en móvil), enlaces del menú, *Nueva aplicación*, los dos filtros, *Limpiar filtros* (si está activo), las cabeceras ordenables, las acciones de cada fila y la paginación. En el filtro de estado, *Intro* o *Espacio* abren la lista, las flechas recorren las opciones, *Intro* selecciona y el foco vuelve al disparador. En el disparador verás `role="combobox"`, `aria-expanded="false"` que pasa a `"true"` al abrir y `aria-controls` apuntando al `id` de la lista.

En el menú móvil, el foco no sale del panel aunque sigas pulsando *Tab* (*focus trap*), *Escape* lo cierra y el foco vuelve al botón que lo abrió. Todo ello lo aporta el `Dialog` de Radix que hay debajo del `Sheet`.

Respuestas · capítulo 11

← Volver a las preguntas

1. Que implementa comportamiento, accesibilidad y estado de los componentes, pero no trae ningún estilo.
2. Por ejemplo: apertura y navegación con teclado, cierre con *Escape* o al hacer clic fuera, devolución del foco, atributos ARIA y posicionamiento de la lista.
3. Renderiza el contenido al final del `<body>` para que no lo recorte un `overflow: hidden` ni lo tape otro elemento por los `z-index`.
4. Porque no es una dependencia: su CLI copia el código de cada componente en el proyecto.
5. `cva` define las clases comunes y las de cada variante; `VariantProps` deriva de esa definición los tipos de las props `variant` y `size`.
6. Porque sus props se tipan como `React.ComponentProps<'button'>` y se propagan al elemento con `{...props}`.
7. Porque el `Select` de Radix solo trabaja con cadenas.
8. El `Dialog` de Radix: hereda la captura del foco, el cierre con *Escape*, el bloqueo del desplazamiento y la accesibilidad de un diálogo modal.
9. Que las mejoras del catálogo no llegan solas: hay que reaplicarlas a mano sobre el código propio.

A.12 Capítulo 12 · El layout y el sistema visual

↑ Índice

Solución 12.1 · Activa un módulo en el menú

← Volver al ejercicio

Con `basePath: '/incidencias'`, `comingSoon` eliminado y dos `items`, el grupo aparece en el menú y los enlaces navegan, pero cada uno muestra la página 404 de la aplicación: el router no tiene rutas para `/incidencias`, así que encaja la ruta comodín `{ path: '*', element: <NotFoundPage /> }`. Faltaría crear el módulo con sus páginas, declarar `incidentRoutes` en su `routes.ts` y registrarlas en `app/router.tsx`, además de la regla del proxy si lo sirve otro servicio.

Solución 12.2 · Recuerda los grupos plegados

← Volver al ejercicio

```
const storageKey = `nav-open:${module.id}`
const [open, setOpen] = useState(() => {
  try {
    return localStorage.getItem(storageKey) !== 'false'
  } catch {
    return true
  }
})

const handleOpenChange = (next: boolean) => {
  setOpen(next)
  try { localStorage.setItem(storageKey, String(next)) } catch { /* almacenamiento no
disponible */ }
}
// <Collapsible open={open} onOpenChange={handleOpenChange}>
```

El valor se guarda **en el manejador**: es la respuesta a una acción del usuario, no una sincronización con el renderizado (capítulo 8). La inicialización con una función hace que `localStorage` se lea una sola vez, en el primer renderizado. Los `try/catch` cubren navegadores con el almacenamiento bloqueado. Como `SidebarNav` también se usa en el panel móvil, la preferencia se comparte entre ambos.

Solución 12.3 · Elimina una duplicación

← Volver al ejercicio

```
import { DetailField, LongText } from '@/components/common/detail-field'

export function ApplicationSummary({ application, columns = 2, className }: { /* … */ }) {
  const twoColumns = columns === 2
  return (
    <dl className={cn('grid gap-x-8 gap-y-5', twoColumns && 'sm:grid-cols-2', className)}>
      <DetailField label="Identificador"><span className="font-mono text-sm">#{application.id}
</span></DetailField>
      {/* … */}
      <DetailField label="Descripción" className={twoColumns ? 'sm:col-span-2' : undefined}>
        <LongText value={application.description} />
      </DetailField>
    </dl>
  )
}
```

Se gana una única definición del aspecto de las fichas: si cambia el estilo de las etiquetas, cambia en aplicaciones y demandas a la vez. El riesgo es pequeño pero real: hay que comprobar que el aspecto es idénti-

co (clases, texto vacío por defecto «Sin descripción») y que nadie dependía del componente privado. Es una refactorización sin cambio de comportamiento, que conviene hacer en un *commit* separado.

Respuestas · capítulo 12 ← Volver a las preguntas

1. Menú lateral, cabecera y cabecera de página son fijas en su forma; la zona variable es el `<Outlet />` con la página activa.
2. Porque en móviles la barra del navegador aparece y desaparece; `svh` usa la altura visible real y evita saltos.
3. La estructura de navegación es un dato que se puede leer y cambiar sin tocar la presentación, y añadir un módulo es añadir datos.
4. Para exigir coincidencia exacta de la ruta y que un enlace no quede activo en las rutas que empiezan por la suya.
5. El panel deslizante (`Sheet`) contiene el mismo `SidebarNav` , al que se pasa `onNavigate` para cerrarlo al navegar.
6. Para que guíe la vista hacia lo importante: la acción principal, la ubicación actual y el foco.
7. Que el nivel se lee por el número de barras, sin depender del color.
8. Mantiene los datos anteriores y muestra una franja de aviso encima en lugar de sustituirlos por el error.

A.13 Capítulo 13 · Organización por funcionalidad

↑ Índice

Solución 13.1 · Dibuja el grafo de un fichero ← Volver al ejercicio

`demand-edit-page.tsx` importa de: React y librerías (`react` , `react-router` , `react-hook-form` , `@hookform/resolvers/zod` , `sonner` , `lucide-react`); `components/ui` (alert, button, card, form, input, label, separator); `components/common` (page-header, resource-not-found, state-views); `lib` (api-client, form-errors, route-params); y del propio módulo, unas ocho importaciones repartidas entre `api/` (consultas), `components/` (campos, ficha, ciclo de vida), y `model/` (constantes, esquemas, reglas de estado, tipos). Cumple todas las reglas: ninguna importación de otro módulo ni de `app/` . La dependencia de aplicaciones llega indirectamente a través del campo *Aplicación*, que usa `application-options.ts` .

Solución 13.2 · Haz cumplir las reglas con ESLint ← Volver al ejercicio

`eslint.config.js (bloques añadidos)`

```
{
  files: ['src/components/**/*.{ts,tsx}', 'src/lib/**/*.{ts,tsx}'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{ group: ['@/features/*'], message:
'components/ y lib/ no pueden depender de los módulos.' }],
    }],
  },
},
{
  files: ['src/features/applications/**/*.{ts,tsx}'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{ group: ['@/features/demands/*'], message: 'Aplicaciones no puede depender
de Demandas.' }],
    }],
```

```
  },
},
```

`npm run lint` pasa sin errores con el código actual. Si añades en `pagination-bar.tsx` una importación de `@/features/applications/model/types`, ESLint muestra el mensaje configurado. Para reglas más completas (por ejemplo, «cada módulo solo puede importar la API pública de otro») existen plugins dedicados, como `eslint-plugin-boundaries`.

Solución 13.3 · Decide dónde va ← Volver al ejercicio

(a) `components/common` (o una función en `lib/format.ts` y un componente común que la use): no sabe nada del dominio. (b) `features/incidents/model/types.ts`. (c) La apariencia del semáforo en `components/common` si otros módulos pueden necesitarla (como `StatusPill`); la correspondencia entre niveles de SLA y colores, en el módulo. (d) `features/incidents/model/` (por ejemplo, `sla-rules.ts`), como función pura con sus pruebas, igual que `status-rules.ts`. (e) El hook que obtiene los equipos debería vivir en el módulo de equipos (`features/teams/api/`) y exponerse para otros módulos, como hace `useApplicationOptions`; mientras no exista ese módulo, puede empezar en incidencias y moverse después.

Respuestas · capítulo 13 ← Volver a las preguntas

1. Por tipo técnico, el código de una funcionalidad se reparte en carpetas por clase de artefacto; por funcionalidad, cada módulo agrupa todo lo suyo y solo lo transversal vive fuera.
2. `app/`, para registrar las rutas de todos los módulos en el router.
3. Porque dejaría de ser común: dependería del dominio y crearía dependencias en ambas direcciones.
4. Es unidireccional, estrecha (un hook y una constante pensados para ello), está concentrada en un único fichero y refleja una relación real del dominio.
5. Si necesita saber algo del dominio, si lo usan varios módulos o es un patrón claro, y separar la apariencia del significado.
6. `StatusPill` y `LevelBadge` (comunes) definen el aspecto de un estado o un nivel; `application-badges.tsx` (del módulo) decide qué tono o nivel corresponde a cada valor del dominio.
7. Porque las funciones normales se prueban y reutilizan sin montar componentes, también fuera de React.
8. Valores que llegan como `undefined` en ejecución y errores del tipo *Cannot access 'X' before initialization*.

A.14 Capítulo 14 · Anatomía de un módulo

↑ Índice

Solución 14.1 · Compara dos ficheros paralelos ← Volver al ejercicio

Las diferencias **de dominio**: el *hook* de datos (`useDemands`), los componentes de filtros y tabla, las funciones de la URL, los textos y los filtros que cuentan para `hasFilters` (aplicación, estado y prioridad). Las **accidentales**: los nombres de las funciones de la URL (`parseDemandListParams` frente a `parseListParams`) y pequeñas diferencias de redacción u orden. La estructura es idéntica: lectura de la URL con `useMemo`, `updateParams` con `useCallback`, el efecto de corrección de página, la cadena de estados y la paginación.

Extraer más código común (por ejemplo, un hook `useUrlListState(parse, serialize)` que devuelva `params` y `updateParams`, o un componente `ListPageLayout`) eliminaría duplicación. A favor: un único sitio que corregir. En contra: más abstracción que entender, y el riesgo de que el tercer módulo no encaje. Con

dos módulos casi idénticos y un tercero en camino, el hook de estado en la URL sí parece justificado; un componente de página genérico, probablemente todavía no.

Solución 14.2 · Verifica el contrato de ordenación

← Volver al ejercicio

El frontend ofrece `id`, `title`, `application.name`, `priority`, `status`, `createdAt` y `updatedAt`. El backend acepta `id`, `title`, `priority`, `status` y `updatedAt`. Por tanto, **ordenar por *Aplicación* o por fecha de creación produce un `400 Invalid demand sort field`**. El usuario ve el estado de error del listado con ese mensaje, y la URL queda con `?orden=createdAt,asc`, así que al recargar se repite el error.

El origen está en la separación de las aplicaciones en otro servicio: la ordenación por nombre de aplicación necesitaba una consulta que unía ambas tablas, y ya no es posible en la base de datos de demandas. Soluciones: en el **frontend**, alinear `DEMAND_SORT_FIELDS` con la lista del backend (quitar `application.name` y `createdAt`, o hacer esas columnas no ordenables) y, de paso, que una URL antigua con esos valores se descarte al leerla; en el **backend**, volver a admitir `createdAt` (es una columna propia de `Demand`, fácil de añadir) y decidir cómo ordenar por nombre de aplicación (por ejemplo, guardando una copia del nombre en la demanda). Lo razonable es ambas: corregir ya el frontend para no ofrecer lo que el backend rechaza, y decidir en el backend si se recupera la funcionalidad. Es una de las notas de integración del capítulo 25.

Solución 14.3 · Diseña el `model/` de Incidencias

← Volver al ejercicio

`features/incidents/model/types.ts (borrador)`

```ts
export const INCIDENT_SEVERITIES = ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL'] as const
export type IncidentSeverity = (typeof INCIDENT_SEVERITIES)[number]

export const INCIDENT_STATUSES = ['OPEN', 'IN_PROGRESS', 'RESOLVED', 'CLOSED'] as const
export type IncidentStatus = (typeof INCIDENT_STATUSES)[number]

/** IncidentResponse (PENDIENTE: confirmar nombres con el DTO del backend). */
export interface Incident {
  id: number
  title: string
  description: string | null
  severity: IncidentSeverity
  status: IncidentStatus
  applicationId: number
  applicationName: string      // PENDIENTE: ¿lo devuelve el backend, como en demandas?
  createdAt: string            // Instant ISO-8601
  resolvedAt: string | null
}

export interface IncidentListParams {
  applicationId?: number
  status?: IncidentStatus
  severity?: IncidentSeverity
  page: number
  size: number
  sortField: IncidentSortField  // PENDIENTE: lista validada por el backend
  sortDirection: SortDirection
}
```

`features/incidents/model/constants.ts (borrador)`

```ts
export const INCIDENT_STATUS_LABELS: Record<IncidentStatus, string> = {
  OPEN: 'Abierta', IN_PROGRESS: 'En curso', RESOLVED: 'Resuelta', CLOSED: 'Cerrada',
}
export const SEVERITY_LABELS: Record<IncidentSeverity, string> = {
```

```
  LOW: 'Baja', MEDIUM: 'Media', HIGH: 'Alta', CRITICAL: 'Crítica',
}
// PENDIENTE: alinear con @PageableDefault y @Size del backend
export const DEFAULT_INCIDENT_LIST_PARAMS: IncidentListParams = { page: 0, size: 10,
sortField: 'id', sortDirection: 'desc' }
```

Las decisiones sin backend (nombres de campos, si llega el nombre de la aplicación, campos ordenables, valores por defecto, longitudes) se marcan con comentarios `PENDIENTE` y se resuelven leyendo el controlador y los DTO cuando existan. Lo que sí puede decidirse ya es la estructura y las etiquetas.

Respuestas · capítulo 14 ← Volver a las preguntas

1. `model/` (tipos, constantes, esquemas, reglas, URL), `api/` (endpoints y consultas), `components/` (presentación del módulo) y `pages/` (orquestación). Las páginas usan todo; los componentes y la API usan `model/`; `model/` no usa a nadie.
2. Para ser una base estable, reutilizable y fácil de probar con pruebas unitarias sencillas.
3. Los que reflejan los DTO del backend y los propios del frontend, como los criterios del listado.
4. La URL (la ve y comparte el usuario), el objeto de la interfaz (lo usan los componentes) y la consulta a la API (la espera Spring Data).
5. `-api.ts` contiene un método por endpoint, sin lógica; `-queries.ts` envuelve esos métodos en hooks de TanStack Query con sus claves de caché e invalidaciones.
6. Para ser reutilizables con datos de cualquier origen y mantener la carga de datos en un único sitio por pantalla.
7. Leer y validar la URL, pedir los datos, decidir qué estado mostrar, componer los componentes y gestionar las acciones del usuario.
8. Crear el módulo, declarar sus rutas y registrarlas en el router, añadir su entrada al menú y, si lo sirve otro servicio, la regla del proxy.

A.15 Capítulo 15 · El cliente HTTP y la gestión de errores

↑ Índice

Solución 15.1 · Recorre los formatos de error ← Volver al ejercicio

(a) `GET /api/applications/999999` → **404** con cuerpo `{ status: 404, error: 'RESOURCE_ERROR', message: 'Application not found: 999999', fieldErrors: {} }` (si el servicio de aplicaciones usa el mismo formato que el de demandas). `kind: 'not-found'`; el usuario ve el aviso de aplicación inexistente. En *Network* verás una sola petición: los 404 no se reintentan.

(b) `POST /api/applications` con nombre vacío → **400** con `error: 'VALIDATION_ERROR'`, `message: 'La petición contiene campos inválidos'` y `fieldErrors: { name: '…' }` con el mensaje de Bean Validation. `kind: 'validation'`. Si llegara desde el formulario, `applyServerFieldErrors` lo colocaría bajo el campo *Nombre*.

(c) Con el servicio de aplicaciones detenido, *Nueva demanda* llama a `/api/applications` para el desplegable: el proxy de Vite responde **500** (o 502) sin el formato del backend y la terminal muestra `ECONNREFUSED`; `kind: 'server'` y mensaje por defecto, tras dos reintentos. Si intentas guardar la demanda, el servicio de demandas no puede validar la aplicación y responde **503** con `message: 'Application Catalog is unavailable'`: también `kind: 'server'`, esta vez con el mensaje del backend.

Solución 15.2 · Añade el método `delete`

← Volver al ejercicio

`src/lib/api-client.ts`

```
export const apiClient = {
  // …
  delete: <T = void>(path: string) => apiRequest<T>(path, { method: 'DELETE' }),
}
```

`src/lib/api-client.test.ts`

```
import { afterEach, describe, expect, it, vi } from 'vitest'
import { apiClient } from './api-client'

describe('apiClient.delete', () => {
  afterEach(() => vi.unstubAllGlobals())

  it('devuelve undefined con una respuesta 204', async () => {
    const fetchMock = vi.fn().mockResolvedValue(new Response(null, { status: 204 }))
    vi.stubGlobal('fetch', fetchMock)

    await expect(apiClient.delete('/applications/7')).resolves.toBeUndefined()
    expect(fetchMock).toHaveBeenCalledWith('/api/applications/7', expect.objectContaining({ me
thod: 'DELETE' }))
  })
})
```

El genérico con valor por defecto `T = void` documenta que normalmente no se espera cuerpo. La segunda aserción comprueba además la URL construida y el método.

Solución 15.3 · Prepara la autenticación

← Volver al ejercicio

En `apiRequest`, se obtendría el *token* de un único sitio (por ejemplo, una función `getAccessToken()` de un módulo de autenticación) y se añadiría a las cabeceras una entrada `Authorization` con el valor `Bearer` seguido del *token*, solo cuando exista (con el mismo *spread* condicional que ya se usa para `Content-Type`). Para el 401, tras `toApiError`, si `status === 401` se notificaría a un único gestor (un evento o una función registrada por el módulo de autenticación) que limpie la sesión, vacíe la caché de TanStack Query y redirija al inicio de sesión; además, se añadiría `401` a la lista de errores que no se reintentan (ya lo cubre la regla de 4xx).

Sobre el almacenamiento: **en memoria** es lo más seguro frente a XSS, pero se pierde al recargar; `localStorage` sobrevive a recargas, pero cualquier script inyectado puede leerlo; una **cookie** `HttpOnly` emitida por el backend no es accesible desde JavaScript (la opción más robusta frente a XSS), y con la topología de mismo origen (capítulo 16) funciona sin CORS, aunque exige protección CSRF. En aplicaciones empresariales, la opción habitual es la *cookie* `HttpOnly` gestionada por el backend o por una pasarela.

Respuestas · capítulo 15

← Volver a las preguntas

1. La URL base, las cabeceras, la serialización a JSON, los parámetros vacíos, la comprobación de `ok`, la interpretación de los errores, los fallos de red y la cancelación.
2. Porque el cuerpo puede estar vacío o no ser JSON; leerlo como texto permite decidir antes de parsear sin provocar excepciones.
3. Se omite de la cadena de consulta, y para Spring un parámetro ausente significa «sin filtro».

4. `network` (sin conexión), `validation` (errores por campo o datos no válidos), `not-found` (aviso de inexistente), `client` (mensaje; en demandas el 409 tiene tratamiento propio) y `server` (error con reintento).
5. Porque puede recibir JSON del backend, el formato por defecto de Spring, HTML de un proxy o nada, y siempre debe producir un error válido.
6. Un 400 con `VALIDATION_ERROR` y `fieldErrors`; el frontend coloca cada mensaje en su campo con `applyServerFieldErrors`.
7. Porque repetir una petición incorrecta da el mismo resultado; solo tiene sentido reintentar fallos transitorios.
8. En `apiRequest`, porque es el único punto por el que pasan todas las peticiones.

A.16 Capítulo 16 · CORS, proxy y despliegue

↑ Índice

Solución 16.1 · Provoca un error de CORS

← Volver al ejercicio

Con `VITE_API_BASE_URL=http://localhost:8081/api`, el navegador llama directamente al servicio de aplicaciones, que es otro origen. La consola muestra *Access to fetch at 'http://localhost:8081/api/applications?...' from origin 'http://localhost:5173' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource*. En *Network* la petición aparece como fallida (*CORS error*), aunque el servidor sí la ha recibido y respondido. `fetch` rechaza la promesa, así que la interfaz recibe un `ApiError` de tipo `network` («No se puede conectar con el servidor»). El proxy ya no interviene porque solo actúa sobre rutas relativas a `localhost:5173` que empiezan por `/api/...`.

Solución 16.2 · Sirve el build como en producción

← Volver al ejercicio

```
npm run build
docker run --rm -p 8088:80 \
  -v "$PWD/dist:/usr/share/nginx/html:ro" \
  -v "$PWD/nginx.conf:/etc/nginx/conf.d/default.conf:ro" \
  nginx:1.27-alpine
```

En `nginx.conf`, los destinos son `proxy_pass http://host.docker.internal:8081;` y `http://host.docker.internal:8080;`. En Linux puede hacer falta añadir `--add-host=host.docker.internal:host-gateway`. Al abrir `http://localhost:8088/demandas/1` y recargar, `try_files` devuelve `index.html` y React Router pinta la ficha. Los ficheros de `/assets/` llegan con `Cache-Control: public, max-age=31536000, immutable`, e `index.html` con `no-cache`.

Solución 16.3 · Elige una opción de despliegue

← Volver al ejercicio

Una propuesta razonable: **opción A con el balanceador**. Reglas por prefijo: `/api/applications/*` al servicio de aplicaciones, `/api/demands/*` al de demandas, y todo lo demás a un contenedor Nginx con el `dist/` (o al almacenamiento estático, si el balanceador puede enrutar a él). El frontend se construye una vez con `/api` relativo y la imagen se promociona entre entornos. Configuración necesaria: las reglas del balanceador, el Nginx con *fallback* y cabeceras de caché, HTTPS en el balanceador y cabeceras de seguridad. Riesgos: que el orden de las reglas del balanceador envíe las rutas de la API al contenedor de la SPA (se probaría con una batería de rutas tras cada cambio), y la caché de `index.html` tras un despliegue. La

CDN con dominio propio (opción B) se descartaría salvo que el rendimiento global lo exigiera, porque obligaría a configurar CORS en los dos servicios y a construir por entorno.

Respuestas · capítulo 16 ← Volver a las preguntas

1. Esquema, host y puerto. No: cambia el puerto.
2. Al usuario: impide que una web maliciosa lea, con las credenciales del usuario, datos de otra web en la que está autenticado.
3. Un `OPTIONS` automático que el navegador envía antes de ciertas peticiones entre orígenes (métodos distintos de GET/HEAD/POST, cabeceras o tipos de contenido no simples) para preguntar si están permitidas.
4. Porque el navegador solo habla con Vite (mismo origen) y Vite reenvía al backend desde Node, donde no aplica la política del mismo origen.
5. Proxy inverso con mismo origen (la recomendada), orígenes distintos con CORS en el backend, y la SPA servida por el propio backend.
6. Porque el JavaScript llama a `/api` de su propio origen y es el proxy de cada entorno quien sabe dónde están los servicios; no hay URLs del entorno incrustadas en el *build*.
7. `index.html`: `no-cache`. `assets/`: caché de larga duración e inmutable, porque sus nombres cambian con el contenido.
8. No. CORS solo controla qué páginas pueden leer las respuestas en un navegador; cualquier cliente fuera del navegador puede llamar a la API, y la protección es la autenticación y la autorización del backend.

A.17 Capítulo 17 · TanStack Query

↑ Índice

Solución 17.1 · Instala las herramientas de TanStack Query ← Volver al ejercicio

```
npm install -D @tanstack/react-query-devtools
```

`src/app/app.tsx`

```tsx
import { ReactQueryDevtools } from '@tanstack/react-query-devtools'
// …
<QueryClientProvider client={queryClient}>
  {/* … */}
  <ReactQueryDevtools initialIsOpen={false} />
</QueryClientProvider>
```

Aparece un botón flotante que abre el panel. Verás `["applications","list",{…}]` en verde (fresca) y, a los 30 segundos, en amarillo (obsoleta); al salir de la pantalla, en gris (inactiva). Al crear una aplicación, `["applications","detail",<id>]` aparece ya con datos sin haber hecho ninguna petición (`setQueryData`), y los listados pasan a obsoletos y se refrescan. Ten en cuenta que el paquete se instala como dependencia de desarrollo pero se importa desde `src`, así que Vite lo procesa; la librería devuelve un componente vacío en producción.

Solución 17.2 · Mide el efecto de `staleTime` ← Volver al ejercicio

Dentro de los 30 segundos, solo hay una petición al listado (la primera) y una al detalle: al volver, el listado sale de la caché sin petición. Pasados los 30 segundos, al volver se muestra al instante el listado en caché y se lanza una petición en segundo plano. Con `staleTime: 0`, cada vuelta al listado provoca una petición (aunque la pantalla se pinta igualmente al instante con los datos en caché). El compromiso: un `staleTime` alto reduce peticiones y carga del backend a costa de mostrar datos de hasta ese tiempo de antigüedad; uno bajo maximiza la frescura con más tráfico. Para datos de gestión que cambian con poca frecuencia, 30 segundos es un buen punto medio, y las invalidaciones tras escribir garantizan que lo que cambia el propio usuario se ve al momento.

Solución 17.3 · Precarga el detalle al pasar el ratón

← Volver al ejercicio

`src/features/applications/components/applications-table.tsx (idea)`

```
const queryClient = useQueryClient()
const prefetchDetail = (id: number) =>
  queryClient.prefetchQuery({
    queryKey: applicationKeys.detail(id),
    queryFn: ({ signal }) => applicationsApi.getById(id, signal),
  })
// …
<div onMouseEnter={() => prefetchDetail(app.id)}>
  <RowAction to={`/aplicaciones/${app.id}`} label={`Ver detalle de ${app.name}`}><Eye /></RowA
ction>
</div>
```

`prefetchQuery` respeta el `staleTime`: si el detalle ya está fresco, no hace nada. Importar `applicationKeys` y `applicationsApi` desde un componente del propio módulo es correcto; para no acoplar la tabla a la caché, lo más limpio sería exponer un hook `usePrefetchApplication()` en `applications-queries.ts`. No compensa cuando la petición es cara, cuando hay muchas filas y el usuario recorre la tabla con el ratón (decenas de peticiones inútiles), o en dispositivos táctiles, donde no existe el evento de pasar el ratón.

Respuestas · capítulo 17

← Volver a las preguntas

1. No es del cliente (solo hay una copia), es asíncrono, se comparte entre pantallas y queda obsoleto cuando cambia en el servidor.
2. `isPending`: no hay datos que mostrar; `isFetching`: hay una petición en curso. El listado muestra el esqueleto con el primero y, con el segundo, atenúa la tabla y desactiva la paginación.
3. Fresca: se sirve sin pedir durante `staleTime`. Obsoleta: se sirve y se refresca al volver a usarse. Inactiva: sin observadores, se conserva durante `gcTime` y después se elimina.
4. Para no repetir peticiones al navegar entre pantallas y para evitar refrescos y parpadeos inesperados al volver a la pestaña.
5. Los 4xx no cambian por repetirlos; las mutaciones podrían duplicar escrituras.
6. Todas las consultas cuya clave empieza por `['applications', 'list']`: todos los listados con cualquier parámetro y el catálogo de opciones, cuya clave cuelga de ese prefijo.
7. `setQueryData` guarda la respuesta como detalle (disponible al instante) e `invalidateQueries` marca los listados como obsoletos para que se refresquen. Insertar a mano exigiría replicar filtros, orden y paginación del servidor.
8. Cada respuesta se guarda en la entrada de su propia clave, y la pantalla solo observa la clave actual; además, la petición que ya no interesa se cancela con su `signal`.

A.18 Capítulo 18 · React Router

↑ Índice

Solución 18.1 · Añade una ruta de informes

← Volver al ejercicio

`src/features/applications/pages/applications-report-page.tsx`

```tsx
import { PageHeader } from '@/components/common/page-header'

export function ApplicationsReportPage() {
  return <PageHeader eyebrow="Aplicaciones" title="Informe de aplicaciones" description="Próxi
mamente." />
}
```

`src/features/applications/routes.ts (nueva hija)`

```ts
{
  path: 'informe',
  handle: { title: 'Informe de aplicaciones' },
  lazy: () =>
    import('./pages/applications-report-page').then((m) => ({ Component: m.ApplicationsReportP
age })),
},
```

Al visitar `/aplicaciones/informe`, la pestaña muestra «Informe de aplicaciones · IT Service Manager» y en *Network* aparece un *chunk* nuevo (en desarrollo, el módulo de la página). No se confunde con `:id` y **da igual dónde la declares**: React Router puntúa las rutas y un segmento estático gana a uno dinámico, sea cual sea el orden del array.

Solución 18.2 · Migas de pan automáticas

← Volver al ejercicio

`src/components/layout/breadcrumbs.tsx`

```tsx
import { Link, useMatches } from 'react-router'

export function Breadcrumbs() {
  const crumbs = useMatches()
    .map((m) => ({ pathname: m.pathname, crumb: (m.handle as { crumb?: string } |
undefined)?.crumb }))
    .filter((m): m is { pathname: string; crumb: string } => Boolean(m.crumb))

  return (
    <nav aria-label="Migas de pan" className="mb-2 text-xs text-muted-foreground">
      {crumbs.map((c, i) => (
        <span key={c.pathname}>
          {i > 0 && ' / '}
          {i < crumbs.length - 1 ? <Link to={c.pathname} className="hover:underline">{c.crumb}
</Link> : c.crumb}
        </span>
      ))}
    </nav>
  )
}
```

El filtro usa una guarda de tipo para que TypeScript sepa que `crumb` ya no es `undefined`. El último elemento no es enlace (es la página actual). Para el nombre de la aplicación, el layout no tiene sus datos: los tiene la página. Opciones: que la miga lea el detalle de la caché con `useQueryClient().getQueryData(applicationKeys.detail(id))` usando el parámetro de la ruta, o que el `hand-`

`le.crumb` sea una función que reciba los parámetros (`crumb: (params) => …`). La primera es sencilla porque el detalle ya está en caché cuando se pinta la página.

Solución 18.3 · Navega desde código con estado ← Volver al ejercicio

```
const navigate = useNavigate()
// en onSubmit, tras mutateAsync:
toast.success('Aplicación creada', { description: `«${result.name}» con identificador #${resul
t.id}.` })
navigate(`/aplicaciones/${result.id}`)
```

La ficha aparece **sin estado de carga**, porque `useCreateApplication` ya guardó la aplicación creada en `applicationKeys.detail(id)` con `setQueryData`: `useApplication(id)` encuentra los datos frescos en la caché. Sobre la experiencia: ir al detalle es más directo si lo habitual es crear una aplicación y trabajar con ella; la tarjeta de resultado original es mejor si se crean varias seguidas (*Crear otra*). Una opción intermedia es navegar al detalle y ofrecer en la notificación una acción «Crear otra».

Respuestas · capítulo 18 ← Volver a las preguntas

1. Declarativo, de datos y *framework*. El proyecto usa el modo de datos, sin cargadores.
2. Para agrupar rutas bajo un elemento común, aquí un `errorElement`, sin consumir ningún segmento de la URL.
3. Marca dónde se pinta la ruta hija activa; lo usa `AppLayout` dentro de `<main>`.
4. Porque React Router puntúa las rutas y un segmento estático (`nueva`) vale más que uno dinámico (`:id`).
5. Porque la URL la puede escribir cualquiera y los parámetros llegan como texto. `parseEntityId` solo acepta enteros seguros positivos y devuelve `undefined` en otro caso, evitando peticiones inútiles.
6. `Link` para enlaces que pulsa el usuario; `NavLink` para enlaces que deben saber si están activos (menú); `useNavigate` para navegar desde código tras una acción; `<Navigate />` para redirigir al renderizar.
7. Sin `replace` se añade una entrada al historial; con `replace` se sustituye la actual, de modo que *atrás* no vuelve a ella.
8. En la carga inicial, `hydrateFallbackElement` (el `LoadingState` a pantalla completa); en una navegación posterior, mantiene la pantalla actual hasta que la nueva está lista.
9. URL sin ruta (ruta comodín → `NotFoundPage`), recurso inexistente o `id` inválido (la página → `ResourceNotFound`) y error de renderizado (`errorElement` → `RouteErrorPage`).

A.19 Capítulo 19 · El estado del listado en la URL

↑ Índice

Solución 19.1 · Añade un filtro de texto a la URL ← Volver al ejercicio

Se añadiría `search?: string` a `ApplicationListParams`; en `parseListParams`, `search: search.get('buscar')?.trim() || undefined` (quizá con una longitud máxima); en `serializeListParams`, `if (params.search) search.set('buscar', params.search)`; y en `toApiQuery`, `name: params.search`. Las pruebas: lectura, omisión cuando está vacío y reversibilidad.

No conviene que cada pulsación cree una entrada en el historial ni una petición. Un ***debounce*** retrasa la acción hasta que el usuario deja de escribir un tiempo (por ejemplo, 300 ms). Una forma de hacerlo: el campo

de texto tiene su propio `useState` (valor inmediato, para que escribir sea fluido) y un efecto con `setTimeout` que, pasado ese tiempo sin cambios, llama a `updateParams({ search: valor, page: 0 }, { replace: true })`, con su limpieza (`clearTimeout`). El `replace` evita llenar el historial con cada palabra a medio escribir.

Solución 19.2 · Escribe una prueba que falle

← Volver al ejercicio

`src/features/demands/model/list-params.test.ts (prueba añadida)`

```
it('descarta campos de ordenación que el backend no admite', () => {
  const params = parseDemandListParams(new URLSearchParams('orden=createdAt,asc'))
  expect(params.sortField).toBe(DEFAULT_DEMAND_LIST_PARAMS.sortField)
})
```

Con el código actual **falla**: `createdAt` está en `DEMAND_SORT_FIELDS`, así que `parseSortParam` lo acepta como válido y el listado acabaría pidiendo un orden que el backend rechaza con un 400. Al quitar `createdAt` (y `application.name`) de `DEMAND_SORT_FIELDS`, la validación lo descarta y la prueba pasa. Es un ejemplo de prueba que documenta un contrato con el backend: si la lista vuelve a divergir, la prueba lo dirá.

Solución 19.3 · Extrae un hook genérico

← Volver al ejercicio

`src/hooks/use-url-list-state.ts`

```
import { useCallback, useMemo } from 'react'
import { useSearchParams } from 'react-router'

export function useUrlListState<P extends object>(
  parse: (search: URLSearchParams) => P,
  serialize: (params: P) => URLSearchParams,
) {
  const [searchParams, setSearchParams] = useSearchParams()
  const params = useMemo(() => parse(searchParams), [parse, searchParams])
  const updateParams = useCallback(
    (patch: Partial<P>, options?: { replace?: boolean }) =>
      setSearchParams(serialize({ ...params, ...patch }), options),
    [params, serialize, setSearchParams],
  )
  return { params, updateParams }
}
```

En la página: `const { params, updateParams } = useUrlListState(parseListParams, serializeListParams)`. Funciona sin más precauciones porque `parseListParams` y `serializeListParams` son funciones de módulo, con referencia estable; si se pasaran funciones creadas en el componente, la memorización dejaría de servir. El efecto de corrección de página puede ir en otro hook, `useClampPage(totalPages, page, updateParams)`, o quedarse en la página: depende de los datos de la consulta (`totalPages`), que el hook de la URL no conoce, y mezclar ambas cosas lo acoplaría a TanStack Query.

Respuestas · capítulo 19

← Volver a las preguntas

1. Recargar la página, compartir el enlace, guardarlo en marcadores y volver con *atrás* desde un detalle conservando filtros y página (además de recorrer las páginas con el historial).
2. Porque los códigos son estables e inequívocos; las etiquetas pueden cambiar o traducirse y romperían los enlaces guardados.
3. Para que la URL del listado sin cambios sea limpia (`/aplicaciones`) y estable.

4. Que la página no guarda ninguna copia de los parámetros: los calcula de la URL en cada renderizado y los cambia escribiendo en la URL.
5. Ambos valores se descartan al leer: se usan el tamaño por defecto (10) y ningún filtro de estado.
6. Porque cambian el conjunto de resultados o su orden, y la página actual deja de tener sentido (o de existir).
7. En la corrección automática de una página fuera de rango, porque no es una acción del usuario y volver a ella con *atrás* no tendría sentido.
8. Con un enlace a `/demandas?aplicacion=7` : el listado de demandas lee el parámetro como su filtro.
9. Que los valores por defecto producen una URL vacía y que serializar y volver a leer devuelve exactamente los mismos parámetros.

A.20 Capítulo 20 · React Hook Form y Zod

↑ Índice

Solución 20.1 · Añade un campo con validación

← Volver al ejercicio

`src/features/applications/model/schemas.ts (campo añadido)`

```
const ownerField = z
  .string()
  .trim()
  .email('Introduce un correo electrónico válido.')
  .or(z.literal(''))

export const applicationCreateSchema = z.object({
  name: nameField,
  description: descriptionField,
  criticality: criticalityField,
  owner: ownerField,
})
```

`OwnerField` sería un `TextInputField` con `name="owner"` , etiqueta «Responsable» y descripción «Opcional.» (y `maxLength` , por ejemplo 254). Pruebas: vacío válido, correo válido, `'no-es-un-correo'` inválido. Hay que añadir `owner: ''` a `EMPTY_FORM` y a `toFormValues` .

Si se enviara en el DTO, depende de la configuración de Jackson en el backend: por defecto, Spring Boot **ignora** las propiedades desconocidas (`FAIL_ON_UNKNOWN_PROPERTIES` desactivado), así que el campo se perdería en silencio y el usuario creería haberlo guardado. Si estuviera activado, la petición fallaría con un 400. En ambos casos, enviar campos que el contrato no define es un error.

Solución 20.2 · Valida al salir del campo

← Volver al ejercicio

- **`onSubmit`** : no molesta mientras se rellena el formulario por primera vez; los errores aparecen todos al enviar. Ideal para formularios cortos como estos.
- **`onTouched`** : valida cada campo al salir de él y, después, al cambiar; da información temprana sin interrumpir mientras se escribe, pero puede mostrar errores en campos que el usuario solo ha atravesado con el tabulador.
- **`onChange`** : valida en cada pulsación; da la respuesta más inmediata, pero muestra errores mientras el usuario aún está escribiendo (por ejemplo, «correo no válido» tras la primera letra) y provoca más renderizados.

Solución 20.3 · Simula un error de servidor por campo ← Volver al ejercicio

Con el error simulado, el mensaje «ya existe una aplicación con ese nombre» aparece bajo el campo *Nombre*, con el campo marcado como inválido, y la alerta general muestra «La petición contiene campos inválidos». `applyServerFieldErrors` solo actúa porque `kind` es `validation` y `name` está en la lista de campos conocidos; si el error viniera para un campo que no existe en el formulario (por ejemplo, `code`), no se asignaría a ningún campo y solo quedaría la alerta. En cuanto el usuario modifica el nombre, React Hook Form revalida el campo con el esquema y el error de servidor desaparece.

Respuestas · capítulo 20 ← Volver a las preguntas

1. Zod describe las reglas, valida y transforma los datos y deriva sus tipos; React Hook Form gestiona valores, errores, estado de modificación, envío y reinicio.
2. `z.string().trim().min(1, '…')`. `trim()` es necesario para que una cadena de solo espacios cuente como vacía, igual que en `@NotBlank`.
3. `z.input` es el tipo que maneja el formulario antes de validar; `z.output`, el que se obtiene tras validar y transformar. `applicationId` entra como `string` (el desplegable) y sale como `number` (el DTO).
4. Porque guarda el estado en un almacén propio y solo renderiza los componentes suscritos a lo que cambia (cada campo, el contador con `useWatch`).
5. Valida al enviar y, tras el primer intento, al cambiar cada campo. Se eligió para no mostrar errores antes de que el usuario haya intentado enviar.
6. Porque no es un `<select>` nativo: hay que conectar `value`, `onValueChange`, `onBlur` y `ref` por separado.
7. `null` en la API se convierte en `''` al cargar (`?? ''`), y una descripción vacía o de espacios se envía como `null` (`normalizeDescription`).
8. Para que el formulario refleje lo que realmente quedó guardado y para que esos valores sean la nueva referencia de `isDirty`.
9. Para añadir una validación que depende del estado actual de la demanda: rechazar las transiciones no permitidas con un error en el campo `status`.

A.21 Capítulo 21 · El módulo de aplicaciones de principio a fin

↑ Índice

Solución 21.1 · Traza una sesión completa ← Volver al ejercicio

Una traza típica: `GET list` (página 1) → al filtrar, `GET list?status=INACTIVE` → página 2, `GET list?status=INACTIVE&page=1` → detalle, `GET /api/applications/7` → *Modificar*: **ninguna petición** si no han pasado 30 segundos (el detalle está en caché y fresco) → guardar: `PUT /api/applications/7` y, como no hay ningún listado en pantalla, ningún `GET` inmediato: los listados se marcan como obsoletos → *Volver al listado*: se muestra al instante la versión en caché de `/aplicaciones` y se refresca en segundo plano (`GET list`), porque fue invalidada. Si la aplicación modificada dejara de cumplir el filtro, desaparecería de la lista tras ese refresco.

Solución 21.2 · Mejora la confirmación del alta ← Volver al ejercicio

`src/features/applications/pages/application-create-page.tsx (en CreatedResult)`

```
<Button asChild variant="outline">
  <Link to={`/demandas/nueva?aplicacion=${application.id}`}>
    <ClipboardList /> Crear demanda para esta aplicación
```

```
  </Link>
</Button>
```

Lo hace posible el parámetro `aplicacion` de la URL, que lee `DemandCreatePage` con `parseIdParam(searchParams.get('aplicacion'))` para inicializar el formulario con la aplicación preseleccionada. El desplegable encontrará la nueva aplicación en el catálogo porque la invalidación de `lists()` tras el alta refresca también la clave de opciones.

Solución 21.3 · Avisa de los cambios sin guardar

← Volver al ejercicio

`src/features/applications/pages/application-edit-page.tsx (idea)`

```
const blocker = useBlocker(
  ({ currentLocation, nextLocation }) => isDirty && !mutation.isPending && currentLocation.pathname !== nextLocation.pathname,
)

useEffect(() => {
  if (!isDirty) return
  const onBeforeUnload = (e: BeforeUnloadEvent) => e.preventDefault()
  window.addEventListener('beforeunload', onBeforeUnload)
  return () => window.removeEventListener('beforeunload', onBeforeUnload)
}, [isDirty])

// Si blocker.state === 'blocked', se muestra un diálogo con blocker.proceed() y blocker.reset()
```

`useBlocker` recibe una función que decide si bloquear cada navegación interna; cuando bloquea, `blocker.state` vale `'blocked'` y se puede mostrar un diálogo (el del ejercicio 11.1) que llame a `proceed()` o `reset()`. `beforeunload` cubre el cierre de la pestaña o la recarga, donde el navegador muestra su propio aviso genérico. No debe bloquearse **justo después de guardar**: como `form.reset(result)` deja `isDirty` en `false`, la navegación posterior ya no se bloquea; y tampoco mientras se está guardando, para no interferir con una navegación tras el éxito.

Respuestas · capítulo 21

← Volver a las preguntas

1. `/aplicaciones` (GET listado), `/aplicaciones/nueva` (POST), `/aplicaciones/modificar` (GET detalle y GET listado), `/aplicaciones/:id` (GET detalle) y `/aplicaciones/:id/editar` (GET y PUT).
2. El router carga la página con `lazy`, la página interpreta la URL y llama a `useApplications`, TanStack Query detecta que no hay datos y ejecuta la `queryFn`, que usa el cliente HTTP; la respuesta vuelve a la caché y la página se renderiza con los datos.
3. Porque la URL anterior produce los mismos parámetros, es decir, la misma clave de caché, y sus datos siguen frescos o, al menos, disponibles.
4. `handleSubmit` valida con Zod, muestra «Selecciona una criticidad.» bajo el campo, pone el foco en él y no hace ninguna petición.
5. Porque la mutación de alta guardó la respuesta en la clave del detalle con `setQueryData`.
6. Buscar por identificador (estado local del campo y `searchedId`, que activa una consulta dependiente) y seleccionar del catálogo (parámetros del listado en `useState`).
7. Porque el contrato del backend (`ApplicationUpdateRequest`) define la modificación completa del recurso con todos sus campos validados.
8. No: cada pestaña tiene su propia caché. Los verá cuando los datos se consideren obsoletos y se refresquen (a partir de 30 segundos, al volver a usarlos).

9. La ordenación alfabética de la criticidad (se guarda como texto) y la ausencia de búsqueda por nombre.

A.22 Capítulo 22 · El módulo de demandas

↑ Índice

Solución 22.1 · Reproduce un conflicto de transición

← Volver al ejercicio

En la segunda pestaña, que cree que la demanda está *Abierta*, *Completada* aparece desactivada: `OPEN → COMPLETED` no es una transición válida y el desplegable no la permite (capa 1). En el segundo experimento, la segunda pestaña envía *Cancelada* creyendo que el estado es `OPEN`: para el cliente, una transición válida. El servidor, que tiene la demanda en `COMPLETED`, responde **409** con `message: 'Invalid demand status transition: COMPLETED -> CANCELLED'`. El usuario ve «Cambio rechazado por el servidor · El servidor no permite pasar de «Completada» a «Cancelada». Puede que la demanda haya cambiado mientras la editabas.» Al pulsar *Recargar datos actuales*, se pide de nuevo la demanda, el formulario muestra el estado *Completada*, el desplegable queda desactivado (estado final) y desaparece la alerta.

Solución 22.2 · Corrige el desajuste de ordenación

← Volver al ejercicio

`src/features/demands/model/types.ts`

```ts
export const DEMAND_SORT_FIELDS = ['id', 'title', 'priority', 'status', 'updatedAt'] as const
```

`npm run typecheck` señala `demands-table.tsx`, donde `SortableTableHead field="application.name"` y `field="createdAt"` ya no son valores de `DemandSortField`. Esas columnas pasan a `TableHead` normales. Hay que revisar también las pruebas de `list-params.test.ts` que usaran esos campos, y añadir la del ejercicio 19.2. Con el cambio, `?orden=createdAt,asc` se descarta al leer la URL y se usa el orden por defecto (`id` descendente). Conviene recoger en las notas de integración la pregunta abierta para el backend: ¿se vuelve a admitir `createdAt`?

Solución 22.3 · Muestra las transiciones como acciones

← Volver al ejercicio

`src/features/demands/components/status-actions.tsx (idea)`

```tsx
const ACTION_LABELS: Partial<Record<DemandStatus, string>> = {
  IN_PROGRESS: 'Iniciar', COMPLETED: 'Completar', CANCELLED: 'Cancelar',
}

export function StatusActions({ demand }: { demand: Demand }) {
  const mutation = useUpdateDemand(demand.id)
  const change = async (status: DemandStatus) => {
    if (isFinalStatus(status) && !window.confirm(`¿Seguro? «${DEMAND_STATUS_LABELS[status]}» es un estado final.`)) return
    try {
      await mutation.mutateAsync({
        title: demand.title, description: demand.description, priority: demand.priority,
        applicationId: demand.applicationId, status,
      })
      toast.success('Estado actualizado')
    } catch {
      // el error se muestra a partir de mutation.error
    }
  }
  return (
    <div className="flex gap-2">
      {getNextStatuses(demand.status).map((s) => (
```

```
          <Button key={s} size="sm" variant={s === 'CANCELLED' ? 'outline' : 'default'}
                  disabled={mutation.isPending} onClick={() => change(s)}>
            {ACTION_LABELS[s]}
          </Button>
        ))}
      </div>
    )
  }
```

`window.confirm` sirve para una prueba rápida; en la aplicación debería usarse el diálogo de shadcn/ui (ejercicio 11.1). Con un 409, lo adecuado es mostrar el mensaje de `describeConflict` y refrescar la demanda (`refetch` de `useDemand` o invalidar su clave), porque la ficha está mostrando un estado obsoleto. Como la mutación ya actualiza el detalle con `setQueryData` en caso de éxito, la ficha y `StatusFlow` cambian solos al nuevo estado.

Respuestas · capítulo 22 ← Volver a las preguntas

1. Un ciclo de vida con transiciones restringidas, la pertenencia a una aplicación de otro servicio y las fechas de creación y modificación.
2. En `Demand.changeStatus()` del backend; su reflejo en `features/demands/model/status-rules.ts` .
3. Porque es exhaustiva (un estado nuevo obliga a declarar sus transiciones), fácil de comparar con el backend y de la que se derivan todas las preguntas de la interfaz.
4. El desplegable (prevenir: solo habilita lo permitido), el esquema de Zod (comprobar en el cliente antes de enviar) y el backend (autoridad final, que responde 409 y la interfaz explica con opción de recargar).
5. Porque el cliente decide con los datos que cargó, y otro usuario puede haber cambiado el estado entretanto.
6. Vuelve a pedir la demanda, reinicia el formulario con los datos reales, limpia el error de la mutación y recalcula las transiciones permitidas.
7. Para garantizar que la aplicación asociada a la demanda que se edita aparezca en el desplegable aunque no esté entre las mil primeras del catálogo.
8. Porque un instante en UTC no depende de la zona del servidor ni del usuario; la zona solo se aplica al mostrar, con la del navegador.
9. El frontend ofrece ordenar por `application.name` y `createdAt` , que el backend rechaza con un 400. Su origen es la separación de las aplicaciones en otro servicio, que impidió ordenar por su nombre, sin actualizar la lista del frontend.

A.23 Capítulo 23 · Pruebas con Vitest

↑ Índice

Solución 23.1 · Convierte una prueba en tabla ← Volver al ejercicio

`src/lib/list-params.test.ts`

```
it.each([
  ['3', 2], ['1', 0], ['0', 0], [null, 0], ['-1', 0], ['abc', 0], ['2.5', 1], ['', 0],
])('parsePageParam(%j) → %i', (input, expected) => {
  expect(parsePageParam(input)).toBe(expected)
})
```

`parsePageParam('2.5')` devuelve **1**: `Number.parseInt('2.5', 10)` lee los dígitos iniciales y se detiene en el punto, devolviendo 2, que como página 1-based corresponde a la 1 en base 0. Es un comportamiento to-

lerante y aceptable para una URL (`?pagina=2.5` muestra la página 2); si se quisiera rechazarlo, habría que validar con una expresión regular `^\d+$`, como hace `parseEntityId`. El formato `%j` imprime los valores como JSON, de modo que `null` y `''` se distinguen en el nombre de la prueba.

Solución 23.2 · Prueba `formatDateTime`

← Volver al ejercicio

`src/lib/format.test.ts`

```ts
import { describe, expect, it } from 'vitest'
import { formatDateTime } from './format'

describe('formatDateTime', () => {
  it('muestra una raya si no hay fecha', () => {
    expect(formatDateTime(null)).toBe('—')
    expect(formatDateTime(undefined)).toBe('—')
  })

  it('devuelve el texto original si no es una fecha válida', () => {
    expect(formatDateTime('no-es-fecha')).toBe('no-es-fecha')
  })

  it('formatea un instante en hora local', () => {
    expect(formatDateTime('2026-10-24T08:15:30Z')).toBe('24/10/26, 10:15')
  })
})
```

La última prueba depende de la zona horaria: en Madrid en octubre (UTC+2) da `10:15`; en una máquina en UTC daría `08:15`. Para que sea estable, fija la zona en la configuración de Vitest (`test: { env: { TZ: 'Europe/Madrid' } }`) o en el script (`TZ=Europe/Madrid vitest run`). El formato exacto también depende de los datos de idioma de la versión de Node, así que para pruebas muy estrictas se puede comparar con el resultado de un `Intl.DateTimeFormat` construido en la propia prueba.

Solución 23.3 · Tu primera prueba de componente

← Volver al ejercicio

`src/features/demands/components/status-flow.test.tsx`

```tsx
// @vitest-environment jsdom
import { render, screen } from '@testing-library/react'
import '@testing-library/jest-dom/vitest'
import { describe, expect, it } from 'vitest'
import { StatusFlow } from './status-flow'

describe('StatusFlow', () => {
  it('indica las transiciones posibles desde Abierta', () => {
    render(<StatusFlow current="OPEN" />)
    expect(screen.getByText(/Desde «Abierta» puede pasar a «En curso» o «Cancelada»/)).toBeInT
heDocument()
    expect(screen.getByText('Abierta')).toHaveAttribute('aria-current', 'step')
  })
})
```

`@testing-library/jest-dom/vitest` registra los *matchers* adicionales (`toBeInTheDocument`, `toHaveAttribute`) en Vitest. Para que TypeScript los conozca en todas las pruebas, lo habitual es importarlo en un fichero de preparación (`setupFiles` en la configuración de Vitest) en lugar de en cada prueba. Si ESLint se queja del fichero `.tsx` con React Refresh, es porque la regla solo debería aplicarse a componentes, no a pruebas: se puede excluir `**/*.test.tsx` de esa regla.

Respuestas · capítulo 23 ← Volver a las preguntas

1. Unitario, componente, integración y extremo a extremo. El proyecto cubre el nivel unitario.
2. Porque esas funciones concentran las reglas que más fácilmente se rompen sin notarse, y sus pruebas son rápidas, estables y sin infraestructura.
3. No hay que configurar aparte TypeScript, el alias `@/` ni los módulos: las pruebas se transforman igual que la aplicación.
4. `toBe` compara con `Object.is` (primitivos o misma referencia); `toEqual` compara el contenido de objetos y arrays.
5. Una prueba que fija lo que el frontend espera del backend; por ejemplo, las transiciones copiadas de `Demand.changeStatus()`, el texto del 409 y los formatos de error de `api-client.test.ts`.
6. Porque recibe un `Response` en lugar de hacer la petición, de modo que se prueba con objetos `Response` construidos en la prueba, sin red.
7. Por rol accesible, etiqueta o texto, como lo haría un usuario; así las pruebas no dependen de clases ni de la estructura del DOM.
8. No: la norma se refiere a la aplicación, que siempre usa la API real. En pruebas automáticas, simular respuestas es legítimo para cubrir casos difíciles de reproducir.
9. `typecheck`, `lint`, `test` y `build`: de lo más barato y rápido a lo más caro, para fallar cuanto antes.

A.24 Capítulo 24 · Registro de decisiones de arquitectura

↑ Índice

Solución 24.1 · Escribe un ADR sobre la ordenación de demandas ← Volver al ejercicio

`docs/adr/0024-campos-ordenacion-demandas.md`

```
# ADR-024: Los campos de ordenación de demandas siguen la lista del backend

- Estado: propuesta
- Fecha: 2026-10-24

## Contexto
Al extraer las aplicaciones a un servicio propio, la entidad Demand solo guarda
applicationId. DemandService valida sort contra VALID_SORT_FIELDS
(id, title, priority, status, updatedAt). El frontend ofrece además
application.name y createdAt, que producen un 400.

## Decisión
El frontend solo ofrece los campos que el backend valida. Las columnas
Aplicación y Creación se muestran sin ordenación. El backend estudiará
admitir createdAt (columna propia, sin coste).

## Consecuencias
- Desaparece el error 400 y las URL antiguas con esos órdenes caen al orden por defecto.
- Se pierde la ordenación por nombre de aplicación mientras no exista una
  copia del nombre en la demanda.
- Una prueba de list-params fija la lista para detectar futuros desajustes.

## Alternativas consideradas
- Guardar applicationName en Demand y mantenerlo sincronizado: permite ordenar,
  pero exige sincronización entre servicios (eventos o refresco periódico).
- Ordenar en el frontend: incorrecto, solo ordenaría la página recibida.
- Ordenar en el servicio de demandas cargando todo el catálogo: no escala.
```

Solución 24.2 · Evalúa una decisión con otro contexto ← Volver al ejercicio

Tres revisiones razonables: **ADR-001** (SPA pura): con SEO y primera carga críticas, convendría renderizado en servidor o generación estática para las páginas públicas (un *framework* como React Router en modo *framework* o Next.js), manteniendo la SPA para las zonas privadas. **ADR-011** (mismo origen): con miles de usuarios, servir los estáticos desde una CDN con dominio propio cobra sentido, lo que obligaría a configurar CORS o a usar la CDN también como proxy de la API. **ADR-012** (TanStack Query, `staleTime` de 30 s): con datos públicos y mucha concurrencia, revisar las políticas de caché, añadir caché HTTP en la API y quizá actualización en tiempo real. También habría que revisar ADR-020 (autenticación de clientes externos) y reforzar las pruebas (ADR-019, ADR-021).

Solución 24.3 · Crea el registro en el repositorio ← Volver al ejercicio

La estructura esperada:

```
docs/adr/
├── README.md                       ← formato, estados y cómo proponer uno nuevo
├── 0001-spa-con-vite.md
├── 0012-tanstack-query.md
└── 0016-estado-listados-en-url.md
```

El `README.md` debería explicar la plantilla, los estados posibles, la numeración (correlativa, nunca se reutiliza un número) y que los ADR sustituidos no se borran. En el `README.md` del proyecto basta una línea en la sección de arquitectura: «Las decisiones de arquitectura están en `docs/adr/`.».

Respuestas · capítulo 24 ← Volver a las preguntas

1. Título con número, estado, contexto, decisión y consecuencias.
2. No se modifica: se escribe uno nuevo que lo sustituye y se marca el antiguo como «sustituido por ADR-NNN».
3. Que sea difícil de revertir, que afecte a varios módulos o a la forma de trabajar, y que se hayan descartado alternativas razonables.
4. ADR-012 (TanStack Query para todo el estado del servidor).
5. La desincronización manual con el backend: los campos de ordenación de demandas.
6. Porque responden a la pregunta «¿y por qué no X?» que alguien volverá a hacer, y evitan repetir debates ya resueltos.

A.25 Capítulo 25 · Notas de integración con el backend

↑ Índice

Solución 25.1 · Prioriza con tu criterio ← Volver al ejercicio

Una propuesta (hay otras igualmente válidas):

- **Iteración 1 · Corregir lo que falla y lo trivial.** Frontend: nota 1 (alinear ordenación con prueba), nota 12 (`.env.example`). Backend: decidir si se admite `createdAt` (nota 1) y manejador genérico de excepciones (nota 9). Sin dependencias entre equipos salvo la decisión de `createdAt`.
- **Iteración 2 · Contrato de errores y concurrencia.** Backend: códigos estables y detalles (notas 2 y 4), `@Version` (nota 3). Frontend, después: adaptar `toApiError` a ambos formatos y generalizar el patrón

de conflicto con recarga. Dependencia: el frontend necesita el nuevo formato desplegado (o compatible hacia atrás).

- **Iteración 3 · Escalabilidad y contrato automatizado.** Backend: orden lógico de enumerados (nota 5), búsqueda por nombre (nota 6), springdoc-openapi (nota 8). Frontend: buscador con *debounce*, selector con búsqueda (nota 7) y generación de tipos. Dependencia: la búsqueda del backend antes que el selector.

Las notas 10, 11 y 13 se planifican según convenga: la 13 cuando se prepare el primer despliegue.

Solución 25.2 · Diseña el contrato de errores

← Volver al ejercicio

`ApiErrorResponse v2 (propuesta)`

```
public record ApiErrorResponse(
        int status,
        String error,                    // código estable: VALIDATION_ERROR,
APPLICATION_NOT_FOUND, INVALID_STATUS_TRANSITION…
        String message,                  // texto para personas, en español
        Map<String, String> fieldErrors,
        Map<String, Object> details      // datos estructurados: { "from": "COMPLETED", "to":
"CANCELLED" }
) { }
```

`src/lib/api-client.ts (tipo ampliado)`

```
export interface ApiErrorBody {
  status: number
  error: string
  message: string | null
  fieldErrors?: Record<string, string>
  details?: Record<string, unknown>
}
```

`ApiError` añadiría un campo `details`, copiado en `toApiError` si existe. La interpretación del 409 quedaría compatible con ambos formatos:

```
export function getTransition(error: ApiError): { from: DemandStatus; to: DemandStatus } |
undefined {
  if (error.code === 'INVALID_STATUS_TRANSITION' && error.details) {
    const { from, to } = error.details as { from?: string; to?: string }
    if (isDemandStatus(from) && isDemandStatus(to)) return { from, to }
  }
  return parseTransitionError(error.message)   // formato antiguo, mientras dure la transición
}
```

Con el diccionario de códigos (`APPLICATION_NOT_FOUND → 'La aplicación no existe.'`), `getErrorMessage` podría traducir por código y usar `message` solo como respaldo.

Solución 25.3 · Prueba el procedimiento

← Volver al ejercicio

Para ampliar el título a 200 caracteres: en el **backend**, `@Size(max = 200)` en los dos DTO y una migración de Flyway que amplíe la columna (`length = 150` en la entidad). En el **frontend**: primero la prueba de `schemas.test.ts` (cambiar `150/151` por `200/201`; falla); después `TITLE_MAX_LENGTH = 200` en `model/constants.ts`; la prueba pasa. Solo cambian **dos ficheros** del frontend gracias a la constante, que usan a la vez el esquema y el contador de caracteres. El compilador **no** lo habría detectado si te olvidaras: un nú-

mero no tiene tipo que lo relacione con el `@Size` del backend. Por eso la prueba de valores límite es la red de seguridad, y la generación desde OpenAPI (que puede incluir `maxLength`) la automatizaría.

Respuestas · capítulo 25 ← Volver a las preguntas

1. Los tipos de `model/types.ts`, las constantes de `constants.ts`, las listas de campos de ordenación, las reglas de `status-rules.ts` y la interpretación de errores de `api-client.ts`.
2. La separación de las aplicaciones en otro servicio, que eliminó la relación JPA y la posibilidad de ordenar por su nombre, sin actualizar la lista del frontend; en el fondo, que el contrato se replica a mano.
3. Porque extrae los estados de un texto con una expresión regular; si el texto cambia, deja de reconocerlo. Se resolvería con un código de error estable y los estados como campos estructurados.
4. Detectar modificaciones simultáneas comparando una versión del recurso; la alerta de conflicto con *Recargar datos actuales* de la edición de demandas ya está preparada.
5. Porque el código es estable y lo interpretan las máquinas (para traducir, para decidir), mientras que el mensaje puede cambiar de redacción o de idioma.
6. El servicio de demandas responde 503 al consultar, crear o modificar demandas, porque necesita el catálogo; el frontend lo muestra como error del servidor, con reintento en las consultas.
7. La consulta N+1 al resolver el nombre de la aplicación: ahora se consulta el catálogo una sola vez por página con `findByIds`.
8. Leer el cambio del backend, localizar su reflejo en el frontend, actualizar primero las pruebas, actualizar el código hasta que pasen (con `typecheck`), probar a mano contra el backend real y registrar el cambio (ADR o nota).

A.26 Capítulo 26 · Ejercicio guiado: el módulo de Incidencias

↑ Índice

Solución 26.1 · Pruebas de las reglas de incidencias ← Volver al ejercicio

`src/features/incidents/model/status-rules.test.ts`

```
import { describe, expect, it } from 'vitest'
import { canTransition, getSelectableStatuses, isFinalStatus } from './status-rules'
import { INCIDENT_STATUSES } from './types'

describe('reglas de transición de incidencias', () => {
  it('permite las transiciones del contrato', () => {
    expect(canTransition('OPEN', 'IN_PROGRESS')).toBe(true)
    expect(canTransition('OPEN', 'CLOSED')).toBe(true)
    expect(canTransition('IN_PROGRESS', 'RESOLVED')).toBe(true)
    expect(canTransition('RESOLVED', 'CLOSED')).toBe(true)
  })

  it('rechaza las no definidas', () => {
    expect(canTransition('IN_PROGRESS', 'CLOSED')).toBe(false)
    expect(canTransition('OPEN', 'RESOLVED')).toBe(false)
    expect(canTransition('CLOSED', 'OPEN')).toBe(false)
  })

  it('permite reabrir una incidencia resuelta', () => {
    expect(canTransition('RESOLVED', 'IN_PROGRESS')).toBe(true)
  })

  it('solo CLOSED es final', () => {
```

```
    expect(INCIDENT_STATUSES.filter(isFinalStatus)).toEqual(['CLOSED'])
  })

  it('desde RESOLVED se puede mantener, cerrar o reabrir', () => {
    expect(getSelectableStatuses('RESOLVED')).toEqual(['RESOLVED', 'CLOSED', 'IN_PROGRESS'])
  })
})
```

La cuarta prueba expresa la propiedad «solo `CLOSED` es final» de una vez, filtrando todos los estados con la propia función.

Solución 26.2 · Duración de la resolución

← Volver al ejercicio

`src/lib/format.ts`

```
/** Duración legible entre dos instantes ISO-8601: «3 h 25 min», «2 d 4 h», «12 min». */
export function formatDuration(fromIso: string, toIso: string): string {
  const ms = new Date(toIso).getTime() - new Date(fromIso).getTime()
  if (!Number.isFinite(ms) || ms < 0) return '—'
  const totalMinutes = Math.floor(ms / 60_000)
  const days = Math.floor(totalMinutes / 1440)
  const hours = Math.floor((totalMinutes % 1440) / 60)
  const minutes = totalMinutes % 60
  if (days > 0) return hours > 0 ? `${days} d ${hours} h` : `${days} d`
  if (hours > 0) return minutes > 0 ? `${hours} h ${minutes} min` : `${hours} h`
  return `${minutes} min`
}
```

`src/lib/format.test.ts (añadido)`

```
it.each([
  ['2026-10-24T08:00:00Z', '2026-10-24T08:12:00Z', '12 min'],
  ['2026-10-24T08:00:00Z', '2026-10-24T11:25:00Z', '3 h 25 min'],
  ['2026-10-22T08:00:00Z', '2026-10-24T12:00:00Z', '2 d 4 h'],
  ['2026-10-24T08:00:00Z', 'no-es-fecha', '—'],
])('formatDuration(%s, %s) → %s', (from, to, expected) => {
  expect(formatDuration(from, to)).toBe(expected)
})
```

No depende de la zona horaria porque opera con la diferencia entre dos instantes absolutos (milisegundos desde 1970 en UTC); la zona solo interviene al **mostrar** una fecha, no al restar dos. Ojo: es duración de reloj, no de horario laboral; un SLA en horas laborables necesitaría otra lógica (y probablemente calcularse en el backend).

Solución 26.3 · Mueve el catálogo de aplicaciones

← Volver al ejercicio

Pasos de la refactorización:

1. Mover `useApplicationSelectOptions` (y la interfaz `ApplicationOption`) a `src/features/applications/api/application-options.ts`, junto a `useApplicationOptions`, o a `applications-queries.ts`.
2. Mover el componente `ApplicationField` a `src/features/applications/components/application-select-field.tsx`, o dejarlo en cada módulo consumidor si sus textos difieren (por ejemplo, «antes de registrar demandas»). Una opción intermedia: el componente común recibe por prop el texto del caso vacío.

3. Actualizar las importaciones de `demand-filters.tsx` y `demand-form-fields.tsx` a `@/features/applications/…`.
4. Ejecutar `npm run typecheck` (detecta cualquier importación rota), `npm run lint` y `npm run test`, y probar el alta y la edición de una demanda.
5. Si existe la regla de ESLint del ejercicio 13.2, ajustarla para permitir que otros módulos importen solo estos ficheros de aplicaciones (lo ideal sería exponerlos en un `features/applications/index.ts` y prohibir el resto de rutas internas).

El resultado: demandas e incidencias dependen ambas de aplicaciones (relación de dominio real) y no entre sí.

Respuestas · capítulo 26 ← Volver a las preguntas

1. Porque el contrato manda: los tipos, constantes, reglas y validaciones del frontend reflejan el backend, y sin endpoint real el módulo no puede funcionar (no se usan datos simulados).
2. Una incidencia resuelta puede reabrirse (`RESOLVED → IN_PROGRESS`), es decir, un estado no final puede volver atrás; en `status-rules.ts` basta con incluir `IN_PROGRESS` entre los siguientes de `RESOLVED`.
3. La validación genérica de la URL, los campos de formulario, los indicadores, la paginación, los estados de carga y error, la cabecera de página y el cliente HTTP.
4. Porque crearía una dependencia entre dos módulos sin relación de dominio; el catálogo de aplicaciones pertenece al módulo de aplicaciones.
5. Registrar las rutas en `app/router.tsx`, activar el módulo en `navigation.ts` y añadir la regla del proxy en `vite.config.ts` (más, opcionalmente, el enlace desde el detalle de una aplicación).
6. Tipos, lint, pruebas, *build*, estados de cada pantalla, accesibilidad con teclado, visualización en móvil y documentación (ADR y notas).

ANEXO B

Glosario

↑ Volver al índice

Términos técnicos usados en la guía, en orden alfabético. Entre paréntesis, el capítulo donde se explican con más detalle.

B.1 A – D

↑ Índice

Término	Definición
ADR (*Architecture Decision Record*)	Documento breve que recoge una decisión de arquitectura, su contexto, alternativas y consecuencias (24).
`AbortSignal`	Señal que permite cancelar una petición `fetch` en curso; TanStack Query la crea por consulta (2, 17).
`ApiError`	Clase de error propia del proyecto que unifica cualquier fallo de la API con `kind`, `status`, mensaje y errores por campo (15).
`asChild`	Prop de los componentes de shadcn/ui que presta su comportamiento y estilos al único hijo, mediante `Slot` de Radix (9, 11).
`async` / `await`	Sintaxis para escribir código asíncrono con promesas como si fuera secuencial, sin bloquear el hilo (2).
Bucle de eventos (*event loop*)	Mecanismo por el que el hilo único de JavaScript atiende, una tras otra, las tareas pendientes (1).
Build	Proceso que transforma el código fuente en los ficheros optimizados de `dist/` (1, 4).
Bundle	Fichero JavaScript resultante de empaquetar muchos módulos; también, el conjunto de ficheros del *build* (1).
Caché de consultas	Almacén de TanStack Query que guarda las respuestas de la API por clave (17).
Callback	Función que se pasa a otra para que la llame cuando ocurra algo, como `onPageChange` (2, 6).
Children	Prop especial con el contenido escrito entre las etiquetas de un componente (6).
Chunk	Fragmento del *bundle* que se descarga por separado, por ejemplo, una página con carga diferida (1, 18).
Clave de consulta (*query key*)	Array que identifica una entrada de la caché de TanStack Query y permite invalidarla por prefijo (17).
`cn()`	Utilidad del proyecto que combina clases con `clsx` y resuelve conflictos de Tailwind con `tailwind-merge` (10).
Code splitting	División del código en *chunks* que se cargan bajo demanda (1).
Commit (de React)	Fase en la que React aplica al DOM los cambios calculados en el renderizado (7).

Término	Definición
CommonJS	Sistema de módulos antiguo de Node (`require` , `module.exports`) (3).
Componente	Función que recibe props y devuelve una descripción de la interfaz en JSX (6).
Componente controlado	Campo cuyo valor lo decide el estado de React mediante `value` y `onChange` (7).
Componentes compuestos	Familia de componentes pensados para usarse juntos, como `Card` , `CardHeader` , `CardContent` (9).
Concurrencia optimista	Técnica que detecta modificaciones simultáneas comparando una versión del recurso (22, 25).
Contexto (*context*)	Mecanismo de React para que un proveedor ofrezca un valor a todo un subárbol (9).
CORS	Mecanismo por el que un servidor autoriza a páginas de otro origen a leer sus respuestas (16).
`cva`	Librería *class-variance-authority*, que define variantes de clases para un componente (11).
Debounce	Retrasar una acción hasta que deja de repetirse durante un tiempo, por ejemplo al escribir en un buscador (19).
Desestructuración	Sintaxis que extrae propiedades o posiciones en variables: `const { id } = params` (2).
DOM	Árbol de objetos en memoria que representa la página; lo que se ve es su reflejo (1).
DTO	Objeto de transferencia de datos entre backend y frontend; en el frontend se describe con tipos de TypeScript (2, 14).

B.2 E – L

↑ Índice

Término	Definición
Efecto (`useEffect`)	Código que sincroniza el componente con un sistema externo tras el renderizado (8).
`errorElement`	Componente que React Router pinta si una ruta falla al renderizarse (9, 18).
ESLint	Herramienta de análisis estático que detecta patrones problemáticos en el código (5).
ESM (módulos ES)	Sistema de módulos estándar de JavaScript: `import` y `export` (2, 3).
Estado (*state*)	Datos que un componente recuerda entre renderizados y cuyo cambio provoca un nuevo renderizado (7).
Estado del servidor	Copia en el navegador de datos que pertenecen al backend; la gestiona TanStack Query (17).
Fallback a `index.html`	Regla del servidor web que devuelve `index.html` para cualquier ruta, necesaria en una SPA (1, 16).
Fast Refresh	Integración de React con la recarga en caliente que conserva el estado de los componentes al editar (4).

Término	Definición
`fetch`	API estándar del navegador para hacer peticiones HTTP; solo rechaza la promesa ante errores de red (2, 15).
Fragmento	`<>…</>`: agrupa varios elementos sin añadir un nodo al DOM (6).
Función flecha	Sintaxis concisa de función: `(x) => x * 2` (2).
Función pura	Función cuyo resultado depende solo de sus argumentos y no tiene efectos secundarios (6, 13).
Genérico	Tipo parametrizado, como `apiRequest<T>` o `FilterSelect<V>` (2).
Guarda de tipo	Función que devuelve `valor is Tipo` y estrecha el tipo en TypeScript (2).
Handle	Objeto de metadatos de una ruta de React Router; el proyecto lo usa para los títulos (18).
Headless	Componente sin estilos que solo aporta comportamiento y accesibilidad, como los de Radix (11).
HMR	*Hot Module Replacement*: sustitución en caliente de un módulo modificado sin recargar la página (4).
Hook	Función de React cuyo nombre empieza por `use` y que da acceso a estado, efectos, contexto… (8).
Hook personalizado	Hook propio que encapsula lógica reutilizable con otros hooks (8).
Inferencia de tipos	Capacidad de TypeScript de deducir tipos sin anotarlos (2).
Inmutabilidad	Tratar los datos como no modificables y crear copias con los cambios (7).
Invalidación	Marcar entradas de la caché como obsoletas para que se vuelvan a pedir (17).
JSX	Sintaxis parecida a HTML dentro de JavaScript que se transforma en llamadas a funciones (6).
`key`	Prop que identifica un elemento entre renderizados; en listas, y también para reiniciar un componente (6, 7).
`lazy`	Opción de una ruta que carga su componente bajo demanda (18).
Lockfile	`package-lock.json`: versiones exactas de todas las dependencias instaladas (3).

B.3 M – R

↑ Índice

Término	Definición
Matcher	Comprobación de una prueba, como `toBe` o `toEqual` (23).
Memorización	Guardar un valor o función entre renderizados con `useMemo` o `useCallback` (8).
Modo estricto (TypeScript)	Conjunto de comprobaciones de `strict: true`, entre ellas la de nulos (5).
Módulo (del proyecto)	Carpeta de `features/` con todo lo de un dominio de negocio (13).

Término	Definición
MSW	*Mock Service Worker*: librería que intercepta peticiones en pruebas (23).
Mutación	Escritura contra la API gestionada con `useMutation` (17).
Node.js	Entorno de ejecución de JavaScript fuera del navegador; en el proyecto, solo para herramientas (3).
npm	Gestor de paquetes de Node (3).
OKLCH	Espacio de color perceptualmente uniforme usado en los tokens (10).
Origen	Combinación de esquema, host y puerto; base de la política del mismo origen (16).
`Outlet`	Hueco donde una ruta padre pinta a su ruta hija activa (18).
Petición de verificación previa (*preflight*)	`OPTIONS` automático que el navegador envía antes de ciertas peticiones entre orígenes (16).
Placeholder data	Datos provisionales que muestra una consulta mientras carga una clave nueva (17).
Portal	Renderizado de un componente al final del `<body>` aunque pertenezca a otra parte del árbol (11).
Promesa	Objeto que representa un valor futuro; equivalente a `CompletableFuture` (2).
Propagación (*spread*)	Operador `...` que copia propiedades o elementos en un objeto o array nuevo (2).
Props	Datos que un componente recibe de su padre; son de solo lectura (6).
Proveedor (*provider*)	Componente que ofrece un valor de contexto a su subárbol (9).
Proxy de desarrollo	Función de Vite que reenvía `/api/*` al backend para evitar CORS (4, 16).
Proxy inverso	Servidor que recibe las peticiones y las reparte entre servicios (16).
Radix UI	Librería de componentes accesibles sin estilos (11).
React	Librería para construir interfaces a partir de componentes (6).
React Hook Form	Librería que gestiona el estado de los formularios (20).
React Router	Librería de navegación entre pantallas (18).
Reconciliación	Comparación entre el árbol de elementos anterior y el nuevo para calcular los cambios del DOM (7).
Record (TypeScript)	Tipo de objeto con claves de un tipo y valores de otro; con uniones, es exhaustivo (2).
Renderizar	En React, ejecutar los componentes para obtener la descripción de la interfaz (7).
Render props	Patrón en el que un componente recibe una función que devuelve JSX (9).
Resolver	Adaptador que conecta un esquema de validación con React Hook Form (`zodResolver`) (20).

Término	Definición
Rolldown	Empaquetador escrito en Rust que usa Vite 8 (4).
Ruta índice	Ruta que se pinta cuando la URL coincide exactamente con la de su padre (18).

B.4 S – Z

↑ Índice

Término	Definición
Semver	Versionado semántico `MAYOR.MENOR.PARCHE` (3).
shadcn/ui	Catálogo de componentes que se copian al proyecto, construidos con Radix y Tailwind (11).
Slot	Componente de Radix que fusiona sus props en su hijo; base de `asChild` (9, 11).
Source map	Fichero que relaciona el código generado con el original para depurar (1).
SPA	*Single-Page Application*: aplicación que se carga una vez y construye la interfaz en el navegador (1).
Stale-while-revalidate	Estrategia de mostrar los datos en caché y refrescarlos en segundo plano (17).
`staleTime`	Tiempo durante el que TanStack Query considera frescos unos datos (17).
`StrictMode`	Modo de desarrollo de React que renderiza dos veces y remonta componentes para detectar errores (8).
Tailwind CSS	Motor de estilos basado en clases de utilidad (10).
TanStack Query	Librería de caché y sincronización del estado del servidor (17).
Testing Library	Librería para probar componentes como los usaría un usuario (23).
Tipado estructural	Compatibilidad de tipos por su forma, no por su nombre (2).
Tipo literal	Tipo que solo admite un valor concreto, como `'asc'` (2).
Tipo unión	Tipo que admite uno entre varios tipos o valores, que se escriben separados por una barra vertical; por ejemplo, `SortDirection` admite `'asc'` o `'desc'` (2).
Token de diseño	Variable con un valor del sistema visual (color, radio, fuente) (10).
Tree shaking	Eliminación del código importado que no se usa (1).
TypeScript	Superconjunto de JavaScript con tipos estáticos que se eliminan al compilar (2).
`unknown`	Tipo de lo que no se conoce; obliga a comprobarlo antes de usarlo (2).
`URLSearchParams`	API estándar para leer y construir cadenas de consulta (15, 19).
Valor derivado	Valor calculado durante el renderizado a partir de props, estado o datos (7).
Variante (Tailwind)	Prefijo que condiciona una clase: `hover:`, `sm:`, `data-[state=open]:` (10).
Vite	Herramienta de desarrollo y construcción del proyecto (4).
Vitest	Ejecutor de pruebas del ecosistema Vite (23).

Término	Definición
Zod	Librería de esquemas para validar datos y derivar sus tipos (20).

ANEXO C

Equivalencias Java ↔ frontend

↑ Volver al índice

Una tabla de consulta rápida con los conceptos del mundo Java/Spring que tienen un equivalente razonable en el frontend del proyecto. Las analogías ayudan a orientarse, pero ninguna es exacta: la última columna recoge la diferencia que más conviene recordar.

C.1 Lenguaje

↑ Índice

Java	TypeScript / JavaScript	Diferencia clave
Clase DTO o `record`	`interface` o `type`	Los tipos desaparecen en ejecución: no validan nada
Tipado nominal (`implements`)	Tipado estructural	Basta con tener la forma; no hay que declarar nada
`enum`	Unión de literales + `as const`	En ejecución son cadenas, exactamente como en el JSON
`final`	`const`	`const` impide reasignar, no modificar el objeto
`Optional.map` / `orElse`	`?.` / `??`	Integrado en el lenguaje, sin envolver el valor
`null`	`null` **y** `undefined`	Dos valores de ausencia con usos distintos
`switch` exhaustivo sobre un enum sellado	`Record<Unión, V>`	El compilador exige todas las claves
Lambda + interfaz funcional	Función flecha	El tipo es la firma; puede modificar variables externas
`Stream.map/filter/findFirst`	`array.map/filter/find`	Sin `stream()` ni `collect()` ; devuelven arrays
`CompletableFuture<T>`	`Promise<T>`	`await` no bloquea el hilo
Hilos, `synchronized`	Un solo hilo + bucle de eventos	Nunca bloquees el hilo
Paquete + `public`	Módulo + `export`	Se pueden exportar funciones y constantes sueltas
Genéricos (`<T extends Foo>`)	Genéricos (`<T extends Foo>`)	Parecidos; los de TypeScript se infieren más
`Object` / `?`	`unknown`	`any` desactiva la comprobación; evítalo
Excepción comprobada	No existen	Cualquier cosa puede lanzarse; lo capturado es `unknown`

C.2 Construcción y ecosistema

↑ Índice

Java	Frontend	Diferencia clave
JDK en la máquina de desarrollo	Node.js	No se despliega: solo ejecuta herramientas
Maven / Gradle	npm + Vite	Las dependencias usan rangos; el *lockfile* da reproducibilidad
`pom.xml`	`package.json`	`scripts` sustituye a las fases de Maven
Ámbitos `compile` / `test`	`dependencies` / `devDependencies`	El criterio es si el código acaba en el *bundle*
`~/.m2`	`node_modules` (por proyecto)	Grande; nunca se versiona
`mvn package`	`npm run build`	Resultado: ficheros estáticos en `dist/`
`mvn compile`	`npm run typecheck`	Vite transpila sin comprobar tipos
Proyecto multimódulo	Referencias de proyecto de TypeScript	Separan código de navegador y de Node
Checkstyle / PMD / SpotBugs	ESLint	Las reglas de *hooks* no tienen equivalente directo
Spring Boot DevTools	HMR + React Fast Refresh	Conserva el estado de la pantalla
Arquetipo / Spring Initializr	CLI de shadcn/ui	Copia código que pasa a ser tuyo

C.3 Arquitectura y Spring

↑ Índice

Spring / backend	Frontend del proyecto	Diferencia clave
Paquete por funcionalidad	`features/<módulo>`	Misma idea, mismas fronteras de dominio
`@RequestMapping`	Tabla de rutas de React Router	Las rutas se anidan y aportan armazones (*layouts*)
Controlador	Página (`pages/`)	Lee la URL, pide datos y elige la vista
`@PathVariable` / `@RequestParam`	`useParams` / `useSearchParams`	Llegan como texto y hay que validarlos a mano
Vista (JSP / Thymeleaf)	Componente	Se re-evalúa en el navegador cada vez que cambian los datos
Fragmento parametrizado de Thymeleaf	Props de tipo `ReactNode` , `children`	Composición en lugar de herencia de plantillas
`@Configuration` + beans	Proveedores en `app.tsx`	Inyección por posición en el árbol, no por tipo

Spring / backend	Frontend del proyecto	Diferencia clave
`@Autowired`	`useContext` / hooks	Busca el proveedor más cercano hacia arriba
Servicio	Hook personalizado	No es un *singleton*: la lógica se instala en cada componente
Dominio / entidad con reglas	`model/` (tipos, reglas, esquemas)	Refleja reglas del backend, que sigue siendo la autoridad
`RestClient` / Feign	`api-client.ts` + `-api.ts`	Construido sobre `fetch`
`ResponseErrorHandler`	`toApiError`	Interpreta varios formatos de error
`@RestControllerAdvice`	`ErrorState`, `applyServerFieldErrors`, `errorElement`	La presentación del error la decide cada pantalla
Caché de segundo nivel / `@Cacheable`	TanStack Query	Además notifica a la interfaz y refresca en segundo plano
`@CacheEvict`	`invalidateQueries`	Invalidación por prefijo de clave
Comprobación de cambios de Hibernate	Reconciliación de React	Tú describes el estado; el motor aplica solo las diferencias
Entidad gestionada mutable	Estado inmutable	Se sustituye por copias; mutarlo no provoca renderizado
`FetchType.LAZY` / carga perezosa de clases	`lazy` + `import()` dinámico	La primera vez hay una pequeña espera
Bean Validation (`@NotBlank`, `@Size`)	Esquemas de Zod	Validación de comodidad; el backend vuelve a validar
`application.properties` / perfiles	`.env` + `VITE_*`	Se incrustan en el *bundle* al construir: nunca secretos
Hilo virtual que espera E/S	`await`	Mismo efecto: esperar sin bloquear

C.4 Pruebas

↑ Índice

Java	Frontend	Diferencia clave
JUnit 5	Vitest (`describe`, `it`)	Misma estructura; sin anotaciones
AssertJ	`expect(…).toBe/toEqual`	`toBe` compara referencias; `toEqual`, contenido
`@ParameterizedTest`	`it.each`	
Mockito	`vi.fn`, `vi.spyOn`, `vi.mock`	Úsalos con moderación
MockMvc	Testing Library	Se interactúa como un usuario: por rol y texto

Java	Frontend	Diferencia clave
WireMock	MSW	Intercepta `fetch` en las pruebas
Testcontainers + pruebas de extremo a extremo	Playwright contra el backend real	Navegador real
ArchUnit	`no-restricted-imports`, `eslint-plugin-boundaries`	Reglas de dependencia entre carpetas
Spring Cloud Contract / Pact	Pruebas de contrato en `model/`, OpenAPI	Hoy manuales en el proyecto

C.5 Despliegue

↑ Índice

Java	Frontend	Diferencia clave
JAR ejecutable con JVM	Ficheros estáticos en un servidor web	No hay nada que ejecutar en el servidor
Imagen Docker multietapa con Maven y JRE	Imagen multietapa con Node y Nginx	La imagen final no lleva Node
CORS en Spring (`addCorsMappings`)	Mismo origen con proxy (recomendado)	Sin proxy, CORS debe configurarse en cada servicio
Construir una vez, desplegar muchas	URL de API relativa `/api`	Evita reconstruir por entorno

ANEXO D

Chuleta de comandos y atajos

↑ Volver al índice

Los comandos y atajos que usarás a diario con el proyecto, agrupados por herramienta.

D.1 Arranque diario

↑ Índice

```
# 1. Backend: servicios de aplicaciones (8081) y demandas (8080) arrancados
# 2. Frontend
cd C:\Workspace\it-service-manager-web
npm install          # solo si cambió package.json o tras cambiar de rama
npm run dev          # http://localhost:5173
```

D.2 Scripts del proyecto

↑ Índice

Comando	Qué hace
`npm run dev`	Servidor de desarrollo con recarga en caliente y proxy a la API
`npm run build`	Comprueba tipos y genera la versión de producción en `dist/`
`npm run preview`	Sirve `dist/` en local para probar la versión de producción
`npm run typecheck`	Solo comprobación de tipos (`tsc -b`)
`npm run lint`	ESLint sobre todo el proyecto
`npm run test`	Todas las pruebas una vez (`vitest run`)

Antes de dar algo por terminado: `npm run typecheck && npm run lint && npm run test`.

D.3 npm

↑ Índice

Comando	Qué hace
`npm install`	Instala dependencias según `package.json` y el *lockfile*
`npm ci`	Instalación limpia y exacta desde el *lockfile* (integración continua)
`npm install paquete` / `npm install -D paquete`	Añade una dependencia / de desarrollo
`npm uninstall paquete`	Elimina una dependencia
`npm outdated`	Muestra dependencias desactualizadas
`npm update`	Actualiza dentro de los rangos permitidos

Comando	Qué hace
`npm ls paquete` / `npm explain paquete`	Quién depende de un paquete y por qué está instalado
`npm audit`	Vulnerabilidades conocidas
`npx herramienta`	Ejecuta una herramienta instalada en el proyecto
`node -v` / `npm -v`	Versiones instaladas

D.4 Vite

↑ Índice

Comando	Qué hace
`npx vite --force`	Arranca regenerando la caché de dependencias (`node_modules/.vite`)
`npx vite --port 5180`	Arranca en otro puerto
`npx vite build --mode production`	*Build* con un modo concreto (ficheros `.env.<modo>`)

D.5 Vitest

↑ Índice

Comando	Qué hace
`npx vitest`	Modo vigilancia
`npx vitest run fichero`	Solo los ficheros que contienen el texto
`npx vitest run -t "texto"`	Solo las pruebas cuyo nombre contiene el texto
`npx vitest run --reporter=verbose`	Lista cada prueba
`npx vitest run --coverage`	Cobertura (requiere `@vitest/coverage-v8`)
`npx vitest --ui`	Interfaz web (requiere `@vitest/ui`)

D.6 shadcn/ui

↑ Índice

Comando	Qué hace
`npx shadcn@latest add dialog`	Copia el componente en `src/components/ui` e instala sus dependencias
`npx shadcn@latest add dialog --overwrite`	Vuelve a copiarlo (para actualizar; compara con Git después)

D.7 Git (flujo recomendado para los ejercicios)

↑ Índice

```
git switch -c ejercicio/19-1-buscador      # rama para el ejercicio
git add -p && git commit -m "…"            # commits pequeños
git switch main && git branch -D ejercicio/19-1-buscador   # descartar al terminar
```

D.8 Herramientas del navegador

↑ Índice

Atajo (Chrome/Edge en Windows)	Acción
F12 o Ctrl+Mayús+I	Abrir las DevTools
Ctrl+Mayús+C	Seleccionar un elemento de la página
Ctrl+Mayús+M	Modo de dispositivo (móvil)
Ctrl+P (en DevTools)	Abrir un fichero fuente para depurar
Ctrl+Mayús+P (en DevTools)	Paleta de comandos de las DevTools
Ctrl+F5 / Ctrl+Mayús+R	Recarga sin caché
Pestaña *Network* → *Fetch/XHR*	Solo las llamadas a la API
Pestaña *Network* → *Disable cache*	Desactiva la caché mientras están abiertas
Pestaña *Components* (React DevTools)	Árbol de componentes, props, estado y hooks
Pestaña *Profiler* (React DevTools)	Graba y mide los renderizados

D.9 Depurar en el código

↑ Índice

Técnica	Uso
`console.log(valor)`	Lo más rápido; en desarrollo aparece dos veces por `StrictMode`
`console.table(array)`	Muestra un array de objetos como tabla
`debugger;`	Detiene la ejecución en ese punto si las DevTools están abiertas
Punto de ruptura en *Sources*	Clic en el número de línea
Copy as fetch en *Network*	Copia una petición para repetirla o modificarla en la consola

D.10 Comprobar la API desde la consola del navegador

↑ Índice

```
// Listado (pasa por el proxy de Vite: mismo origen)
await fetch('/api/applications?page=0&size=5').then((r) => r.json())

// Alta
await fetch('/api/applications', {
  method: 'POST',
```

```
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'PRUEBA', description: null, criticality: 'LOW' }),
}).then((r) => r.json())
```

D.11 Docker (ejemplo del capítulo 16)

↑ Índice

```
npm run build
docker run --rm -p 8088:80 \
  -v "$PWD/dist:/usr/share/nginx/html:ro" \
  -v "$PWD/nginx.conf:/etc/nginx/conf.d/default.conf:ro" \
  nginx:1.27-alpine
```

D.12 Regenerar esta guía

↑ Índice

```
cd docs/guia-aprendizaje/fuente
pip install weasyprint markdown pygments
python build.py                                    # genera build/guia-frontend-it-service-
manager.pdf
python check_links.py build/guia-frontend-it-service-manager.pdf
```

ANEXO E

Mapa comentado de ficheros

↑ Volver al índice

Todos los ficheros del proyecto, con una línea que explica para qué sirven y el capítulo de la guía donde se estudian. Úsalo como índice para encontrar código o como lista de comprobación al crear un módulo nuevo.

E.1 Raíz del proyecto

↑ Índice

Fichero	Para qué sirve	Cap.
`package.json`	Nombre, scripts, dependencias y versión mínima de Node	3
`package-lock.json`	Versiones exactas de todas las dependencias	3
`index.html`	HTML de partida con `div#root` y el script de entrada	1
`vite.config.ts`	Plugins, alias `@`, puerto, proxy a los dos servicios y configuración de Vitest	4, 23
`tsconfig.json`	Referencias a los dos subproyectos de TypeScript	5
`tsconfig.app.json`	TypeScript del código de navegador (`src/`), modo estricto	5
`tsconfig.node.json`	TypeScript de `vite.config.ts` (entorno Node)	5
`eslint.config.js`	Reglas de ESLint (JS, TypeScript, *hooks*, React Refresh)	5
`components.json`	Configuración de la CLI de shadcn/ui	11
`.env.example`	Variables de entorno documentadas (pendiente de actualizar)	4, 25
`.gitignore`	Ficheros excluidos de Git (`node_modules` , `dist` , `*.local` …)	3
`README.md`	Puesta en marcha y resumen de la arquitectura	—
`public/favicon.svg`	Icono de la pestaña, copiado tal cual a `dist/`	4
`docs/guia-aprendizaje/`	Esta guía y su fuente	—

E.2 `src/` (raíz)

↑ Índice

Fichero	Para qué sirve	Cap.
`main.tsx`	Punto de entrada: monta `App` en `#root` dentro de `StrictMode`	1, 8
`index.css`	Tailwind, animaciones, fuente Inter, tokens de diseño y estilos base	10
`vite-env.d.ts`	Tipos de `import.meta.env` y de Vite	4

E.3 `src/app/`: arranque

↑ Índice

Fichero	Para qué sirve	Cap.
`app.tsx`	Proveedores globales: TanStack Query, *tooltips*, router y notificaciones	1, 9
`router.tsx`	Tabla de rutas: layout, límite de error, redirección inicial, módulos y 404	18
`query-client.ts`	Configuración global de TanStack Query (`staleTime` , reintentos)	17

E.4 `src/components/layout/`: armazón

↑ Índice

Fichero	Para qué sirve	Cap.
`app-layout.tsx`	Armazón con menú, cabecera y `<Outlet />` ; título de la pestaña	12, 8
`sidebar-nav.tsx`	Menú lateral: grupos desplegables por módulo y módulos «Pronto»	12
`navigation.ts`	Definición declarativa de los módulos y acciones del menú	12
`app-header.tsx`	Cabecera fija y menú móvil en panel deslizante	12
`brand.tsx`	Logotipo en SVG	12

E.5 `src/components/common/`: piezas compartidas

↑ Índice

Fichero	Para qué sirve	Cap.
`page-header.tsx`	Cabecera de página: contexto, título, descripción y acciones	6
`state-views.tsx`	`LoadingState` , `EmptyState` y `ErrorState`	6, 12, 15
`table-skeleton.tsx`	Esqueleto de carga de las tablas	12
`resource-not-found.tsx`	Aviso de recurso inexistente con enlace al listado	12, 18
`pagination-bar.tsx`	Paginación con filas por página y rango mostrado	6, 19
`sortable-table-head.tsx`	Cabecera de columna ordenable con `aria-sort`	6
`filter-select.tsx`	Desplegable de filtro con opción «Todos» (genérico)	2, 11
`badges.tsx`	`StatusPill` (estados) y `LevelBadge` (niveles 1 a 4)	10, 12
`form-fields.tsx`	Campos de formulario genéricos y contador de caracteres	20
`detail-field.tsx`	Par etiqueta/valor y texto largo para fichas	12
`row-action.tsx`	Botón de icono con *tooltip* para acciones de fila	9

E.6 `src/components/ui/`: componentes de shadcn/ui

↑ Índice

Fichero	Para qué sirve	Cap.
`button.tsx`	Botón con variantes (incluida `brand`) y `asChild`	11
`badge.tsx`	Etiqueta genérica	11
`card.tsx`	Tarjeta y sus partes	9
`table.tsx`	Tabla y sus partes	11
`input.tsx`, `textarea.tsx`, `label.tsx`	Controles de formulario	11
`select.tsx`	Desplegable sobre Radix	11
`form.tsx`	Integración con React Hook Form mediante contextos	9, 20
`alert.tsx`	Alertas (incluidas las variantes `success` e `info`)	11
`separator.tsx`, `skeleton.tsx`	Separador y bloque de carga	11
`collapsible.tsx`	Sección plegable (grupos del menú)	12
`sheet.tsx`	Panel lateral (diálogo de Radix)	11, 12
`tooltip.tsx`	*Tooltips* y su proveedor	11
`sonner.tsx`	Contenedor de notificaciones configurado con el tema	11

E.7 `src/lib/`: utilidades sin interfaz

↑ Índice

Fichero	Para qué sirve	Cap.
`api-client.ts`	Cliente HTTP, `ApiError` e interpretación de errores	15
`api-client.test.ts`	Pruebas de la consulta y de los formatos de error	23
`list-params.ts`	Utilidades genéricas de paginación, orden y enumerados en la URL	19
`list-params.test.ts`	Sus pruebas	23
`route-params.ts`	Validación de identificadores de ruta	18
`form-errors.ts`	Errores de validación del servidor → campos del formulario	15, 20
`format.ts`	Formato de fechas en hora local	22
`utils.ts`	`cn()`	10

E.8 `src/pages/`: páginas globales

↑ Índice

Fichero	Para qué sirve	Cap.
`not-found-page.tsx`	Página 404 (ruta comodín)	18

Fichero	Para qué sirve	Cap.
`route-error-page.tsx`	Página de error de renderizado (`errorElement`)	9, 18

E.9 `src/features/applications/`: módulo de aplicaciones

↑ Índice

Fichero	Para qué sirve	Cap.
`routes.ts`	Rutas del módulo con carga diferida y títulos	14, 18
`model/types.ts`	Tipos alineados con los DTO, enumerados y parámetros del listado	2, 14
`model/constants.ts`	Etiquetas, valores por defecto y longitudes máximas	14
`model/list-params.ts` (+ `.test.ts`)	URL ↔ parámetros ↔ consulta de Spring	19
`model/schemas.ts` (+ `.test.ts`)	Esquemas de Zod de alta y modificación	20
`api/applications-api.ts`	Un método por endpoint	14, 15
`api/applications-queries.ts`	Claves de caché y hooks de consulta y mutación; catálogo para otros módulos	17
`components/applications-table.tsx`	Tabla con cabeceras ordenables y modos gestión/selección	6
`components/application-filters.tsx`	Filtros de estado y criticidad	6, 19
`components/application-badges.tsx`	Estado y criticidad → indicadores comunes	12
`components/application-form-fields.tsx`	Campos configurados del formulario	9, 20
`components/application-summary.tsx`	Ficha de solo lectura	12
`pages/applications-list-page.tsx`	Consulta con filtros, orden y paginación en la URL	19, 21
`pages/application-create-page.tsx`	Alta y resultado del servidor	20, 21
`pages/application-detail-page.tsx`	Detalle y enlace a sus demandas	21
`pages/application-edit-selector-page.tsx`	Búsqueda por ID y selección en el catálogo	7, 21
`pages/application-edit-page.tsx`	Modificación con `key` , cambios pendientes y resultado	7, 20, 21

E.10 `src/features/demands/`: módulo de demandas

↑ Índice

Fichero	Para qué sirve	Cap.
`routes.ts`	Rutas del módulo	18
`model/types.ts`	Tipos, enumerados y campos de ordenación (ver nota 1 del cap. 25)	14, 25

Fichero	Para qué sirve	Cap.
`model/constants.ts`	Etiquetas, valores por defecto (orden descendente) y longitudes	22
`model/list-params.ts` (+ `.test.ts`)	URL del listado con filtro por aplicación	19
`model/schemas.ts` (+ `.test.ts`)	Esquemas de Zod; el de modificación depende del estado actual	20, 22
`model/status-rules.ts` (+ `.test.ts`)	Máquina de estados e interpretación del 409	22
`api/demands-api.ts`	Endpoints	14
`api/demands-queries.ts`	Claves de caché y hooks	17
`components/application-options.ts`	Catálogo de aplicaciones para desplegables (dependencia entre módulos)	13, 22
`components/demand-filters.tsx`	Filtros de aplicación, estado y prioridad	22
`components/demand-badges.tsx`	Estado y prioridad → indicadores comunes	12
`components/demand-form-fields.tsx`	Campos, selector de aplicación y selector de estado con transiciones	20, 22
`components/demand-summary.tsx`	Ficha con fechas	22
`components/demands-table.tsx`	Tabla con enlace a la aplicación y fecha de modificación	22
`components/status-flow.tsx`	Diagrama del ciclo de vida	22
`pages/demands-list-page.tsx`	Consulta	22
`pages/demand-create-page.tsx`	Alta con aplicación preseleccionable por URL	22
`pages/demand-detail-page.tsx`	Detalle y ciclo de vida	22
`pages/demand-edit-selector-page.tsx`	Búsqueda para modificar	21, 22
`pages/demand-edit-page.tsx`	Modificación, conflicto 409 y recarga	22

E.11 Para un módulo nuevo

↑ Índice

La estructura de cualquier módulo nuevo debería reproducir la de la última tabla. Usa esta lista como comprobación (capítulos 14 y 26):

```
src/features/<módulo>/
├── routes.ts
├── model/   types.ts · constants.ts · list-params.ts · schemas.ts · (reglas).ts · *.test.ts
├── api/     <módulo>-api.ts · <módulo>-queries.ts
├── components/  badges · filters · form-fields · summary · table
└── pages/   list · create · detail · edit-selector · edit
+ app/router.tsx · components/layout/navigation.ts · vite.config.ts (proxy)
```